



microBee

RV32IMC Soft-Core Mikrodenetleyici IP Ailesi

EHB326E-RISCV-MCU-Book

AUTHORS

Süleyman Emre Önsay^{1,2}, Fatih Abdullahcan Güler²,
Alp Eren Kıyak¹, Müştak Erhan Yalçın¹

¹İstanbul Teknik Üniversitesi, Elektrik-Elektronik Fakültesi

²TÜBİTAK — Tümdevre Tasarım ve Eğitim Laboratuvarı (TÜTEL)

onsay20@itu.edu.tr, fatih.guler@bursiyer.tubitak.gov.tr,
kiyak20@itu.edu.tr, mustak.yalcin@itu.edu.tr

Belge numarası:	microBee-DS-001
Revizyon:	P1.0
Ürün tipi:	Sentezlenebilir SystemVerilog IP
İşlemci mimarisi:	32-bit RISC-V RV32IMC
Referans profil:	microBee-REF32
Yayın tarihi:	9 Eylül 2026

Ürün kapsamı. microBee paketlenmiş bir yarı iletken değil, FPGA tasarımına eklenen bir mikrodenetleyici IP'sidir. Bu sistem referansı işlemci davranışını, mantıksal portları, bus protokolünü, bellek ve register haritalarını, interrupt modelini ve yapılandırma kurallarını tanımlar. Gerilim, sıcaklık, I/O standardı, güç ve azami saat frekansı seçilen FPGA, kart, constraint ve implementation sonucuna göre ayrıca doğrulanır.

Belge Kullanım Kuralları

Bu belge, microBee RTL tasarımcısı ve bu örnek üzerinde çalışan öğrencinin iletişim kurmasını amaçlayan bir teknik anlatım ve ders materyalidir.

Tablo 1: Veri sınıfları

Sınıf	Anlam
Sabit	microBee mimarisinin bütün uyumlu örneklerinde değişmeyen özelliktir.
Yapılandırılabilir	Sentez ya da SoC entegrasyonu sırasında seçilir ve bitstream boyunca sabittir.
REF32 profili	Kullanıma hazır microBee-REF32 profilinde sabitlenen örnek değerdir.
Hedefe bağlı	FPGA, kart, bellek makrosu, araç sürümü veya timing constraint'i tarafından belirlenir.
Ayrılmış	İleri kullanım için ayrılmıştır; yazılım aksi belirtilmedikçe sıfır yazmalıdır.

Normatif kaynak sırası.

microBee Gereksinim ve Mimari Karar Matrisi ürünün üst düzey mimari sözleşmesidir. ISA ve machine-mode davranışında RISC-V International'ın ratified spesifikasyonları geçerlidir [1, 5]. Bu veri sayfası microBee'ye özgü port, register ve entegrasyon ayrıntılarını tanımlar. Bir profil manifesti yalnızca bu belgede yapılandırılabilir bırakılmış değerleri sabitleyebilir.

İçindekiler

Belge Kullanım Kuralları	i
1 Tasarım Özeti	1
1.1 Açıklama	1
2 Giriş	2
2.1 Nasıl Kullanılır?	2
2.1.1 Block Design	2
2.2 Xilinx Vivado	3
2.2.1 Nedir?	3
2.2.2 Neyi Amaçlar?	5
2.3 IP Bloğu Nedir?	5
2.3.1 IP Repository'yi IP Catalog'a Ekleme	6
2.4 Hazırlanmış Bloğu IP'ye Dönüştürme	8
2.5 Block Design Oluşturma ve microBee Ekleme	8
2.5.1 Boş Block Design	8
2.5.2 microBee SoC'yi Yerleştirme	9
2.5.3 AXI UART Lite Ekleme	10
2.6 Connection Automation ile Blokların Bağlanması	12
2.6.1 Birinci Aşama: UART Lite'ın AXI ve Dış Port Bağlantıları	12
2.6.2 İkinci Aşama: Clock ve Reset Dış Girişleri	14
2.7 External Interrupt Vektörünün Oluşturulması	15
2.7.1 Modülün Görevi	16
2.7.2 Block Design'a RTL Modül Ekleme	17
2.8 IMEM / DMEM Başlatma Dosyaları	20
2.9 HDL Wrapper	24
2.10 Simülasyon Sistemi	26
2.10.1 Nedir?	26
2.10.2 Testbench Ekleme	26
2.10.3 Simulation Top'u Ayarlama	29
2.10.4 Sınırları Nelerdir?	31
2.11 Vivado Çıktılarının İzlenmesi	31
2.11.1 Waveform	31
2.11.2 Debug	33
2.12 Kısa Kontrol Listesi	33
3 Mimari	34

3.1	Mimari - Bilgisayar Mimarisi	34
3.1.1	Mimari nedir?	34
3.1.2	Mimari neleri etkiler?	34
3.1.3	Neden Mimariyi bilmelisiniz?	34
3.2	Genel Mimari	34
3.2.1	Top	34
3.2.2	I/O	36
3.2.3	Pipeline	38
3.3	Register File	39
3.3.1	Özellikleri	40
3.3.2	Kullanım Örneği	40
3.3.3	Debug Nasıl Yapılır?	40
3.4	Instruction Kavramı	40
3.4.1	Nedir?	40
3.4.2	Nerede Depolanır	41
3.4.3	Nasıl Oluşur	41
3.4.4	Kim Belirliyor	41
3.5	Aritmetik Mantık Birimi (Arithmetic Logic Unit)	41
3.6	MUL/DIV	42
3.6.1	MUL	42
3.6.2	DIV	42
3.6.3	Neden Ayrı Birimler ve Pipeline Kullanılıyor?	42
3.7	Dallanma (Branch)	42
3.7.1	Nedir?	42
3.7.2	Nasıl İfade Edilir?	43
3.7.3	Nasıl Çözülür?	43
3.8	Stall ve Flush Kavramları	44
3.8.1	Stall Nedir?	44
3.8.2	microBee'de Stall Oluşturan Durumlar	44
3.8.3	Flush Nedir?	45
3.8.4	microBee'de Branch Flush	45
3.8.5	Stall ve Flush Arasındaki Fark	45
3.8.6	Bubble Kavramı	46
3.9	Dallanma Öngörüsü (Branch Prediction)	46
3.9.1	Nedir?	46
3.9.2	Neden Gereklidir?	46
3.9.3	Taken ve Not-Taken	46
3.9.4	Bimodal Branch Prediction	47
3.9.5	Prediction Doğruysa Ne Olur?	47
3.9.6	Prediction Yanlıysa Ne Olur?	47
3.9.7	Embedded Sistemlerde Neden Basit Predictor Kullanılır?	47
3.9.8	Branch Prediction Performansı	48
3.9.9	microBee Açısından Branch Prediction	48
3.9.10	Örnek	48
3.10	DMEM	49
3.10.1	Nedir?	49

3.10.2	Load ve Store	49
3.10.3	Adres Nasıl Hesaplanır?	49
3.10.4	Veri Boyutları	50
3.10.5	Byte Enable	50
3.10.6	Memory Stall	51
3.10.7	DMEM ve Pipeline	51
3.10.8	Load-Use Hazard	51
3.10.9	Embedded Sistem Açısından DMEM	52
4	Instruction Set	53
4.1	ISA Nedir?	53
4.1.1	Tanım	53
4.1.2	Kim Belirler?	53
4.1.3	Ne İşe Yarar?	54
4.1.4	Neden Sabit ISA?	54
4.2	I Seti	55
4.2.1	R-Type Formatı	55
4.2.2	I-Type Arithmetic Instruction'ları	57
4.2.3	Shift Immediate Instruction'ları	58
4.2.4	Load Instruction'ları	58
4.2.5	Store Instruction'ları	59
4.2.6	Branch Instruction'ları	59
4.2.7	Jump Instruction'ları	60
4.2.8	Upper Immediate Instruction'ları	61
4.2.9	FENCE	61
4.2.10	Environment Instruction'ları	61
4.3	M Seti	62
4.3.1	MUL	62
4.3.2	MULH	62
4.3.3	MULHSU	62
4.3.4	MULHU	62
4.3.5	DIV	62
4.3.6	DIVU	63
4.3.7	REM	63
4.3.8	REMU	63
4.4	C Seti	63
4.4.1	C.ADDI4SPN	64
4.4.2	C.LW	64
4.4.3	C.SW	64
4.4.4	C.NOP	64
4.4.5	C.ADDI	64
4.4.6	C.JAL	64
4.4.7	C.LI	65
4.4.8	C.ADDI16SP	65
4.4.9	C.LUI	65
4.4.10	C.SRLI	65

4.4.11	C.SRAI	65
4.4.12	C.ANDI	65
4.4.13	C.SUB	65
4.4.14	C.XOR	65
4.4.15	C.OR	65
4.4.16	C.AND	66
4.4.17	C.J	66
4.4.18	C.BEQZ	66
4.4.19	C.BNEZ	66
4.4.20	C.SLLI	66
4.4.21	C.LWSP	66
4.4.22	C.JR	66
4.4.23	C.MV	67
4.4.24	C.EBREAK	67
4.4.25	C.JALR	67
4.4.26	C.ADD	67
4.4.27	C.SWSP	67
4.5	NOP	67
4.6	ZicSr	68
4.6.1	CSRrW	68
4.6.2	CSRrS	69
4.6.3	CSRrC	69
4.6.4	CSRrWI	69
4.6.5	CSRrSI	69
4.6.6	CSRrCI	69
4.7	Zifencei	70
4.7.1	FENCE.I	70
5	Interrupts	71
5.1	Nedir?	71
5.2	Ne İşe Yarar?	71
5.3	microBee'de Interrupt Kaynakları	72
5.4	Interrupt Kabul Koşulu	72
5.4.1	microBee'de mip Davranışı	72
5.5	Interrupt Önceliği	73
5.6	Precise Interrupt ve Pipeline Drain	73
5.7	Interrupt Sırasında mepc	73
5.8	Trap Girişinde mstatus	74
5.9	mret	74
5.10	Nested Interrupt	74
5.11	mtvec	75
5.12	CLINT	75
5.12.1	Software Interrupt	75
5.12.2	Timer Interrupt	76
5.13	PLIC	77
5.13.1	Interrupt ID	78

5.13.2	Priority ve Enable	78
5.13.3	Trigger Modları	79
5.13.4	Claim ve Complete	79
5.14	Shadow Register Bank	80
5.15	Interrupt Çözme Mekanizması	80
5.16	Interrupt ve Exception Farkı	80
5.17	Öğrenci Kurulum Örneği (PLIC kaynak 1)	81
5.18	Software Interrupt Örneği	83
5.19	Hardware Interrupt Akışı (Özet)	83
5.20	Özet	83
6	Genel Amaçlı Giriş/Çıkış Portları (GPIO) ve Çevre Birimleri	85
6.1	GPIO Tasarımı	85
6.1.1	GPIO Portları	85
6.1.2	Input Senkronizasyonu	86
6.1.3	Reset Davranışı	86
6.1.4	Bus Erişimi	86
6.2	GPIO'nun Görevleri	87
6.3	GPIO Adres Haritası	87
6.3.1	IN	87
6.3.2	OUT	88
6.3.3	OUT_SET	88
6.3.4	OUT_CLR	88
6.3.5	OUT_TGL	88
6.3.6	OE	89
6.3.7	VALID	89
6.3.8	Birden Fazla GPIO Bankası	89
6.4	GPIO_ENABLE = 0 Davranışı	89
6.5	Register Güncelleme Zamanı	90
6.6	GPIO Nasıl Kullanılır?	90
6.6.1	Örnek: GPIO Bit 0 ile LED Kontrolü	90
6.6.2	Örnek: GPIO Bit 1 Giriş Okuma	91
6.7	GPIO Doğrulama İpuçları	91
6.8	Diğer Native Çevre Birimleri	91
6.9	Native Timer	91
6.10	Native UART	92
6.11	AXI UART Lite	93
6.11.1	AXI UART Lite register özeti (PG142)	93
6.11.2	Uygulama: sıfırdan kurulum, PLIC, AXI yaz/oku	94
7	Konfigürasyonlar	97
7.1	Sabit Konfigürasyonlar	97
7.1.1	RV32C	97
7.1.2	Register file ve LUT/FF	98
7.2	Yapılandırılabilir Konfigürasyonlar	98
7.2.1	REF32 özeti	98

7.2.2	MUL/DIV	98
7.2.3	Dallanma tahmincisi	98
7.2.4	Gölge bank	99
7.2.5	IMEM / DMEM boyutu ve ön yükleme	99
7.2.6	PLIC	99
7.2.7	GPIO	99
7.2.8	AXI	100
7.2.9	CLINT, timer, native UART, monitoring	100
7.2.10	TRACE_ENABLE ve DEBUG_ENABLE	100
7.2.11	Reset vektörü ve adres pencereleri	100
7.2.12	Tek parametre değiştirme prensibi	100
7.3	Sabit Özellikler ve Sık Karıştırılan Noktalar	100
8	Monitoring ve Performans Sayaçları	102
8.1	Nedir?	102
8.2	Sayaç Modeli	102
8.3	MON_CTRL	103
8.3.1	ENABLE	103
8.3.2	FREEZE	103
8.3.3	CLEAR	103
8.3.4	SNAPSHOT	104
8.4	Kontrol Önceliği	104
8.5	Monitoring Register Haritası	104
8.6	Temel Sayaçların Anlamı	106
8.6.1	CYCLES	106
8.6.2	RETIRED	106
8.6.3	STALL	106
8.6.4	FLUSH	106
8.6.5	BUBBLE	106
8.6.6	HAZARD ve LOAD_USE	106
8.7	Branch Sayaçları	107
8.7.1	BRANCH	107
8.7.2	BRANCH_TAKEN	107
8.7.3	MISPREDICT	107
8.7.4	PREDICT_CORRECT	107
8.7.5	SPEC_REDIRECT	108
8.7.6	BRANCH_STALL	108
8.8	Memory Bekleme Sayaçları	108
8.8.1	IMEM_WAIT	108
8.8.2	DMEM_WAIT	108
8.8.3	MEM_WAIT_TOTAL	108
8.8.4	AXI_WAIT	109
8.9	BUS_ERROR ve IRQ_TAKEN	109
8.9.1	BUS_ERROR	109
8.9.2	IRQ_TAKEN	109
8.10	Execution Birimi Kullanımı	109

8.11	Load ve Store Sayaçları	109
8.12	CAPABILITY Register'ı	110
8.13	Performans Ölçümleri	110
8.13.1	IPC	110
8.13.2	CPI	110
8.13.3	Stall Oranı	111
8.13.4	Branch Prediction Doğruluğu	111
8.14	Sayaçlar Birlikte Nasıl Yorumlanır?	111
8.15	Ölçüm Yaparken Dikkat Edilmesi Gerekenler	112
8.16	CPI Hedefinin Yorumlanması	112
8.17	Örnek Ölçüm Akışı	112
8.17.1	Örnek: CYCLES sayacını AXI UART Lite'a yazmak	113
9	Performans Raporunun Hazırlanması	115
9.1	Sistem Nasıl İzlenir?	115
9.1.1	Waveform ve Debug Sinyalleri	116
9.1.2	Monitoring Sayaçları	116
9.1.3	Simülasyon Log ve Trace	117
9.2	Hangi Yöntem Ne Zaman Kullanılmalı?	117
9.3	Sistem İzleme Araçları	117
9.3.1	AXI UART Lite ve Host Terminal	117
9.3.2	Vivado Waveform	118
9.3.3	Hesaplama	118
9.4	Performans Raporu Nasıl Hazırlanır?	118
9.4.1	Kimlik Bilgileri	119
9.4.2	Ham Sayaçlar	119
9.4.3	Türetilen Metrikler	120
9.4.4	CPI ve IPC Nasıl Yorumlanır?	120
9.5	Sayaçlar Birlikte Nasıl Yorumlanır?	121
9.6	Örnek Performans Raporu	121
9.6.1	Host Çıktısı	121
9.7	Konfigürasyon Karşılaştırması	122
9.7.1	Neden Tek Parametre Değiştirilir?	122
9.7.2	Deney 1: predictor kapasitesi (GShare kapalı)	122
9.7.3	Deney 2: GShare, BP_ENTRIES=128	123
9.7.4	Deney Sonucunun Özeti	123
9.8	Sık Yapılan Hatalar	123
9.8.1	MONITOR_ENABLE = 0	123
9.8.2	Sayaç Doyumu (Saturation)	124
9.8.3	Freeze veya Snapshot Kullanmamak	124
9.8.4	UART Çıktısını Ölçüme Dahil Etmek	124
9.8.5	Farklı Binary'leri Core Karşılaştırması Sanmak	124
9.8.6	Simülasyon Cycle'ını Gerçek Zaman Sanmak	125
9.8.7	Tek Metriğe Bakmak	125
9.8.8	Live ve Shadow Sayaçları Karıştırmak	125
9.9	Minimum Geçerli Performans Raporu	125

10 Debug Portları	126
10.1 Debug Portlarının Yapısı	126
10.1.1 Boru hattı PC gözlemi	126
10.1.2 Yazmaç dosyası yazma portu	127
10.1.3 CSR anlık görüntüsü	127
10.1.4 İsteğe bağlı iz (TRACE_ENABLE)	128
10.2 Debug Portlarının Kullanımı	128
10.2.1 Ne zaman dbg_*, ne zaman monitoring?	128
10.2.2 Vivado Block Design simülasyonu	128
10.2.3 Verilator / tmcu_top	129
10.2.4 Okuma düzeni	129
10.3 Debug Portlarının Örnekleri	129
10.3.1 Reset sonrası ilk getirme	129
10.3.2 Bir ALU yazması	129
10.3.3 Yükleme	129
10.3.4 Dallanma	129
10.3.5 Trap	130
10.3.6 UART'a yazılan bir karakter	130
10.4 Debug Portlarının Sınırlamaları	130
11 rv32 Emülatörü ve Assembly Geliştirme Ortamı	131
11.1 Giriş	131
11.1.1 Nedir?	131
11.1.2 Neyi Amaçlar?	132
11.1.3 Nasıl Kullanılır?	132
11.1.4 Sınırları Nelerdir?	132
11.2 Kurulum ve İlk Açılış	132
11.2.1 Windows	132
11.2.2 Linux	132
11.2.3 Çalışma Dosyalarının Düzeni	132
11.2.4 Açılış Sorunları Nasıl Çözülür?	133
11.3 Arayüz ve Çalıştırma Denetimleri	133
11.4 İlk Program: Aritmetik, Store ve Load	134
11.4.1 Kullanım Örneği	134
11.4.2 Nasıl Çalıştırılır?	134
11.4.3 Öğrenci Uygulaması	134
11.5 Yazmaç, Bellek ve Instruction Gözlemi	135
11.5.1 Register File Nasıl İzlenir?	135
11.5.2 DMEM Adresi Nasıl Hesaplanır?	135
11.5.3 Encoding ve Symbols Ne Gösterir?	135
11.6 Pipeline ve Hata Ayıklama	135
11.6.1 Pipeline Nasıl Okunur?	135
11.6.2 Bir Adım Ne Anlama Gelir?	136
11.6.3 Donanım Pipeline'ından Farkı Nedir?	136
11.7 Breakpoint, Geri Adım ve Tam Mesajları	136
11.7.1 Döngü Nasıl İncelenir?	136

11.7.2 Kod Tamamlama Nasıl Kullanılır?	136
11.8 Çevre Birimleri ve Interrupt Çalışmaları	137
11.8.1 UART Kullanım Örneği	137
11.8.2 Konfigürasyon Nasıl Yüklenir?	137
11.8.3 Interrupt Çalışırken Neye Dikkat Edilir?	137
11.9 Bellek Görüntüsü Üretme	138
11.9.1 Generate Nasıl Kullanılır?	138
11.9.2 Dosya Nasıl Okunur?	138
11.9.3 Vivado'ya Geçiş Koşulu	138
11.10 microBee-REF32 ile Birlikte Kullanım	138
11.10.1 Bir Board Dosyası REF32 Uyumu Sağlar mı?	139
11.10.2 Taşıma Nasıl Hazırlanır?	139
11.11 Komut Satırı ve Tekrarlanabilir Deney	139
11.11.1 Temel Komutlar	139
11.11.2 Debugger Nasıl Kullanılır?	139
11.12 Öğrenci Uygulamaları ve Sonuç Kaydı	140
11.12.1 Uygulama Sırası	140
11.12.2 Sonuç Nasıl Raporlanır?	140
11.12.3 Kontrol Soruları	140
11.13 Kaynaklar ve Bölümler Arası İlişki	141
A Adres Haritası ve Sistem Kapasitesi	143
A.1 REF32 Kapasite Özeti	143
A.2 Global Bellek Haritası	144
A.3 DMEM — 0x20000000	144
A.4 CLINT — 0x40000000	145
A.5 PLIC — 0x40001000	145
A.6 GPIO — 0x40002000	146
A.7 TIMER — 0x40003000	146
A.8 Native UART — 0x40004000	146
A.9 MONITOR — 0x40005000	147
A.10 AXI UART Lite — 0x60000000	148
A.11 IMEM — 0x80000000	148
A.11.1 Hizalama	149
A.12 CSR Numaraları	149
A.12.1 mcause Kodları	150

Şekil Listesi

2.1	Örnek laboratuvar Block Design'ının nihai hali: <code>tmcu_soc_0</code> , AXI Interconnect, AXI UART Lite, Clocking Wizard, Processor System Reset ve <code>interrupt_order_manager</code> .	3
2.2	Vivado Project Summary ekranı: hedef FPGA, üst modül ve synthesis/implementation durumu.	4
2.3	Flow Navigator içerisinde Project Manager, IP Integrator, Simulation ve sonraki FPGA tasarım adımları.	5
2.4	Project Manager altında IP Catalog.	6
2.5	IP Catalog üzerinde Add Repository seçimi.	7
2.6	<code>tmcu_soc</code> IP repository klasörünün seçilmesi.	7
2.7	Create Block Design penceresi.	8
2.8	Boş Block Design çalışma alanı.	9
2.9	Block Design içerisinde Add IP seçimi.	9
2.10	Add IP aramasında microBee SoC RV32IMC.	10
2.11	Block Design içerisinde yalnızca <code>tmcu_soc_0</code> .	10
2.12	AXI UART Lite IP'sinin Block Design'a eklenmesi.	11
2.13	microBee ve AXI UART Lite henüz birbirine bağlanmadan önce.	11
2.14	Designer Assistance üzerinden Run Connection Automation.	12
2.15	İlk Connection Automation penceresinde AXI UART Lite <code>S_AXI</code> ve UART bağlantılarının seçilmesi.	13
2.16	İlk Connection Automation sonrasında oluşan sistem altyapısı.	14
2.17	İkinci Connection Automation: clock ve reset dış bağlantıları.	14
2.18	Clock ve reset otomasyonu tamamlandıktan sonraki Block Design.	15
2.19	<code>interrupt_order_manager</code> : 16 adet tek-bit interrupt girişinin 16-bit external interrupt vektöründe birleştirilmesi.	16
2.20	Block Design üzerinde Add Module seçimi.	17
2.21	Add Module penceresinde <code>interrupt_order_manager</code> modülünün seçilmesi.	18
2.22	<code>interrupt_order_manager</code> modülünün Block Design'a eklenmesi.	18
2.23	<code>interrupt_array[15:0]</code> çıkışının microBee <code>ext_irq_i[15:0]</code> girişine bağlanması.	19
2.24	AXI UART Lite <code>interrupt</code> çıkışının <code>in_0</code> girişine bağlanması.	20
2.25	Customize IP penceresinde memory ve peripheral parametreleri ile Imem Init File alanı.	21
2.26	Imem Init File yolunun girilmesi.	22
2.27	Customize IP penceresinde Dmem Init File ve AXI adres parametreleri.	23
2.28	Dmem Init File örneği: <code>soc_integration_dmem_poll.hex</code> .	24
2.29	<code>design_1.bd</code> için Create HDL Wrapper işlemi.	25
2.30	Design Sources altında <code>design_1_wrapper</code> modülü.	25
2.31	Add Sources içerisinde simulation source seçimi.	27
2.32	<code>tb_design_1_hello.sv</code> dosyasının simulation source olarak eklenmesi.	27
2.33	Define Module ekranında testbench modülünün oluşturulması.	28
2.34	Testbench kaynağında clock, reset, UART ve beklenen "Hello" mesajının tanımlanması.	28

2.35	Simulation Sources içerisinde Set as Top seçimi.	29
2.36	tb_design_1_hello modülünün simulation top olarak seçilmesi.	30
2.37	Flow Navigator içerisinde Run Simulation.	30
2.38	Behavioral simülasyon waveform'unda UART üzerinden "Hello" karakterlerinin alınması.	32
2.39	Tcl Console: UART "Hello" çıktısı ve PASS: tb_design_1_hello mesajı.	32
3.1	microBee SoC üst düzey mimarisi (tmcu_soc, REF32).	36
3.2	microBee SoC dış I/O portları ve işlevsel grupları (tmcu_soc).	38
3.3	ARM mimarisi ABI haritası [6].	40
3.4	C kaynak kodunun derleyici tarafından RISC-V assembly çıktısına dönüştürülmesi.	41
11.1	İlk instruction write-back aşamasına ulaştığında a0 yazmacı 5 olur. Encoding sekmesi gerçekleşen işlemi gösterir.	133
11.2	İlk program tamamlandıktan sonra veri belleğinde result = 8 gözlenir. Satır adresi ile sütun offset'i birlikte okunur.	135
11.3	sum_loop.s programında pipeline görünümü. Son çevrimler instruction bazında izlenebilir.	136

DRAFT

Tablo Listesi

1	Veri sınıfları	i
3.1	RISC-V integer register'ları [1]	39
3.2	Register file debug sinyalleri ve açıklamaları	40
5.1	microBee interrupt kaynakları ve mcause değerleri.	72
5.2	CLINT timer register adresleri.	76
5.3	PLIC konfigürasyon adresleri ve erişim durumları.	77
6.1	GPIO register adres haritası ve erişim türleri.	87
6.2	AXI UART Lite register adresleri ve temel görevleri.	93
6.3	AXI UART Lite STAT ve CTRL bitleri (PG142).	94
7.1	Sentezden bağımsız sabit mimari.	97
7.2	REF32'de açık birimler ve çekirdek anahtarları. tmcu_soc varsayılını, aksi belirtilmedikçe.	98
8.1	Monitoring register'ları	105
8.2	Türetilen performans ölçümleri	110
9.1	microBee performans gözlem yöntemleri.	115
9.2	Performans raporunda tutulması gereken kimlik bilgileri.	119
9.3	Performans raporunda önerilen temel monitoring sayaçları.	119
9.4	Performans raporunda kullanılan temel metrikler.	120
9.5	Örnek monitoring çıktısının ondalık karşılıkları.	121
9.6	Embench ağırlıklı sonuçlar, GShare kapalı, yalnızca BP_ENTRIES değişir.	122
9.7	Embench ağırlıklı sonuçlar, BP_ENTRIES=128 , GShare taraması.	123
9.8	Predictor deneylerinin kısa özeti.	123
10.1	Debug PC portları ve RTL kaynağı.	126
10.2	Debug geçerli bitleri.	127
10.3	Yazmaç dosyası debug yazma portu.	127
10.4	Debug CSR portları.	128
11.1	Bu bölümde kullanılan bilgi sınıfları	131
11.2	Kurulum sırasında sık görülen durumlar	133
11.3	Temel kısayollar	133
11.4	Program sonunda beklenen değerler	134
11.5	Sık yapılan hatalar ve düzeltme yolu	137
11.6	rv32 varsayılan çevre birimi adresleri	137
11.7	Dışa aktarma işlemlerinin kapsamı	138

11.8 REF32 ve rv32 arasında taşımayı etkileyen farklar	139
11.9 Terminal debugger komutları	140
11.10Başlangıç uygulamaları	140
11.11İşlevsel deney kaydı şablonu	140
11.12Kullanılan kaynaklar	141
A.1 REF32 sistem kapasitesi.	143
A.2 REF32 bellek pencereleri, adres sırasıyla.	144
A.3 CLINT register haritası.	145
A.4 PLIC register haritası.	145
A.5 GPIO register haritası.	146
A.6 Native Timer register haritası.	146
A.7 Native UART register haritası.	147
A.8 Monitoring register haritası.	148
A.9 AXI UART Lite register haritası.	148
A.10 microBee machine-mode CSR kümesi, numara sırasıyla [5].	149
A.11 Exception cause kodları, bit31=0.	150
A.12 Interrupt cause kodları, bit31=1.	150

DRAFT

Bölüm 1

Tasarım Özeti

1.1 Açıklama

microBee, TÜBİTAK Tümden Tasarım ve Eğitim Laboratuvarı (TÜTEL) bursiyer laboratuvarı ve İTÜ Gömülü Sistem Tasarım Laboratuvarı (GSTL) ortaklığında geliştirilen 32-bit bir RISC-V soft-core mikrodenele-yicidir. Gömülü sistem eğitimi, bilgisayar mimarisi laboratuvarları, FPGA tabanlı kontrol sistemleri ve küçük gerçek zamanlı uygulamalar için tasarlanmıştır. Tek bir IP ailesi içinde RV32I, birbirinden bağımsız olarak etkinleştirilebilen multiply ve divide birimleri, RV32C predecoder, machine-mode trap sistemi, yerel interrupt kontrolü, bellek eşlemeli çevre birimleri, performans izleme ve AXI4-Lite bağlantısı bulunur.

Çekirdek single-issue ve in-order çalışır. Ana uygulama düzeni beş aşamalı IF-ID-EX-MEM-WB pipeline'dır. Instruction ve data erişimleri, ayrı Harvard instruction/data yolları üzerinden gerçekleştirilir; her iki yol da aynı valid/ready request-response arayüz protokolünü kullanır. Bu yapı, FPGA block RAM'lerine doğrudan bağlanan küçük sistemlerle AXI4-Lite üzerinden daha geniş SoC entegrasyonlarını aynı işlemci çekirdeğiyle destekler.

Bölüm 2

Giriş

Bu bölümde microBee'nin genel kullanım yapısı ve Xilinx Vivado üzerinde bir öğrenci laboratuvarı kapsamında nasıl çalıştırılacağı anlatılmaktadır.

Amaç, çekirdek RTL'sini satır satır incelemek değil; paketlenmiş `tmcu_soc` IP'sini bir Block Design içerisine yerleştirmek, AXI UART Lite ile haberleştirmek ve behavioral simülasyon üzerinde "Hello" çıktısını doğrulamaktır.

microBee, RV32IMC tabanlı bir soft-core tasarımıdır. FPGA üzerinde kullanılmak üzere IP olarak paketlenir. Clock, reset, AXI master, GPIO, Native UART ve external interrupt gibi arayüzler Block Design üzerinde görünür.

Çekirdek içerisinde bulunan pipeline, memory ve peripheral yapılarının konfigürasyonu ise Customize IP üzerinden verilen parametrelerle belirlenir.

Not

Bu bölümdeki ekran görüntüleri örnek bir Vivado projesinden (`tmcu_ip_deneme_1`, Artix-7 `xc7a100tcsg324-1`) alınmıştır.

IP repository yolu kullanılan bilgisayara ve çalışma dizinine göre değişebilir. Repoda paketlenen microBee IP'si genellikle:

```
work/fpga_synth/ip/tmcu_soc
```

dizini altında bulunur.

Bu bölümde kullanılan laboratuvar örneğinde Block Design ve testbench 100 MHz sistem clock'u ve 9600 baud UART ayarı üzerinden hazırlanmıştır. REF32 profilinde kullanılan 50 MHz / 115200 konfigürasyonu bundan ayrı bir FPGA konfigürasyonudur. Clock ve baud değerleri birlikte değiştirilmelidir.

2.1 Nasıl Kullanılır?

Laboratuvar akışı genel olarak aşağıdaki adımlardan oluşur:

1. Vivado projesi açılır ve microBee IP repository'si IP Catalog'a eklenir.
2. Boş bir Block Design oluşturulur. Ardından `tmcu_soc` ve AXI UART Lite IP'leri tasarıma eklenir.
3. *Connection Automation* kullanılarak AXI, clock ve reset bağlantıları oluşturulur.
4. İsteğe bağlı olarak external interrupt kaynaklarını `ext_irq_i[15:0]` hattına yerleştirmek için `interrupt_order_manager` adlı yardımcı RTL modülü eklenir.
5. IMEM ve DMEM initialization dosyaları Customize IP üzerinden tanımlanır.
6. HDL wrapper oluşturulur. Testbench Simulation Sources altına eklenir ve simulation top olarak seçilir.
7. Behavioral simülasyon çalıştırılır. UART üzerinden gönderilen beklenen metin waveform ve simülasyon konsolunda doğrulanır.

2.1.1 Block Design

Hedef sistem, microBee'nin AXI master olarak çalıştığı küçük bir SoC yapısıdır.

Clocking Wizard sistem clock'unu üretirken Processor System Reset gerekli reset sinyallerini oluşturur. micro-

Bee, AXI Interconnect üzerinden AXI UART Lite peripheral'ına erişir.

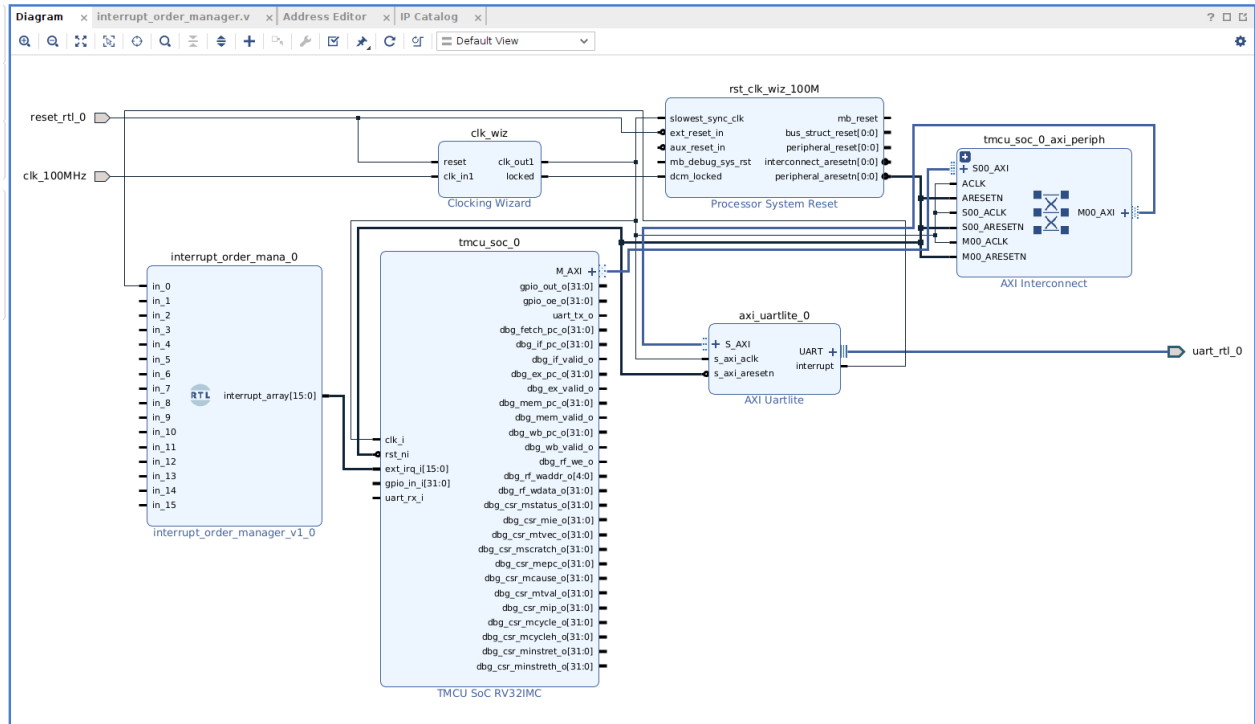
AXI UART Lite interrupt çıkışı, yardımcı `interrupt_order_manager` modülü üzerinden microBee'nin `ext_irq_i[15:0]` external interrupt vektörüne bağlanabilir.

Block Design'ın dış dünya ile temel bağlantıları örnek projede:

- `clk_100MHz`,
- `reset_rtl_0`,
- `uart_rtl_0`

şeklinde dir.

`reset_rtl_0` dış giriş seviyesinde active-high reset olarak kullanılır. microBee'nin active-low `rst_ni` girişi gerekli reset alt yapısı üzerinden üretilir.



Şekil 2.1: Örnek laboratuvar Block Design'ının nihai hali: `tmcu_soc_0`, AXI Interconnect, AXI UART Lite, Clocking Wizard, Processor System Reset ve `interrupt_order_manager`.

Aşağıdaki bölümlerde bu Block Design'ın adım adım nasıl oluşturulduğu anlatılacaktır.

2.2 Xilinx Vivado

2.2.1 Nedir?

Vivado, AMD/Xilinx FPGA'leri için kullanılan entegre FPGA tasarım ortamıdır.

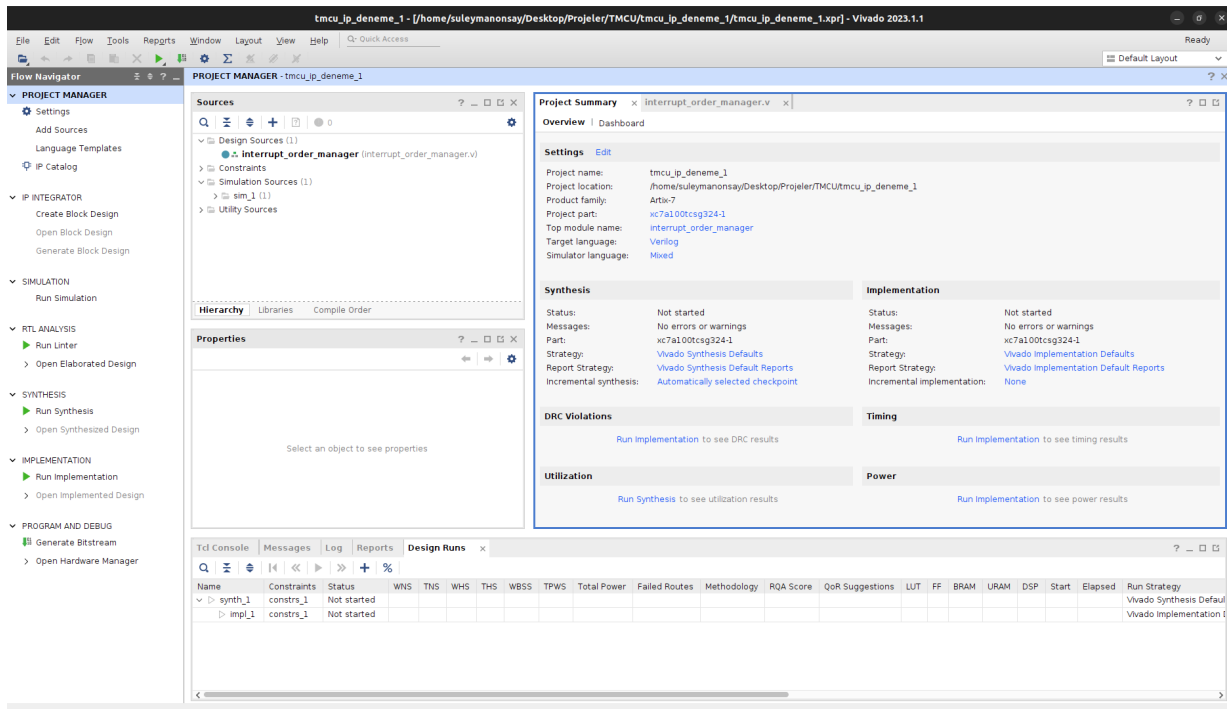
Tek bir proje içerisinde:

- RTL kaynakları,
- IP Catalog,
- IP Integrator ve Block Design,
- behavioral simülasyon,
- synthesis,
- placement ve routing,
- bitstream oluşturma

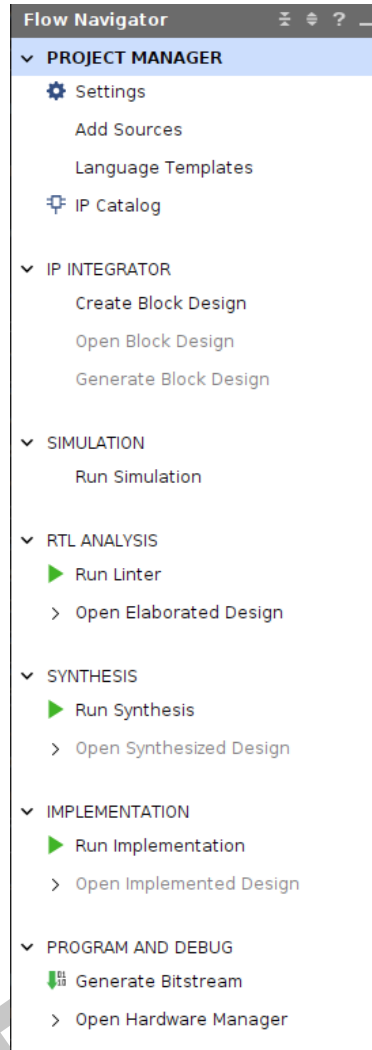
işlemleri yönetilebilir.

Yeni veya boş bir projede ana görünüm genellikle *Project Summary* ekranıdır.

Sol tarafta bulunan *Flow Navigator*, projenin farklı tasarım aşamalarına erişmek için kullanılır.



Şekil 2.2: Vivado Project Summary ekranı: hedef FPGA, üst modül ve synthesis/implementation durumu.



Şekil 2.3: Flow Navigator içerisinde Project Manager, IP Integrator, Simulation ve sonraki FPGA tasarım adımları.

2.2.2 Neyi Amaçlar?

Bu laboratuvarında Vivado'nun üç temel bölümü kullanılacaktır:

- **IP Catalog ve Repository:** Hazır veya kullanıcı tarafından paketlenmiş IP'leri projeye tanıtmak için kullanılır. Repository, IP'nin dosya konumudur; IP Catalog ise Vivado'nun bu IP'leri listelediği arayüzdür.
- **IP Integrator / Block Design:** microBee, clock, reset, AXI peripheral ve yardımcı RTL bloklarını görsel olarak birbirine bağlamak için kullanılır.
- **Simulation:** HDL wrapper ve testbench üzerinden sistemin fonksiyonel davranışını doğrulamak için kullanılır. UART ve gerekli durumlarda `dbg_*` sinyalleri waveform üzerinde izlenebilir.

Bitstream oluşturma ve FPGA'ya yükleme aynı tasarım akışının sonraki aşamalarıdır.

Bu bölümde öncelikli amaç fiziksel FPGA'ya geçmeden önce sistemin behavioral simülasyonda doğru şekilde çalıştığını göstermektir.

2.3 IP Bloğu Nedir?

IP, yani *Intellectual Property*, tekrar kullanılabilir bir donanım bloğudur.

Bir Vivado IP'si genel olarak:

- belirli giriş/çıkış portlarına,
- konfigürasyon parametrelerine,
- RTL kaynaklarına,

- `component.xml` gibi IP-XACT tanımlarına,
- XGUI konfigürasyon dosyalarına

sahiptir.

AMD/Xilinx AXI UART Lite bir IP'dir. microBee SoC da benzer şekilde paketlenmiş bir kullanıcı IP'si olarak Vivado IP Catalog içerisinde görünür.

microBee IP'si projeye RTL dosyalarının tek tek eklenmesi yerine bir IP repository olarak tanıtılır.

Böylece Block Design içerisinde:

Add IP

komutu ile:

```
microBee SoC RV32IMC
```

aranabilir ve Block Design içerisine bir IP instance olarak eklenebilir.

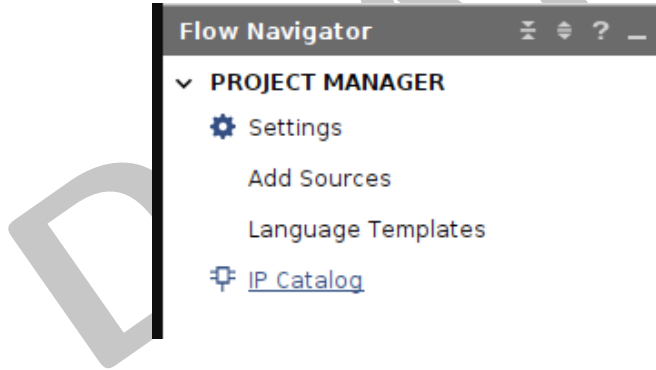
2.3.1 IP Repository'yi IP Catalog'a Ekleme

1. Flow Navigator içerisinde **IP Catalog** açılır (Şekil 2.4).
2. IP Catalog içerisinde boş bir alana sağ tıklanır ve **Add Repository...** seçilir (Şekil 2.5).
3. Paketlenmiş microBee IP klasörü seçilir:

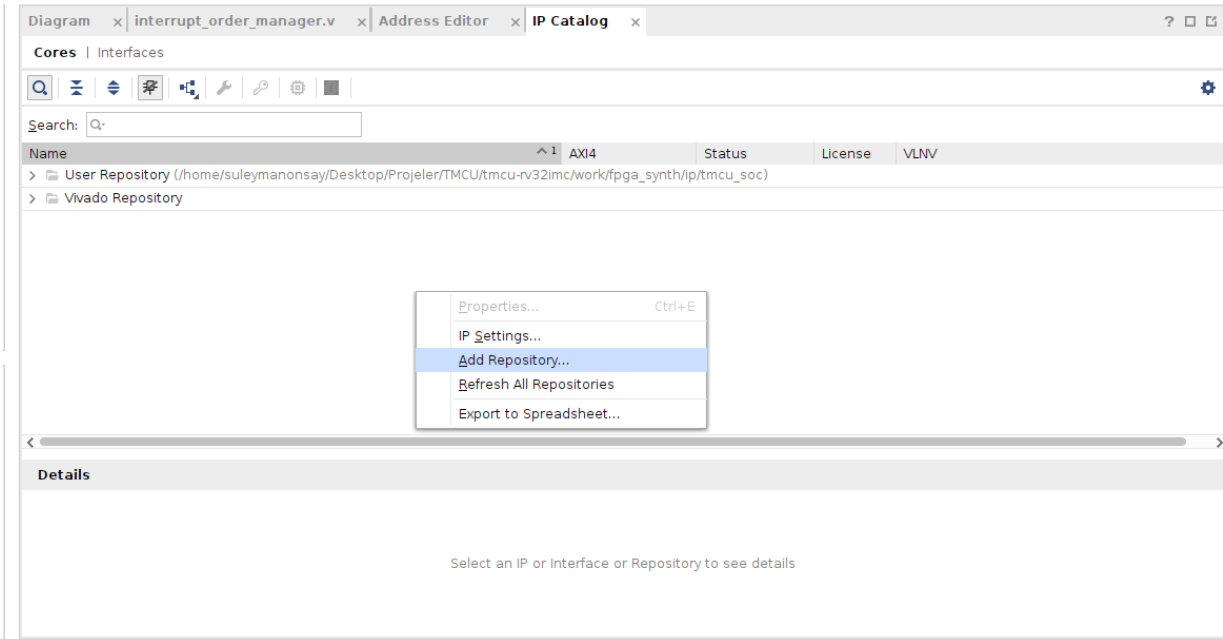
```
.../work/fpga_synth/ip/tmcu_soc
```

(Şekil 2.6).

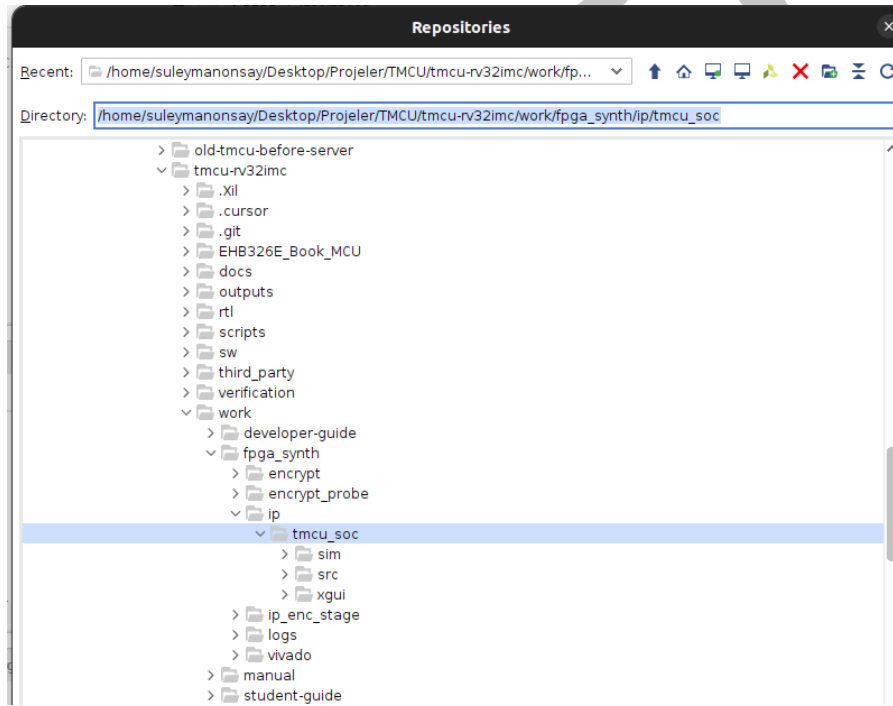
IP klasörü içerisinde `src`, `sim`, `xgui` ve ilgili paket dosyalarının bulunması beklenir.



Şekil 2.4: Project Manager altında IP Catalog.



Şekil 2.5: IP Catalog üzerinde Add Repository seçimi.



Şekil 2.6: tmcu_soc IP repository klasörünün seçilmesi.

Not

Repository yolu, paketlenmiş IP'nin bulunduğu `tmcu_soc` klasörünü göstermelidir. IP yeniden paketleniyse Vivado içerisinde:

Refresh All Repositories

işlemi uygulanabilir.

Projede daha eski bir microBee IP instance'ı bulunuyorsa ayrıca **Upgrade IP** işlemi gerekebilir.

2.4 Hazırlanmış Bloğu IP'ye Dönüştürme

Öğrenci laboratuvarında çoğunlukla önceden paketlenmiş microBee IP'si kullanılır.

Bu IP, repository içerisindeki RTL kaynaklarının Vivado tarafından tanınabilecek IP-XACT paketine dönüştürülmüş halidir.

Repository içerisinde tipik paketleme seçenekleri:

- Şifrelenmemiş IP:

```
scripts/package_tmcu_ip.tcl → work/fpga_synth/ip/tmcu_soc
```

- İsteğe bağlı şifreli IP:

```
make ip-encrypted → work/fpga_synth/ip_enc/tmcu_soc
```

Paketlenmiş IP içerisinde `tmcu_soc` üst modülü ve kullanıcıya açılan parametreler Customize IP penceresinde görünür.

Bu sayede Block Design kullanıcısının çekirdeğe ait bütün RTL dosyalarını projeye tek tek eklemesi gerekmez.

Bu bölümün geri kalanında IP'nin paketlenmesinden çok kullanımı üzerinde durulacaktır.

2.5 Block Design Oluşturma ve microBee Ekleme

2.5.1 Boş Block Design

Flow Navigator içerisinde:

IP Integrator → Create Block Design

seçilir.

Laboratuvar için Block Design adı:

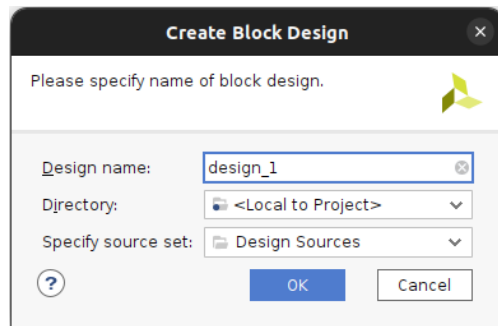
`design_1`

olarak bırakılabilir (Şekil 2.7).

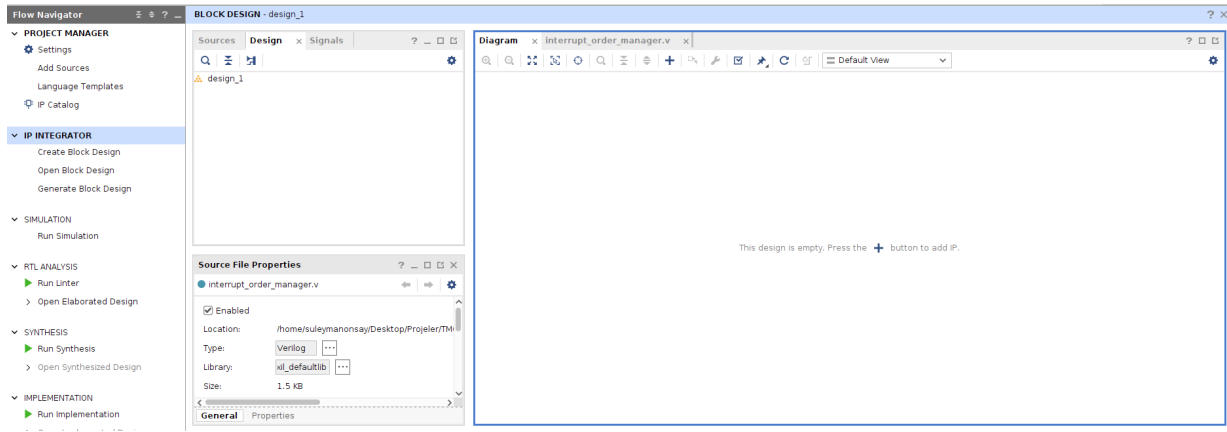
Boş Block Design açıldığında Vivado:

Press the + button to add IP

uyarısını gösterir (Şekil 2.8).



Şekil 2.7: Create Block Design penceresi.



Şekil 2.8: Boş Block Design çalışma alanı.

2.5.2 microBee SoC'yi Yerleştirme

Block Design içerisinde sağ tıklanır ve:

Add IP...

seçilir.

Klavye kısayolu olarak **Ctrl+I** kullanılabilir (Şekil 2.9).

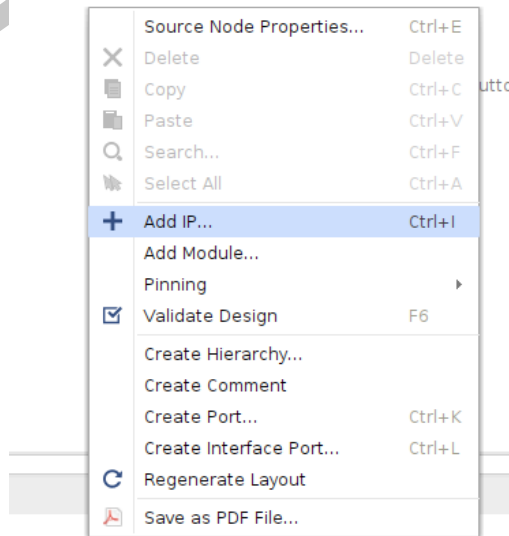
Arama kutusuna:

microBee

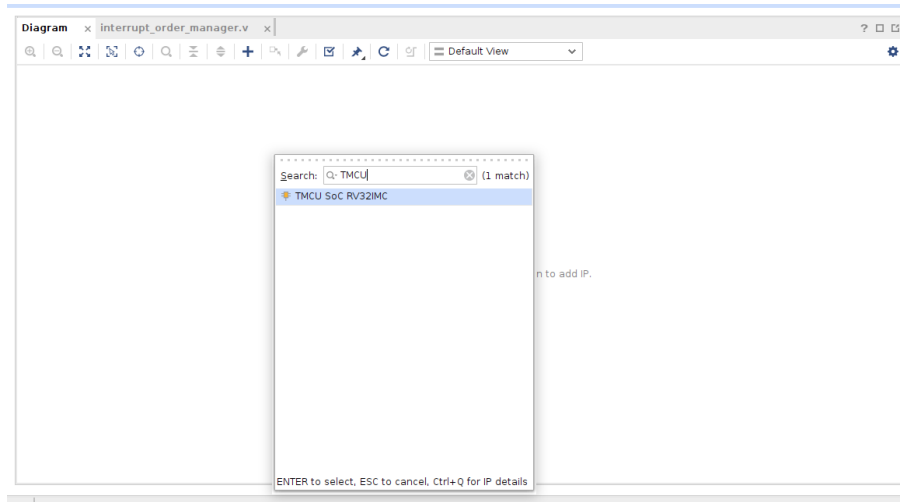
yazılır ve:

microBee SoC RV32IMC

seçilir (Şekil 2.10).



Şekil 2.9: Block Design içerisinde Add IP seçimi.



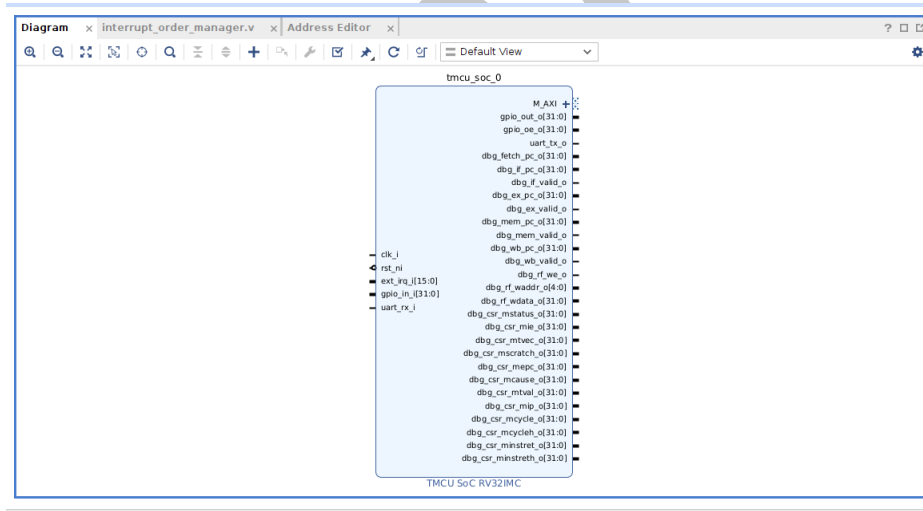
Şekil 2.10: Add IP aramasında microBee SoC RV32IMC.

Block Design içerisine eklenen ilk microBee instance'ı genellikle:

`tmcu_soc_0`

adını alır.

IP üzerinde clock, reset, interrupt, GPIO ve Native UART gibi sinyaller ile M_AXI master interface görünür. DEBUG_ENABLE=1 olduğunda dbg_* debug portları da Block Design seviyesinde kullanılabilir (Şekil 2.11).



Şekil 2.11: Block Design içerisinde yalnızca tmcu_soc_0.

2.5.3 AXI UART Lite Ekleme

microBee'nin AXI üzerinden erişebileceği bir UART eklemek için tekrar **Add IP** açılır.

Arama kutusunda:

UARTlite

aranır ve:

AXI UART Lite

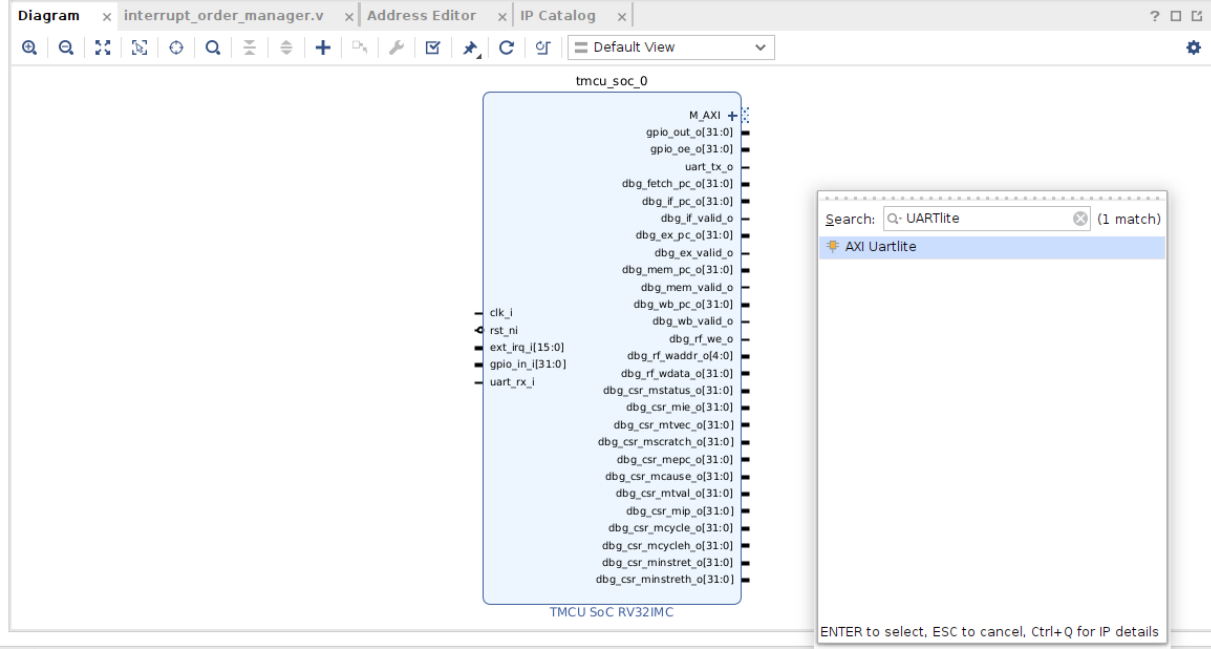
seçilir (Şekil 2.12).

Bu aşamada microBee ile AXI UART Lite henüz birbirine bağlı değildir.

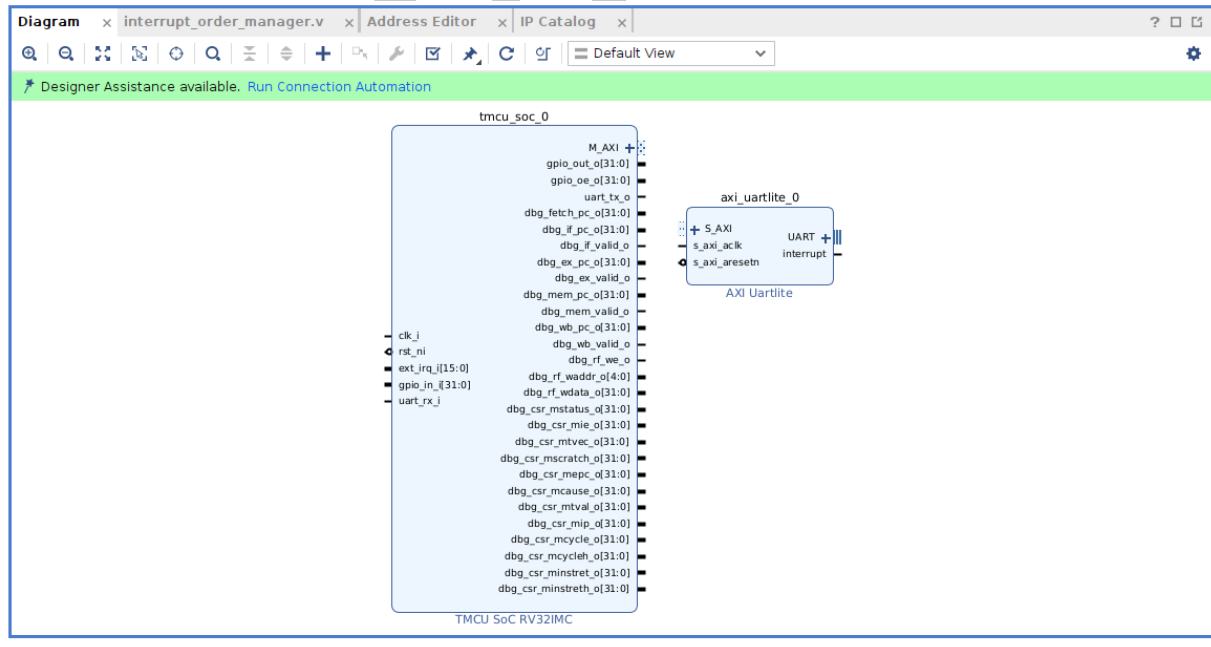
Vivado, bağlantı yapılabilecek interface'leri algıladığında Block Design üzerinde:

Designer Assistance available. Run Connection Automation

uyarısını gösterebilir (Şekil 2.13).



Şekil 2.12: AXI UART Lite IP'sinin Block Design'a eklenmesi.



Şekil 2.13: microBee ve AXI UART Lite henüz birbirine bağlanmadan önce.

Not

microBee'nin kendi Native UART birimi ile AXI UART Lite birbirinden bağımsız iki peripheral'dır. Native UART:

`uart_rx_i / uart_tx_o`

portlarını kullanır.

Bu laboratuvar da seri haberleşme için ağırlıklı olarak AXI adres penceresindeki AXI UART Lite kullanılmaktadır.

AXI UART Lite için örnek taban adresi:

`0x60000000`

olarak seçilir.

Harici RX/TX bağlantısı Block Design seviyesinde `uart_rt1_0` interface'i üzerinden dışarı çıkarılır.

2.6 Connection Automation ile Blokların Bağlanması

Connection Automation, Vivado'nun AXI, clock ve reset bağlantılarının önemli bir bölümünü otomatik olarak oluşturmasını sağlar.

Bu laboratuvar tasarımında Connection Automation iki aşamada kullanılmaktadır.

2.6.1 Birinci Aşama: UART Lite'nin AXI ve Dış Port Bağlantıları

Block Design üzerinde görünen:

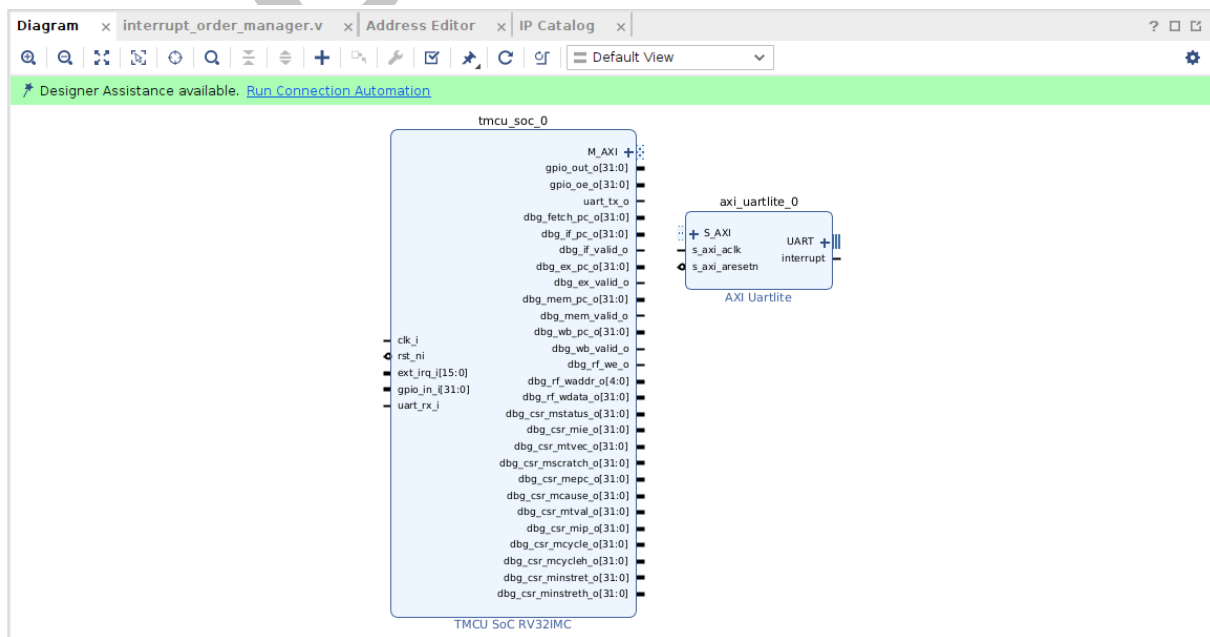
Run Connection Automation

seçeneği çalıştırılır (Şekil 2.14).

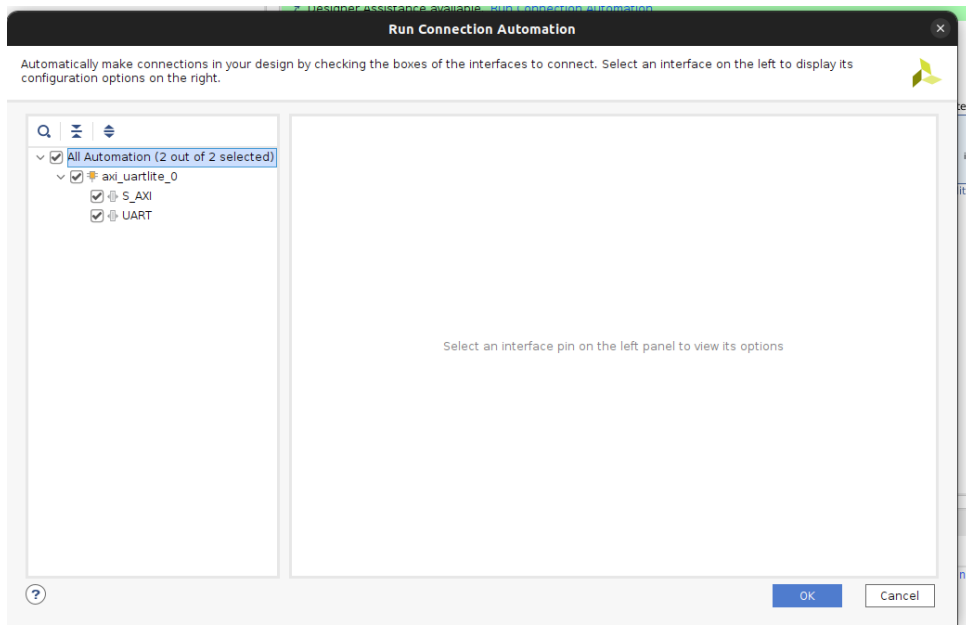
Açılan pencerede `axi_uartlite_0` için:

- S_AXI,
- UART

bağlantıları seçilir (Şekil 2.15).



Şekil 2.14: Designer Assistance üzerinden Run Connection Automation.



Şekil 2.15: İlk Connection Automation penceresinde AXI UART Lite S_AXI ve UART bağlantılarının seçilmesi.

İlk otomasyon sonrasında Vivado tasarıma gerekli AXI ve clock/reset altyapı bloklarını ekleyebilir.

Örnek olarak:

- AXI Interconnect,
- Clocking Wizard,
- Processor System Reset

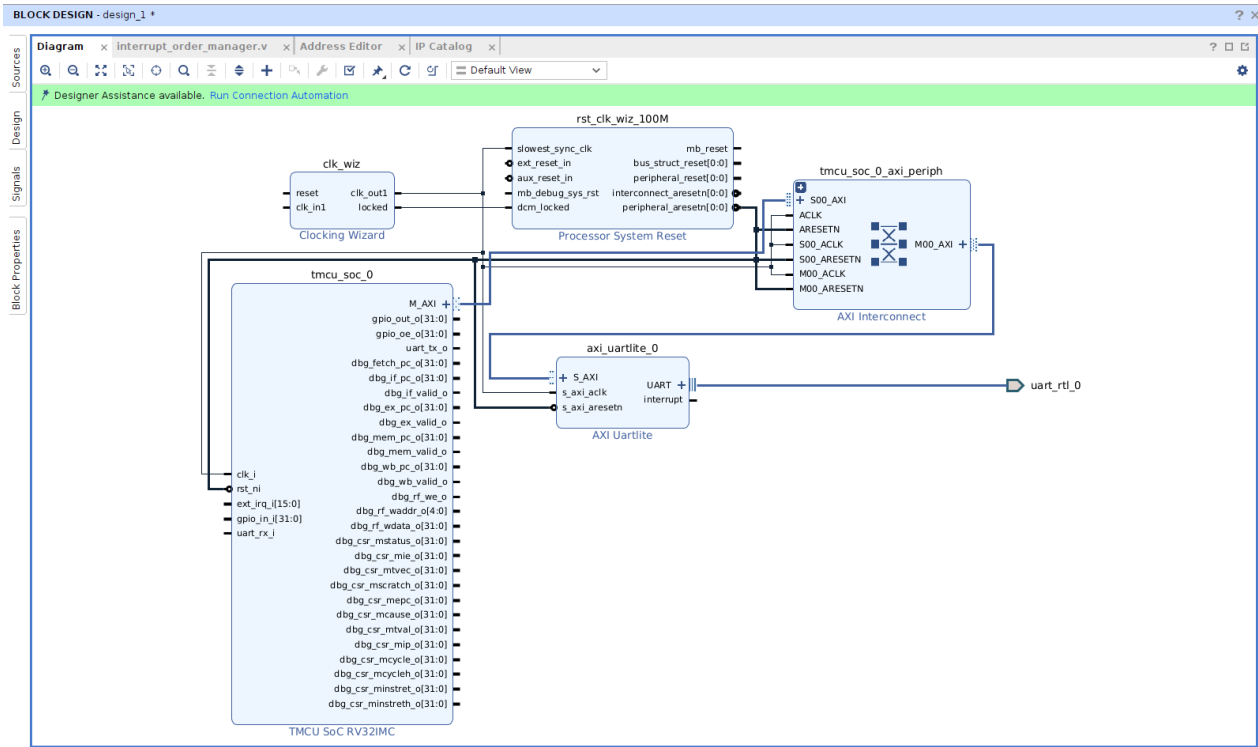
blokları oluşturulur.

microBee'nin M_AXI master interface'i AXI Interconnect üzerinden AXI UART Lite'a bağlanır.

UART interface'i ise:

`uart_rtl_0`

adıyla dış port olarak oluşturulabilir (Şekil 2.16).



Şekil 2.16: İlk Connection Automation sonrasında oluşan sistem altyapısı.

2.6.2 İkinci Aşama: Clock ve Reset Dış Girişleri

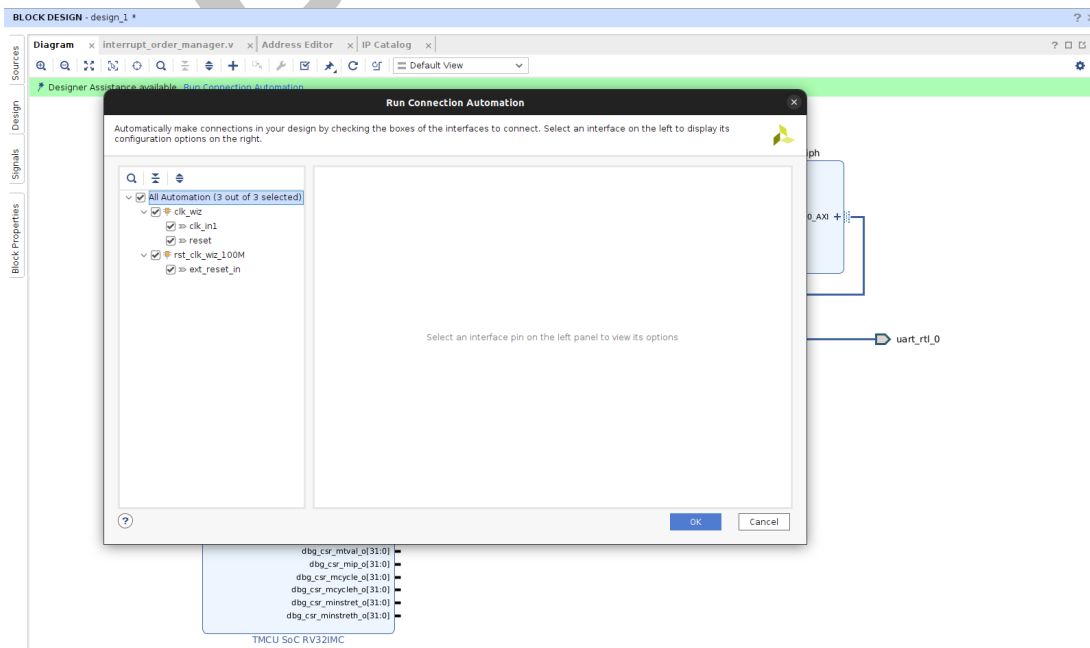
İlk otomasyon sonrasında Clocking Wizard ve reset altyapısının dış portları için Vivado tekrar Connection Automation önerebilir.

Run Connection Automation tekrar çalıştırılır.

Örnek tasarımda:

- `clk_wiz` için `clk_in1` ve `reset`,
- `rst_clk_wiz_100M` için `ext_reset_in`

seçilir (Şekil 2.17).



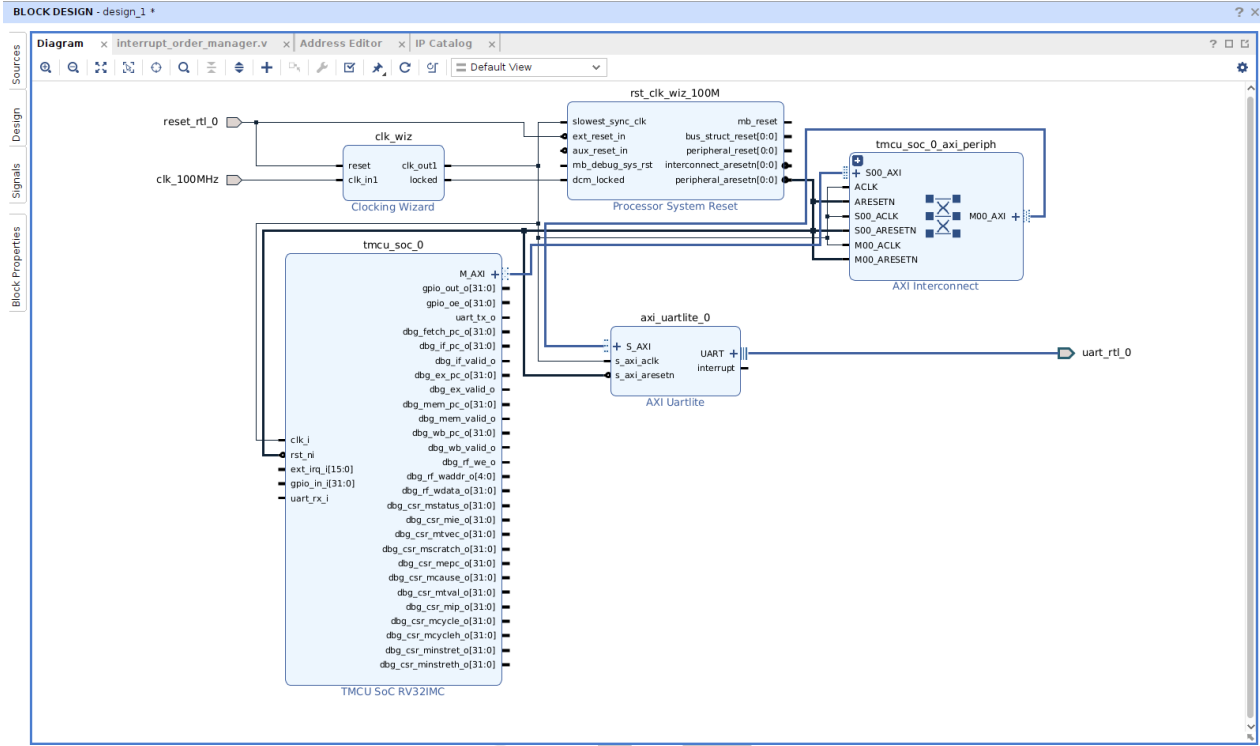
Şekil 2.17: İkinci Connection Automation: clock ve reset dış bağlantıları.

Bu aşamadan sonra üst seviyede örnek olarak:

- `clk_100MHz`,
- `reset_rtl_0`

portları görülür.

microBee'nin `clk_i` ve `rst_ni` girişleri bu altyapı üzerinden beslenir (Şekil 2.18).



Şekil 2.18: Clock ve reset otomasyonu tamamlandıktan sonraki Block Design.

Address Editor içerisinde AXI UART Lite'in taban adresi, microBee'nin AXI adres penceresi içerisinde seçilmelidir.

Bu laboratuvar örneğinde:

0x60000000

kullanılabilir.

UART baud ayarı ile sistem clock'u birbiriyle uyumlu olmalıdır.

Bu bölümde kullanılan örnek testbench:

100 MHz / 9600 baud

konfigürasyonuna göre hazırlanmıştır.

Farklı bir clock veya baud değeri kullanılıyorsa hem AXI UART Lite IP ayarı hem de testbench aynı değere göre güncellenmelidir.

2.7 External Interrupt Vektörünün Oluşturulması

microBee external interrupt girişinde 16-bit bir vektör bekler:

`ext_irq_i[15:0]`

AXI UART Lite ise tek bitlik:

interrupt

çıkışı üretir.

Bu ve benzeri tek-bit interrupt kaynaklarını microBee'nin external interrupt vektörüne yerleştirmek için:

interrupt_order_manager

adlı yardımcı RTL modülü kullanılabilir.

2.7.1 Modülün Görevi

interrupt_order_manager modülü:

in_0, in_1, ..., in_15

şeklindeki 16 adet tek-bit giriş sinyalini:

interrupt_array[15:0]

vektörü içerisinde birleştirir.

Burada:

in_0 → interrupt_array[0]

olacak şekilde bit sırası korunur.

Modül kaynak kodu Design Sources içerisine eklenmiş olmalıdır (Şekil 2.19).

```

15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module interrupt_order_manager(
24     input wire in_0 , in_1 , in_2,
25     input wire in_3 , in_4 , in_5,
26     input wire in_6 , in_7 , in_8,
27     input wire in_9 , in_10, in_11,
28     input wire in_12, in_13, in_14,
29     input wire in_15,
30     output wire [15:0] interrupt_array
31 );
32
33     assign interrupt_array = {in_15, // array[15]
34                             in_14, // array[14]
35                             in_13, // array[13]
36                             in_12, // array[12]
37                             in_11, // array[11]
38                             in_10, // array[10]
39                             in_9,  // array[9]
40                             in_8,  // array[8]
41                             in_7,  // array[7]
42                             in_6,  // array[6]
43                             in_5,  // array[5]
44                             in_4,  // array[4]
45                             in_3,  // array[3]
46                             in_2,  // array[2]
47                             in_1,  // array[1]
48                             in_0}; // array[0]
49
50 endmodule
51

```

Şekil 2.19: interrupt_order_manager: 16 adet tek-bit interrupt girişinin 16-bit external interrupt vektöründe birleştirilmesi.

2.7.2 Block Design'a RTL Modül Ekleme

`interrupt_order_manager` bir Vivado IP'si değil, doğrudan proje RTL kaynağıdır.

Bu nedenle Block Design içerisinde:

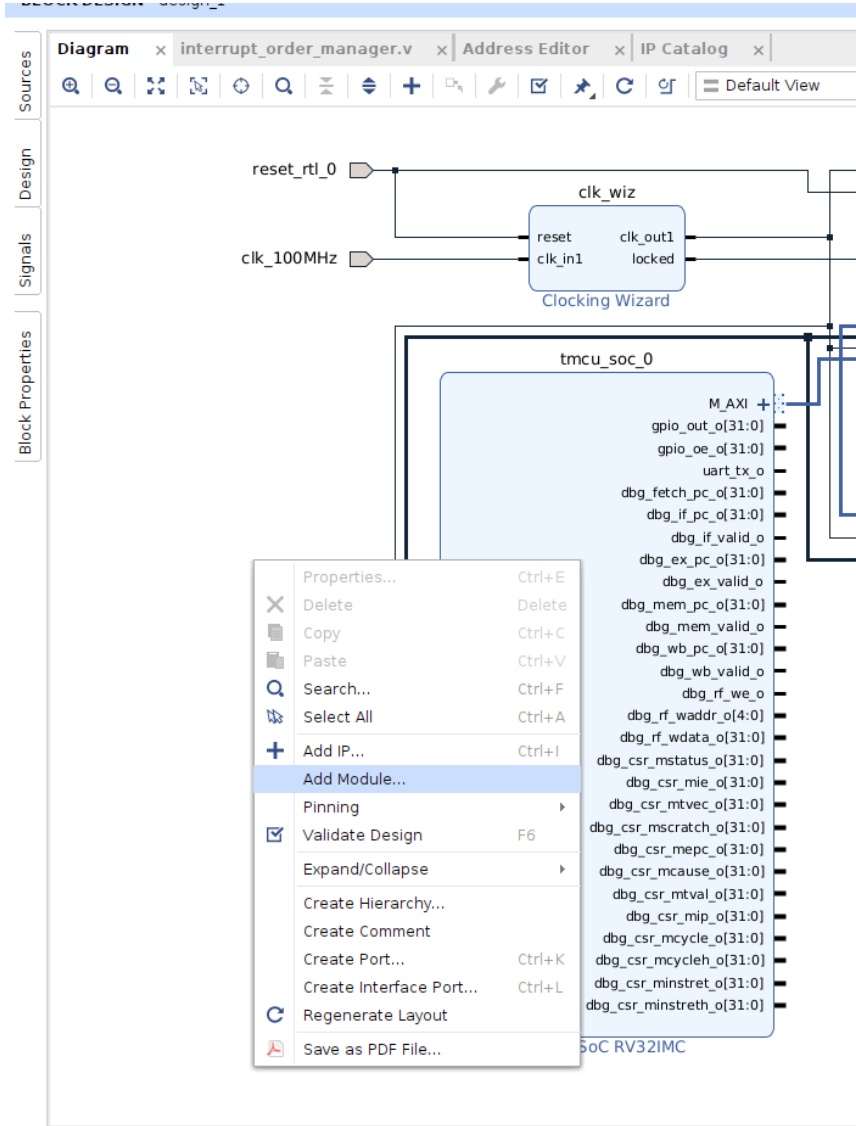
sağ tık → **Add Module...**

kullanılır (Şekil 2.20).

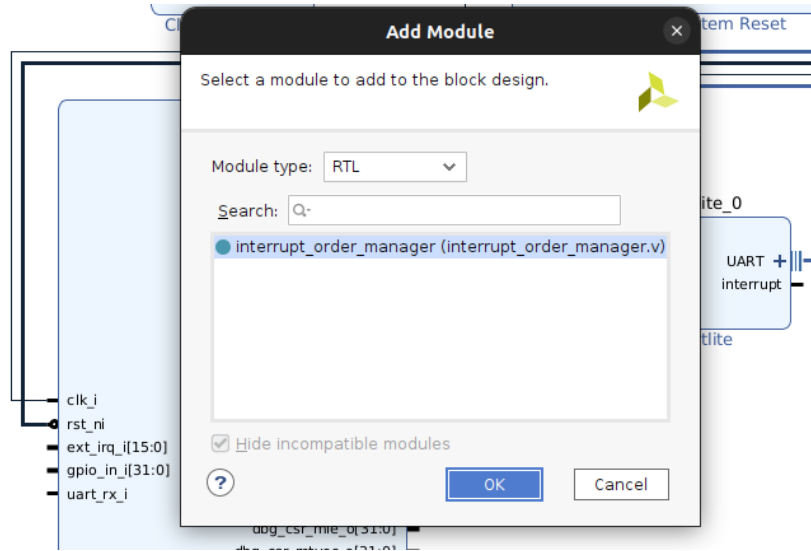
Açılan listeden:

`interrupt_order_manager`

seçilir (Şekil 2.21).



Şekil 2.20: Block Design üzerinde Add Module seçimi.

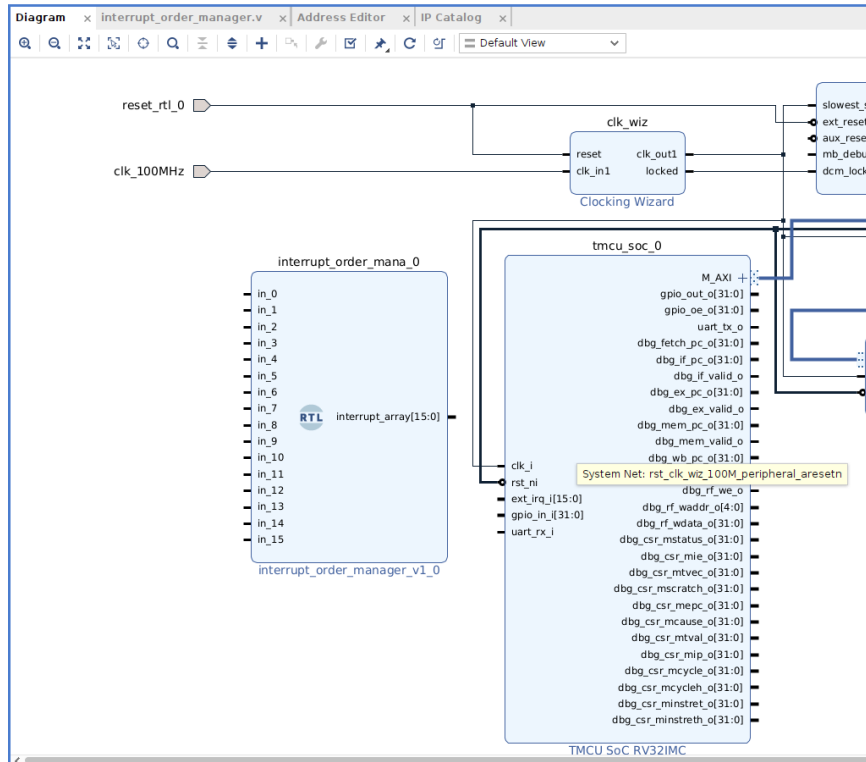


Şekil 2.21: Add Module penceresinde `interrupt_order_manager` modülünün seçilmesi.

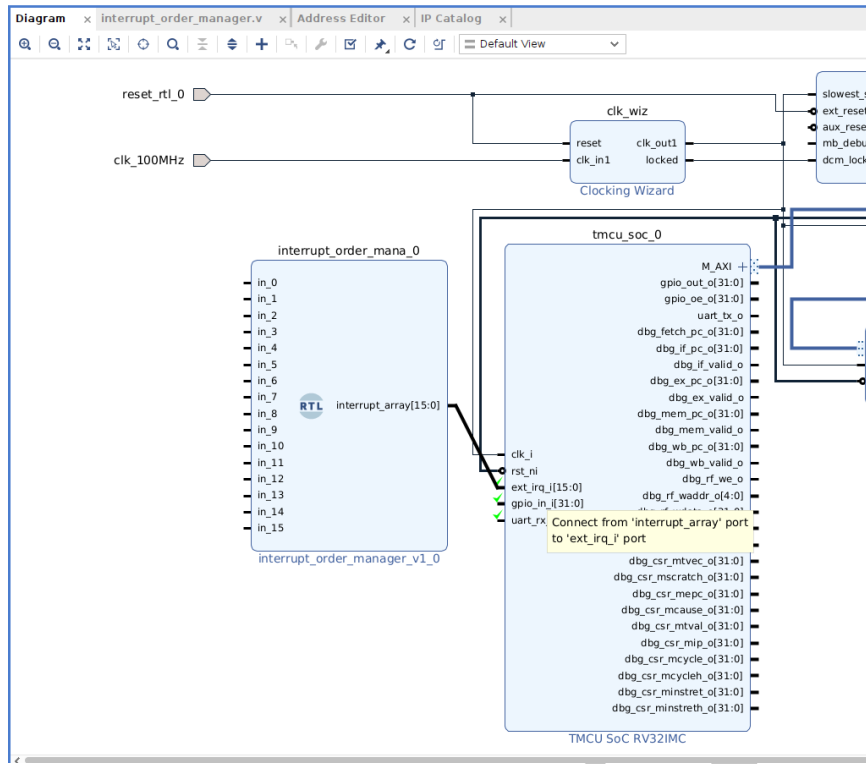
Modül eklendikten sonra:

`interrupt_array` → `ext_irq_i`

bağlantısı yapılır (Şekil 2.22 ve Şekil 2.23).



Şekil 2.22: `interrupt_order_manager` modülünün Block Design'a eklenmesi.

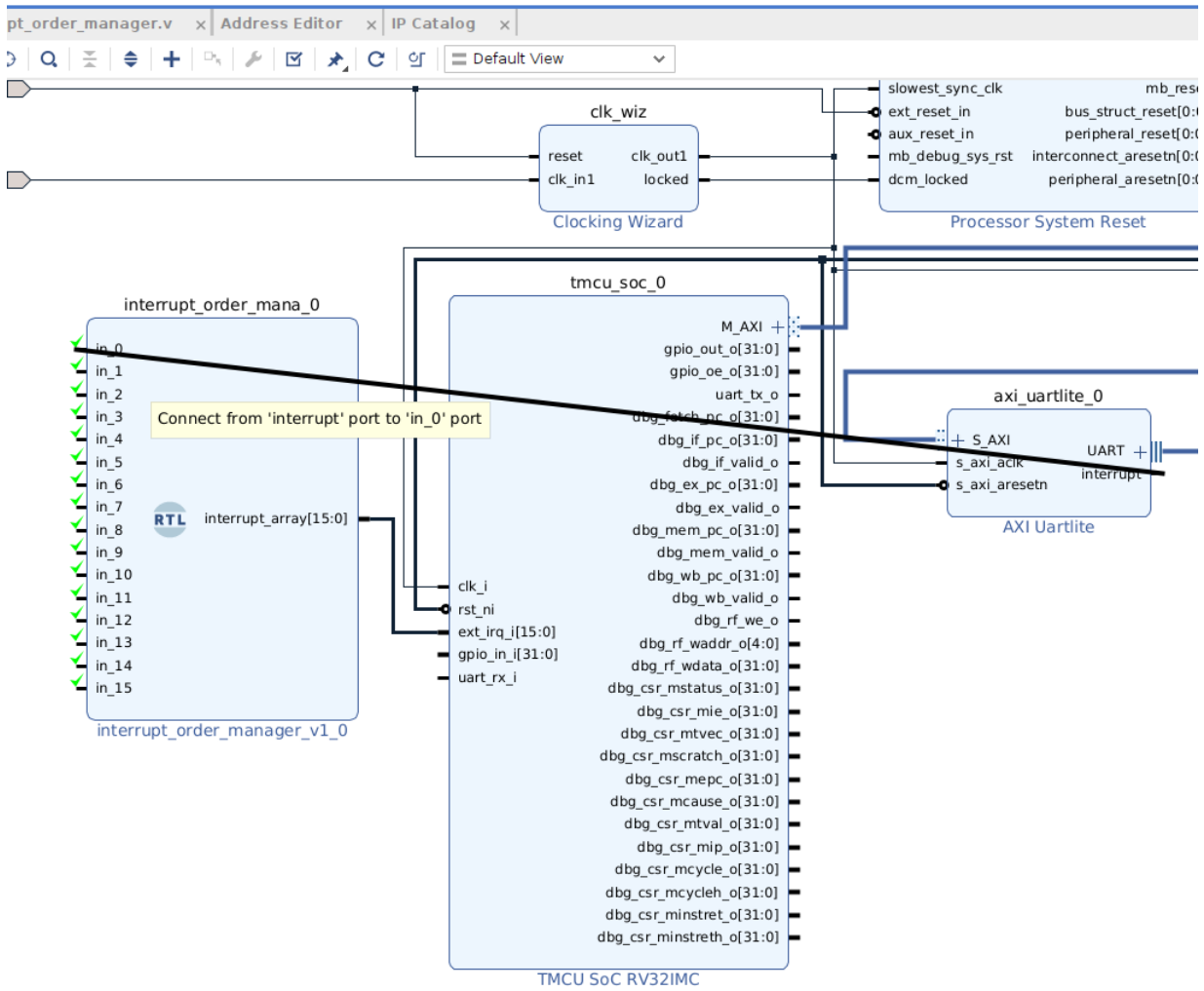


Şekil 2.23: `interrupt_array[15:0]` çıkışının microBee `ext_irq_i[15:0]` girişine bağlanması.

AXI UART Lite interrupt çıkışı:

`interrupt` → `in_0`

olarak bağlanır (Şekil 2.24).



Şekil 2.24: AXI UART Lite interrupt çıkışının in_0 girişine bağlanması.

Böylece AXI UART Lite interrupt kaynağı:

`ext_irq_i[0]`

hattına ve dolayısıyla microBee PLIC içerisinde ID 1'e bağlanmış olur.

Kullanılmayan `in_*` girişleri floating (boşta) bırakılmamalıdır.

Bu girişler 0'a sabitlenebilir veya Constant IP ile bağlanabilir. Aksi durumda behavioral simülasyonda bilinmeyen X değerleri external interrupt vektörüne yayılabilir.

Nihai Block Design Şekil 2.1'de verilen yapıya ulaşır.

2.8 IMEM / DMEM Başlatma Dosyaları

Program ve başlangıç verileri, microBee Customize IP penceresinde verilen IMEM ve DMEM initialization dosyaları ile belleklere yerleştirilebilir.

Block Design içerisinde:

`tmcu_soc_0`

üzerine çift tıklanarak **Re-customize IP** penceresi açılır.

- **Imem Init File**

Örnek:

verification/system/soc_integration_imem.hex

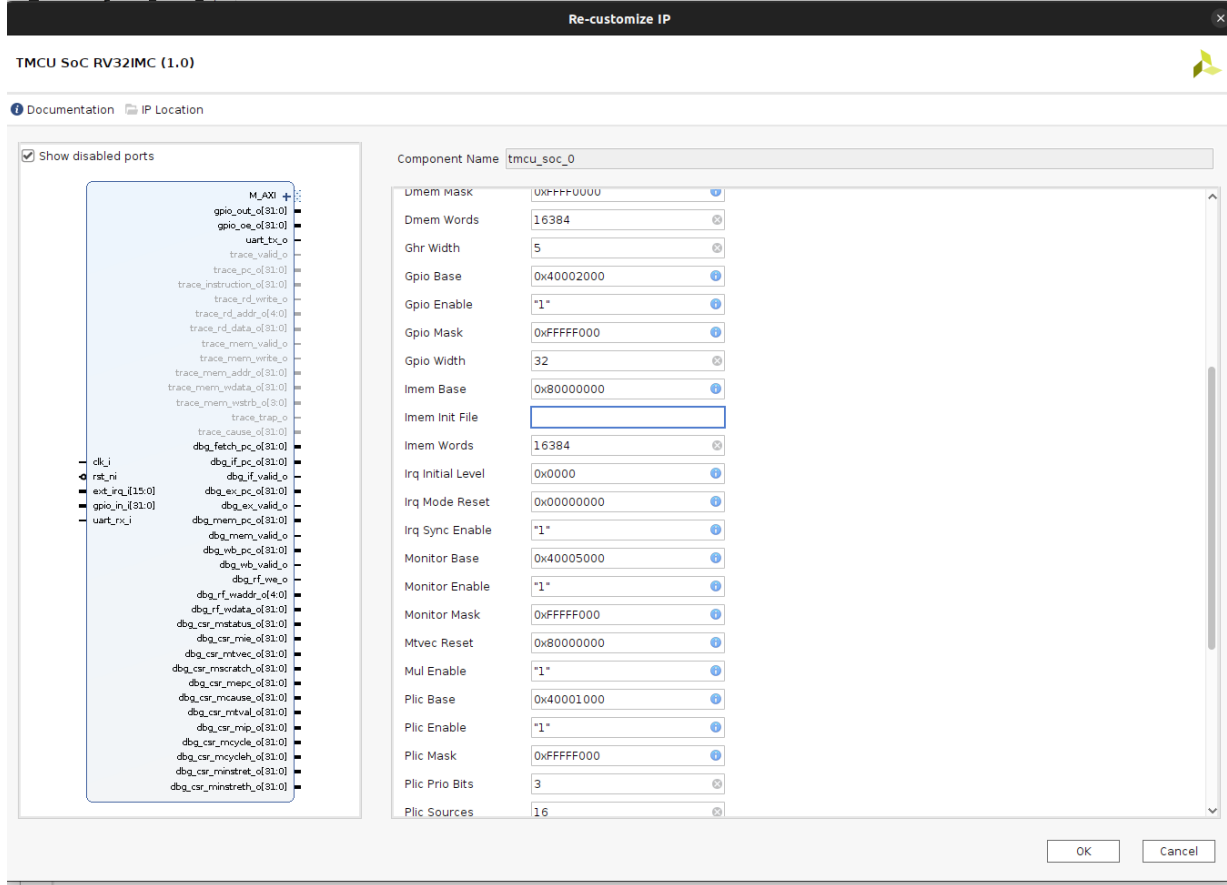
(Şekil 2.25 ve Şekil 2.26).

- **Dmem Init File**

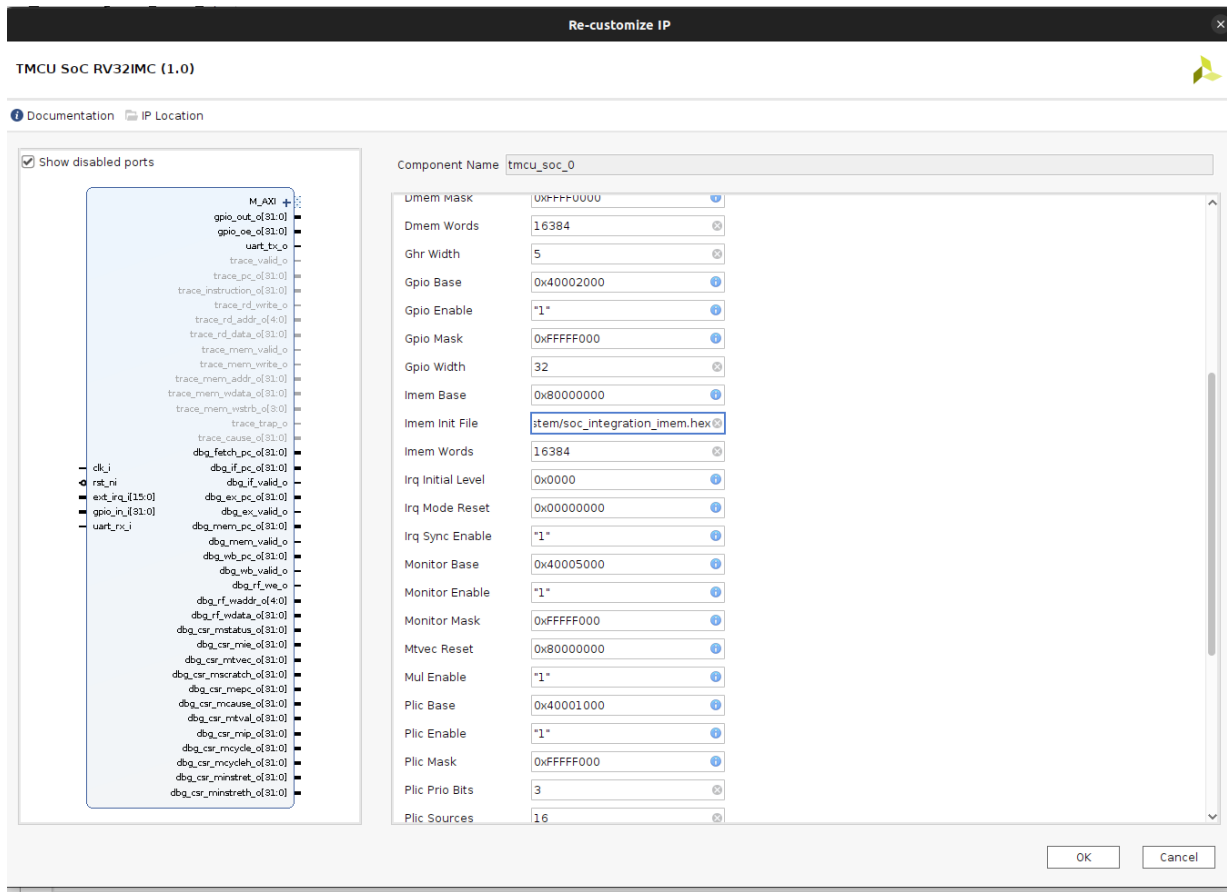
Örnek:

verification/system/soc_integration_dmem_poll.hex

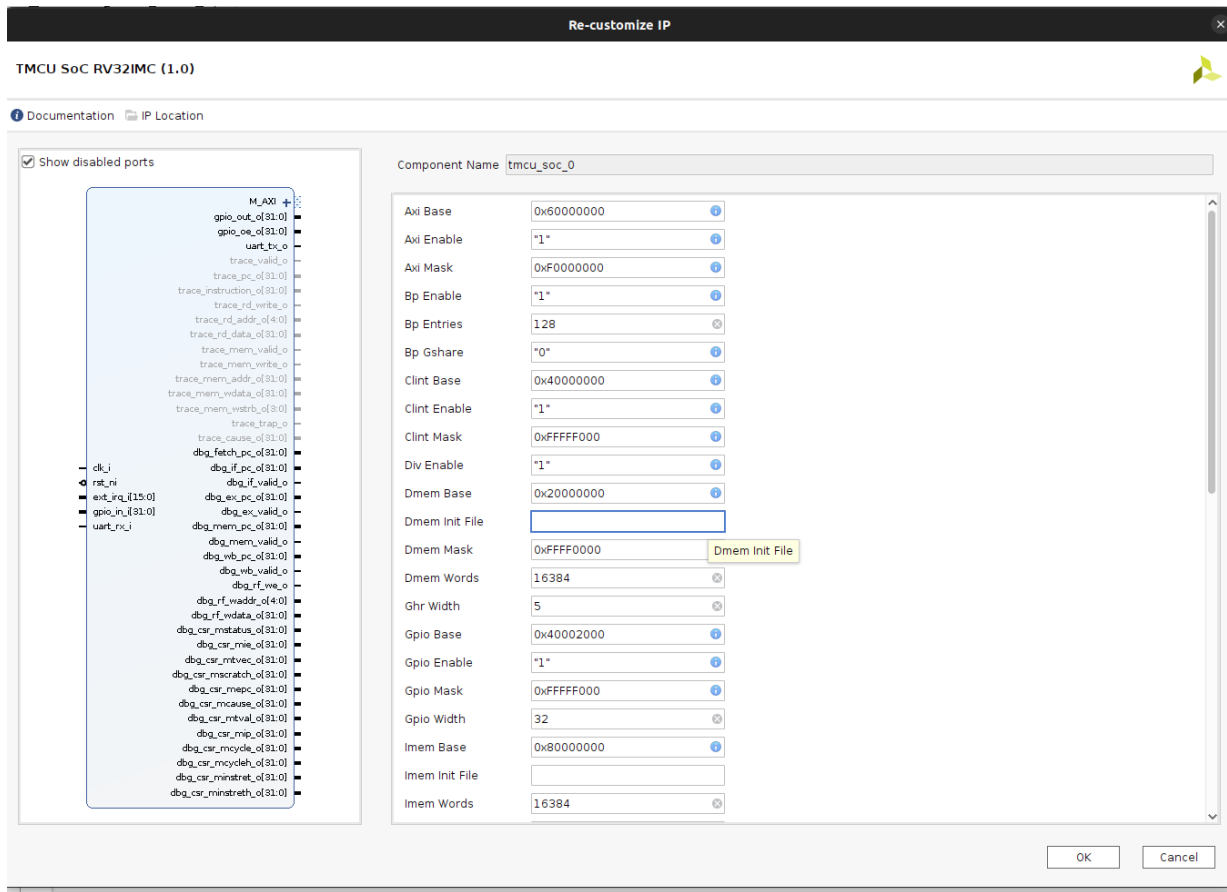
(Şekil 2.27 ve Şekil 2.28).



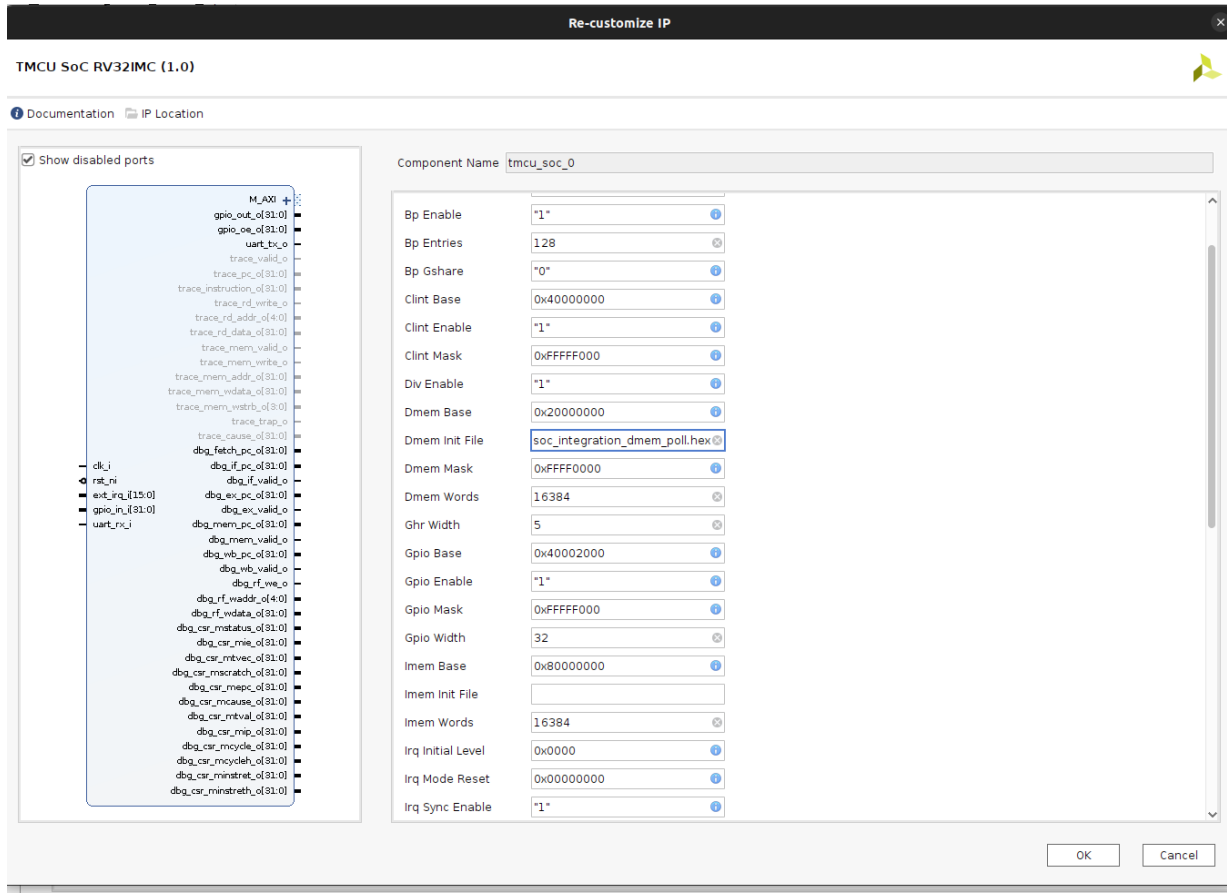
Şekil 2.25: Customize IP penceresinde memory ve peripheral parametreleri ile Imem Init File alanı.



Şekil 2.26: Imem Init File yolunun girilmesi.



Şekil 2.27: Customize IP penceresinde Dmem Init File ve AXI adres parametreleri.



Şekil 2.28: Dmem Init File örneği: `soc_integration_dmem_poll.hex`.

Not

Initialization dosyalarının yolları Vivado'nun çalışma dizinine göre görelî veya mutlak olarak verilebilir. Initialization dosyası bulunamaz veya doğru şekilde yüklenemezse program ya da veri belleği beklenen içerikle başlatılamaz. Bu durumda simülasyonda beklenen UART çıktısı alınmayabilir.

AXI taban adresi, Address Editor içerisinde AXI UART Lite için kullanılan adres ile eşleşmelidir.

Bu laboratuvar örneğinde:

0x60000000

kullanılmaktadır.

2.9 HDL Wrapper

Block Design tek başına proje top-level RTL modülü olarak kullanılmaz.

Vivado, Block Design içerisindeki IP ve interface bağlantılarını RTL hiyerarşisine taşımak için bir HDL wrapper üretir.

Sources penceresinde:

`design_1.bd`

üzerinde sağ tıklanır ve:

Create HDL Wrapper...

seçilir (Şekil 2.29).

Laboratuvar için:

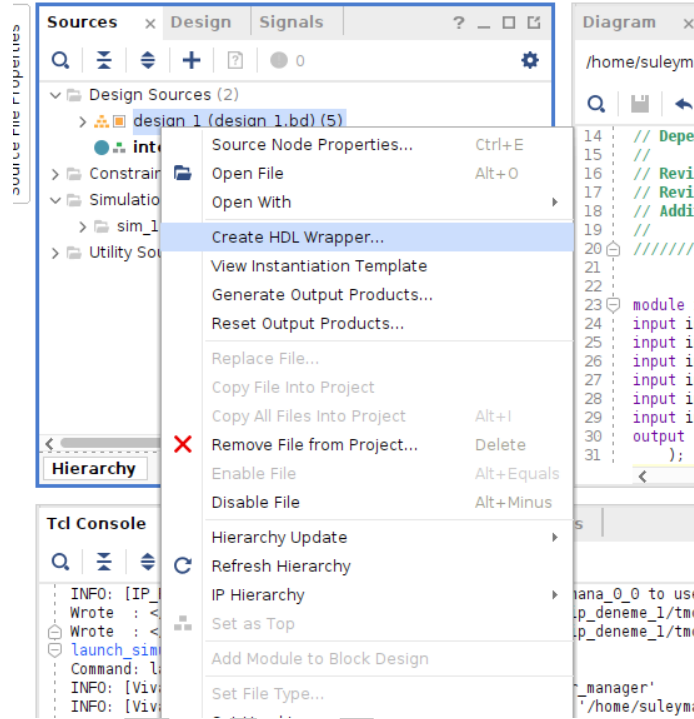
Let Vivado manage wrapper and auto-update

seçeneği kullanılabilir.

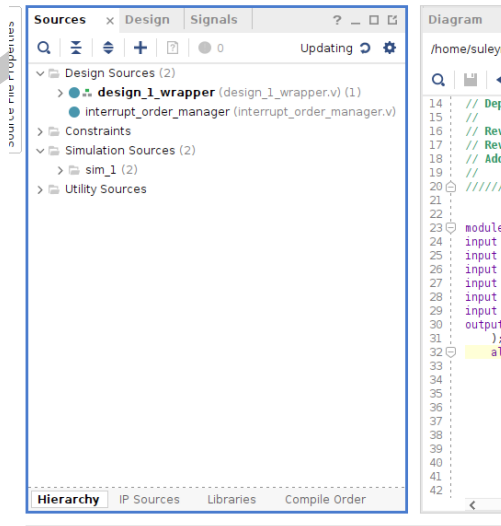
Bundan sonra Design Sources altında:

design_1_wrapper

modülü görülür (Şekil 2.30).



Şekil 2.29: design_1.bd için Create HDL Wrapper işlemi.



Şekil 2.30: Design Sources altında design_1_wrapper modülü.

Wrapper'ın dış portları daha sonra testbench tarafından sürülür veya izlenir.

Örnek portlar:

- clk_100MHz,

- reset_rtl_0,
- uart_rtl_0_rxd,
- uart_rtl_0_txd.

2.10 Simülasyon Sistemi

2.10.1 Nedir?

Simülasyon ve emülasyon araçları, program ve sistem davranışını farklı soyutlama düzeylerinde inceler. Bu laboratuvar bölümünde asıl konu Vivado xsim behavioral simülasyonudur. rv32 ise assembly kaynağını assemble edip yazmaç, veri belleği ve instruction etkilerini adım adım gözlemlemeye yarar; kurulumu ve öğrenci uygulamaları Bölüm 11’de anlatılmıştır.

Vivado’nun xsim simülatörü behavioral simülasyon sırasında synthesis ve physical implementation öncesindeki fonksiyonel modeli çalıştırır.

Testbench:

- clock üretir,
- reset uygular,
- DUT olarak `design_1_wrapper` modülünü instantiate eder,
- UART TX hattındaki seri veriyi baud zamanlamasına göre örnekler,
- alınan karakterleri beklenen ASCII dizisi ile karşılaştırır.

Hazır testbench örneği:

```
verification/system/tb_design_1_hello.sv
```

olarak kullanılabilir.

Bu laboratuvar testbench’inde beklenen mesaj:

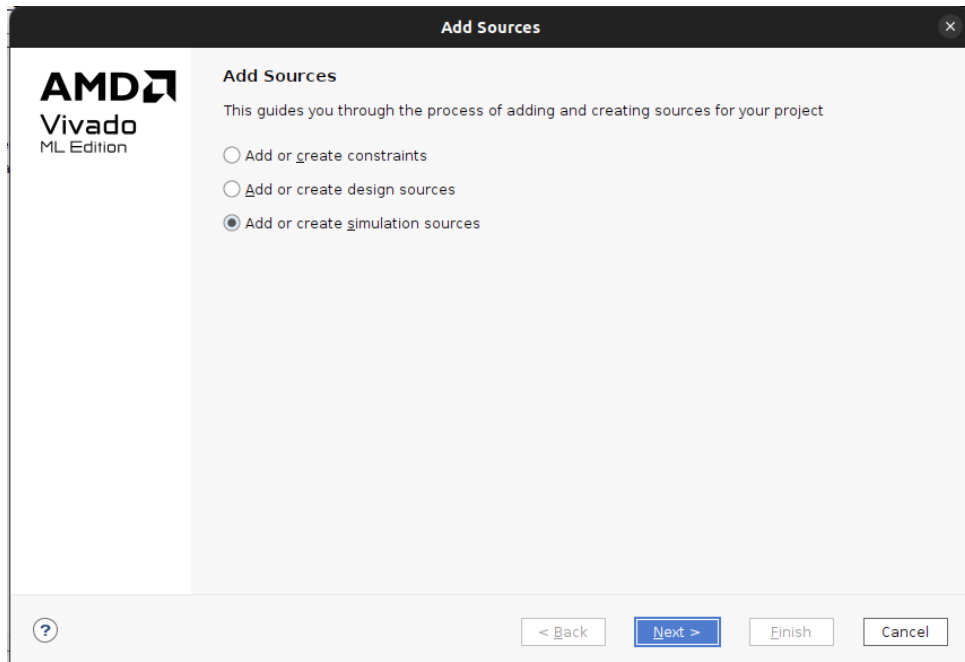
```
"Hello from microBee on Arty A7\r\n"
```

şeklindedir.

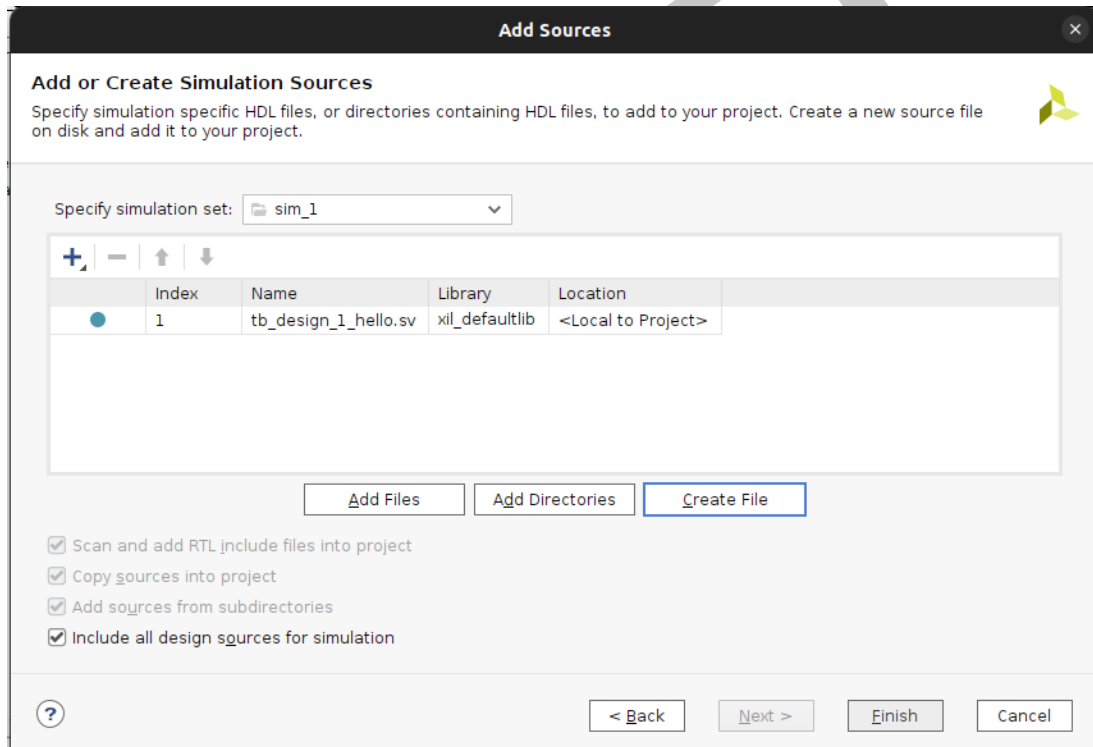
Bu örnek 28 karakterlik mesajı 100 MHz sistem clock’u ve 9600 baud konfigürasyonu ile doğrular.

2.10.2 Testbench Ekleme

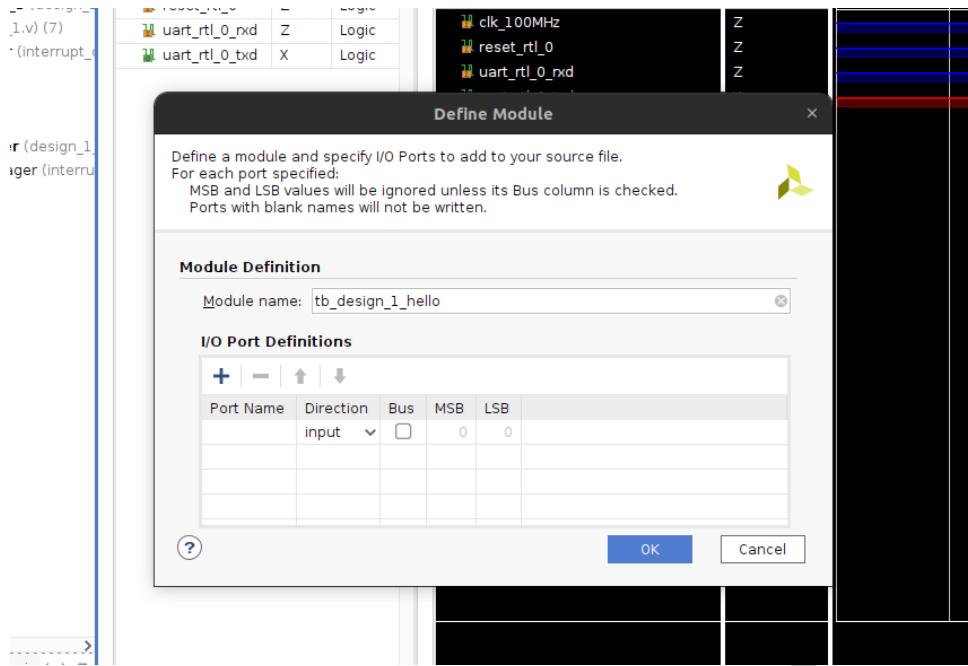
1. **Add Sources** seçilir.
2. **Add or create simulation sources** seçilir (Şekil 2.31).
3. `tb_design_1_hello.sv` dosyası projeye eklenir veya **Create File** ile oluşturulur (Şekil 2.32 ve Şekil 2.33).
4. Testbench port bağlantıları kullanılan wrapper portları ile uyumlu hale getirilir (Şekil 2.34).



Şekil 2.31: Add Sources içerisinde simulation source seçimi.



Şekil 2.32: tb_design_1_hello.sv dosyasının simulation source olarak eklenmesi.



Şekil 2.33: Define Module ekranında testbench modülünün oluşturulması.

```

interrupt_order_manager.v x tb_design_1_hello.sv x Untitled 2 x
/home/suleymanonsay/Desktop/Projeler/TMCU/tmcu_ip_deneme_1/tmcu_ip_deneme_1.srscs/sim_1/new/tb_design_1_hello.sv

1 timescale 1ns/1ps
2
3 // Behavioral TB for student Vivado BD wrapper `design_1_wrapper`.
4 //
5 // Ports:
6 //   clk_100MHz, reset_rtl_0 (ACTIVE_HIGH), uart_rtl_0_rxd, uart_rtl_0_txd
7 //
8 // IMEM/DMEM init on tmcu_soc (Customize IP):
9 //   verification/system/soc_integration_imem.hex
10 //   verification/system/soc_integration_dmem_poll.hex
11 //
12 // UART Lite: 9600 baud @ 100 MHz ACLK
13 // Expected: "Hello from TMCU on Arty A7\r\n" (28 bytes)
14 //
15 // Vivado: Simulation top = tb_design_1_hello
16 // Prefer file type SystemVerilog; written to also parse cleanly as Verilog-style SV.
17
18 module tb_design_1_hello;
19   localparam integer CLK_HZ      = 100000000;
20   localparam integer BAUD        = 9600;
21   localparam integer BIT_TICKS   = CLK_HZ / BAUD;
22   localparam integer HALF_BIT    = BIT_TICKS / 2;
23   localparam integer EXPECT_CHARS = 28;
24   localparam integer RESET_CYCLES = 20000;
25   localparam integer TIMEOUT_MS  = 200;
26
27   // ASCII: "Hello from TMCU on Arty A7\r\n"
28   reg [7:0] HELLO [0:27];
29
30   reg clk_100MHz = 1'b0;
31   reg reset_rtl_0 = 1'b1; // ACTIVE_HIGH
32   reg uart_rxd    = 1'b1;
33   wire uart_txd;
34
35   integer i;
36
37   initial begin
38     HELLO[ 0] = 8'h48; HELLO[ 1] = 8'h65; HELLO[ 2] = 8'h6C; HELLO[ 3] = 8'h6C;
39     HELLO[ 4] = 8'h6F; HELLO[ 5] = 8'h20; HELLO[ 6] = 8'h66; HELLO[ 7] = 8'h72;
40     HELLO[ 8] = 8'h6F; HELLO[ 9] = 8'h6D; HELLO[10] = 8'h20; HELLO[11] = 8'h54;

```

Şekil 2.34: Testbench kaynağında clock, reset, UART ve beklenen “Hello” mesajının tanımlanması.

2.10.3 Simulation Top' u Ayarlama

Bu laboratuvar akışında Design Sources içindeki wrapper, sentez ve implementation için top-level modüldür. Behavioral simülasyonda ise top-level modül testbench olmalıdır.

Simulation Sources altında:

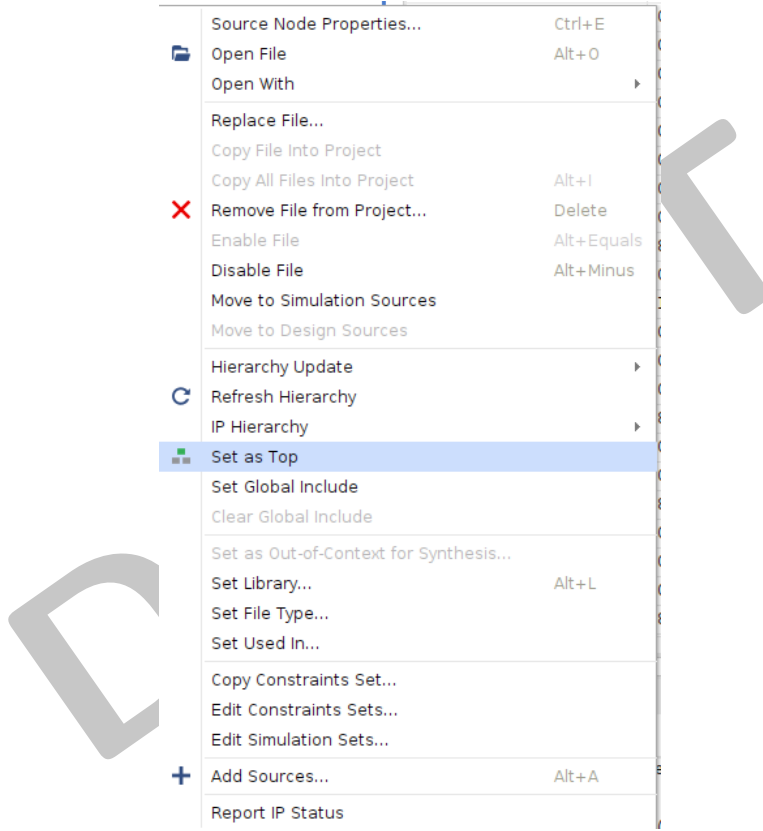
```
tb_design_1_hello
```

üzerinde sağ tıklanır ve:

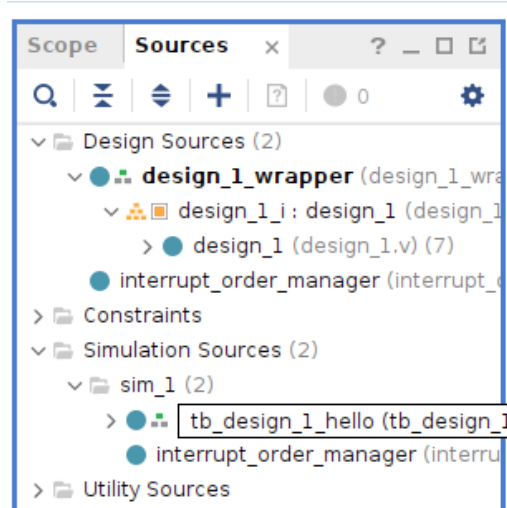
Set as Top

seçilir (Şekil 2.35).

Doğru hiyerarşide testbench simülasyonun en üst modülü olur (Şekil 2.36).



Şekil 2.35: Simulation Sources içerisinde Set as Top seçimi.

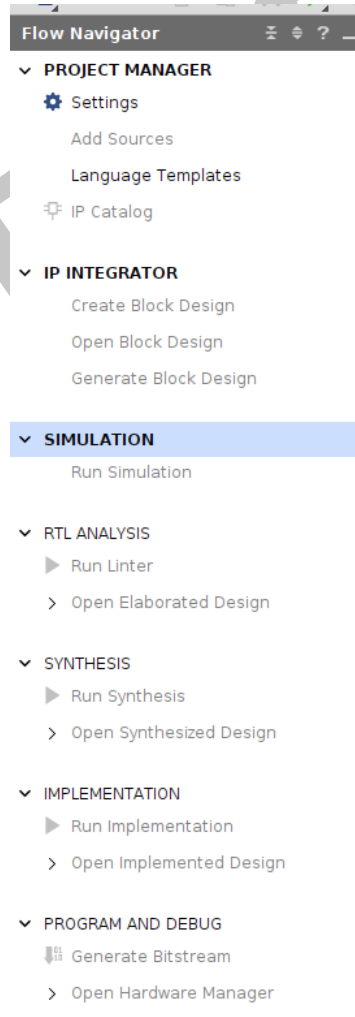


Şekil 2.36: tb_design_1_hello modülünün simulation top olarak seçilmesi.

Ardından:

Flow Navigator → **Run Simulation** → **Run Behavioral Simulation**

seçilir (Şekil 2.37).



Şekil 2.37: Flow Navigator içerisinde Run Simulation.

2.10.4 Sınırları Nelerdir?

Behavioral simülasyon güçlü bir fonksiyonel doğrulama aracıdır ancak fiziksel FPGA davranışının tamamını temsil etmez.

- Behavioral simülasyon placement ve routing sonrasında oluşan gerçek FPGA gecikmelerini modellemez. Temel olarak fonksiyonel davranış doğrular.
- UART baud değeri ile testbench'in kullandığı bit süresi uyumlu değilse karakterler yanlış örneklenebilir ve test FAIL ile sonuçlanabilir.
- IMEM/DMEM initialization dosyaları doğru yüklenmezse program beklenen şekilde çalışmayabilir.
- AXI UART Lite taban adresi yazılımdaki adres ile uyuşmazsa UART erişimleri doğru peripheral'a ulaşmaz.
- DEBUG_ENABLE=1 olduğunda dbg_* portları waveform üzerinde kullanılabilir. Debug özelliğinin etkinleştirilmesi ek gözlem mantığı sebebiyle synthesis kaynak kullanımını artırabilir, ancak programın mimari davranışını değiştirmemelidir.
- Bu akış Verilator tabanlı microBee doğrulama akışından farklıdır. Burada Vivado Block Design ve xsim üzerinden, FPGA entegrasyon yapısını içeren bir behavioral doğrulama yapılmaktadır.
- rv32'nin Pipeline görünümü öğretim amaçlıdır; microBee'nin hazard, stall, forwarding ve fiziksel zamanlama modelini uygulamaz. Emülatör bellek başlangıçları ve çevre birimi register'ları da REF32 haritasıyla birebir aynı değildir. Ortamlar arası karşılaştırma Bölüm 11.10'da verilmiştir.

2.11 Vivado Çıktılarının İzlenmesi

2.11.1 Waveform

Behavioral simülasyon başladıktan sonra Waveform penceresinde en azından:

- clk_100MHz,
- reset_rtl_0,
- UART TX sinyali,
- testbench içerisinde oluşturulan char_byte,
- beklenen HELLO dizisi

izlenebilir.

Reset bırakıldıktan sonra testbench UART TX hattını decode eder.

Doğru çalışmada char_byte sırasıyla:

0x48, 0x65, 0x6C, 0x6C, 0x6F, ...

değerlerini alır.

Bu değerlerin ASCII karşılığı:

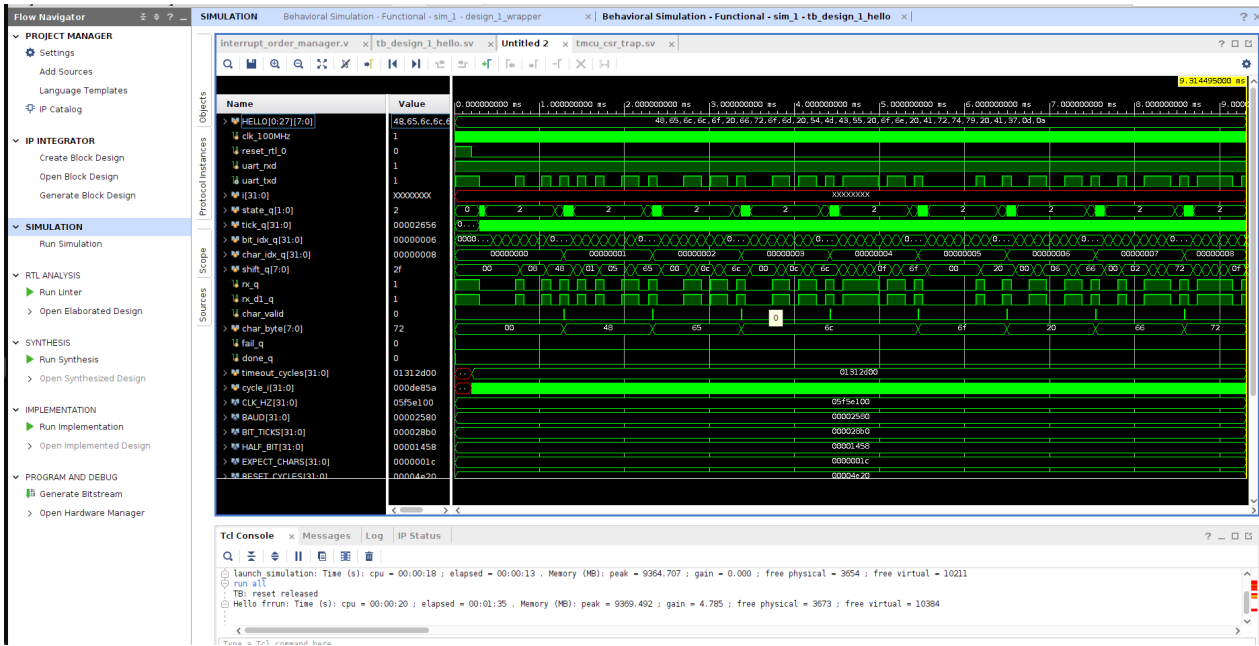
Hello...

mesajını oluşturur.

Tcl Console içerisinde de testbench tarafından üretilen:

TB: reset released

gibi mesajlar ve UART çıktısı görülebilir (Şekil 2.38).



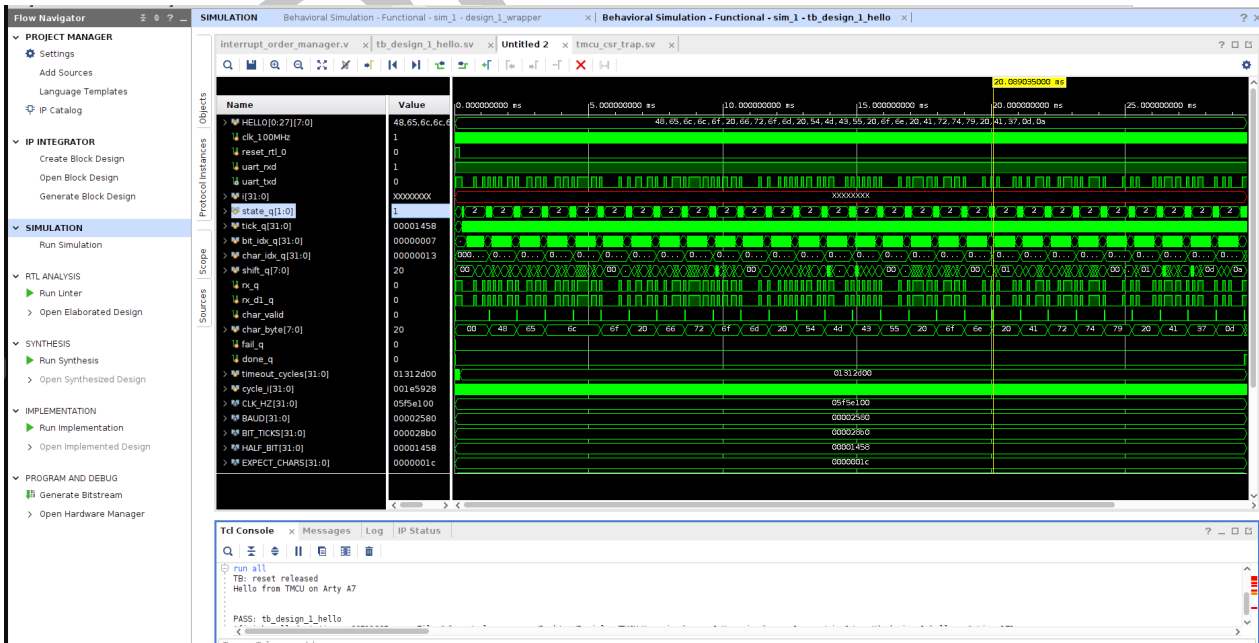
Şekil 2.38: Behavioral simülasyon waveform’unda UART üzerinden “Hello” karakterlerinin alınması.

Simülasyon sonuna gelindiğinde testbench beklenen 28 karakterin tamamını doğru almışsa konsolda hem UART metni hem de başarı mesajı yazılır (Şekil 2.39):

```
Hello from microBee on Arty A7
PASS: tb_design_1_hello
```

Bu çıktı, IMEM/DMEM init dosyalarının yüklendiğini, microBee’nin programı çalıştırdığını ve AXI UART Lite → uart_rtl_0_txd yolunun testbench alıcısı ile uyumlu olduğunu gösterir.

Karakter uyumsuzluğu veya zaman aşımında FAIL satırı görülür; o durumda baud/saat eşleşmesi, Address Editor tabanı ve init yolları yeniden kontrol edilmelidir.



Şekil 2.39: Tcl Console: UART “Hello” çıktısı ve PASS: tb_design_1_hello mesajı.

2.11.2 Debug

Block Design simülasyonunda çekirdeğin iç RTL hiyerarşisine girmeden mikro-mimari davranışı gözlemlemek için microBee debug portları kullanılabilir.

DEBUG_ENABLE=1 olduğunda tmcu_soc_0 üzerinde dbg_* çıkışları kullanılabilir.

Bu portlar üzerinden örneğin:

- pipeline aşamalarındaki PC değerleri,
- pipeline valid bilgileri,
- register file write enable,
- register file write address,
- register file write data,
- bazı CSR durumları

waveform üzerinde izlenebilir.

Debug portlarının amacı normal program davranışını değiştirmek değil, çekirdeğin iç davranışını dışarıdan gözlemlenebilir hale getirmektir.

Ayrıntılı dbg_* port listesi Debug Portları bölümünde verilmiştir.

FPGA üzerinde ILA veya MARK_DEBUG kullanımı bu giriş laboratuvarının zorunlu bir parçası değildir.

İlk aşamada xsim içerisinde UART çıktısının doğrulanması ve gerektiğinde dbg_* portlarının waveform üzerinden incelenmesi yeterlidir.

2.12 Kısa Kontrol Listesi

Laboratuvar sonunda aşağıdaki maddeler kontrol edilmelidir:

1. microBee IP repository yolu doğru mu?

work/fpga_synth/ip/tmcu_soc

2. Block Design içerisinde microBee ve AXI UART Lite bulunuyor mu?
3. AXI, clock ve reset bağlantıları Connection Automation ile tamamlandı mı?
4. AXI UART Lite Address Editor içerisinde microBee'nin AXI penceresinde bulunuyor mu?
5. Laboratuvar testbench'i kullanılıyorsa clock ve baud ayarları 100 MHz / 9600 ile uyumlu mu?
6. External interrupt kullanılacaksa AXI UART Lite interrupt çıkışı interrupt_order_manager üzerinden ext_irq_i[0] hattına bağlandı mı?
7. Kullanılmayan external interrupt girişleri 0'a sabitlendi mi?
8. IMEM ve DMEM initialization dosyalarının yolları doğru mu?
9. HDL wrapper oluşturuldu mu?
10. tb_design_1_hello simulation top olarak seçildi mi?
11. Behavioral simülasyon çalıştırıldığında UART üzerinden beklenen "Hello" mesajı alınıyor mu?
12. Gerekirse DEBUG_ENABLE=1 yapılarak dbg_* sinyalleri waveform üzerinde doğrulandı mı?

Waveform (Şekil 2.38) veya simülasyon konsolunda PASS: tb_design_1_hello ile birlikte beklenen "Hello" mesajının görülmesi (Şekil 2.39); microBee'nin program belleğinden komut getirebildiğini, yazılımın çalıştığını ve temel AXI UART Lite haberleşme yolunun doğru kurulduğunu gösteren önemli bir ilk doğrulamadır.

Bu test sistemdeki bütün peripheral ve mikro-mimari özelliklerin eksiksiz doğrulandığı anlamına gelmez. Daha ayrıntılı doğrulamalar sonraki laboratuvarlarda monitoring, interrupt, GPIO ve debug mekanizmaları kullanılarak yapılmalıdır.

Bölüm 3

Mimari

3.1 Mimari - Bilgisayar Mimarisi

3.1.1 Mimari nedir?

Bilgisayar mimarisi, bir bilgisayar sisteminin tasarımını ve bu tasarımda yapılan temel seçimleri tanımlayan çerçevedir. ISA, mikro-mimari, bellek yapısı, pipeline, bus ve I/O gibi alanlarda hangi seçimlerin yapıldığı ve hangi trade-off'ların kabul edildiği mimarinin kapsamına girer.

3.1.2 Mimari neleri etkiler?

Mimari, sistemin ne yapabileceğini ve performans, alan, güç gibi ölçütleri büyük ölçüde şekillendirir. Bununla birlikte bu ölçütler yalnızca mimariye bağlı değildir; implementasyon, üretim teknolojisi, compiler, bellek hiyerarşisi ve workload gibi etkenler de sonucu belirler. Yazılım ve toolchain desteği de mimari seçimlerden etkilenir. Örneğin bir GPU'nun işlem birimi düzeni, LLM çıkarım (inference) hızını önemli ölçüde etkileyebilir; ancak nihai hız, yazılım yığını ve çalışma zamanı katmanlarına da bağlıdır. Mimari, sistemin temel sınırlarını ve fırsatlarını belirler.

3.1.3 Neden Mimariyi bilmelisiniz?

Mimari bilmek bir gömülü sistem tasarımcısı için vazgeçilmezdir. Piyasada bulunan binlerce çeşit kontrolcünün her birinin kendisine özgü özellikleri vardır. Bazı kontrolcüler yüksek hız için, bazıları ise yüksek I/O kapasitesi veya düşük güç için tasarlanmış olabilir. Bu farkları bilmek, tasarımcının seçilen kontrolcüdeki mimarinin sınırlarını sonuna kadar kullanmasını sağlar.

3.1.3.1 Örnek:

Sistem superscalar ise, uygun bağımsız instruction'ları aynı cycle içerisinde birden fazla issue ederek paralel yürütmeyi hedefler. Bu, yalnızca birden fazla fonksiyon birimine (örneğin iki toplayıcıya) sahip olmak anlamına gelmez; kritik nokta, bağımlılık olmayan instruction'ların aynı cycle'da başlatılabilmesidir. Bağımlılık, hazard veya kaynak çakışması varlığında her cycle iki sonuç üretilmesi garanti değildir. Tasarımcı bunu bilmezse bağımlılık içeren işlemleri gereksiz yere sıralayıp paralel yürütme fırsatını kaçırabilir.

Bu bölüme kadar genel bilgisayar mimarisi kavramları özetlenmiştir. Bundan sonraki kısımlar aynı kavramları microBee SoC'sinin üst düzey blok diyagramı, pipeline'ı ve bellek yolları üzerinden somutlaştırır.

3.2 Genel Mimari

3.2.1 Top

microBee üst seviyede Harvard mimarisine sahip bir SoC yapısıdır. Instruction memory (IMEM) ve data memory (DMEM) birbirinden ayrıdır. Bu sayede instruction fetch işlemleri ile load/store işlemleri farklı bellek yolları üzerinden gerçekleştirilebilir.

Çekirdek beş aşamalı bir pipeline kullanır:

IF → ID → EX → MEM → WB

Pipeline içerisinde RV32IMC instruction'ları işlenir.

Instruction fetch tarafı native IMEM istek/yanıt arayüzü üzerinden doğrudan IMEM'e erişir. Data tarafındaki load/store ve MMIO istekleri ise native valid/ready arayüzü üzerinden `tmcu_request_router` bloğuna gönderilir.

Request router, erişim adresine göre veri isteğinin hangi hedefe yönlendirileceğini belirler. REF32 profilinde temel yönlendirme sırası:

CLINT → PLIC → GPIO → TIMER → UART → MONITOR → DMEM → AXI → Error

şeklindedir.

Bu nedenle bir veri isteği:

- CLINT'e,
- PLIC'e,
- GPIO'ya,
- Native Timer'a,
- Native UART'a,
- Monitoring birimine,
- DMEM'e,
- veya AXI adres penceresine

yönlendirilebilir.

microBee çekirdeği AXI protokolünü doğrudan kullanmaz. AXI adres penceresine yönlendirilen native istekler `tmcu_native_to_axi4lite` köprüsü (bridge) tarafından AXI4-Lite transaction'larına dönüştürülür.

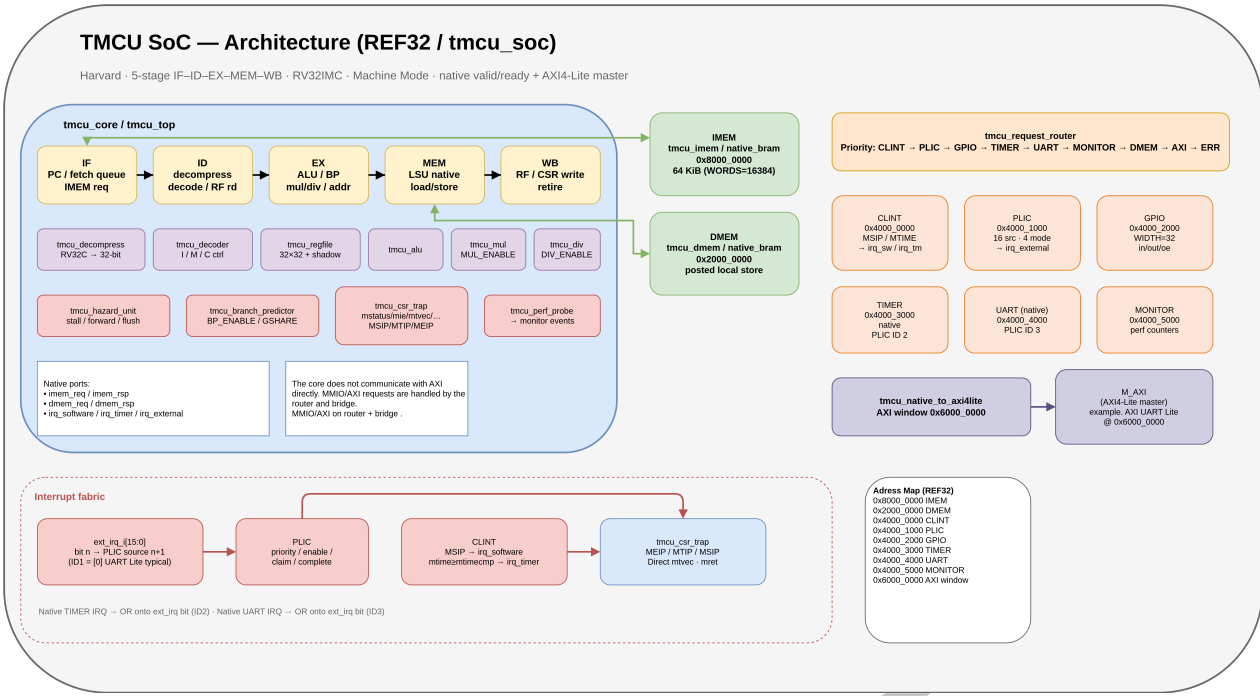
REF32 profilinde AXI penceresi:

0x60000000

adresinden başlar.

Bu yapı sayesinde çekirdek ve native peripheral'lar kendi basit valid/ready arayüzlerini kullanırken, FPGA Block Design içerisine eklenen AXI tabanlı IP'lere de erişilebilir.

Şekil 3.1, `tmcu_soc` içerisindeki ana blokları göstermektedir. Şekilde beş aşamalı pipeline, IMEM ve DMEM, request router, CLINT, PLIC, GPIO, Native Timer, Native UART, Monitoring birimi, AXI4-Lite bridge ve interrupt yapısı birlikte gösterilmiştir.



Şekil 3.1: microBee SoC üst düzey mimarisi (tmcu_soc, REF32).

3.2.2 I/O

microBee'nin dış dünya ile bağlantısını sağlayan portlar tmcu_soc seviyesinde bulunur.

Portların görünürlüğü ve kullanımı bazı konfigürasyon parametrelerine bağlıdır. Özellikle DEBUG_ENABLE, TRACE_ENABLE ve AXI_ENABLE ilgili port gruplarının kullanımını etkiler.

Temel I/O grupları aşağıdaki gibidir.

- **Clock ve Reset**

clk_i, microBee'nin ana clock girişidir.

rst_ni ise active-low reset girişidir.

FPGA Block Design içerisinde bu sinyaller genellikle doğrudan kart pinlerinden microBee'ye bağlanmaz. Giriş clock'u Clocking Wizard üzerinden işlenirken reset üretimi ve senkronizasyonu genellikle Processor System Reset (proc_sys_reset) bloğu tarafından gerçekleştirilir.

- **External Interrupt**

microBee dış interrupt kaynakları için:

ext_irq_i[15:0]

girişini kullanır.

Bit ile PLIC source ID arasındaki ilişki:

ext_irq_i[n] → PLIC source ID $n + 1$

şeklinde.

Buna göre:

ext_irq_i[0] → PLIC ID 1

olur.

FPGA Block Design içerisinde AXI UART Lite interrupt çıkışı tipik olarak ext_irq_i[0] hattına bağlanır ve PLIC ID 1 olarak kullanılır.

Native Timer ve Native UART interrupt kaynakları ise SoC içerisinde PLIC girişlerine dahil edilir. REF32 profilinde:

- Native Timer → PLIC ID 2,
- Native UART → PLIC ID 3

olarak kullanılır.

Bu kaynaklar ilgili external interrupt hatları ile SoC içerisinde birleştirilebildiği için aynı PLIC ID'yi kullanan başka bir kaynağın bağlantısı tasarlanırken dikkat edilmelidir.

- **GPIO**

GPIO arayüzü üç temel port grubundan oluşur:

- `gpio_in_i`: fiziksel GPIO giriş değerlerini microBee'ye taşır.
- `gpio_out_o`: GPIO çıkış verisini dışarı verir.
- `gpio_oe_o`: ilgili GPIO hattının output olarak sürülüp sürülmeyeceğini belirler.

REF32 profilinde:

`GPIO_WIDTH = 32`

kullanılır.

`gpio_oe_o=1` olduğunda ilgili GPIO hattı output olarak sürülür; 0 olduğunda output sürücüsü devre dışı bırakılır.

- **Native UART**

microBee RTL içerisinde doğrudan bulunan Native UART:

`uart_rx_i      uart_tx_o`

portlarını kullanır.

Native UART'ın MMIO base adresi REF32 profilinde:

`0x40004000`

olarak tanımlanmıştır.

Native UART ile Vivado Block Design içerisine eklenen AXI UART Lite birbirinden farklı peripheral'lardır. AXI UART Lite, microBee RTL'nin bir parçası değildir. FPGA Block Design içerisinde ayrı bir AXI IP olarak eklenir ve microBee'nin `M_AXI` interface'i üzerinden erişilir.

Block Design içerisinde dış RX/TX interface'i genellikle:

`uart_rtl_0`

adıyla görülür.

- **AXI4-Lite Master**

`M_AXI`, microBee'nin FPGA üzerindeki AXI4-Lite peripheral'lara erişmesini sağlayan master interface'tir.

Bu interface:

- AW: write address,
- W: write data,
- B: write response,
- AR: read address,
- R: read data

kanallarından oluşur.

microBee çekirdeği bu AXI sinyallerini doğrudan üretmez. Native veri isteği, request router üzerinden AXI penceresine yönlendirildiğinde AXI köprüsü gerekli AXI4-Lite işlemi oluşturur.

REF32 profilinde AXI adres penceresi:

`0x60000000–0x6FFFFFFF`

aralığındadır.

Örneğin AXI UART Lite:

`0x60000000`

adresine yerleştirilebilir.

`AXI_ENABLE=0` olduğunda AXI erişim yolu devre dışıdır ve `M_AXI` interface'i aktif olarak kullanılmaz.

- **Debug Portları**

`DEBUG_ENABLE=1` olduğunda microBee çeşitli mikro-mimari bilgileri `dbg_*` portları üzerinden dışarı verir.

Bu portlar arasında örneğin:

- pipeline PC değerleri,
- pipeline valid sinyalleri,
- register file write enable,
- register file write address,
- register file write data,
- CSR snapshot bilgileri

bulunur.

Debug portları normal program execution'ını değiştirmek için değil, waveform ve simülasyon sırasında çekirdeğin iç durumunu gözlemlemek için kullanılır.

`DEBUG_ENABLE=0` olduğunda debug çıkışları aktif gözlem amacıyla kullanılmaz; IP paketleme ayarına bağlı olarak Block Design seviyesinde gizlenebilir veya sabit değerlere çekilebilir.

• Trace Portları

TRACE_ENABLE=1 olduğunda instruction commit bilgileri **trace_*** portları üzerinden dışarı aktarılabilir. Trace interface'i örneğin:

- retired instruction PC'si,
- instruction değeri,
- register write bilgisi,
- memory access bilgisi,
- trap ve cause bilgisi

gibi commit seviyesindeki olayların izlenmesini sağlar.

Debug portları daha çok anlık mikro-mimari gözlem için kullanılırken, trace portları instruction seviyesindeki mimari olay akışını takip etmek için kullanılır.

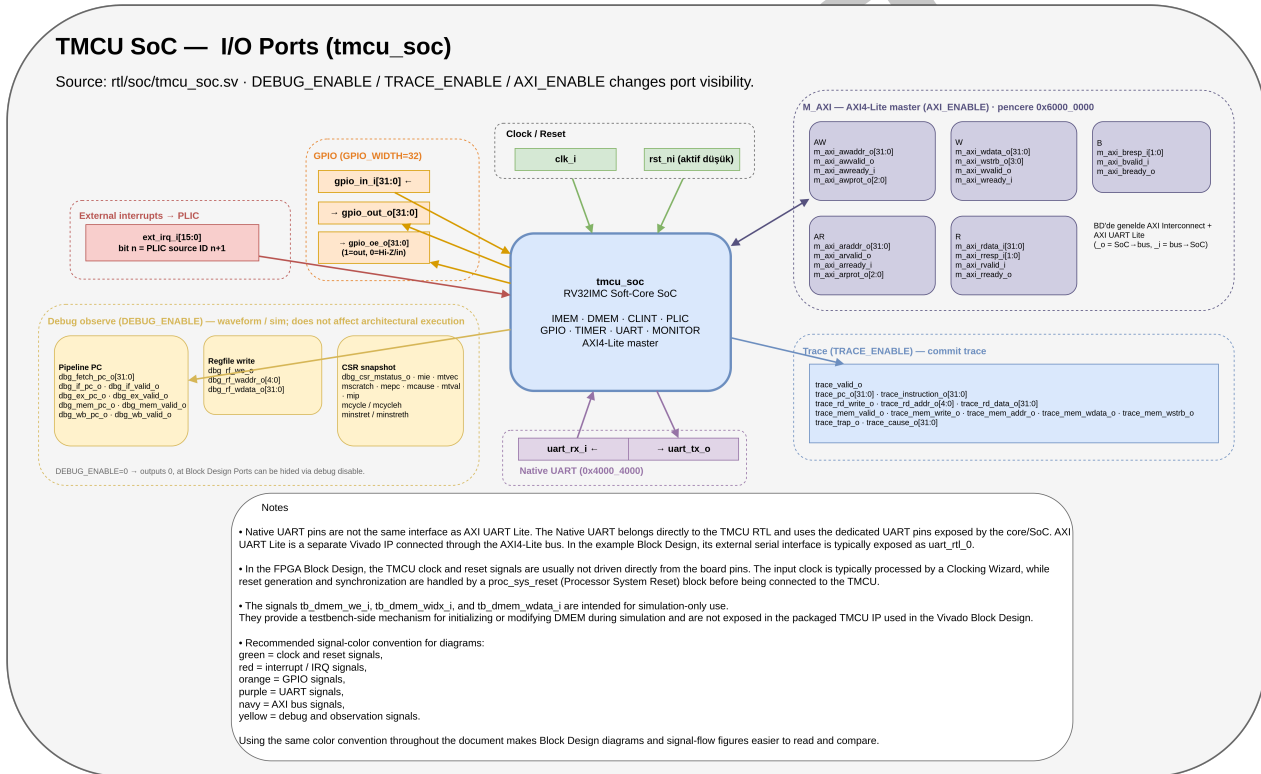
Simülasyona özel portlar.

tb_dmem_we_i, tb_dmem_widx_i ve tb_dmem_wdata_i yalnızca simülasyon amacıyla kullanılan testbench portlarıdır.

Bu portlar testbench tarafından DMEM içeriğinin doğrudan hazırlanması veya değiştirilmesi için kullanılabilir.

Paketlenmiş microBee FPGA IP'sinin normal Block Design interface'inde bu portlar bulunmaz.

Şekil 3.2, tmcu_soc seviyesinde dışarı açılan temel I/O gruplarını göstermektedir.



Şekil 3.2: microBee SoC dış I/O portları ve işlevsel grupları (tmcu_soc).

3.2.3 Pipeline

microBee çekirdeğinin pipeline'ı beş stage'den oluşur. Bu stage'ler sırasıyla Fetch, Decode, Execute, Memory ve Writeback'tir.

IF -> IF/ID -> ID -> ID/EX -> EX -> EX/MEM -> MEM -> MEM/WB -> WB

IF/ID, ID/EX gibi yapılar pipeline register'larıdır. Ayrı birer stage değildir; stage'ler arası geçiş register'larını ifade eder.

3.2.3.1 Fetch

Fetch, sıradaki komutun bellekten alınmasını yönetir. Program sayacı (PC) ile komut belleğine istek gönderilir; gelen kelimeler fetch kuyruğunda tutulur ve gerekirse 16/32-bit hizalama ile bir sonraki aşamaya hazırlanır.

3.2.3.2 Decode

Decode, alınan komutu yorumlar ve kaynak register'lar (**rs1/rs2**) register file'dan okunur. Çıktı kontrol sinyalleri ve operandlardır.

3.2.3.3 Execute

Execute, komutun asıl hesabını yapar. ALU işlemleri, branch/jump kararı, çarpma/bölme (etkinse) ve load/store için bellek adresi burada üretilir. Sonuç bir sonraki aşamaya aktarılır.

3.2.3.4 Memory

Memory, veri yolu erişimini yürütür. Load/store isteği, byte lane seçimi ve yanıt bekleme bu aşamadır. Bellek işlemi yoksa aşama sonuçları olduğu gibi iletir.

3.2.3.5 Writeback

Writeback, mimari durumu günceller. Register file ve gerekirse CSR yazımı bu saat kenarında tamamlanır; komut retired sayılır ve izleme/performans olayları buradan üretilir. Store gibi register file'a yazmayan instruction'lar da bu aşamada mimari olarak tamamlanmış sayılabilir.

3.3 Register File

microBee içerisinde 32 adet 32-bit register bulunur. Bu kümenin toplamına register file denir. İşlemci çalışırken operand ve sonuç değerlerini burada tutar. Örneğin bir toplama işleminde $A + B = C$ için operandlar ve sonuç sıklıkla register'larda tutulur; ancak bu yalnız tipik bir örnektir. Operand immediate, forwarding yolu veya bellek üzerinden de gelebilir. İşlemci, gelen komuttaki bilgilere göre okuyacağı ve yazacağı adresleri çözerek register file üzerinde manipülasyon yapar. **x0** her zaman sıfır okunur ve ona yapılan yazmalar yok sayılır.

Tablo 3.1: RISC-V integer register'ları [1]

Register	ABI adı	Geleneksel kullanım	Donanım davranışı
x0	zero	Sabit sıfır	Okuma 0; yazma yok sayılır.
x1	ra	Dönüş adresi	Genel amaçlı depolama.
x2	sp	Stack pointer	Genel amaçlı depolama.
x3	gp	Global pointer	Genel amaçlı depolama.
x4	tp	Thread pointer	Genel amaçlı depolama.
x5-x7	t0-t2	Temporary	Genel amaçlı depolama.
x8-x9	s0-s1	Saved/frame pointer	Genel amaçlı depolama.
x10-x17	a0-a7	Argüman/dönüş	Genel amaçlı depolama.
x18-x27	s2-s11	Saved	Genel amaçlı depolama.
x28-x31	t3-t6	Temporary	Genel amaçlı depolama.

Not

ABI (Application Binary Interface), binary seviyesinde yazılım bileşenleri arasındaki sözleşmedir: calling convention, register kullanımı, stack düzeni ve benzeri kurallar burada tanımlanır. Compiler, programı makine koduna çevirirken ABI'de tanımlanan register rollerine göre kod üretir. Donanım açısından RISC-V integer register'larının çoğu genel amaçlıdır; **x0** sabit sıfır gibi istisnalar mimari olarak sabittir. ARM gibi bazı mimarilerde de belirli register'lar (örneğin link register) geleneksel olarak sabit roller taşır.

x0-x7	x8-x15	x16-x23	x24-x30
Parameter and Result Registers (x0-x7)	XR (x8)	IP0 (x16)	Callee-saved Registers (x24-x28)
	Corruptible Registers (x9-x15)	IP1 (x17)	
		PR (x18)	
		Callee-saved Registers (x19-x23)	FP (x29)
			LR (x30)

Şekil 3.3: ARM mimarisi ABI haritası [6].

3.3.1 Özellikleri

İki combinational read port ve bir synchronous write port bulunur. Okuma combinational, yazma ise sequential (saat kenarında) gerçekleşir. Her register 32-bit veri tutar; uygun koşullarda aynı cycle içinde okuma ve yazma yapılabilir.

3.3.2 Kullanım Örneği

add x1, x2, x3

x1 adresine x2 ve x3'ün toplamı yazılır.

addi x1, x2, 5

x1 adresine, x2'de depolanan değer ile immediate 5'in toplamı yazılır.

3.3.3 Debug Nasıl Yapılır?

Sistemde izlenebilir debug portları bulunur; işlemci çalışırken bu portlar üzerinden register file'a yapılan yazmalar gözlemlenebilir. Reset sonrası başlangıç durumu biliniyor ve bütün yazmalar eksiksiz izleniyorsa write-port takibi, register file içeriğini yeniden oluşturmak için pratik bir yoldur. Shadow bank veya yazmaların kaçırıldığı durumlarda yalnız write-port izlemesi yeterli olmayabilir.

Tablo 3.2: Register file debug sinyalleri ve açıklamaları

Sinyal	Açıklama
dbg_rf_we_o	İlgili clock çevriminde register file'a yazma işlemi yapıldığını belirtir.
dbg_rf_waddr_o[4:0]	Yazma işleminin hedef register adresini (x0-x31) belirtir.
dbg_rf_wdata_o[31:0]	Register file'a yazılan 32-bit veriyi belirtir.

3.4 Instruction Kavramı

3.4.1 Nedir?

Instruction, işlemcinin decode edip yürüteceği komuttur. Komutlar işlemci içerisinde çözümlenerek ilgili birimlere yönlendirilir. RISC-V çerçevesinde instruction'lar, programın ifade edilebileceği temel işlemleri ve bunların bit seviyesindeki encoding'lerini tanımlar.

3.4.1.1 Örnek

RISC ve CISC iki temel felsefedir. RISC, sık kullanılan işlemleri daha basit instruction'larla ifade etmeyi tercih eder. CISC ise daha karmaşık işlemleri tek instruction içinde tanımlayabilir. RISC mimarisinde **and**, **or**, **xor**, **add**, **sub** gibi temel instruction'lar tanımlanmıştır. Bunlar ISA seviyesinde temel işlemlerdir; mikro-mimari içinde daha küçük adımlara ayrılabilirler.

3.4.2 Nerede Depolanır

Instruction'lar mimariler arasında farklılık gösterebilse de Harvard mimarilerinde IMEM adı verilen instruction memory bloğunda tutulur ve program counter ile sırayla seçilerek işlenir.

3.4.3 Nasıl Oluşur

Instruction'lar, kullanıcıların yazdığı kodların compiler'lar aracılığıyla dönüştürülmesi ile elde edilir. Compiler konusu çok derin ve incelenmesi gömülü sistem tasarımcıları için önemli bir konudur. Ders çerçevesinde bahsedilmesi karmaşık ve uzun olacağı için atlanmıştır. İlgisi olana ufak bir örnek eklenmiştir.

3.4.3.1 Örnek-1

Compiler girdi ve çıktı örneği aşağıda verilmiştir.

```

1 // Type your code here, or load an example.
2 int square(int num) {
3     return num * num;
4 }

```

```

1 square(int):
2     addi    sp, sp, -32
3     sw      ra, 28(sp)
4     sw      s0, 24(sp)
5     addi    s0, sp, 32
6     sw      a0, -20(s0)
7     lw      a5, -20(s0)
8     mul     a5, a5, a5
9     mv      a0, a5
10    lw      ra, 28(sp)
11    lw      s0, 24(sp)
12    addi    sp, sp, 32
13    jr      ra

```

Şekil 3.4: C kaynak kodunun derleyici tarafından RISC-V assembly çıktısına dönüştürülmesi.

3.4.3.2 Örnek-2

Compiler bilmenin önemi şu şekilde örneklendirilebilir. Compiler, pointer'ların aynı bellek nesnesine alias olabileceğini varsaymak zorundadır; bu yüzden bazı array işlemlerinde daha agresif optimizasyon uygulamaz. Eğer tasarımcı adreslerin birbiri üzerine binmeyeceğini (alias olmayacağını) garanti ediyorsa, array girişlerini **restrict** niteleyicisi ile işaretler. Bu, güvenlik parametresini kapatmaz; compiler'a alias olmadığı güvencesini verir ve beklenen performansta daha agresif optimizasyona izin verir.

$X[i] = Y[i] + Z[i]$ gibi işlemlerde bu durum görülür. İleri okuma için **Pointer Aliasing** konusuna bakılmalıdır.

3.4.4 Kim Belirliyor

RISC-V instruction set'leri RISC-V International tarafından ratifiye edilen spesifikasyonlarla tanımlanır [1]. Topluluk üyeleri, geliştiriciler ve araştırmacılar uzantı önerileri sunabilir; resmi ISA belgesi vakıf tarafından yayımlanır. Bu instruction set'leri kullanıcıların isteklerine göre genişletilebilir yapılardır. Ancak compiler uyumsuzluğu ve kodun derlenememesi gibi durumlar sebebiyle tasarımcılar genelde compiler'ın desteklediği instruction'lar çevresinde geliştirme yapmaktadır.

3.5 Aritmetik Mantık Birimi (Arithmetic Logic Unit)

Her işlemcide bulunan bir birimdir. Desteklenen instruction'ların hesaplama işlemlerinin yapıldığı kısımdır. Belirli durumlarda sistem frekansını artırmak için bazı işlemlerine pipeline yapılabilir. Ancak çoğunlukla işlemleri tek cycle'da bitirir.

3.6 MUL/DIV

M genişlemesi, çarpma, bölme ve kalan (remainder) işlemlerini sağlar. `MUL_ENABLE` ve `DIV_ENABLE` parametreleri birbirinden bağımsızdır. İlgili birim devre dışı bırakıldığında, bu birime ait opcode'lar *illegal instruction* trap'i üretir. Her iki birim de EX aşamasında çalışır ve sonuç hazır olana kadar EX aşaması stall durumunda tutulur.

3.6.1 MUL

`tmcu_mul`, 2 çevrimlik pipelined bir çarpma birimidir ve `MUL`, `MULH`, `MULHSU` ve `MULHU` komutlarını destekler. Birim, tek bir 33×33 -bit işaretli çarpıcı kullanır. Operasyona bağlı olarak çarpım sonucunun yüksek veya düşük 32-bitlik kısmı seçilir.

Birim müsait olduğu sürece her saat çevriminde yeni bir çarpma işlemi başlatılabilir. Bu, multiplier biriminin kendi throughput özelliğidir. microBee tek-issue bir çekirdek olduğundan, çekirdeğin her cycle yeni bir MUL issue edebildiği anlamına gelmez; EX aşamasında stall ve diğer pipeline kısıtları da geçerlidir.

3.6.2 DIV

`tmcu_div`, yinelemeli restoring bölme algoritmasını kullanan bir bölme birimidir ve `DIV`, `DIVU`, `REM` ve `REMU` komutlarını destekler.

Tipik işlem süresi yaklaşık 34 saat çevrimidir (kurulum + 32 yineleme adımı + sonlandırma). Bölme veya kalan işlemi devam ederken EX aşamasına stall uygulanır. Pipeline flush oluşması durumunda devam eden işlem iptal edilir.

3.6.3 Neden Ayrı Birimler ve Pipeline Kullanılıyor?

ALU işlemleri tek çevrimde tamamlanabilirken, çarpma ve özellikle bölme işlemleri daha uzun sürebilir. MUL ve DIV işlemlerinin ayrı birimlerde gerçekleştirilmesi ve gerektiğinde EX aşamasının stall edilmesi sayesinde:

- tek çevrimlik ALU komutları tek çevrimlik yürütme süresini korur,
- uzun süren MUL/DIV işlemleri yalnızca gerektiğinde pipeline'ı bekletir,
- MUL ve DIV birimleri sentez parametreleri ile bağımsız olarak etkinleştirilebilir veya devre dışı bırakılabilir.

MUL ve DIV işlemlerinin tamamen tek çevrimlik kombinasyonel mantık ile gerçekleştirilmesi kritik yolu uzatarak maksimum çalışma frekansını düşürebilir ve alan kullanımını artırabilir. Buna karşılık tüm çekirdeğin uzun MUL/DIV gecikmelerine göre tasarlanması da tek çevrimlik komutların performansını gereksiz yere düşürür. Bu nedenle microBee'de uzun gecikmeli işlemler ayrı yürütme birimleri üzerinden gerçekleştirilir.

3.7 Dallanma (Branch)

3.7.1 Nedir?

Branch instruction'ları programın normal sıralı akışını değiştirmek için kullanılır. Normal durumda Program Counter sıradaki instruction adresine ilerler. Branch instruction'ında ise belirli bir koşul kontrol edilir ve bu koşul doğruysa program farklı bir adresten çalışmaya devam eder.

Bu yapı özelliklerle:

- if-else yapılarında,
- loop'larda,
- karşılaştırmaya bağlı karar mekanizmalarında

kullanılır.

Örneğin:

beq x1, x2, label

instruction'ı `x1` ve `x2` register'larını karşılaştırır. Eğer iki değer eşitse program `label` adresine dallanır. Eşit değilse program normal sıradaki instruction'dan devam eder.

Branch sonucunun iki temel durumu vardır:

- **Taken:** Koşul sağlanır ve branch hedef adresine gidilir.
- **Not-Taken:** Koşul sağlanmaz ve sıradaki instruction çalıştırılır.

3.7.2 Nasıl İfade Edilir?

RISC-V içerisinde koşullu dallanma işlemleri branch instruction'ları ile ifade edilir.

microBee tarafından desteklenen temel branch instruction'ları şunlardır:

- **BEQ:** Eşitse branch alınır.
- **BNE:** Eşit değilse branch alınır.
- **BLT:** Signed karşılaştırmada küçükse branch alınır.
- **BGE:** Signed karşılaştırmada büyük veya eşitse branch alınır.
- **BLTU:** Unsigned karşılaştırmada küçükse branch alınır.
- **BGEU:** Unsigned karşılaştırmada büyük veya eşitse branch alınır.

Branch instruction'ları genel olarak iki register ve bir immediate bilgisi içerir.

Örneğin:

beq x5, x6, 16

şeklindeki bir instruction'da x5 ve x6 karşılaştırılır. Koşul sağlanırsa branch hedef adresi mevcut Program Counter ile immediate değerinin toplanmasıyla hesaplanır.

Genel olarak:

$$\text{Branch Target} = PC + \text{Immediate}$$

şeklinde ifade edilebilir.

Branch alınmazsa Program Counter normal sıradaki instruction adresine ilerler.

RV32IMC yapısında instruction uzunluğu 16-bit veya 32-bit olabileceği için sıradaki PC değeri instruction uzunluğuna göre belirlenir.

3.7.3 Nasıl Çözülür?

microBee içerisinde branch instruction'larının gerçek sonucu Execute (EX) aşamasında kesinleşir.

Branch pipeline içerisinde ilerlerken Decode aşamasında gerekli register değerleri okunur ve Execute aşamasına gönderilir.

Execute aşamasında iki temel işlem yapılır:

1. kaynak operandlar branch türüne göre karşılaştırılır,
2. branch hedef adresi $PC + \text{immediate}$ ile hesaplanır.

Örneğin BEQ instruction'ında Execute aşamasında:

$$rs1 == rs2$$

karşılaştırması yapılır.

Sonuç doğruysa branch taken, yanlışsa not-taken olarak belirlenir.

BLT ve BGE instruction'larında signed karşılaştırma, BLTU ve BGEU instruction'larında ise unsigned karşılaştırma yapılır.

Branch sonucunun EX aşamasında kesinleşmesi sebebiyle bu aşamaya gelene kadar Fetch ve Decode tarafında başka instruction'lar pipeline içerisine girmiş olabilir.

Eğer branch prediction doğruysa alınan instruction'lar geçerli kalır ve pipeline normal şekilde çalışmaya devam eder.

Eğer prediction yanlışsa Fetch ve Decode tarafında yanlış execution path üzerinden alınmış instruction'lar bulunabilir.

Bu durumda microBee:

- yanlış instruction'ları geçersiz hale getirir,
- IF/ID tarafını flush eder,
- Program Counter'ı doğru branch adresi veya fall-through adresi ile günceller,
- Fetch işlemini doğru adresten tekrar başlatır.

Bu işlem branch misprediction penalty oluşmasına neden olur.

Branch kararının pipeline içerisinde daha erken verilmesi penalty'yi azaltabilir ancak bu durumda Fetch veya Decode aşamalarına ek karşılaştırma ve adres hesaplama mantığı eklemek gerekebilir.

microBee'de branch çözümünün EX aşamasında yapılması tasarım karmaşıklığı ile performans arasında yapılan bir trade-off'tur.

3.8 Stall ve Flush Kavramları

Pipeline yapısında instruction'ların her cycle bir sonraki aşamaya ilerlemesi ideal durumdur. Ancak bazı durumlarda bir instruction'ın ihtiyaç duyduğu veri veya sonuç henüz hazır olmayabilir.

Bu gibi durumlarda pipeline'ın kontrol edilmesi gerekir.

Bu kontrol için kullanılan temel mekanizmalardan ikisi **stall** ve **flush** işlemleridir.

3.8.1 Stall Nedir?

Stall, pipeline içerisindeki belirli instruction'ların geçici olarak ilerlemesinin durdurulmasıdır.

Bir instruction'ın ihtiyaç duyduğu veri veya donanım kaynağı hazır değilse instruction'ın ilerlemesine izin verilmez.

Bu durumda ilgili pipeline register'ları mevcut değerlerini korur.

Basit olarak:

Normal: IF -> ID -> EX -> MEM -> WB

iken stall durumunda belirli bir instruction aynı stage içerisinde bir veya birden fazla cycle bekleyebilir.

Stall yeni instruction üretmez. Mevcut instruction'ın gerekli koşullar sağlanana kadar beklemesini sağlar.

3.8.2 microBee'de Stall Oluşturan Durumlar

microBee içerisinde stall farklı sebeplerle oluşabilir.

3.8.2.1 Load İşlemleri

Load instruction'ında memory'den okunacak verinin geri dönmesi gerekir.

Memory cevabı hazır değilse instruction'ın Writeback aşamasına ilerlemesi mümkün değildir.

Bu durumda pipeline memory cevabı gelene kadar stall edilir.

3.8.2.2 Load-Use Hazard

Bir load instruction'ından hemen sonra load edilen register'ı kullanan başka bir instruction gelirse veri henüz hazır olmayabilir.

Örneğin:

```
lw  x5, 0(x10)
add x6, x5, x7
```

ikinci instruction x5 değerine ihtiyaç duyar.

Eğer forwarding ile gerekli veri zamanında sağlanamıyorsa pipeline bir veya birden fazla cycle stall edilerek load sonucunun hazır olması beklenir.

3.8.2.3 MUL ve DIV İşlemleri

MUL ve DIV birimleri ALU işlemlerine göre daha uzun sürebilir.

Bu işlemler Execute aşamasında çalışırken sonuç hazır olana kadar EX aşaması stall durumunda tutulur.

Özellikle DIV işlemi yinelemeli çalıştığı için pipeline uzun süre EX aşamasında bekleyebilir.

3.8.2.4 Çekirdek Dışı Store İşlemleri

microBee çekirdeği içerisinde gerçekleştirilen store işlemlerinde çekirdek stall edilmez.

Ancak store işlemi core dışındaki bir bus veya peripheral'a yönlendiriliyorsa karşı taraf isteği hemen kabul etmeyebilir.

Bu durumda dış arayüz hazır olana kadar core stall edilebilir.

3.8.3 Flush Nedir?

Flush, pipeline içerisinde bulunan ancak artık çalıştırılmaması gereken instruction'ların geçersiz hale getirilmesidir.

Stall ile flush arasındaki temel fark şudur:

- **stall**: instruction bekletilir,
- **flush**: instruction tamamen iptal edilir.

Flush özellikle control hazard durumlarında kullanılır.

Branch prediction yanlış olduğunda pipeline içerisine yanlış execution path üzerinden instruction'lar alınmış olabilir.

Bu instruction'ların çalıştırılması programın yanlış sonuç üretmesine sebep olur.

Bu nedenle yanlış yoldaki instruction'lar pipeline içerisinden temizlenir.

3.8.4 microBee'de Branch Flush

microBee'de branch sonucu EX aşamasında kesinleşir.

Eğer branch prediction sonucu gerçek branch sonucu ile uyuşmuyorsa yanlış instruction'lar Fetch ve Decode tarafında bulunabilir.

Bu durumda IF/ID pipeline bölümü flush edilir ve Fetch doğru Program Counter değerinden tekrar başlatılır.

Örneğin predictor branch'i not-taken tahmin etmiş ancak gerçek sonuç taken çıkmışsa:

1. sıradaki instruction'lar yanlış yoldan alınmıştır,
2. EX aşamasında branch'in taken olduğu anlaşılır,
3. yanlış Fetch/Decode instruction'ları flush edilir,
4. PC branch target adresine ayarlanır,
5. Fetch doğru adresten tekrar başlar.

Benzer şekilde predictor taken tahmin etmiş ancak gerçek sonuç not-taken ise PC fall-through adresine yönlendirilir.

3.8.5 Stall ve Flush Arasındaki Fark

Stall ve flush birbirine benzer görünse de kullanım amaçları farklıdır.

	Stall	Flush
Amaç	Bekletmek	İptal etmek
Instruction	Geçerli kalır	Geçersiz olur
PC	Genellikle bekletilir	Yeni adrese yönlendirilebilir
Örnek	Load bekleme	Branch misprediction

Stall daha çok data hazard, uzun süren execution ve memory bekleme durumlarında kullanılır.

Flush ise yanlış instruction path veya artık geçerli olmayan instruction'ları pipeline içerisinden temizlemek için kullanılır.

3.8.6 Bubble Kavramı

Stall veya flush sonucunda pipeline içerisinde bazı cycle'larda geçerli bir instruction bulunmayabilir.

Bu boş duruma **bubble** adı verilir.

Bubble pipeline içerisinde gerçek bir instruction gibi ilerleyebilir ancak mimari durumda herhangi bir değişiklik oluşturmaz.

Örneğin bir load-use hazard sebebiyle bir cycle stall uygulanması pipeline içerisinde bir bubble oluşmasına sebep olabilir.

Branch misprediction sonrasında flush edilen instruction'ların yerleri de pipeline içerisinde geçici boşluklar oluşturur.

Bubble sayısının artması işlemcinin IPC değerini düşürür ve CPI değerini artırır.

Bu sebeple pipeline tasarımında stall, flush ve bubble oluşumlarının mümkün olduğunca azaltılması performans açısından önemlidir.

3.9 Dallanma Öngörüsü (Branch Prediction)

3.9.1 Nedir?

Branch prediction, işlemcinin bir branch instruction'ının sonucunu daha hesaplamadan önce tahmin etmeye çalışmasıdır. Pipeline yapısındaki işlemcilerde bir sonraki instruction'ın hangi adresten alınacağı mümkün olduğunca erken bilinmek istenir.

Normal durumda program counter her instruction sonrasında sıradaki adrese ilerler. Ancak branch instruction'larında iki farklı ihtimal vardır:

- branch alınmazsa program normal sıradaki instruction'dan devam eder,
- branch alınırsa program branch hedef adresinden devam eder.

Branch sonucunun kesin olarak bilinmesi Execute aşamasına kadar sürebilir. Bu süre içerisinde Fetch biriminin bekletilmesi pipeline performansını düşürür. Branch predictor bu bekleme azaltmak için branch sonucunu önceden tahmin eder.

3.9.2 Neden Gereklidir?

5-stage bir pipeline içerisinde instruction Fetch aşamasında alınırken branch kararının Execute aşamasında verilmesi mümkündür. Bu durumda işlemcinin önünde iki seçenek vardır.

Birinci seçenek branch sonucu belli olana kadar yeni instruction almamaktır. Bu yöntem donanım açısından basittir ancak her branch instruction'ında pipeline birkaç cycle boyunca boş kalır.

İkinci seçenek ise branch'in sonucunu tahmin ederek instruction almaya devam etmektir. Tahmin doğruysa pipeline durmadan çalışmaya devam eder. Tahmin yanlışsa yanlış yoldan alınan instruction'lar pipeline içerisinden temizlenir ve doğru adresten Fetch işlemine tekrar başlanır.

Bu sebeple branch prediction temel olarak performans ile donanım karmaşıklığı arasında yapılan bir trade-off'tur.

3.9.3 Taken ve Not-Taken

Branch instruction'larının sonucu genel olarak iki şekilde ifade edilir.

Taken: Branch koşulu sağlanmıştır ve program branch hedef adresine atlar.

Not-Taken: Branch koşulu sağlanmamıştır ve program sıradaki instruction'dan devam eder.

Örneğin aşağıdaki instruction ele alınabilir:

beq x1, x2, label

Eğer x1 ve x2 içerisindeki değerler eşitse branch **taken**, eşit değilse **not-taken** olur.

3.9.4 Bimodal Branch Prediction

Her predictor entry için 2-bitlik bir durum tutulur. Genel olarak dört durum vardır:

Durum	Tahmin
00	Strongly Not-Taken
01	Weakly Not-Taken
10	Weakly Taken
11	Strongly Taken

Branch taken olduğunda sayaç bir artırılır, not-taken olduğunda bir azaltılır. Sayaç 00 değerinin altına veya 11 değerinin üzerine çıkamaz. Bu sebeple *saturating counter* olarak isimlendirilir.

Bu yapıda predictor'ın karar değiştirmesi için genellikle arka arkaya birden fazla ters sonuç görmesi gerekir. Böylece loop'un sonundaki tek bir not-taken sonucu predictor'ın bütün davranışını doğrudan değiştirmez.

3.9.5 Prediction Doğruysa Ne Olur?

Predictor branch'in sonucunu doğru tahmin etmişse Fetch tarafından alınmış instruction'lar doğru execution path üzerindedir.

Bu durumda pipeline'ın temizlenmesine gerek kalmaz ve branch penalty büyük oranda ortadan kaldırılmış olur. Bu durum **doğru tahmin** (correct prediction) olarak düşünülebilir.

3.9.6 Prediction Yanlışsa Ne Olur?

Predictor yanlış sonuç üretmişse işlemci yanlış instruction path üzerinden instruction almaya başlamıştır.

Execute aşamasında gerçek branch sonucu hesaplandığında prediction ile gerçek sonuç karşılaştırılır.

Eğer eşleşmiyorsa:

1. yanlış yoldan alınmış instruction'lar geçersiz hale getirilir,
2. pipeline içerisindeki ilgili stage'ler flush edilir,
3. PC doğru branch hedefi veya fall-through adresi ile güncellenir,
4. Fetch doğru adresten tekrar başlar.

Bu işlemin oluşturduğu kayıp cycle sayısına branch misprediction penalty denilebilir.

Branch'in pipeline içerisinde ne kadar geç çözüldüğü misprediction penalty'yi doğrudan etkiler.

3.9.7 Embedded Sistemlerde Neden Basit Predictor Kullanılır?

Yüksek performanslı işlemcilerde çok büyük ve karmaşık branch predictor yapıları kullanılabilir. Bu yapılarda global history, local history, tournament predictor veya birden fazla predictor'ın birlikte kullanıldığı sistemler bulunabilir.

Embedded işlemcilerde ise yalnızca maksimum performans hedeflenmez.

Aşağıdaki parametreler de önemlidir:

- kullanılan FPGA veya silikon alanı,
- güç tüketimi,
- predictor erişim gecikmesi,
- tasarım karmaşıklığı,
- doğrulama maliyeti,
- çalışma frekansı.

Örneğin çok büyük bir predictor prediction accuracy değerini artırabilir. Ancak predictor sonucunun Fetch aşamasındaki kritik yolu uzatması işlemcinin maksimum frekansını düşürebilir.

Bu durumda prediction accuracy artmasına rağmen toplam sistem performansı beklendiği kadar artmayabilir.

Bu sebeple embedded core tasarımında predictor seçilirken yalnızca prediction accuracy değerine bakmak doğru değildir.

Alan, güç, frekans ve misprediction penalty birlikte değerlendirilmelidir.

3.9.8 Branch Prediction Performansı

Branch predictor performansı genel olarak prediction accuracy üzerinden ölçülebilir.

$$\text{Prediction Accuracy} = \frac{\text{True Guess Count}}{\text{Total Guess Count}}$$

Benzer şekilde misprediction rate:

$$\text{Misprediction Rate} = \frac{\text{Wrong Guess Count}}{\text{Total Guess Count}}$$

şeklinde düşünülebilir.

Ancak yalnızca accuracy değerine bakmak yeterli değildir.

Örneğin branch instruction'larının program içerisindeki oranı düşükse predictor accuracy değerindeki küçük bir artış toplam CPI üzerinde çok az değişiklik oluşturabilir. İlgili durum için **Amdahl Yasası**'na bakılabilir.

Branch penalty'nin yaklaşık etkisi aşağıdaki mantıkla düşünülebilir:

$$\text{CPI}_{\text{branch}} \approx \text{Branch Frequency} \times \text{Misprediction Rate} \times \text{Misprediction Penalty}$$

Bu denklem tam bir işlemci performans modeli değildir ancak branch prediction'ın CPI üzerindeki etkisini anlamak için kullanışlıdır.

3.9.9 microBee Açısından Branch Prediction

microBee tasarımında 128-entry bimodal branch predictor kullanılmaktadır. Predictor içerisinde 128 adet 2-bit saturating counter bulunur. Branch instruction'ının Program Counter değerinin belirli bitleri kullanılarak bu tablo içerisindeki bir entry seçilir.

Her entry aşağıdaki dört durumdan birini tutar:

Durum	Tahmin
00	Strongly Not-Taken
01	Weakly Not-Taken
10	Weakly Taken
11	Strongly Taken

Branch sonucu Execute aşamasında kesinleştiğinde ilgili entry güncellenir. Branch taken olmuşsa sayaç artırılır, not-taken olmuşsa azaltılır.

128-entry kullanılması predictor'ın alan maliyetini düşük tutarken geçmiş branch davranışlarından faydalanmasını sağlar. Ancak farklı Program Counter değerleri aynı entry'ye denk gelebilir. Bu durumda iki farklı branch aynı predictor durumunu paylaşır. Bu durum **aliasing** olarak isimlendirilir.

microBee gibi embedded kullanım hedefleyen bir core için predictor boyutu belirlenirken prediction accuracy ile FPGA alan kullanımı, güç tüketimi ve Fetch aşamasındaki kritik yol birlikte değerlendirilmiştir.

3.9.10 Örnek

Aşağıdaki basit loop ele alınabilir:

```
for (int i = 0; i < 100; i++) {
    sum += array[i];
}
```

Compiler bu yapının sonunda koşullu bir branch oluşturabilir.

Loop 100 defa çalışıyorsa branch yaklaşık olarak:

- ilk iterasyonlarda tekrar tekrar taken,
- loop tamamlanırken bir defa not-taken

davranışı gösterebilir.

Always Not-Taken predictor bu loop sırasında çok sayıda yanlış tahmin üretebilir.

2-bit predictor ise birkaç iterasyondan sonra branch'in taken davranışını öğrenerek loop boyunca doğru tahmin yapabilir. Loop çıkışında bir yanlış tahmin oluşsa bile predictor'ın durumu tamamen tersine dönmez.

Bu örnek dinamik branch prediction'ın özellikle loop yoğun kodlarda neden faydalı olduğunu göstermektedir.

3.10 DMEM

3.10.1 Nedir?

DMEM, Data Memory kelimelerinin kısaltmasıdır ve işlemcinin program çalışırken kullandığı verilerin tutulduğu bellek alanını ifade eder.

Harvard mimarisi kullanılan sistemlerde instruction'ların bulunduğu bellek IMEM, verilerin bulunduğu bellek ise DMEM olarak ayrılır.

Instruction Fetch işlemleri IMEM üzerinden yapılır. Load ve store işlemlerinin hedefi ise adrese göre belirlenir: DMEM'e gidebileceği gibi MMIO çevre birimlerine veya AXI penceresine de yönlendirilebilir. DMEM, özellikle program değişkenleri ve yığın gibi genel amaçlı verilerin tutulduğu yerel veri belleğidir.

Örneğin C dilinde:

```
int a = 10;
int b = 20;
int c = a + b;
```

gibi bir kod çalıştırıldığında compiler'ın oluşturduğu programa bağlı olarak değişkenler register file veya data memory üzerinde tutulabilir.

Register sayısının yeterli olmadığı veya verinin bellekte bulunması gerektiği durumlarda DMEM erişimi yapılır.

3.10.2 Load ve Store

RISC-V mimarisinde işlemcinin memory içerisindeki verilere doğrudan ALU instruction'ları ile işlem yapması beklenmez.

Öncelikle veri register file'a alınır.

Bu işlem **load** instruction'ları ile yapılır.

Register içerisindeki bir verinin belleğe yazılması ise **store** instruction'ları ile yapılır.

Örneğin:

```
lw x5, 0(x10)
```

instruction'ı x10 register'ında bulunan adres üzerinden 32-bit veri okur ve sonucu x5 register'ına yazar.

Benzer şekilde:

```
sw x5, 0(x10)
```

instruction'ı x5 içerisindeki 32-bit veriyi x10 tarafından gösterilen memory adresine yazar.

3.10.3 Adres Nasıl Hesaplanır?

Load ve store instruction'larında kullanılacak adres Execute aşamasında hesaplanır.

Genel yapı:

$$Memory\ Address = rs1 + immediate$$

şeklindedir.

Örneğin:

lw x5, 8(x10)

instruction'ında memory adresi:

$$Address = x10 + 8$$

olarak hesaplanır.

Bu adres ALU üzerinden hesaplandıktan sonra Memory stage'e gönderilir.

3.10.4 Veri Boyutları

RV32 içerisinde memory erişimleri yalnızca 32-bit olmak zorunda değildir. Byte, half-word ve word büyüklüklerinde erişimler yapılabilir.

Temel load instruction'ları:

- LB: 8-bit signed load,
- LBU: 8-bit unsigned load,
- LH: 16-bit signed load,
- LHU: 16-bit unsigned load,
- LW: 32-bit load.

Temel store instruction'ları:

- SB: 8-bit store,
- SH: 16-bit store,
- SW: 32-bit store.

Signed load işlemlerinde okunan küçük veri 32-bit register genişliğine **sign extension** ile genişletilir.

Unsigned load işlemlerinde ise kalan üst bitler sıfır ile doldurulur.

3.10.5 Byte Enable

Memory bus'ı 32-bit olsa bile her store işleminde bütün 32-bit'in değiştirilmesi istenmeyebilir.

Örneğin SB instruction'ında yalnızca tek byte yazılır.

Bu sebeple memory arayüzünde hangi byte'ların yazılacağını belirleyen byte enable veya write strobe sinyalleri kullanılabilir.

32-bit bir veri yolu için dört adet byte lane düşünülebilir:

[31:24] [23:16] [15:8] [7:0]

Her lane 8 bit genişliğindedir.

Örneğin yalnızca en düşük byte yazılacaksa:

WSTRB = 4'b0001

kullanılabilir.

32-bit word yazılacaksa:

WSTRB = 4'b1111

olur.

Bu yapı işlemcinin byte ve half-word store instruction'larını 32-bit memory bus üzerinden gerçekleştirmesini sağlar.

3.10.6 Memory Stall

Load ve store işlemlerinin pipeline üzerindeki etkisi microBee içerisinde aynı değildir.

3.10.6.1 Load İşlemleri

Load instruction'ında memory'den okunacak verinin işlemciye geri dönmesi gerekmektedir. Bu veri hazır olmadan instruction'ın Writeback aşamasında register file'a yazılması mümkün değildir.

Bu sebeple load isteği oluşturulduktan sonra memory cevabı hazır değilse pipeline stall durumunda tutulur.

Basit olarak:

Load Request -> Memory Wait -> Load Response -> Writeback

şeklinde düşünülebilir.

Memory latency arttıkça load instruction'ının oluşturduğu stall süresi de artabilir.

3.10.6.2 Store İşlemleri

Store işlemlerinde durum farklıdır. Store instruction'ı register file'a bir sonuç yazmadığı için işlemcinin yazılan veriyi tekrar beklemesi her durumda gerekmez.

microBee çekirdeği içerisinde gerçekleştirilen store işlemlerinde çekirdek tarafında stall uygulanmaz. Store isteği ilgili memory sistemine iletdikten sonra pipeline çalışmasına devam edebilir.

Ancak store işlemi core alanı dışındaki bir hedefe gönderiliyorsa durum değişebilir. Örneğin harici bir bus veya peripheral üzerinden gerçekleştirilen store işlemi karşı taraf isteği hemen kabul edemeyebilir.

Bu durumda dış arayüz işlemi tamamlanana veya isteği kabul edene kadar core stall durumunda tutulabilir.

Bu sebeple microBee içerisinde memory stall davranışı genel olarak şu şekilde özetlenebilir:

- load işlemlerinde memory cevabı beklenirken stall uygulanabilir,
- core içi store işlemlerinde core stall edilmez,
- core dışına çıkan store işlemlerinde dış bus veya hedef hazır değilse stall uygulanabilir.

Bu ayrım özellikle embedded sistemlerde önemlidir. Çünkü memory mapped peripheral veya harici bus erişimleri core içerisindeki yerel memory erişimlerine göre daha uzun latency oluşturabilir.

3.10.7 DMEM ve Pipeline

microBee'nin 5-stage pipeline yapısında load/store işlemlerinin temel akışı şu şekilde düşünülebilir:

IF -> ID -> EX -> MEM -> WB

IF: Instruction memory'den alınır.

ID: Kaynak register'lar okunur.

EX: Memory adresi hesaplanır.

MEM: DMEM erişimi gerçekleştirilir.

WB: Load işlemi ise okunan veri register file'a yazılır.

Store instruction'larında register file writeback yapılması gerekmez. Veri MEM aşamasında memory sistemine gönderilir.

3.10.8 Load-Use Hazard

Load instruction'ları pipeline açısından önemli bir hazard oluşturabilir.

Örneğin:

```
lw  x5, 0(x10)
add x6, x5, x7
```

ilk instruction $x5$ deęerini memory'den okumaktadır.

İkinci instruction ise hemen sonraki cycle'da $x5$ deęerine ihtiya duyar.

Ancak load sonucu henüz hazır olmayabilir.

Bu duruma **load-use hazard** adı verilir.

İşlemci bu durumda gerekli forwarding mümkün deęilse pipeline'a stall ekleyerek ikinci instruction'ın doęru veri hazır olana kadar beklemesini saęlar.

Bu örnek memory latency ile pipeline hazard yapısının birbirinden baęımsız olmadıęını göstermektedir.

3.10.9 Embedded Sistem Aęısından DMEM

Embedded sistemlerde memory mimarisi toplam sistem performansını büyük ölçüde etkiler.

ok hızlı bir işlemci çekirdeęi tasarlanmış olsa bile sürekli yavaş memory erişimi yapıyorsa işlemci zamanının büyük bölümünü stall durumunda geçirebilir.

Bu sebeple yalnızca işlemci frekansına bakarak sistem performansı deęerlendirilmemelidir.

Aşaęıdaki parametreler birlikte incelenmelidir:

- memory latency,
- memory bandwidth,
- load/store instruction oranı,
- pipeline stall sayısı,
- kullanılan bus protokolü,
- memory hiyerarşisi, interconnect ve erişim latency'si.

FPGA üzerinde işlemciye yakın kullanılan Block RAM genellikle düşük latency saęlarken harici DDR veya bus üzerinden ulaşılan bir peripheral daha uzun cevap süresine sahip olabilir.

Bu yüzden embedded sistem tasarımında memory sistemi işlemci mimarisinin ayrılmaz bir parçası olarak deęerlendirilmelidir.

Bölüm 4

Instruction Set

4.1 ISA Nedir?

4.1.1 Tanım

Instruction set, işlemcinin anlayabildiği ve çalıştırabildiği komutların tamamına verilen isimdir. Assembler, compiler veya toolchain tarafından üretilen instruction'lar işlemci tarafından decode edilir ve ilgili donanım birimlerine yönlendirilir.

Bir instruction set içerisinde yalnızca hangi işlemlerin yapılabileceği değil, bu işlemlerin bit seviyesinde nasıl encode edildiği de tanımlanır.

Örneğin bir toplama instruction'ı için:

```
add x1, x2, x3
```

şeklinde bir komut yazılabilir.

Ancak işlemci bu komutu yazı olarak görmez. Compiler ve assembler tarafından oluşturulan binary encoding işlemciye ulaşır. İşlemci opcode, funct ve register alanlarını decode ederek bunun bir toplama işlemi olduğunu anlar.

Bu sebeple Instruction Set Architecture veya kısaca ISA, yazılım ile işlemci donanımı arasında bulunan temel sözleşme olarak düşünülebilir.

Yazılım ISA'ya uygun instruction üretir; donanım ise aynı ISA'da tanımlanan davranışı uygulamak zorundadır.

4.1.2 Kim Belirler?

RISC-V mimarisinde instruction setleri RISC-V International tarafından belirlenen spesifikasyonlar üzerinden tanımlanır [1].

RISC-V tek parça bir instruction set yerine temel ISA ve extension'lardan oluşan modüler bir yapı kullanır.

Örneğin:

- **I**: Temel integer instruction seti,
- **M**: Çarpma ve bölme işlemleri,
- **C**: Compressed instruction'lar,
- **Zicsr**: CSR erişim instruction'ları,
- **Zifencei**: Instruction fetch senkronizasyonu

gibi farklı instruction grupları bulunmaktadır.

Bir işlemci tasarımcısı ihtiyacı olan extension'ları seçerek kendi işlemcisinin ISA profilini oluşturabilir.

Örneğin microBee:

```
RV32IMC + Zicsr + Zifencei
```

instruction setini desteklemektedir.

Burada:

- RV32: XLEN'in 32 bit olduğunu (register ve integer işlem genişliği),
- I: Temel integer instruction'larını,
- M: Multiply ve divide işlemlerini,
- C: 16-bit compressed instruction'ları

ifade eder.

Bunun yanında RISC-V mimarisi üreticiye özel instruction geliştirmek için custom encoding alanları da bırakmaktadır.

Bu sayede standart ISA korunurken belirli uygulamalara özel accelerator veya instruction'lar sisteme eklenebilir.

4.1.3 Ne İşe Yarar?

Instruction set'in belirlenmesinin temel amacı yazılım ile donanım arasında standart bir arayüz oluşturmaktır. Eğer ISA tanımlı değilse compiler'ın hangi binary kodu üretmesi gerektiği veya işlemcinin gelen bitleri nasıl yorumlaması gerektiği bilinemez.

ISA şu soruların cevabını verir:

- Hangi instruction'lar bulunmaktadır?
- Instruction'ların bit encoding'i nasıldır?
- Kaç adet register vardır?
- Operandlar nasıl ifade edilir?
- Memory erişimi nasıl yapılır?
- Branch ve jump işlemleri nasıl çalışır?
- Exception ve CSR erişimleri nasıl gerçekleştirilir?

Bu yapı sayesinde aynı ISA'yı kullanan farklı işlemciler aynı programları çalıştırabilir.

Örneğin iki farklı üretici birbirinden tamamen farklı pipeline mimarileri tasarlayabilir.

Bir işlemci 3-stage pipeline kullanırken başka bir işlemci 5-stage pipeline, branch predictor veya farklı ALU yapıları kullanabilir.

Ancak her iki işlemci de aynı RISC-V ISA'yı doğru şekilde destekliyorsa yazılım açısından aynı instruction'ları çalıştırabilir.

Bu noktada ISA ile mikro-mimari birbirinden ayrılmalıdır.

ISA işlemcinin ne yapacağını tanımlar; mikro-mimari ise bunu nasıl yapacağını belirler.

Örneğin MUL instruction'ı ISA içerisinde tanımlıdır. Ancak bu instruction'ın tek cycle'da, iki cycle pipelined bir yapıda veya iterative bir çarpıcı ile gerçekleştirilmesi mikro-mimari tasarım kararına bağlıdır.

4.1.4 Neden Sabit ISA?

ISA'nın mümkün olduğunca sabit ve standart olması özellikle yazılım ekosistemi açısından önemlidir.

İşlemci tasarımcısı instruction encoding'lerini sürekli değiştirebilseydi, compiler ve assembler'ın her işlemci için farklı şekilde kod üretmesi gerekirdi.

Örneğin bir işlemcide:

ADD opcode = A

başka bir işlemcide:

ADD opcode = B

olsaydı aynı binary iki işlemcide çalışmazdı.

Bu durumda software compatibility büyük ölçüde ortadan kalkardı.

Sabit bir ISA sayesinde compiler belirli bir hedef mimari için kod üretir.

Örneğin microBee için compiler'a yaklaşık olarak:

-march=rv32imc_zicsr_zifencei

şeklinde hedef mimari bilgisi verilebilir.

Compiler bu bilgiyi kullanarak yalnızca işlemcinin desteklediği instruction'ları üretir.

ISA'nın standart olması compiler optimizasyonları açısından da önemlidir.

Compiler bir instruction'ın davranışını kesin olarak bildiği için:

- instruction scheduling,
- register allocation,
- constant propagation,
- loop optimization,
- instruction selection

gibi işlemleri güvenli bir şekilde gerçekleştirebilir.

Instruction davranışlarının veya encoding'lerinin tasarımdan tasarıma değişmesi compiler'ın bu optimizasyonları uygulamasını zorlaştırır.

Aynı problem işletim sistemi, debugger, assembler, linker ve diğer tool'lar için de geçerlidir.

Bu sebeple standart ISA'nın en büyük avantajlarından biri yalnızca işlemci tasarımını kolaylaştırması değil, büyük bir yazılım ekosisteminin aynı mimari üzerinde çalışabilmesini sağlamasıdır.

Ancak tamamen sabit bir ISA her uygulama için yeterli olmayabilir.

Örneğin görüntü işleme, yapay zeka veya kriptografi gibi belirli alanlarda standart instruction'lara göre daha hızlı çalışan özel instruction'lar geliştirilmek istenebilir.

RISC-V bu problemi custom instruction alanları ile çözmektedir.

Tasarıma özel instruction'lar custom opcode alanlarında tanımlanabilir. Böylece standart RISC-V instruction encoding'leri değiştirilmez ve standart encoding'lere dayanan mevcut compiler/toolchain uyumu bozulmaz. Custom instruction kullanan yazılım için ise toolchain desteği ayrıca gerekir.

Eğer compiler'ın bu yeni instruction'ları otomatik kullanması isteniyorsa compiler backend'ine ilgili instruction desteğinin ayrıca eklenmesi gerekir.

Aksi durumda custom instruction'lar assembly veya intrinsic gibi yöntemlerle manuel olarak kullanılabilir.

Bu sebeple yeni bir instruction eklemek yalnızca RTL tasarımı değildir. Instruction'ın yazılım tarafından verimli şekilde kullanılabilmesi için compiler ve toolchain desteği de tasarımın önemli bir parçasıdır.

4.2 I Seti

I seti RV32I mimarisinin temel integer instruction setidir [2]. microBee 32-bit bir işlemci olduğu için register genişliği (XLEN) 32 bittir. I setindeki instruction'lar 32-bit uzunluğundadır.

Instruction encoding içerisinde kullanılan temel alanlar şunlardır:

- **opcode**: Instruction'ın temel sınıfını belirler.
- **rd**: Sonucun yazılacağı destination register'dır.
- **rs1**: Birinci source register'dır.
- **rs2**: İkinci source register'dır.
- **funct3**: Aynı opcode içerisindeki işlemleri ayırır.
- **funct7**: Özellikle R-type işlemlerinde ek ayırım sağlar.
- **imm**: Instruction içerisinde taşınan sabit değerdir.

RV32I içerisinde R, I, S, B, U ve J olmak üzere farklı instruction formatları kullanılır.

4.2.1 R-Type Formatı

R-Type instruction'lar iki register üzerinde işlem yaparak sonucu başka bir register'a yazar.

31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	opcode

Genel işlem:

$$rd \leftarrow operation(rs1, rs2)$$

4.2.1.1 ADD

İki register'ın değerini toplar.

add rd, rs1, rs2

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	000	rd	0110011

$$rd = rs1 + rs2$$

4.2.1.2 SUB

sub rd, rs1, rs2

31:25	24:20	19:15	14:12	11:7	6:0
0100000	rs2	rs1	000	rd	0110011

$$rd = rs1 - rs2$$

4.2.1.3 SLL

Logical left shift yapar. Shift miktarı **rs2[4:0]** ile belirlenir.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	001	rd	0110011

$$rd = rs1 \ll rs2[4:0]$$

4.2.1.4 SLT

Signed karşılaştırma yapar.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	010	rd	0110011

$$rd = \begin{cases} 1, & \text{signed}(rs1) < \text{signed}(rs2) \\ 0, & \text{otherwise} \end{cases}$$

4.2.1.5 SLTU

Unsigned karşılaştırma yapar.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	011	rd	0110011

$$rd = \begin{cases} 1, & \text{unsigned}(rs1) < \text{unsigned}(rs2) \\ 0, & \text{otherwise} \end{cases}$$

4.2.1.6 XOR

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	100	rd	0110011

$$rd = rs1 \oplus rs2$$

4.2.1.7 SRL

Logical right shift yapar ve üst bitleri sıfır ile doldurur. Shift miktarı **rs2[4:0]** ile belirlenir.

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	101	rd	0110011

$$rd = rs1 \gg rs2[4 : 0]$$

4.2.1.8 SRA

Arithmetic right shift yapar. İşaret biti korunur. Shift miktarı **rs2[4:0]** ile belirlenir.

31:25	24:20	19:15	14:12	11:7	6:0
0100000	rs2	rs1	101	rd	0110011

$$rd = signed(rs1) \gg rs2[4 : 0]$$

(İşaret biti korunarak sağa kaydırma.)

4.2.1.9 OR

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	110	rd	0110011

$$rd = rs1 | rs2$$

4.2.1.10 AND

31:25	24:20	19:15	14:12	11:7	6:0
0000000	rs2	rs1	111	rd	0110011

$$rd = rs1 \& rs2$$

4.2.2 I-Type Arithmetic Instruction'ları

Genel I-Type formatı:

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	funct3	rd	opcode

Immediate 12 bittir ve arithmetic işlemlerde 32 bite sign-extend edilir.

4.2.2.1 ADDI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	000	rd	0010011

$$rd = rs1 + signextend(imm)$$

4.2.2.2 SLTI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	010	rd	0010011

Signed karşılaştırma yapar.

4.2.2.3 SLTIU

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	011	rd	0010011

Unsigned karşılaştırma yapar. Immediate önce 32 bite sign-extend edilir, daha sonra karşılaştırma unsigned olarak gerçekleştirilir.

4.2.2.4 XORI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	100	rd	0010011

4.2.2.5 ORI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	110	rd	0010011

4.2.2.6 ANDI

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	111	rd	0010011

4.2.3 Shift Immediate Instruction'ları

4.2.3.1 SLLI

31:25	24:20	19:15	14:12	11:7	6:0
0000000	shamt[4:0]	rs1	001	rd	0010011

4.2.3.2 SRLI

31:25	24:20	19:15	14:12	11:7	6:0
0000000	shamt[4:0]	rs1	101	rd	0010011

4.2.3.3 SRAI

31:25	24:20	19:15	14:12	11:7	6:0
0100000	shamt[4:0]	rs1	101	rd	0010011

4.2.4 Load Instruction'ları

Load instruction'ları I-Type formatını kullanır.

Effective address:

$$\text{Address} = rs1 + \text{signextend}(imm[11 : 0])$$

4.2.4.1 LB

8-bit signed veri okur ve 32 bite sign-extend eder.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	000	rd	0000011

4.2.4.2 LH

16-bit signed veri okur ve 32 bite sign-extend eder.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	001	rd	0000011

4.2.4.3 LW

32-bit word okur.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	010	rd	0000011

4.2.4.4 LBU

8-bit unsigned veri okur ve zero-extension uygular.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	100	rd	0000011

4.2.4.5 LHU

16-bit unsigned veri okur ve zero-extension ile 32 bite genişletir.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	101	rd	0000011

4.2.5 Store Instruction'ları

Store instruction'ları S-Type formatını kullanır.

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode

4.2.5.1 SB

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011

rs2[7:0] memory'ye yazılır.

4.2.5.2 SH

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011

rs2[15:0] memory'ye yazılır.

4.2.5.3 SW

31:25	24:20	19:15	14:12	11:7	6:0
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011

32-bit rs2 değeri memory'ye yazılır.

4.2.6 Branch Instruction'ları

Branch instruction'ları B-Type formatını kullanır.

31	30:25	24:20	19:15	14:12	11:8	7	6:0
imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	1100011

Branch target:

$$\text{Target} = \text{PC} + \text{signextend}(\{\text{imm}[12 : 1], 1'b0\})$$

4.2.6.1 BEQ

$rs1 == rs2$ ise branch alınır.

31	30:25	24:20	19:15	14:12	11:8	7	6:0
imm[12]	imm[10:5]	rs2	rs1	000	imm[4:1]	imm[11]	1100011

4.2.6.2 BNE

$rs1 != rs2$ ise branch alınır.

funct3 = 001

4.2.6.3 BLT

Signed $rs1 < rs2$.

funct3 = 100

4.2.6.4 BGE

Signed $rs1 >= rs2$.

funct3 = 101

4.2.6.5 BLTU

Unsigned $rs1 < rs2$.

funct3 = 110

4.2.6.6 BGEU

Unsigned $rs1 >= rs2$.

funct3 = 111

Bütün branch instruction'larında diğer bit alanları BEQ ile aynıdır ve yalnızca funct3 alanı değişir.

4.2.7 Jump Instruction'ları

4.2.7.1 JAL

J-Type formatını kullanır.

31	30:21	20	19:12	11:7	6:0
imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	1101111

$$rd = PC + 4$$

$$PC = PC + \text{signextend}(\{imm[20 : 1], 1'b0\})$$

JAL instruction'ı 32-bit olduğundan link değeri her zaman PC+4'tür.

4.2.7.2 JALR

I-Type formatını kullanır.

31:20	19:15	14:12	11:7	6:0
imm[11:0]	rs1	000	rd	1100111

$$rd = PC + 4$$

$$PC = (rs1 + \text{signextend}(imm)) \& \sim 1$$

Hedef adresin en düşük biti sıfırlanır.

4.2.8 Upper Immediate Instruction'ları

4.2.8.1 LUI

U-Type formatındadır.

31:12	11:7	6:0
imm[31:12]	rd	0110111

$$rd = imm[31 : 12] \ll 12$$

4.2.8.2 AUIPC

31:12	11:7	6:0
imm[31:12]	rd	0010111

$$rd = PC + (imm[31 : 12] \ll 12)$$

4.2.9 FENCE

Memory işlemlerinin gözlemlenme sırasını düzenlemek için kullanılır.

31:28	27:24	23:20	19:15	14:12	11:7	6:0
fm	pred	succ	00000	000	00000	0001111

Standart bir FENCE instruction'ında fm=0000 kullanılır.

pred önceki memory işlemlerinin, succ ise sonraki memory işlemlerinin sınıflarını belirtir.

4.2.10 Environment Instruction'ları

4.2.10.1 ECALL

Execution environment'a servis isteği oluşturur.

31:20	19:15	14:12	11:7	6:0
000000000000	00000	000	00000	1110011

32-bit encoding:

0x00000073

4.2.10.2 EBREAK

Debugger veya execution environment'a breakpoint trap oluşturmak için kullanılır.

31:20	19:15	14:12	11:7	6:0
000000000001	00000	000	00000	1110011

32-bit encoding:

0x00100073

4.3 M Seti

M extension integer çarpma, bölme ve kalan işlemlerini tanımlar [3].

Bütün RV32M instruction'ları R-Type formatını kullanır ve:

funct7 = 0000001

değerine sahiptir.

4.3.1 MUL

32×32-bit çarpımın düşük 32 bitini döndürür.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	000	rd	0110011

$$rd = (rs1 \times rs2)[31 : 0]$$

4.3.2 MULH

Signed × signed çarpımın yüksek 32 bitini verir.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	001	rd	0110011

4.3.3 MULHSU

rs1 signed, rs2 unsigned kabul edilir.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	010	rd	0110011

4.3.4 MULHU

Unsigned × unsigned çarpımın yüksek 32 bitini verir.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	011	rd	0110011

4.3.5 DIV

Signed integer division yapar.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	100	rd	0110011

$$rd = signed(rs1)/signed(rs2)$$

Özel durumlar:

- Sıfıra bölme: sonuç 0xFFFFFFFF olur.
- Signed overflow (INT_MIN / -1): sonuç rs1 (0x80000000) olur.

4.3.6 DIVU

Unsigned division yapar.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	101	rd	0110011

Sıfıra bölmede sonuç 0xFFFFFFFF olur.

4.3.7 REM

Signed division sonucunun kalanını verir.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	110	rd	0110011

Özel durumlar:

- Sıfıra bölme: kalan **rs1** olur.
- Signed overflow ($\text{INT_MIN} \% -1$): kalan 0 olur.

4.3.8 REMU

Unsigned division sonucunun kalanını verir.

31:25	24:20	19:15	14:12	11:7	6:0
0000001	rs2	rs1	111	rd	0110011

Sıfıra bölmede kalan **rs1** olur.

4.4 C Seti

C extension sık kullanılan RISC-V instruction'larının 16-bit encoding ile ifade edilmesini sağlar [4]. Temel amacı program boyutunu ve instruction fetch bandwidth ihtiyacını azaltmaktır.

microBee RV32C kullandığı için 16-bit ve 32-bit instruction'lar aynı program içerisinde birlikte bulunabilir.

16-bit instruction'ın en düşük iki biti instruction quadrant'ını belirler:

- 00: Quadrant 0
- 01: Quadrant 1
- 10: Quadrant 2
- 11: 32-bit instruction (microBee/RV32IMC kapsamında)

Bazı compressed instruction'larda yalnızca 8 register kullanılabilir:

Compressed adres	Gerçek register
000	x8
001	x9
010	x10
011	x11
100	x12
101	x13
110	x14
111	x15

Bu register'lar **rd**, **rs1** ve **rs2** şeklinde gösterilir.

4.4.1 C.ADDI4SPN

Stack pointer üzerine pozitif immediate ekleyerek **x8-x15** register'larından birine yazar.

15:13	12:11	10:7	6	5	4:2	1:0
000	nzuimm[5:4]	nzuimm[9:6]	nzuimm[2]	nzuimm[3]	rd'	00

$$rd' = x2 + nzuimm$$

Immediate sıfır olamaz.

4.4.2 C.LW

15:13	12:10	9:7	6	5	4:2	1:0
010	uimm[5:3]	rs1'	uimm[2]	uimm[6]	rd'	00

$$rd' = MEM[rs1' + uimm]$$

4.4.3 C.SW

15:13	12:10	9:7	6	5	4:2	1:0
110	uimm[5:3]	rs1'	uimm[2]	uimm[6]	rs2'	00

4.4.4 C.NOP

C.NOP, compressed ADDI **x0,x0,0** instruction'ıdır.

15:13	12	11:7	6:2	1:0
000	0	00000	00000	01

Encoding:

0x0001

4.4.5 C.ADDI

15:13	12	11:7	6:2	1:0
000	imm[5]	rd/rs1	imm[4:0]	01

$$rd = rd + \text{signextend}(imm)$$

4.4.6 C.JAL

RV32C içerisinde kısa mesafeli subroutine çağrısı için kullanılır.

15:13	12	11	10:9	8	7	6	5:3	2	1:0
001	imm[11]	imm[4]	imm[9:8]	imm[10]	imm[6]	imm[7]	imm[3:1]	imm[5]	01

$$x1 = PC + 2$$

$$PC = PC + \text{offset}$$

4.4.7 C.LI

15:13	12	11:7	6:2	1:0
010	imm[5]	rd	imm[4:0]	01

$$rd = \text{signextend}(imm)$$

4.4.8 C.ADDI16SP

Stack pointer'ı 16'nın katları şeklindeki immediate ile değiştirir.

15:13	12	11:7	6	5	4:3	2	1:0
011	nzimm[9]	00010	nzimm[4]	nzimm[6]	nzimm[8:7]	nzimm[5]	01

$$x2 = x2 + \text{signextend}(nzimm)$$

4.4.9 C.LUI

15:13	12	11:7	6:2	1:0
011	nzimm[17]	rd	nzimm[16:12]	01

rd değeri x0 veya x2 olamaz.

4.4.10 C.SRLI

15:13	12	11:10	9:7	6:2	1:0
100	0	00	rd'/rs1'	shamt[4:0]	01

RV32C içerisinde shamt[5]=0'dır.

4.4.11 C.SRAI

15:13	12	11:10	9:7	6:2	1:0
100	0	01	rd'/rs1'	shamt[4:0]	01

4.4.12 C.ANDI

15:13	12	11:10	9:7	6:2	1:0
100	imm[5]	10	rd'/rs1'	imm[4:0]	01

4.4.13 C.SUB

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	00	rs2'	01

4.4.14 C.XOR

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	01	rs2'	01

4.4.15 C.OR

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	10	rs2'	01

4.4.16 C.AND

15:13	12	11:10	9:7	6:5	4:2	1:0
100	0	11	rd'/rs1'	11	rs2'	01

4.4.17 C.J

15:13	12	11	10:9	8	7	6	5:3	2	1:0
101	imm[11]	imm[4]	imm[9:8]	imm[10]	imm[6]	imm[7]	imm[3:1]	imm[5]	01

$$PC = PC + \text{offset}$$

4.4.18 C.BEQZ

15:13	12	11:10	9:7	6:5	4:3	2	1:0
110	imm[8]	imm[4:3]	rs1'	imm[7:6]	imm[2:1]	imm[5]	01

rs1' == 0 ise branch alınır.

4.4.19 C.BNEZ

15:13	12	11:10	9:7	6:5	4:3	2	1:0
111	imm[8]	imm[4:3]	rs1'	imm[7:6]	imm[2:1]	imm[5]	01

rs1' != 0 ise branch alınır.

4.4.20 C.SLLI

15:13	12	11:7	6:2	1:0
000	0	rd/rs1	shamt[4:0]	10

4.4.21 C.LWSP

Stack pointer tabanlı word load yapar.

15:13	12	11:7	6:4	3:2	1:0
010	uimm[5]	rd	uimm[4:2]	uimm[7:6]	10

$$rd = MEM[x2 + uimm]$$

rd=x0 encoding'i reserved'tır.

4.4.22 C.JR

15:12	11:7	6:2	1:0
1000	rs1	00000	10

rs1 sıfır olamaz.

$$PC = rs1 \& \sim 1$$

Hedef adresin en düşük biti JALR semantiğine uygun olarak temizlenir.

4.4.23 C.MV

15:12	11:7	6:2	1:0
1000	rd	rs2	10

$$rd = rs2$$

rd ve rs2 sıfır olmamalıdır.

4.4.24 C.EBREAK

15:12	11:7	6:2	1:0
1001	00000	00000	10

Encoding:

$$0x9002$$

4.4.25 C.JALR

15:12	11:7	6:2	1:0
1001	rs1	00000	10

$$x1 = PC + 2$$

$$PC = rs1 \& \sim 1$$

Hedef adresin en düşük biti JALR semantiğine uygun olarak temizlenir.

4.4.26 C.ADD

15:12	11:7	6:2	1:0
1001	rd/rs1	rs2	10

$$rd = rd + rs2$$

4.4.27 C.SWSP

Stack pointer tabanlı word store yapar.

15:13	12:9	8:7	6:2	1:0
110	uimm[5:2]	uimm[7:6]	rs2	10

$$MEM[x2 + uimm] = rs2$$

4.5 NOP

NOP, No Operation kelimelerinin kısaltmasıdır. Instruction işlemci tarafından çalıştırılır ancak programın mimari durumunda herhangi bir değişiklik oluşturmaz.

RISC-V içerisinde NOP için ayrı bir temel opcode ayrılmamıştır. Standart 32-bit NOP instruction'ı:

$$\text{ADDI } x0, x0, 0$$

olarak encode edilir.

Bit karşılığı:

31:20	19:15	14:12	11:7	6:0
000000000000	00000	000	00000	0010011

Hex karşılığı:

0x00000013

x0 register'ına yapılan yazmalar yok sayıldığı için mimari durumda değişiklik oluşmaz.

C extension içerisinde ayrıca 16-bit C.NOP bulunur:

0x0001

NOP instruction'ları pipeline kontrolü, instruction hizalama, debug ve bazı compiler işlemlerinde kullanılabilir.

4.6 Zicsr

Zicsr extension, Control and Status Register (CSR) alanına erişmek için kullanılan instruction'ları tanımlar.

CSR adres alanı 12 bittir. Bu nedenle:

$$2^{12} = 4096$$

olası CSR encoding'i vardır. Bunların tamamı implemented CSR değildir; işlemci yalnızca spesifikasyonda tanımlı ve tasarımı bulunan CSR'ları gerçekleştirir.

Bütün CSR instruction'larında:

opcode = 1110011

kullanılır.

Temel encoding:

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1/uimm	funct3	rd	1110011

4.6.1 CSRRW

CSR Read and Write.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1	001	rd	1110011

Temel davranış:

$$old = CSR$$

$$CSR = rs1$$

$$rd = old$$

rd=x0 olduğunda CSR değeri okunsa bile sonucun register file'a yazılması gerekmez.

4.6.2 CSRRS

CSR Read and Set.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1	010	rd	1110011

$$old = CSR$$

$$CSR = old \mid rs1$$

$$rd = old$$

Eğer $rs1=x0$ ise CSR'a yazma yapılmaz ve instruction yalnızca okuma işlemi olarak kullanılabilir.

4.6.3 CSRRC

CSR Read and Clear.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	rs1	011	rd	1110011

$$old = CSR$$

$$CSR = old \& \sim rs1$$

$$rd = old$$

$rs1$ içerisinde 1 olan bitler CSR içerisinde temizlenir.

4.6.4 CSRRWI

CSRRW instruction'ının immediate versiyonudur.

31:20	19:15	14:12	11:7	6:0
csr[11:0]	uimm[4:0]	101	rd	1110011

$uimm[4:0]$ zero-extend edilerek CSR'a yazılır.

$$CSR = zeroextend(uimm)$$

4.6.5 CSRRSI

31:20	19:15	14:12	11:7	6:0
csr[11:0]	uimm[4:0]	110	rd	1110011

$$CSR = CSR \mid zeroextend(uimm)$$

$uimm=0$ olduğunda CSR'a yazma işlemi yapılmaz.

4.6.6 CSRRCI

31:20	19:15	14:12	11:7	6:0
csr[11:0]	uimm[4:0]	111	rd	1110011

$$CSR = CSR \& \sim zeroextend(uimm)$$

$uimm=0$ olduğunda CSR'a yazma yapılmaz.

4.7 Zifencei

Zifencei extension **FENCE.I** instruction'ını tanımlar.

İşlemci instruction ve data erişimlerini farklı yollar üzerinden gerçekleştirebilir. Genel ISA semantiğinde bir program kendi instruction memory alanını data-store'larla değiştirdiğinde, daha önce Fetch tarafında bulunan instruction bilgisinin artık geçersiz olması mümkündür.

FENCE.I, daha önce gerçekleştirilen instruction memory güncellemelerinin daha sonra gerçekleştirilecek instruction fetch işlemleri tarafından görülebilmesini sağlar.

microBee davranışı

Zifencei opcode desteği mevcuttur. **FENCE.I** instruction'ı geçerli kabul edilir ancak mevcut cache'siz mikro-mimaride no-op olarak uygulanır.

microBee memory map'inde IMEM'e normal data-store erişimi bulunmaz. Ayrıca instruction cache invalidate veya instruction-fetch tutarlılığı için özel bir donanım mekanizması da yoktur. Bu nedenle **FENCE.I** çalıştırılması mevcut tasarımda pipeline veya fetch tarafında ek bir operasyon oluşturmaz.

4.7.1 FENCE.I

Encoding:

31:20	19:15	14:12	11:7	6:0
000000000000	00000	001	00000	0001111

Hex karşılığı:

0x0000100F

Genel ISA semantiğinde instruction'ın amacı:

Önceki instruction memory güncellemeleri

⇓

FENCE.I

⇓

Yeni instruction fetch'leri

şeklinde düşünülebilir.

Genel bir implementasyonda **FENCE.I** çalıştırıldığında instruction fetch tarafındaki geçici durumlar veya instruction cache temizlenebilir ve pipeline yeniden fetch işlemine başlayabilir. Bu davranış microBee'nin mevcut no-op uygulamasından farklıdır; yukarıdaki microBee kutusu referans alınmalıdır.

Bu instruction özellikle self-modifying code, bootloader, program loader ve JIT compiler gibi instruction memory üzerinde çalışma sırasında değişiklik yapılan sistemlerde önemlidir.

Bölüm 5

Interrupts

5.1 Nedir?

Interrupt (kesme), işlemcinin normal program akışını geçici olarak bırakıp önceden tanımlanmış bir handler'a geçmesini sağlayan olaydır. Kaynaklar genellikle normal instruction akışından bağımsız kabul edilir; timer veya harici hat gibi asenkron kaynaklar olabileceği gibi software interrupt gibi yazılım tarafından senkron tetiklenen kaynaklar da olabilir.

Interrupt ile exception birbirine benzer şekilde trap mekanizmasını kullanır ancak kaynakları farklıdır.

- **Exception:** Çalıştırılan instruction veya işlem sonucunda oluşur. Illegal instruction veya erişim hatası örnek verilebilir.
- **Interrupt:** İşlemcinin normal instruction akışından bağımsız olarak kabul edilen bir olaydır (timer, software veya harici kaynak).

microBee yalnızca Machine Mode içerisinde çalışır [5]. Interrupt kabul edildiğinde işlemci normal program akışından çıkar ve `mtvec` register'ında bulunan trap handler adresine gider.

Interrupt sırasında temel olarak:

- dönüş adresi `mepc`'ye,
- interrupt nedeni `mcause`'a,
- `mtval` sıfırlanır (interrupt'ta ek fault bilgisi yoktur),
- `mstatus` içerisinde `MIE`→`MPIE`, `MIE`=0 ve `MPP`=11 güncellemeleri

yapılır. Bu güncellemelerin ayrıntısı ilerideki `mstatus` bölümünde anlatılmıştır.

Handler tamamlandığında `mret` instruction'ı kullanılarak normal program akışına geri dönülür.

5.2 Ne İşe Yarar?

Interrupt mekanizmasının temel amacı işlemcinin bir olay gerçekleşip gerçekleşmediğini sürekli kontrol etmek zorunda kalmasını önlemektir.

Interrupt kullanılmayan bir sistemde işlemci örneğin UART üzerinden veri gelip gelmediğini sürekli kontrol edebilir:

```
while (1) {
    if (uart_data_ready()) {
        read_uart();
    }
}
```

Bu yapıya **polling** adı verilir.

Polling basit bir yöntemdir ancak işlemci zamanının önemli bir kısmını herhangi bir olay gerçekleşmemiş olsa bile çevre birimini kontrol ederek harcar.

Interrupt kullanıldığında ise işlemci normal işini yapmaya devam eder. Olay gerçekleştiğinde ilgili donanım interrupt üretir ve işlemci handler'a geçer.

Bu sayede:

- çevre birimleri sürekli kontrol edilmek zorunda kalmaz,
- işlemci zamanı daha verimli kullanılabilir,
- gerçek zamanlı olaylara daha hızlı tepki verilebilir,
- düşük güç sistemlerinde olay güdümlü çalışma uygulanabilir.

5.3 microBee’de Interrupt Kaynakları

microBee core üç temel interrupt türünü destekler:

Tablo 5.1: microBee interrupt kaynakları ve `mcause` değerleri.

Interrupt	Kaynak	mcause
Software Interrupt	<code>irq_software_i</code>	0x80000003
Timer Interrupt	<code>irq_timer_i</code>	0x80000007
External Interrupt	<code>irq_external_i</code>	0x8000000B

Software ve timer interrupt’ları CLINT tarafından üretilir.

External interrupt ise PLIC tarafından oluşturulur.

5.4 Interrupt Kabul Koşulu

Bir interrupt kaynağının yalnızca pending olması interrupt’ın alınması için yeterli değildir.

Interrupt’ın kabul edilmesi için üç temel şart birlikte sağlanmalıdır:

- ilgili interrupt `mip` içerisinde pending olmalıdır,
- ilgili enable biti `mie` içerisinde aktif olmalıdır,
- global interrupt enable biti `mstatus.MIE` aktif olmalıdır.

Örneğin external interrupt için:

```
mip.MEIP = 1
mie.MEIE = 1
mstatus.MIE = 1
```

olması gerekir.

Bu şartlar sağlandığında CSR tarafında interrupt request oluşturulur.

Ancak microBee interrupt’ı doğrudan aynı cycle içerisinde trap olarak kabul etmez.

Not

mip: Machine Interrupt Pending register’ıdır. Hangi interrupt kaynaklarının pending durumda olduğunu gösterir.

mie: Machine Interrupt Enable register’ıdır. Hangi interrupt türlerinin core tarafından kabul edilebileceğini belirler.

mstatus: Machine Mode içerisindeki genel işlemci durumunu tutan CSR’lardan biridir. Interrupt açısından en önemli biti MIE’dir.

MEIP: Machine External Interrupt Pending bitidir ve `mip` register’ı içerisinde bulunur.

MEIE: Machine External Interrupt Enable bitidir ve `mie` register’ı içerisinde bulunur.

5.4.1 microBee’de mip Davranışı

microBee içerisinde yazılımdan okunan `mip` değeri yalnızca CSR içerisinde tutulan yazılabilir pending bitlerinden oluşmaz.

Okunan değer genel olarak:

```
mip_read = mip_shadow | irq_level
```

şeklinde düşünülebilir.

Yani `irq_software_i`, `irq_timer_i` veya `irq_external_i` hattı aktif olduğu sürece ilgili pending biti `mip` içerisinde 1 olarak görünmeye devam eder.

Bu sebeple örneğin external interrupt kaynağı halen aktifken `mip` CSR'ına 0 yazılması interrupt'ı temizlemek için yeterli değildir.

Interrupt'ın asıl kaynağının CLINT, PLIC veya ilgili peripheral tarafında temizlenmesi gerekir.

5.5 Interrupt Önceliği

Birden fazla interrupt aynı anda pending ve enable durumda olabilir.

microBee içerisinde öncelik sırası:

External > Software > Timer

şeklindedir.

Yani aynı anda external ve timer interrupt pending ise external interrupt önce alınır.

Aynı anda software ve timer interrupt pending ise software interrupt önce işlenir.

Synchronous exception ile interrupt aynı anda oluşursa synchronous exception daha yüksek önceliğe sahiptir.

Exception trap'inde `mstatus.MIE` sıfırlanır. Interrupt kaynağı pending durumda kaldığı sürece bilgi CLINT, PLIC veya `mip` tarafında korunur. `mret` sonrası veya handler içerisinde `MIE` tekrar açıldığında interrupt uygun zamanda alınabilir.

5.6 Precise Interrupt ve Pipeline Drain

microBee içerisinde interrupt mekanizmasının önemli özelliklerinden biri **precise interrupt** davranışdır.

Interrupt request olduğu anda pipeline içerisindeki instruction'lar doğrudan silinmez.

Bunun yerine işlemci:

1. yeni IMEM isteği oluşturmayı durdurur,
2. pipeline içerisinde daha önce alınmış instruction'ların tamamlanmasını bekler,
3. pipeline tamamen boşaldığında interrupt trap'ini kabul eder.

RTL seviyesinde temel koşul şu şekilde düşünülebilir:

```
interrupt_trap_valid = csr_interrupt_request & pipeline_empty
```

Bu yöntem sayesinde interrupt geldiğinde daha önce execution'a başlamış instruction'ların yarıda bırakılması engellenir.

Pipeline boşaldığında artık mimari durum belirli ve tutarlı bir noktadadır.

Trap kabulündeki `pipeline_empty` koşulu.

`pipeline_empty` yalnızca klasik pipeline stage'lerinin boş olmasını ifade eden basit bir sinyal değildir. microBee RTL içerisinde fetch, memory ve sunulmuş fakat henüz tamamlanmamış işlem durumları da bu koşulun parçasıdır. Bu sebeple interrupt ancak çekirdek gerçekten güvenli bir mimari sınıra ulaştığında kabul edilir.

5.7 Interrupt Sırasında mepc

Interrupt ile exception arasındaki önemli farklardan biri `mepc` değerinin nasıl oluşturulduğudur.

Interrupt durumunda pipeline tamamen drain edildiği için aktif bir faulting instruction bulunmaz.

Bu durumda:

$$\text{mepc} \leftarrow \text{fetch_pc_q}$$

yapılır.

Buradaki `fetch_pc_q`, normal execution devam etseydi çalıştırılacak olan **bir sonraki mimari instruction'ın adresidir**.

Handler tamamlandıktan sonra `mret` çalıştırıldığında program bu adresten devam eder.

Exception durumunda ise belirli bir instruction hata üretmiştir. `mepc`, fault oluşturan instruction'ın PC değerini tutar.

Faulting instruction normal şekilde retire edilmez ve mimari sonucu commit edilmez. Instruction trap kaynağı olarak kullanılır, daha genç instruction'lar ise flush edilerek geçersiz hale getirilir.

5.8 Trap Girişinde mstatus

Interrupt veya exception kabul edildiğinde `mstatus` içerisindeki interrupt kontrol bitleri otomatik olarak güncellenir.

Trap girişinde:

$$\begin{aligned} \text{MPIE} &\leftarrow \text{MIE} \\ \text{MIE} &\leftarrow 0 \\ \text{MPP} &\leftarrow 2'b11 \end{aligned}$$

işlemleri yapılır.

microBee yalnız Machine Mode kullandığı için:

$$\text{MPP} = 2'b11$$

olarak tutulur.

Interrupt kabulünde ayrıca:

$$\text{mtval} \leftarrow 0$$

yapılır. Exception kabulünde `mtval` olay türüne göre fault adresi, illegal instruction kodu veya ilgili değeri alır; her trap'te sıfırlanmaz.

5.9 mret

Interrupt veya exception handler tamamlandığında normal programa dönmek için `mret` instruction'ı kullanılır.

`mret` çalıştırıldığında:

$$\begin{aligned} \text{PC} &\leftarrow \text{mepc} \\ \text{MIE} &\leftarrow \text{MPIE} \\ \text{MPIE} &\leftarrow 1 \\ \text{MPP} &\leftarrow 2'b11 \end{aligned}$$

işlemleri yapılır.

Bu sayede trap öncesindeki global interrupt durumu geri yüklenir.

5.10 Nested Interrupt

microBee içerisinde nested interrupt'ı özel olarak engelleyen ayrı bir donanım kilidi bulunmaz.

Normal trap girişinde:

```
mstatus.MIE = 0
```

yapıldığı için handler çalışırken yeni interrupt otomatik olarak alınmaz.

Ancak handler yazılımı tekrar:

```
mstatus.MIE = 1
```

yaparsa başka bir interrupt kabul edilebilir.

Yeni interrupt için yine pipeline drain koşulu uygulanır.

Ancak nested interrupt alındığında:

- mepc,
- mcause,
- ilgili trap state

tekrar yazılır.

Bu sebeple nested interrupt kullanımı yazılım tarafından dikkatli şekilde yönetilmelidir.

5.11 mtvec

mtvec, trap oluştuğunda handler'ın hangi adreste bulunduğunu belirler.

microBee yalnızca **Direct Mode** destekler.

Handler adresi:

```
PC <- {mtvec[31:2], 2'b00}
```

şeklinde oluşturulur.

mtvec[1:0] alanına farklı bir değer yazılmaya çalışılırsa bu bitler WARL davranışı ile sıfıra zorlanır.

Yani:

```
mtvec[1:0] = 00
```

olarak kalır.

Vectored mode yazılması illegal instruction oluşturmaz. Mode bitleri yok sayılır ve Direct Mode kullanılır.

5.12 CLINT

CLINT (tarihsel adıyla Core-Local Interruptor), microBee içerisinde yerel software ve timer interrupt birimidir.

microBee içerisinde iki temel interrupt türünü üretir:

- Software Interrupt
- Timer Interrupt

CLINT base adresi:

```
0x40000000
```

olarak kullanılır.

5.12.1 Software Interrupt

Software interrupt MSIP register'ı üzerinden oluşturulur.

Register:

```
MSIP @ 0x40000000
```

adresindedir.

Bit 0'a:

1

yazıldığında software interrupt pending olur.

Interrupt temizlemek için aynı bite:

0

yazılır.

Machine-mode software interrupt'ın kabul edilmesi için:

- `mie.MSIE = 1`,
- `mstatus.MIE = 1`

olmalıdır.

Software interrupt kabul edildiğinde:

`mcause = 0x80000003`

olur.

5.12.2 Timer Interrupt

microBee içerisinde `mtime` ve `mtimecmp` 64-bit register'lardır.

RV32 kullanıldığı için bu register'lara iki adet 32-bit MMIO register üzerinden erişilir.

Tablo 5.2: CLINT timer register adresleri.

Register	Offset	Adres
MTIMECMP LO	0x400	0x40000400
MTIMECMP HI	0x404	0x40000404
MTIME LO	0xFF8	0x40000FF8
MTIME HI	0xFFC	0x40000FFC

Timer interrupt koşulu:

$$mtime \geq mtimecmp$$

olduğunda oluşur.

Timer interrupt'ın kabul edilebilmesi için:

- `mie.MTIE = 1`,
- `mstatus.MIE = 1`

olmalıdır.

Interrupt alındıktan sonra handler genellikle bir sonraki timer olayını belirlemek için `mtimecmp` değerini ileri bir zamana yazar.

Timer interrupt cause:

`mcause = 0x80000007`

değeridir.

5.13 PLIC

PLIC, Platform-Level Interrupt Controller yapısıdır.

Core dışındaki peripheral ve donanım kaynaklarından gelen interrupt'ları toplamak, önceliklendirmek ve core'a tek bir external interrupt hattı üzerinden iletmek için kullanılır.

microBee PLIC içerisinde:

- 16 gerçek external interrupt kaynağı,
- 3-bit priority,
- threshold,
- enable maskesi,
- claim/complete mekanizması,
- dört farklı trigger mode

bulunmaktadır.

PLIC base adresi:

0x40001000

olarak kullanılır.

Tablo 5.3: PLIC konfigürasyon adresleri ve erişim durumları.

Offset	Mutlak adres	Register	Erişim
0x000	0x40001000	CAPABILITY	RO
0x004	0x40001004	PRIORITY[1]	RW
0x008	0x40001008	PRIORITY[2]	RW
0x00C	0x4000100C	PRIORITY[3]	RW
0x010	0x40001010	PRIORITY[4]	RW
0x014	0x40001014	PRIORITY[5]	RW
0x018	0x40001018	PRIORITY[6]	RW
0x01C	0x4000101C	PRIORITY[7]	RW
0x020	0x40001020	PRIORITY[8]	RW
0x024	0x40001024	PRIORITY[9]	RW
0x028	0x40001028	PRIORITY[10]	RW
0x02C	0x4000102C	PRIORITY[11]	RW
0x030	0x40001030	PRIORITY[12]	RW
0x034	0x40001034	PRIORITY[13]	RW
0x038	0x40001038	PRIORITY[14]	RW
0x03C	0x4000103C	PRIORITY[15]	RW
0x040	0x40001040	PRIORITY[16]	RW
0x100	0x40001100	PENDING	RO
0x104	0x40001104	SOURCE_MODE	RW
0x108	0x40001108	INPUT_STATE	RO
0x10C	0x4000110C	RESET_LEVEL	RO
0x180	0x40001180	ENABLE	RW
0x300	0x40001300	THRESHOLD	RW
0x304	0x40001304	CLAIM	RO (yan etkili)
0x308	0x40001308	COMPLETE	WO

Not

RO Yalnız okunur. Yazma yok sayılır veya register'a göre bus response üretilir; mimari durum değişmez (CAPABILITY, PENDING, INPUT_STATE, RESET_LEVEL).

RW Okuma ve yazma. PRIORITY[i] ve THRESHOLD alanlarının üst bitleri RAZ/WI'dır (3-bit priority). ENABLE bit 0 ayrılmıştır; bit k kaynak k 'yı açar. SOURCE_MODE her kaynak için 2 bit: $((i-1)*2) + 2$.

RO (yan etkili) — **CLAIM** Okuma en yüksek etkin kaynak ID'sini döndürür (yoksa 0); aynı anda pending temizlenir ve kaynak in-service olur. irq_external_o başka etkin kaynak yoksa düşer.

WO — **COMPLETE** Yazılan ID in-service ise tamamlanır. Okuma 0 döner. Level kaynakta hat hâlâ aktifse pending yeniden kurulabilir; bu yüzden önce peripheral temizlenmeli, sonra COMPLETE yazılmalıdır.

PENDING Bit 0 ayrılmış; bit k kaynak k pending durumudur. Yazılımla doğrudan temizlenmez; gateway / CLAIM yolu kullanılır.

PRIORITY formülü PRIORITY[i] @ $0x40001000 + 0x004 + 4(i-1)$, $i = 1..16$. Priority 0 seçime girmez; seçim için ayrıca enable=1 ve priority > threshold gerekir.

5.13.1 Interrupt ID

microBee içerisinde:

ext_irq_i[15:0]

olmak üzere 16 gerçek external interrupt kaynağı bulunur.

Mapping:

$$ext_irq_i[n] \rightarrow ID = n + 1$$

şeklindedir.

Yani:

- ext_irq_i[0] → ID 1,
- ext_irq_i[15] → ID 16.

ID 0 gerçek bir interrupt kaynağı değildir ve:

no interrupt

anlamında ayrılmıştır.

5.13.2 Priority ve Enable

Her external interrupt kaynağı için 3-bit priority değeri bulunur.

Bu nedenle priority aralığı:

0...7

şeklindedir.

Priority değeri 0 olan kaynak seçime girmez ve pratik olarak disable edilmiş gibi davranır.

Ancak yalnızca priority değerinin 0'dan büyük olması yeterli değildir.

Bir kaynağın PLIC tarafından seçilebilmesi için:

- ilgili kaynak enable edilmiş olmalıdır,
- priority değeri 0'dan büyük olmalıdır,
- priority değeri threshold değerinden büyük olmalıdır.

Yani temel seçim koşulu:

```
enable[i]    = 1
priority[i]  > 0
priority[i]  > threshold
```

şeklindedir.

Enable kapalıysa priority değeri 7 olsa bile kaynak seçilemez.

Aynı priority değerine sahip birden fazla interrupt aynı anda pending ise **düşük ID değerine sahip kaynak** öncelikli seçilir.

5.13.3 Trigger Modları

Her PLIC kaynağı için 2-bit SOURCE_MODE alanı bulunur.

Kaynak i için ilgili mode alanı:

$$((i-1)*2) +: 2$$

bitlerinde tutulur.

Desteklenen modlar:

Kod	Mod
00	Rising edge
01	Falling edge
10	Active-low level
11	Active-high level

5.13.4 Claim ve Complete

PLIC tarafından external interrupt oluşturulduğunda handler hangi kaynağın interrupt ürettiğini öğrenmek için CLAIM register'ını okur.

microBee içerisinde:

CLAIM @ 0x40001304

adresindedir.

CLAIM en yüksek öncelikli pending interrupt'ın ID değerini verir.

Handler ilgili peripheral üzerindeki interrupt sebebini temizledikten sonra aynı ID değeri:

COMPLETE @ 0x40001308

register'ına yazılır.

Temel akış:

Pending -> Priority Selection -> CLAIM -> Handler -> COMPLETE

şeklindedir.

Level-triggered interrupt.

Level-triggered bir interrupt kaynağında yalnızca COMPLETE yazılması yeterli değildir.

Peripheral üzerindeki interrupt sebebi önce temizlenmelidir.

Eğer kaynak hattı halen active seviyedeysen COMPLETE yazılırsa PLIC kaynağı tekrar pending olarak görebilir ve aynı interrupt yeniden alınabilir.

Bu sebeple level-triggered interrupt için temel sıra:

CLAIM -> Peripheral Clear -> COMPLETE

şeklinde uygulanmalıdır.

5.14 Shadow Register Bank

microBee içerisinde opsiyonel shadow register bank desteği bulunmaktadır.

`SHADOW_BANK_ENABLE = 1` olduğunda her trap girişinde normal register bankasından shadow register bankasına geçilir.

Bu davranış:

- interrupt,
- synchronous exception

için aynıdır.

Shadow bank içerisinde:

- `x1-x31` için ikinci register seti bulunur,
- `x0` her durumda hardwired zero davranışını korur.

Normal register bankasının içeriği shadow bankasına kopyalanmaz.

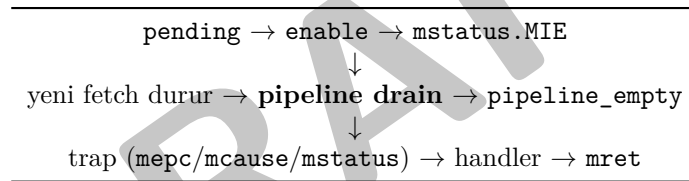
Yalnızca aktif register bankası değiştirilir.

Bu sayede handler çalışmaya başladığında normal programın register değerlerini kaydetmek için gereken bazı yazılımsal save/restore (register kaydetme/geri yükleme) işlemleri azaltılabilir.

`mret` çalıştırıldığında tekrar normal register bankası seçilir.

5.15 Interrupt Çözme Mekanizması

microBee içerisinde bir interrupt'ın baştan sona işlenmesi aşağıdaki akışla özetlenebilir. Ayırt edici nokta: interrupt geldiği anda pipeline flush edilmez; önce **drain** edilir, sonra trap kabul edilir.



Adım adım:

1. CLINT veya PLIC interrupt kaynağını pending hale getirir.
2. Pending bilgisi CSR tarafındaki ilgili `mip` bitine yansır.
3. `mie` içerisindeki ilgili `enable` biti ve `mstatus.MIE` kontrol edilir.
4. Birden fazla interrupt varsa:

External > Software > Timer

öncelik sırası uygulanır.

5. Interrupt request oluştuğunda yeni instruction fetch işlemleri durdurulur.
6. Pipeline içerisindeki mevcut instruction'ların tamamlanması beklenir.
7. `pipeline_empty` olduğunda interrupt trap kabul edilir.
8. `mepc` bir sonraki mimari PC ile güncellenir.
9. `mcause` interrupt türüne göre yazılır.
10. Interrupt ise `mtval = 0` yapılır.
11. `MPIE <- MIE, MIE <- 0` işlemleri yapılır.
12. Shadow bank aktifse shadow register bankasına geçilir.
13. PC `mtvec` adresine yönlendirilir.
14. Handler interrupt kaynağını çözer ve temizler.
15. External interrupt ise PLIC `CLAIM` okunur, peripheral temizlenir ve `COMPLETE` yazılır.
16. `mret` çalıştırılır.
17. Normal register bankasına dönülür, MIE geri yüklenir ve program `mepc` adresinden devam eder.

5.16 Interrupt ve Exception Farkı

Interrupt ve exception aynı trap altyapısını kullanmasına rağmen pipeline davranışları aynı değildir.

Interrupt durumunda microBee önce pipeline'ın boşalmasını bekler.

Bu sebeple interrupt kabul edildiği anda:

- IF/ID,
- ID/EX,
- EX/MEM,
- MEM/WB,
- fetch queue,
- outstanding fetch,
- outstanding memory işlemleri,
- posted işlemler,
- MUL/DIV işlemleri

tamamlanmış veya boş durumdadır.

Bu nedenle interrupt kabulü branch misprediction'daki gibi aktif pipeline stage'lerini anlık olarak flush etmek şeklinde gerçekleştirilmez.

Pipeline önce **drain** edilir, ardından PC **mtvec**'e yönlendirilir.

Exception durumunda ise fault belirli bir instruction yürütülürken ortaya çıkabilir.

Faulting instruction trap kaynağı olur ancak normal şekilde retire edilmez ve mimari sonucu commit edilmez.

mepc faulting instruction'ın PC değerini tutar ve daha genç instruction'lar flush edilerek geçersiz hale getirilir.

Bu fark precise interrupt ve precise exception davranışının önemli bir parçasıdır.

5.17 Öğrenci Kurulum Örneği (PLIC kaynak 1)

Sistem reset sonrası çalışmaya başladığında interrupt'ı kullanmak için önce handler adresi, PLIC ayarları ve CSR enable bitleri kurulmalıdır.

Bu örnek `ext_irq_i[0]` hattını (PLIC ID 1) active-high level modunda açar, priority=2 ve threshold=0 atar, handler adresini `mtvec`'e yazar.

Örnek varsayımları.

- Bu örnek **özel konfigürasyon** varsayar: `SHADOW_BANK_ENABLE=1`. REF32 profilinde ise `SHADOW_BANK_ENABLE=0`'dır; REF32'de handler `t0/t1` için yazılımsal save/restore gerekir.
- Kurulum sırasında kaynak henüz enable edilmeden önce mode/priority yazılır; enable en sonda açılır.
- Interrupt, `ext_irq_i[0]=1` olduğunda gelir (simülasyonda TB veya FPGA'da bağlı peripheral).
- Level kaynak: handler önce hattı düşüren peripheral temizliğini yapmalı, sonra **COMPLETE** yazmalıdır. Aşağıdaki handler eğitim için CLAIM+COMPLETE gösterir; gerçek UART vb. için peripheral clear eklenmelidir.

```
# -----
# microBee öğrenci örneği: PLIC ID1 (ext_irq_i[0]) kurulum + handler
# Adresler: CLINT 0x40000000, PLIC 0x40001000
# -----

.section .text
.globl _start
_start:
# 1) Handler adresini mtvec'e yaz (Direct mode)
la    t0, irq_handler
csrw  mtvec, t0

# 2) PLIC: PRIORITY[1] = 2 @ 0x40001004
li    t0, 0x40001004
li    t1, 2
sw    t1, 0(t0)
```

```

# 3) PLIC: THRESHOLD = 0 @ 0x40001300
li    t0, 0x40001300
sw    x0, 0(t0)

# 4) PLIC: SOURCE_MODE kaynak1 = 11 (active-high) @ 0x40001104
# Kaynak1 bitleri [1:0]; diğer kaynaklar 00 kalsın.
li    t0, 0x40001104
li    t1, 0x3
sw    t1, 0(t0)

# 5) PLIC: ENABLE bit1 (ID1) @ 0x40001180
li    t0, 0x40001180
li    t1, 0x2          # bit 1 = kaynak 1
sw    t1, 0(t0)

# 6) Core: mie.MEIE = 1 (bit 11)
li    t0, (1 << 11)
csrs mie, t0

# 7) Core: mstatus.MIE = 1 (bit 3) -- en son aç
li    t0, (1 << 3)
csrs mstatus, t0

# Ana program: interrupt gelene kadar döngü
main_loop:
    j    main_loop

# -----
# Interrupt anında çalışan kod (mtvec hedefi)
# -----
    .align 4
irq_handler:
    # Shadow bank=1 özel konfigürasyonda t0/t1 ayrı context'tedir.
    # REF32 (shadow=0) için önce save/restore ekleyin.

    # CLAIM oku -> kaynak ID (beklenen: 1)
    li    t0, 0x40001304
    lw    t1, 0(t0)          # t1 = claimed ID

    # (İsteğe bağlı) sayaç: DMEM'de irq geldiğini göster
    # li t0, 0x20000000 ; lw t2,0(t0) ; addi t2,t2,1 ; sw t2,0(t0)

    # Level kaynakta: burada peripheral IRQ sebebini temizleyin
    # (UART/GPIO register clear). Aksi halde COMPLETE sonrası
    # aynı interrupt yeniden pending olabilir.

    # COMPLETE: claimed ID'yi yaz
    li    t0, 0x40001308
    sw    t1, 0(t0)

    mret          # mepc'ye dön, MIE geri yüklenir

```

Kurulum sırasının özeti:

1. mtvec ← irq_handler
2. PRIORITY[1]=2, THRESHOLD=0, mode=active-high
3. ENABLE bit1
4. mie.MEIE, sonra mstatus.MIE
5. ext_irq_i[0] yükselince trap → handler
6. CLAIM → (peripheral clear) → COMPLETE → mret

5.18 Software Interrupt Örneği

Harici pin olmadan denemek için CLINT MSIP kullanılabilir.

Örnek varsayımı — REF32 değil.

Aşağıdaki örnek SHADOW_BANK_ENABLE=1 özel konfigürasyonu içindir; handler t0/t1 için save/restore yapmaz. REF32 profilinde (SHADOW_BANK_ENABLE=0) aynı handler register'ları stack'e kaydedip geri yüklemelidir.

```
la    t0, sw_irq_handler
csrw mtvec, t0

li    t0, (1 << 3)      # mie.MSIE
csrs mie, t0

li    t0, (1 << 3)      # mstatus.MIE
csrs mstatus, t0

li    t0, 0x40000000    # CLINT MSIP
li    t1, 1
sw    t1, 0(t0)         # Software interrupt oluştur

main_sw:
j     main_sw

sw_irq_handler:
li    t0, 0x40000000
sw    x0, 0(t0)         # MSIP = 0
mret
```

Trap kabul edildiğinde mcause = 0x80000003 olur.

5.19 Hardware Interrupt Akışı (Özet)

Harici interrupt kaynakları PLIC üzerinden microBee çekirdeğine iletilir. Kurulum ve handler assembly'si bir önceki “Öğrenci Kurulum Örneği” bölümündedir; burada yalnızca kavramsal akış özetlenir.

ext_irq_i[0] hattı PLIC içinde ID = 1 olarak görülür. Kaynak enable, priority > 0 ve priority > threshold koşulları ile seçildikten sonra irq_external_i = 1 olur; trap kabulünde mcause = 0x8000000B oluşur.

Handler sırası: CLAIM → peripheral clear → COMPLETE → mret. Level-triggered kaynakta peripheral clear, COMPLETE'ten önce yapılmalıdır.

5.20 Özet

microBee interrupt sistemi Machine Mode içerisinde çalışan üç seviyeli bir yapı kullanır:

External > Software > Timer

Software ve timer interrupt'ları CLINT, external interrupt'lar ise PLIC tarafından yönetilir.

Interrupt kabulü sırasında pipeline doğrudan flush edilmez. Önce yeni fetch işlemleri durdurulur ve pipeline tamamen drain edilir. Pipeline boşaldığında trap alınır ve mepc bir sonraki çalıştırılacak instruction'ın PC değerini tutar.

PLIC tarafında bir external kaynağın seçilebilmesi için yalnızca priority değeri yeterli değildir; kaynağın enable edilmiş olması ve priority değerinin threshold değerinden büyük olması gerekir.

mip değeri ise yalnızca CSR içerisindeki yazılabilir pending bitlerinden oluşmaz. Aktif interrupt girişleri de OR edilerek okunan pending durumuna yansıtılır.

Bu yapı sayesinde interrupt'lar precise bir mimari sınırdan alınır ve handler tamamlandığında execution güvenli şekilde kaldığı noktadan devam eder.

DRAFT

Bölüm 6

Genel Amaçlı Giriş/Çıkış Portları (GPIO) ve Çevre Birimleri

Bu bölümde microBee içerisinde bulunan native `tmcu_gpio` bloğu, memory-mapped register yapısı ve kullanım şekli anlatılmaktadır. Bölümün sonunda aynı MMIO adres alanında bulunan native timer ve UART çevre birimleri ile FPGA tarafında kullanılabilen AXI UART Lite yapısı kısaca tanıtılmıştır.

6.1 GPIO Tasarımı

GPIO, SoC (System on Chip) içerisinde:

`GPIO_BASE = 0x40002000`

adresinden başlayan memory-mapped bölgede bulunur.

Router, `GPIO_ENABLE=1` olduğunda bu adres aralığına yapılan load/store isteklerini GPIO birimine yönlendirir.

REF32 / varsayılan SoC konfigürasyonunda:

- `NUM_BANKS = 1`,
- `GPIO_WIDTH = 32`,
- `bank stride = 0x20` byte,
- `INPUT_VALID_MASK = 32'hFFFF_FFFF`,
- `OUTPUT_VALID_MASK = 32'hFFFF_FFFF`

olarak kullanılır.

microBee SoC içerisinde `tmcu_gpio` bloğu temel olarak `BASE_ADDRESS` ve `GPIO_WIDTH` parametreleri ile örneklenmektedir. Input ve output maskeleri modülün varsayılan değerlerinde bırakılmıştır.

`INPUT_VALID_MASK` ve `OUTPUT_VALID_MASK` birbirinden bağımsız parametrelerdir. Bu sayede ileride belirli GPIO bitlerinin yalnız giriş veya yalnız çıkış olarak kullanılacağı farklı konfigürasyonlar oluşturulabilir.

REF32 profilinde ise bütün 32 GPIO biti hem input hem output açısından geçerlidir.

6.1.1 GPIO Portları

GPIO bloğunun SoC tarafındaki temel sinyalleri:

Port	Anlam
<code>gpio_in_i[31:0]</code>	Fiziksel GPIO girişleri
<code>gpio_out_o[31:0]</code>	GPIO çıkış verisi
<code>gpio_oe_o[31:0]</code>	Output enable, 1: çıkış, 0: giriş/Hi-Z

Çıkış ve giriş yolları genel olarak (bit düzeyinde VE / maskeleme):

Sinyal	Hesap
gpio_out_o	OUT & OUTPUT_VALID_MASK
gpio_oe_o	OE & OUTPUT_VALID_MASK
IN	gpio_in_sync_q & INPUT_VALID_MASK

şeklinde çalışır.

Bu yapı sayesinde register içerisinde geçersiz bir GPIO bitine değer yazılsa bile fiziksel çıkış tarafında ilgili maske uygulanır.

6.1.2 Input Senkronizasyonu

GPIO pinlerinden gelen sinyaller işlemci clock domain'ine göre asenkron olabilir. Asenkron bir sinyalin doğrudan senkron mantık içerisinde kullanılması metastability riski oluşturabilir.

Bu nedenle microBee GPIO girişlerinde 2-FF synchronizer kullanılmaktadır.

Temel yapı:

```
gpio_in_i -> FF1 -> FF2 -> gpio_in_sync_q
```

şeklinde düşünülebilir.

Reset sırasında her iki senkronizer register'ı da sıfırlanır. Bu nedenle reset sonrasında GPIO girişinin gerçek seviyesi IN register'ında hemen görülmeyebilir.

Tipik olarak giriş değişimi yaklaşık iki clock sonra yazılım tarafından okunabilir hale gelir.

Dar pulse'lar.

2-FF synchronizer bir pulse-capture devresi değildir. GPIO girişinde işlemci clock'undan daha kısa süren bir pulse oluşursa bu pulse'ın yakalanacağı garanti edilmez. Böyle bir gereksinimde ayrı edge-/pulse detection donanımı kullanılmalıdır.

6.1.3 Reset Davranışı

Reset sonrasında:

OUT = 0

ve:

OE = 0

olarak başlatılır.

Bu durumda SoC içi GPIO çıkış sürücüleri kapalıdır; pinler giriş / yüksek empedans (Hi-Z) niyetindedir. Gerçek FPGA pininde Hi-Z davranışının görülmesi üst seviye IOBUF / pin bağlantısına da bağlıdır.

6.1.4 Bus Erişimi

GPIO erişimleri microBee native valid/ready bus üzerinden gerçekleştirilir.

Adresler 32-bit word-aligned (4-byte hizalı) olmalıdır:

```
address[1:0] = 2'b00
```

Hizasız veya tanımlanmamış bir register adresine erişildiğinde:

```
rsp_err = 1
```

üretilir.

6.2 GPIO'nun Görevleri

GPIO temel olarak üç şekilde kullanılabilir:

- **Çıkış:** LED, röle, enable hattı veya genel dijital çıkış sürmek.
- **Giriş:** Buton, switch veya dijital sensör seviyesini okumak.
- **Atomik bit işlemleri:** OUT_SET, OUT_CLR ve OUT_TGL register'ları üzerinden belirli çıkış bitlerini değiştirmek.

Output olarak kullanılacak bir bit için OE=1, input olarak kullanılacak bit için OE=0 yapılır.

OE işlemleri.

OUT için SET, CLR ve TGL register'ları bulunmasına rağmen OE için benzer register'lar yoktur. Bu nedenle tek bir OE bitinin değiştirilmesi gerekiyorsa read-modify-write işlemi yapılmalıdır.

GPIO bloğu kendi başına PLIC interrupt üretmez. Bir GPIO pininin edge veya level değişiminin interrupt oluşturması isteniyorsa bunun için GPIO dışında ek bir interrupt üretme mantığı kullanılması gerekir.

6.3 GPIO Adres Haritası

GPIO base adresi:

0x40002000

olarak tanımlanmıştır.

REF32 profilinde yalnızca bank 0 bulunur.

Tablo 6.1: GPIO register adres haritası ve erişim türleri.

Offset	Mutlak adres	Register	Erişim
0x00	0x40002000	IN	RO
0x04	0x40002004	OUT	RW
0x08	0x40002008	OUT_SET	WO, okuma 0
0x0C	0x4000200C	OUT_CLR	WO, okuma 0
0x10	0x40002010	OUT_TGL	WO, okuma 0
0x14	0x40002014	OE	RW
0x18	0x40002018	VALID	RO
0x1C	0x4000201C	—	Geçersiz

Geçersiz 0x1C offset'ine erişildiğinde `rsp_err` üretilir.

6.3.1 IN

Adres:

IN @ 0x40002000

GPIO girişlerinin senkronize edilmiş durumunu gösterir.

Okunan değer:

`IN = gpio_in_sync_q & INPUT_VALID_MASK`

şeklindedir.

IN read-only (RO) olarak kullanılır. Bu kitaptaki RO tanımı, yazmanın mutlaka bus error üreteceği anlamına gelmez: IN'e yazılması bus error oluşturmaz; yazma no-op'tur ve GPIO durumunu değiştirmez.

6.3.2 OUT

Adres:

OUT @ 0x40002004

GPIO output data register'ıdır.

Register'a yazılan değer fiziksel output seviyesini belirler.

Yazma sırasında:

```
apply_wstrb(OUT, wdata, wstrb)
```

işlemi uygulanır.

Böylece byte veya half-word store işlemlerindeki write strobe bilgisi korunur.

Ardından:

```
OUT = apply_wstrb(OUT,wdata,wstrb) & OUTPUT_VALID_MASK
```

uygulanır.

6.3.3 OUT_SET

Adres:

OUT_SET @ 0x40002008

Yazılan değerde 1 olan bitleri OUT içerisinde 1 yapar.

```
OUT <- OUT | (wdata & OUTPUT_VALID_MASK)
```

Bu register'da **wstrb** uygulanmaz. Bütün 32-bit **wdata** değerlendirilir.

Bu nedenle bu register'a erişirken **SW** kullanılması önerilir.

6.3.4 OUT_CLR

Adres:

OUT_CLR @ 0x4000200C

Yazılan değerde 1 olan output bitlerini temizler.

```
OUT <- OUT & (wdata & OUTPUT_VALID_MASK)
```

Bu register'da da **wstrb** uygulanmaz.

6.3.5 OUT_TGL

Adres:

OUT_TGL @ 0x40002010

Yazılan değerde 1 olan output bitlerini toggle eder.

```
OUT <- OUT xor (wdata & OUTPUT_VALID_MASK)
```

Bir bit 0 ise 1, 1 ise 0 yapılır.

Bu register'da da **wstrb** uygulanmaz.

6.3.6 OE

Adres:

OE @ 0x40002014

Her GPIO kanalının input veya output olarak kullanılmasını belirler.

- OE[n]=1: ilgili GPIO çıkış sürer,
- OE[n]=0: ilgili GPIO giriş niyetindedir (SoC çıkış sürücüsü kapalı; FPGA pininde Hi-Z, IOBUF bağlantısına bağlıdır).

OE, OUT ile aynı write-strobe davranışını kullanır.

Yani:

apply_wstrb

uygulanır ve ardından OUTPUT_VALID_MASK ile maskelenir.

6.3.7 VALID

Adres:

VALID @ 0x40002018

Okunduğunda:

OUTPUT_VALID_MASK

değerini döndürür.

Bu register yalnız output tarafındaki geçerli GPIO bitlerini gösterir.

INPUT_VALID_MASK için yazılıma görünür ayrı bir register bulunmamaktadır.

VALID register'ına yazma yapılması bus error oluşturmaz. Yazma işlemi no-op olarak kabul edilir.

6.3.8 Birden Fazla GPIO Bankası

Modül birden fazla GPIO bankası oluşturabilecek şekilde parametrize edilmiştir.

Bank b için base adres:

$$\text{GPIO_BASE}_b = 0x40002000 + b \times 0x20$$

şeklindedir.

REF32 profilinde:

NUM_BANKS = 1

olduğu için yalnızca bank 0 kullanılmaktadır.

6.4 GPIO_ENABLE = 0 Davranışı

GPIO_ENABLE=0 olduğunda GPIO bloğu router içerisinde geçerli hedef olarak seçilmez.

Bu durumda 0x40002000 adres aralığı başka bir birime map edilmemişse request router decode sonucunda SEL_ERR hedefini seçer. SEL_ERR, router'ın tanımsız/erişilemeyen adres için ürettiği iç response seçimidir; çekirdek access-fault olarak yansır.

Çekirdek bu durumu:

- load access fault veya

- store access fault

olarak işler.

GPIO modülü sentezden çıkarıldığında SoC tarafındaki:

`gpio_out_o`

ve:

`gpio_oe_o`

çıkışları sıfıra bağlanır.

6.5 Register Güncelleme Zamanı

GPIO register yazmaları GPIO'nun isteği kabul ettiği clock edge'inde gerçekleşir.

Temel kabul koşulu:

`request_fire`

olarak düşünülebilir.

OUT veya OE register'ı ilgili **posedge** üzerinde güncellenir.

Aynı clock edge'inde `rsp_valid` de oluşturulur.

Yani GPIO register'ı, request kabul edildiği clock edge'inde güncellenir; yanıtın bir sonraki cycle'da gelmesini beklemesini beklemesini.

Yazılım açısından store instruction'ı tamamlandığında GPIO register durumu güncellenmiştir. Fiziksel FPGA pinindeki değişim ise IOBUF ve pin yoluna bağlıdır.

6.6 GPIO Nasıl Kullanılır?

Temel kullanım sırası:

1. Output kullanılacak bitler için OE bitlerini 1 yap.
2. OUT, OUT_SET, OUT_CLR veya OUT_TGL ile çıkış seviyesini belirle.
3. Input kullanılacak bitlerde OE=0 olduğundan emin ol.
4. IN register'ını okuyarak giriş seviyesini al.

6.6.1 Örnek: GPIO Bit 0 ile LED Kontrolü

Aşağıdaki örnekte GPIO bit 0 çıkış yapılır ve SET/CLR/TGL register'ları kullanılır.

```
# GPIO base = 0x40002000
```

```
li    t0, 0x40002000
```

```
# OE @ 0x14: bit0 = 1 (çıkış)
```

```
li    t1, 0x1
```

```
sw    t1, 0x14(t0)
```

```
# LED ON: OUT @ 0x04'e doğrudan yaz
```

```
sw    t1, 0x04(t0)
```

```
# LED OFF: OUT_CLR @ 0x0C
```

```
sw    t1, 0x0C(t0)
```

```
# LED ON: OUT_SET @ 0x08
```

```
sw    t1, 0x08(t0)

# LED TOGGLE: OUT_TGL @ 0x10
sw    t1, 0x10(t0)
```

6.6.2 Örnek: GPIO Bit 1 Giriş Okuma

OE için ayrı SET/CLR register'ı bulunmadığından bit değişimi read-modify-write ile yapılır.

```
li    t0, 0x40002000

# OE bit1 = 0 yap, diğer bitleri koru
lw    t1, 0x14(t0)
andi  t1, t1, -3
sw    t1, 0x14(t0)

# IN register'ını oku
lw    t2, 0x00(t0)

# Yalnız bit1'i ayır
andi  t2, t2, 0x2
```

Son işlemten sonra:

- t2 = 0 ise giriş low,
- t2 = 2 ise giriş high

olarak yorumlanabilir.

6.7 GPIO Doğrulama İpuçları

Simülasyon veya FPGA doğrulaması sırasında aşağıdaki noktalar kontrol edilebilir:

- gpio_out_o ve gpio_oe_o waveform üzerinde izlenebilir.
- Output register değişimi (gpio_out_o/gpio_oe_o) request_fire gerçekleşen clock edge'inde görülmelidir. Bu register-level davranıştır; FPGA pinindeki fiziksel değişim aynı anda garanti edilmez.
- Input değişimi 2-FF synchronizer sebebiyle yaklaşık iki clock gecikmeli görülebilir.
- Reset sonrasında senkronizer register'ları sıfır olduğu için ilk GPIO okumaları 0 olabilir.
- 0x4000201C gibi tanımsız offset veya hizasız adres erişimi rsp_err üretmelidir.
- OUT_SET, OUT_CLR ve OUT_TGL register'larında wstrb uygulanmadığı için word store kullanılmalıdır.

6.8 Diğer Native Çevre Birimleri

GPIO dışında aynı 0x40000000 MMIO adres alanında başka native çevre birimleri de bulunmaktadır.

Birim	Base	PLIC ID	Görev
CLINT	0x40000000	—	Software / timer interrupt
PLIC	0x40001000	—	External interrupt yönetimi
GPIO	0x40002000	—	Genel amaçlı I/O
TIMER	0x40003000	2	Native 64-bit timer
UART	0x40004000	3	Native UART
MONITOR	0x40005000	—	Performans sayaçları

6.9 Native Timer

microBee içerisinde CLINT timer'dan bağımsız olarak genel amaçlı bir native timer çevre birimi bulunmaktadır.

Base adresi:

0x40003000

olarak tanımlanmıştır.

Başlıca register'ları (64-bit alanlar LO/HI çiftidir):

Register	Offset	Görev
CTRL	0x00	Timer kontrolü
PRESCALE	0x04	Clock bölme / prescale
COUNT_LO	0x08	Counter düşük 32 bit
COUNT_HI	0x0C	Counter yüksek 32 bit
CMPO_LO	0x10	Compare 0 düşük 32 bit
CMPO_HI	0x14	Compare 0 yüksek 32 bit
CMP1_LO	0x18	Compare 1 düşük 32 bit
CMP1_HI	0x1C	Compare 1 yüksek 32 bit
PERIOD_LO	0x20	Periyot düşük 32 bit
PERIOD_HI	0x24	Periyot yüksek 32 bit
STATUS	0x28	Status, RW1C
CAPABILITY	0x2C	Donanım yetenek bilgisi

COUNT, CMPO, CMP1 ve PERIOD alanları 64-bit'tir; RV32 yazılımı bunlara LO/HI MMIO register'ları üzerinden erişir.

Timer tarafından oluşturulan interrupt SoC içerisinde PLIC kaynak ID 2'ye bağlanır:

TIMER_IRQ_ID = 2

Bu timer, CLINT içerisindeki `mtime/mtimecmp` yapısından ayrıdır. CLINT machine timer interrupt üretirken native timer genel amaçlı peripheral olarak PLIC üzerinden interrupt üretir.

6.10 Native UART

microBee içerisinde temel seri haberleşme için native UART çevre birimi bulunmaktadır.

Base adresi:

0x40004000

olarak tanımlanmıştır.

Başlıca register'ları:

Register	Offset	Görev
DATA	0x00	TX/RX data
STATUS	0x04	UART durum bilgisi
CTRL	0x08	UART kontrolü
BAUDDIV	0x0C	Baud-rate bölücü
IRQ_STATUS	0x10	Interrupt durumu
FIFO_LEVEL	0x14	FIFO kullanım seviyesi
CAPABILITY	0x18	Donanım yetenek bilgisi

UART temel 8N1 seri haberleşme yapısını kullanır.

Baud rate, microBee native UART implementasyonuna özgü yaklaşık formülle:

$$\text{Baud} \approx \frac{f_{clk}}{\text{BAUDDIV}}$$

şeklinde hesaplanır. Bu formül AXI UART Lite için geçerli değildir.

BAUDDIV için minimum değer 8'dir.

Örneğin 50 MHz clock ve 115200 baud için:

$$\text{BAUDDIV} \approx \frac{50\,000\,000}{115\,200} \approx 434$$

değeri kullanılabilir.

Native UART tarafından oluşturulan interrupt PLIC kaynak ID 3'e bağlanır:

$$\text{UART_IRQ_ID} = 3$$

6.11 AXI UART Lite

FPGA implementasyonunda native UART dışında AXI UART Lite çevre birimi de kullanılabilir.

Referans FPGA yapısında AXI UART Lite:

$$0x60000000$$

adresine map edilmiştir (Bölüm 2 laboratuvar akışı ile aynı).

Bu UART, microBee native UART bloğundan farklı bir çevre birimidir.

Referans bağlantıda AXI UART Lite interrupt hattı PLIC kaynak ID 1'e bağlanabilir.

Dolayısıyla örnek FPGA sisteminde:

PLIC ID	Kaynak
1	AXI UART Lite
2	Native Timer
3	Native UART

şeklinde bir bağlantı kullanılabilir.

Native ve FPGA çevre birimleri.

Native GPIO, native timer ve native UART microBee RTL tasarımının parçasıdır. AXI UART Lite ise FPGA Block Design içerisinde microBee dışına eklenen bir IP bloğudur. Bu nedenle AXI UART Lite'in varlığı ve adresi kullanılan FPGA entegrasyonuna bağlıdır.

Native GPIO pinlerinin de FPGA Block Design içerisinde fiziksel FPGA pinlerine ayrıca bağlanması ve ilgili pin constraint'lerinin tanımlanması gerekir.

6.11.1 AXI UART Lite register özeti (PG142)

Tablo 6.2: AXI UART Lite register adresleri ve temel görevleri.

Offset	Mutlak adres	Register	Anlam
0x00	0x60000000	RX_FIFO	Okuma: RX karakteri
0x04	0x60000004	TX_FIFO	Yazma: TX karakteri
0x08	0x60000008	STAT	RX/TX FIFO ve hata durumları
0x0C	0x6000000C	CTRL	FIFO reset ve interrupt enable

Tablo 6.3: AXI UART Lite STAT ve CTRL bitleri (PG142).

Register	Bit	Anlam
STAT	0	RX FIFO valid data
STAT	1	RX FIFO full
STAT	2	TX FIFO empty
STAT	3	TX FIFO full
STAT	4	Interrupt enabled
STAT	5	Overrun error
STAT	6	Frame error
STAT	7	Parity error
CTRL	0	TX FIFO reset
CTRL	1	RX FIFO reset
CTRL	4	Interrupt enable

Baud Vivado IP / BD ayarındadır (REF32: 50 MHz'de tipik 115200); yazılımda baud register'ı yoktur.

6.11.2 Uygulama: sıfırdan kurulum, PLIC, AXI yaz/oku

Aşağıdaki örnek sistemi baştan kurar:

1. Handler adresini `mtvec`'e yazar,
2. PLIC kaynak 1'i (AXI UART Lite) priority / mode / enable ile açar,
3. Core'da `mie.MEIE` ve `mstatus.MIE` açar,
4. AXI UART Lite CTRL ile FIFO reset ve interrupt enable yapar,
5. AXI üzerinden karakter yazar (TX) ve okur (RX),
6. Interrupt gelirse CLAIM → RX drain → COMPLETE yapar.

Bağlantı varsayımı.

- AXI UART Lite @ 0x60000000, interrupt → `ext_irq_i[0]` → PLIC ID 1 (active-high).
- Bu örnek `SHADOW_BANK_ENABLE=1` özel konfigürasyonudur; handler `t0/t1/t2` için save/restore yapmaz. REF32'de (`SHADOW_BANK_ENABLE=0`) yazılımsal save/restore gerekir.
- Aşağıdaki handler eğitim amaçlıdır: interrupt geldiğinde yalnız RX FIFO drain edilir. AXI UART Lite interrupt'ı başka status/error koşullarından da gelebilir; üretim kodunda STAT bitleri ayrıca incelenmelidir.
- RX için host/terminal veya simülasyon BFM'den veri gelmelidir; yalnız TX testinde RX döngüsü bekleyebilir.

```
# -----
# microBee: AXI UART Lite @ 0x60000000 + PLIC kaynak 1
# En baştan kurulum, AXI yazma (TX) ve okuma (RX)
# -----

.section .text
.globl _start
_start:
    li    sp, 0x2000FF00

    # ---- 1) Trap handler ----
    la    t0, irq_handler
    csrwr mtvec, t0

    # ---- 2) PLIC: kaynak 1 (AXI UART Lite) ----
    # PRIORITY[1] = 2 @ 0x40001004
    li    t0, 0x40001004
    li    t1, 2
    sw    t1, 0(t0)
```

```

# THRESHOLD = 0 @ 0x40001300
li    t0, 0x40001300
sw    x0, 0(t0)

# SOURCE_MODE: kaynak1 = 11 (active-high) @ 0x40001104
# (kaynak henüz enable değilken yaz)
li    t0, 0x40001104
li    t1, 0x3
sw    t1, 0(t0)

# ENABLE bit1 (ID1) @ 0x40001180
li    t0, 0x40001180
li    t1, 0x2
sw    t1, 0(t0)

# ---- 3) Core interrupt enable ----
li    t0, (1 << 11)      # mie.MEIE
csrs  mie, t0
li    t0, (1 << 3)       # mstatus.MIE
csrs  mstatus, t0

# ---- 4) AXI UART Lite init @ 0x60000000 ----
li    s0, 0x60000000
li    t1, 0x03           # CTRL: bit0 TX reset + bit1 RX reset
sw    t1, 0x0C(s0)
li    t1, (1 << 4)       # CTRL: IE = 1 (interrupt enable)
sw    t1, 0x0C(s0)

# ---- 5) AXI'ye yaz: "Hi\r\n" ----
li    a0, 'H'
jal   ra, uart_putc
li    a0, 'i'
jal   ra, uart_putc
li    a0, '\r'
jal   ra, uart_putc
li    a0, '\n'
jal   ra, uart_putc

# ---- 6) AXI'den oku: en az bir karakter (polling) ----
jal   ra, uart_getc      # a0 = okunan byte
# (isteğe bağlı) echo:
# jal   ra, uart_putc

main_loop:
    j    main_loop

# -----
# a0 = gönderilecek karakter
# STAT bit3 (TX full)=0 olana kadar bekle, sonra TX_FIFO yaz
# (FIFO tamamen boş olmak zorunda değil; full iken yazma SLVERR)
# -----
uart_putc:
    li    t0, 0x60000000
1:
    lw    t1, 0x08(t0)    # STAT
    andi  t1, t1, 0x08    # TX full?
    bnez  t1, 1b         # full ise bekle
    sw    a0, 0x04(t0)    # TX_FIFO
    ret

# -----

```

```

# STAT bit0 (RX valid) bekle, RX_FIFO oku -> a0
# -----
uart_getc:
    li    t0, 0x60000000
2:
    lw    t1, 0x08(t0)
    andi  t1, t1, 0x01          # RX valid?
    beqz  t1, 2b
    lw    a0, 0x00(t0)          # RX_FIFO
    andi  a0, a0, 0xFF
    ret

# -----
# Eğitim handler'ı: CLAIM sonrası RX drain (yalnız ID==1)
# -----
    .align 4
irq_handler:
    # CLAIM
    li    t0, 0x40001304
    lw    t1, 0(t0)             # t1 = ID (beklenen 1)
    beqz  t1, irq_done          # 0 ise işlenecek yok

    # ID == 1 ise UART Lite RX drain (level IRQ için gerekli)
    li    t0, 1
    bne   t1, t0, irq_complete

    li    t0, 0x60000000
drain_rx:
    lw    t2, 0x08(t0)
    andi  t2, t2, 0x01
    beqz  t2, irq_complete
    lw    t2, 0x00(t0)          # RX oku / FIFO boşalt
    j     drain_rx

irq_complete:
    fence
    li    t0, 0x40001308        # COMPLETE
    sw    t1, 0(t0)

irq_done:
    mret

```

Kurulum sırası özeti:

1. mtvec ← handler,
2. PLIC: PRIORITY[1], THRESHOLD, mode=active-high, ENABLE bit1,
3. mie.MEIE, mstatus.MIE,
4. AXI CTRL: TX+RX FIFO reset (0x03), sonra IE,
5. TX: STAT.TX_FULL=0 bekle → TX_FIFO yaz,
6. RX: STAT.RX_VALID bekle → RX_FIFO oku,
7. IRQ: CLAIM → RX drain → COMPLETE → mret.

Bölüm 7

Konfigürasyonlar

microBee, tek bir sabit donanım konfigürasyonuna sahip değildir; sentez parametreleri ile şekillendirilebilen bir soft-core tasarımıdır.

Bu parametreler sentez veya FPGA IP konfigürasyonu sırasında belirlenir. Bitstream oluşturulduktan sonra yazılım tarafından çalışma anında değiştirilemez. Örneğin `MUL_ENABLE` veya `GPIO_WIDTH` çalışma sırasında değiştirilemez.

Bu belgede **REF32 profili**, derslerde, simülasyonlarda ve FPGA entegrasyonunda kullanılacak belirli bir microBee konfigürasyonunu ifade eder. Değerler `tmcu_soc` varsayılanları ve Arty A7 Block Design betiğindeki (`scripts/create_arty_a7_system.tcl`) ayarlarla uyumludur. Adres haritası Ek A'tedir.

Parametreler genel olarak iki gruba ayrılabilir:

- **Sabit** — RTL içerisinde konfigürasyon parametresi bulunmayan ve bütün microBee örneklerinde aynı kalan mimari özellikler,
- **Yapılandırılabilir** — sentez veya FPGA IP konfigürasyonu sırasında seçilebilen özellikler.

7.1 Sabit Konfigürasyonlar

Aşağıdaki özellikler `tmcu_core` / `tmcu_soc` üzerinde açılıp kapatılmaz.

Tablo 7.1: Sentezden bağımsız sabit mimari.

Özellik	Davranış
Pipeline	Beş aşama, single-issue, in-order: IF-ID-EX-MEM-WB
Hart sayısı	1. İkinci hart RTL'de yoktur.
Taban ISA	RV32I + Zicsr + Zifencei [2]
Sıkıştırılmış komutlar	RV32C her zaman açıktır; <code>C_ENABLE</code> parametresi yoktur.
Ayrıcalık	Yalnız Machine Mode; <code>mtvec</code> Direct kip
Yazmaç dosyası	32×32-bit, <code>x0</code> sabit sıfır; FF dizisi, BRAM seçeneği yok
Bellek düzeni	Harvard: ayrı IMEM ve DMEM native valid/ready yolları
Yerel DMEM store	Posted store: istek kabul edildikten sonra MEM aşaması yanıt beklemeden ilerler; load ve MMIO/AXI store yanıt bekler
Gözlem portları	<code>dbg_*</code> <code>DEBUG_ENABLE</code> ile açılır (Bölüm 10)
IMEM/DMEM tipi	<code>tmcu_native_bram</code> içinde <code>ram_style = "block"</code>

7.1.1 RV32C

Fetch kuyruğu 16-bit birimler halinde (16-bit granülerlikte) çalışır. `tmcu_decompress` 16-bit komutu ortak 32-bit iç biçime açar; decoder tek bir RV32I/M yolundan ilerler [4].

Donanım tarafında C uzantısını kapatan bir parametre bulunmaz. Derleyici `-march=rv32im` gibi bir seçenekle yalnızca 32-bit instruction üretebilir; ancak çekirdek gelen compressed instruction'ları işlemeye devam eder.

M uzantısına ait multiply ve divide/remainder desteği ise `MUL_ENABLE` ve `DIV_ENABLE` parametreleri ile ayrı ayrı kontrol edilebilir [3].

7.1.2 Register file ve LUT/FF

Teknik profil kodundaki “L / B” (LUT-FF veya BRAM yazmaç dosyası) bu RTL’de bir parametre değildir. `tmcu_regfile` iki FF bankası tutar (gölge bank parametresi açıksa). IMEM ve DMEM ise blok RAM olarak çıkarılmak üzere işaretlenmiştir. Alan tasarrufu için değiştirilen şey yazmaç dosyası tipi değil, aşağıdaki bellek WORDS değerleridir.

7.2 Yapılandırılabilir Konfigürasyonlar

Yapılandırılabilir parametreler `tmcu_soc` (FPGA IP) ve `tmcu_core` (`tmcu_top` simülasyonu) üzerindedir. Kapalı bir birim `generate` ile sentezden çıkar. Router o pencereyi seçmez; erişim access fault üretir (Bölüm 6, `GPIO_ENABLE=0` ile aynı model).

7.2.1 REF32 özeti

Tablo 7.2: REF32’de açık birimler ve çekirdek anahtarları. `tmcu_soc` varsayılını, aksi belirtilmedikçe.

Parametre	REF32	Not
<code>MUL_ENABLE</code>	1	2-cycle çarpıcı
<code>DIV_ENABLE</code>	1	Iterative divider, yaklaşık 34 cycle latency
<code>BP_ENABLE</code>	1	2-bit saturating branch predictor
<code>BP_ENTRIES</code>	128	128-entry prediction table
<code>BP_GSHARE</code>	0	Bimodal PC tabanlı indeksleme
<code>SHADOW_BANK_ENABLE</code>	0	SoC/FPGA. <code>tmcu_top</code> varsayılını 1’dir.
<code>AXI_ENABLE</code>	1	0x60000000 penceresi
<code>CLINT_ENABLE</code>	1	MSIP, mtime
<code>PLIC_ENABLE</code>	1	16 kaynak, 3-bit öncelik
<code>GPIO_ENABLE</code>	1	32 bit, 1 bank
<code>TIMER_ENABLE</code>	1	PLIC ID 2
<code>UART_ENABLE</code>	1	Native UART, PLIC ID 3
<code>MONITOR_ENABLE</code>	1	0x40005000
<code>TRACE_ENABLE</code>	0	SoC iz çıkışları bağlı değil
<code>DEBUG_ENABLE</code>	1	<code>dbg_*</code> boru hattı / CSR gözlemi
<code>IMEM_WORDS / DMEM_WORDS</code>	16384	64 KiB + 64 KiB
<code>RESET_VECTOR /</code>	0x80000000	IMEM tabanı
<code>MTVEC_RESET</code>		
FPGA saat / baud	50 MHz / 115200	BD / constraint; SoC parametresi değil

7.2.2 MUL/DIV

`MUL_ENABLE` ve `DIV_ENABLE` birbirinden bağımsızdır.

`MUL_ENABLE=1` iken çarpma birimi 2-cycle pipelined çalışır; birim müsaitken yeni MUL başlatılabilir, ancak tek-issue çekirdekte EX aşaması sonuç hazır olana kadar ilgili instruction için stall edilebilir.

`DIV_ENABLE=1` iken bölme birimi iterative’dir (yaklaşık 34 cycle latency); işlem süresince EX aşaması stall edilir.

İlgili birim kapalı olduğunda o gruba ait instruction’lar illegal instruction trap ile sonuçlanır [3].

İlgili birim kapatıldığında execution bloğu `generate` yapısı ile sentezden çıkarılır. Böylece o birime ait temel donanım kaynakları tasarımda yer almaz. Decode ve exception kontrolüne ait ortak mantık ise çekirdekte bulunmaya devam eder. Yazılım `-march=rv32i` veya `rv32ic` ile o komutları üretmemelidir; aksi halde trap alınır.

7.2.3 Dallanma tahmincisi

- `BP_ENABLE=0`: tablo üretilmez, her dal not-taken tahmin edilir.
- `BP_ENTRIES`: Branch prediction tablosundaki entry sayısını belirler. Her entry 2-bit saturating counter tutar. İndeks genişliği yaklaşık olarak $\lceil \log_2(\text{BP_ENTRIES}) \rceil$ bittir. Pratikte 2’nin kuvveti olan değerler kullanılır.
- `BP_GSHARE=0`: indeks PC-fold (bimodal). REF32 budur.

- **BP_GSHARE=1:** indeks = PC-fold XOR sıfır genişletilmiş GHR. GHR_WIDTH aralığı [1, $\lceil \log_2(\text{BP_ENTRIES}) \rceil$] olmalıdır.

REF32 profilinde 128 girişli bimodal branch predictor kullanılır. Her giriş 2-bit saturating counter tutar. Daha büyük bir tahmin tablosu doğruluğu artırabilir; ancak LUT/FF kullanımı, yerleştirme/routing maliyeti ve maksimum frekans (Fmax) üzerinde ek yük oluşturabilir. Bu nedenle BP_ENTRIES, performans ve alan sonuçları birlikte değerlendirilerek seçilmelidir. Predictor boyutu ve performans karşılaştırması Bölüm 9 içindedir.

7.2.4 Gölge bank

SHADOW_BANK_ENABLE=1 olduğunda trap girişinde normal register bank yerine shadow bank seçilir. Böylece normal bir interrupt handler içerisinde t0, t1 gibi register'ların ayrıca stack'e kaydedilmesi gerekmez.

Ancak handler içerisinde mstatus.MIE tekrar açılarak nested interrupt kabul edilecekse bu avantaj tek başına yeterli değildir. Yeni trap aynı shadow register alanını kullanabileceği için yazılım gerekli register korumasını kendisi sağlamalıdır.

REF32 SoC ve Arty A7 Block Design'da SHADOW_BANK_ENABLE=0 kullanılır. tmcu_core / tmcu_top modül varsayılanı ise 1'dir. FPGA'da Bölüm 5/6 örneklerini REF32 ile çalıştırırken handler'ın caller-saved yazmaçları yığa alması gerekir. Bu bölümlerde SHADOW_BANK_ENABLE=1 gösteren assembly örnekleri özel konfigürasyondur; REF32 varsayımı değildir.

7.2.5 IMEM / DMEM boyutu ve ön yükleme

IMEM_WORDS, DMEM_WORDS, IMEM_INIT_FILE, DMEM_INIT_FILE

REF32'de her iki bellek de 16384 word (64 KiB) kapasitesindedir. Küçültmek BRAM sayısını düşürür; yazılımın .text/.data sığması gerekir. Linker script tabanları IMEM_BASE / DMEM_BASE ile aynı kalmalıdır.

INIT_FILE verilirse FPGA BRAM başlangıç içeriği bitstream içerisine yerleştirilebilir. Init dosyası kullanılmadığında program içeriği simülasyon, boot mekanizması veya farklı bir yükleme yöntemi ile sağlanmalıdır.

7.2.6 PLIC

PLIC_ENABLE=0 harici kesmeyi kapatır: irq_external_i çekirdekte 0 kalır, 0x40001000 penceresi fault olur.

Açıkken:

- **PLIC_SOURCES** — REF32'de 16; bu durumda harici giriş ext_irq_i[15:0] genişliğindedir. Bit n kaynak ID $n + 1$ 'dir.
- **PLIC_PRIO_BITS** — REF32'de 3 (öncelik 0-7).
- **IRQ_SYNC_ENABLE=1** olduğunda her external interrupt kaynağı 2-FF synchronizer üzerinden geçirilir. Bu yapı asenkron girişlerde metastability riskini azaltır. Edge-triggered kısa pulse'ların kaybolmaması için pulse süresinin core clock ile güvenilir biçimde örneklenebilecek kadar uzun tutulması güvenli bir tasarım önerisidir (örneğin birden fazla core clock süren pulse). Asenkron FPGA girişleri kullanılıyorsa IRQ_SYNC_ENABLE=1 seçilmesi önerilir.
- **IRQ_MODE_RESET, IRQ_INITIAL_LEVEL** — reset sonrası tetikleme kipi ve seviye. REF32 her ikisini 0 tutar.
- **TIMER_IRQ_ID=2, UART_IRQ_ID=3** — native timer ve native UART'ın PLIC numaraları. AXI UART Lite BD'de ext_irq_i[0] ile ID 1'e bağlanır.

PLIC ID 0 "interrupt yok" durumunu temsil eder ve gerçek bir interrupt kaynağı olarak kullanılmaz. Ayrıntı Bölüm 5 ve Ek A.

7.2.7 GPIO

GPIO_ENABLE ve GPIO_WIDTH SoC parametresidir. REF32: açık, 32 bit, tek bank. INPUT_VALID_MASK ve OUTPUT_VALID_MASK iki ayrı mask parametresidir; bunlar tmcu_soc seviyesinde ayrıca dışarı açılmamıştır. REF32 örneğinde tmcu_gpio varsayılanları kullanılır: tek bank ve her iki maske için 0xFFFFFFFF.

GPIO_ENABLE=0 olduğunda GPIO peripheral bloğu sentezden çıkarılır ve router 0x40002000 penceresini seçmez. Bu adrese yapılan erişim access fault ile sonuçlanır. SoC seviyesinde GPIO çıkışları (gpio_out_o, gpio_oe_o) sifıra çekilir. Kullanım Bölüm 6'dadır.

7.2.8 AXI

AXI_ENABLE=1 olduğunda AXI4-Lite master köprüsü etkinleşir; native istekler bu köprü üzerinden AXI transaction'larına dönüştürülür. Pencere AXI_BASE/AXI_MASK ile (REF32: 0x60000000-0x6FFFFFFF).

AXI_ENABLE=0 olduğunda AXI bridge aktif değildir ve 0x60000000-0x6FFFFFFF adres penceresine yapılan erişimler fault ile sonuçlanır. Kullanılmayan SoC AXI çıkışları uygun sabit değerlere çekilir. FPGA'daki AXI UART Lite bu pencerededir; native UART (0x40004000) ayrı UART_ENABLE parametresidir.

AXI_WAIT sayacı yalnız AXI erişim yolu aktifken oluşan bekleme cycle'larını sayar. AXI_ENABLE=0 olduğunda bu tür bir AXI bekleme olayı oluşmaz.

7.2.9 CLINT, timer, native UART, monitoring

Bu birimlerin her biri ayrı enable parametresi ile kontrol edilir. REF32 profilinde bu peripheral'ların tamamı aktiftir.

- CLINT kapalıysa yazılım/timer kesmesi üretilmez; mtime MMIO'su yoktur. Çekirdek irq_software_i / irq_timer_i girişleri 0'a çekilir.
- Timer veya native UART kapalıysa ilgili interrupt PLIC girişine aktarılmaz.
- MONITOR_ENABLE=0 izleme bloğunu çıkarır; 0x40005000 fault olur, BUS_ERROR sayacı da yoktur (Bölüm 8).

7.2.10 TRACE_ENABLE ve DEBUG_ENABLE

TRACE_ENABLE SoC sarmalayıcısındadır. DEBUG_ENABLE hem tmcu_soc hem çekirdek üzerindedir (çekirdeğe iletilir).

TRACE_ENABLE=0 olduğunda SoC seviyesindeki trace_* çıkışları sabittir (trace_valid_o = 0). Core içerisindeki commit/trace bilgisi iç yapıda üretilebilir. tmcu_top simülasyonunda trace_* portları her zaman dışarıdadır (SoC sarmalayıcısı yoktur).

DEBUG_ENABLE boru hattı PC, yazmaç yazma ve CSR snapshot/gözlem (dbg_*) portlarını kontrol eder. Varsayılan 1'dir (BD waveform / ders izleme). DEBUG_ENABLE=0 iken:

- SoC ve çekirdek dbg_* çıkışları 0'a çekilir,
- Vivado IP'te bu portlar Customize IP'te gizlenir (enablement_dependency),
- çekirdek yürütme davranışı değişmez (yalnızca gözlem).

Bölüm 10.

7.2.11 Reset vektörü ve adres pencereleri

RESET_VECTOR ilk fetch PC'sidir. MTVEC_RESET, reset sonrasında mtvec register'ına yüklenecek başlangıç değeridir. microBee yalnız Direct mode desteklediği için mtvec[1:0] bitleri 00 olarak tutulur. REF32 ikisini de 0x80000000 yapar.

IMEM_BASE, DMEM_BASE/DMEM_MASK ve MMIO *_BASE/*_MASK router pencereleridir. Değiştirmek linker script, yazılım sabitleri ve Ek A tablolarını birlikte güncellemeyi gerektirir. Ders ölçümlerinde adres pencereleri değiştirilmemelidir; birim kapatmak için *_ENABLE yeterlidir.

Çekirdekteki DMEM_BASE/DMEM_MASK yerel posted-store penceresini de seçer. SoC bu değerleri çekirdeğe iletir.

7.2.12 Tek parametre değiştirme prensibi

Performans karşılaştırmasında aynı anda bir parametre değiştirilir (Bölüm 9). MUL'u kapatıp aynı anda BP tablosunu büyütme hangi etkinin CPI'yi değiştirdiğini gizler.

7.3 Sabit Özellikler ve Sık Karıştırılan Noktalar

1. RV32C'yi kapatacak bir anahtar yoktur.
2. Yazmaç dosyasını BRAM'e almak için parametre yoktur.
3. Saat frekansı SoC parametresi değildir; FPGA clocking yapısı ve timing constraint'leri ile belirlenir.
4. dbg_* çıkışları DEBUG_ENABLE ile kapatılır; TRACE_ENABLE SoC trace_* çıkışlarını kontrol eder.
5. tmcu_top ile tmcu_soc gölge-bank varsayılanı farklıdır (1'e karşı 0).

6. Çift hart, FPU ve vektör uzantısı bu RTL'de yoktur.

DRAFT

Bölüm 8

Monitoring ve Performans Sayaçları

8.1 Nedir?

Monitoring birimi işlemcinin çalışması sırasında oluşan mikro-mimari olayların sayılmasını sağlar.

Bir işlemcinin yalnızca programı doğru çalıştırması yeterli değildir. Tasarım sırasında aşağıdaki soruların da cevaplanması gerekir:

- Bir program kaç cycle'da (saat çevriminde) tamamlandı?
- Kaç instruction retire edildi?
- Pipeline ne kadar süre stall durumunda kaldı?
- Branch predictor ne kadar doğru tahmin yaptı?
- Memory sistemi işlemciyi ne kadar bekletti?
- MUL/DIV gibi uzun gecikmeli birimler ne kadar kullanıldı?
- Interrupt veya bus error gibi olaylar kaç kez oluştu?

Bu bilgilerin yalnız waveform üzerinden takip edilmesi uzun benchmark çalışmalarında pratik değildir.

Bu nedenle microBee içerisinde işlemci olaylarını donanım seviyesinde sayan memory-mapped monitoring register'ları bulunmaktadır.

Yazılım bu sayaçları normal load instruction'ları ile okuyabilir ve sistem performansını ölçebilir.

Bu bölümde **cycle**, aksi belirtilmedikçe çekirdek saat çevrimini ifade eder.

8.2 Sayaç Modeli

Monitoring birimi:

```
MONITOR_ENABLE = 1
```

olduğunda aktif hale gelir.

Mikro-mimari olaylar 32-bit sayaçlar içerisinde tutulur.

REF32 profilinde sayaçlar:

```
0xFFFF_FFFF
```

değerine ulaştığında doyuma ulaşır (saturate).

Yani sayaç maksimum değerine ulaştıktan sonra tekrar sıfıra dönmez.

Örneğin:

```
counter = 0xFFFF_FFFF
counter + 1 -> 0xFFFF_FFFF
```

olarak kalır.

Bu davranış overflow sebebiyle ölçümün yanlış yorumlanması önler.

8.3 MON_CTRL

Monitoring biriminin temel kontrol register'ı:

MON_CTRL

register'idir.

Offset:

0x000

olarak tanımlanmıştır.

Sayaçlar iki kopya halinde tutulabilir: **live** kopya anlık sayımı tutar; **shadow** kopya ise **SNAPSHOT** anındaki live değerlerin dondurulmuş görüntüsüdür.

Başlıca kontrol bitleri:

Bit	Alan	Görev
0	ENABLE	Monitoring sayaçlarını çalıştırır
1	FREEZE	Sayaçları mevcut değerlerinde durdurur
2	CLEAR	Sayaçları temizler, W1P
3	SNAPSHOT	Live sayaçları shadow alana kopyalar, W1P

8.3.1 ENABLE

ENABLE=1 olduğunda monitoring birimi olayları saymaya başlar.

ENABLE=0 olduğunda sayaçlar artmaz ancak mevcut değerlerini korurlar.

Bu özellik belirli bir kod bölgesinin performansını ölçmek için kullanılabilir.

Örneğin:

1. Sayaçlar temizlenir.
2. Monitoring etkinleştirilir (ENABLE=1).
3. Ölçülecek fonksiyon çalıştırılır.
4. Monitoring dondurulur (FREEZE=1).
5. Sayaçlar yazılım tarafından okunur.

8.3.2 FREEZE

FREEZE=1 olduğunda sayaçların mevcut değeri korunur.

Monitoring birimi enable durumunda olsa bile yeni olaylar sayılmaz.

Bu özellikle birden fazla sayaç okunurken ölçümün değişmeye devam etmesi engellenebilir.

Örneğin önce **CYCLES**, sonra **RETIRED** okunurken sistem çalışmaya devam ediyorsa iki sayaç farklı zaman noktalarını temsil edebilir.

FREEZE kullanıldığında bütün sayaçlar aynı ölçüm aralığında sabit kalır.

8.3.3 CLEAR

CLEAR, W1P yani Write-One-Pulse davranışına sahiptir.

Bu bite 1 yazıldığında sayaçlar temizlenir.

Bit sürekli 1 olarak tutulmaz; yazma işlemi bir pulse olarak değerlendirilir.

Temel kullanım:

MON_CTRL.CLEAR = 1

sonrasında sayaçlar:

```
counter <- 0
```

durumuna gelir.

8.3.4 SNAPSHOT

SNAPSHOT yine W1P davranışına sahiptir.

Snapshot alındığında live sayaçların o andaki değerleri shadow register alanına kopyalanır.

Bu sayede monitoring devam ederken birbiriyle aynı zaman noktasına ait sayaç değerleri okunabilir.

Live sayaç offset'lerine:

```
+0x100
```

eklenerek ilgili shadow sayaç okunur.

Örneğin:

```
CYCLES = 0x004
```

ise snapshot karşılığı:

```
CYCLES_SHADOW = 0x104
```

adresinde okunabilir.

Snapshot mekanizmasının avantajı monitoring'i durdurmadan tutarlı bir ölçüm alınabilmesidir.

8.4 Kontrol Önceliği

Aynı cycle içerisinde birden fazla kontrol koşulu oluşursa microBee aşağıdaki önceliği uygular:

```
Reset > Clear > Snapshot > Count
```

Sayım yalnızca:

```
ENABLE = 1
```

```
FREEZE = 0
```

olduğunda gerçekleştirilir.

Bu nedenle genel davranış:

```
ENABLE & !FREEZE
```

şartına bağlıdır.

8.5 Monitoring Register Haritası

Live sayaçlar 0x0004 ile 0x0064 arasındaki memory-mapped register alanında bulunur.

Aynı sayaçların snapshot kopyaları +0x100 offset eklenerek okunabilir.

Tablo 8.1: Monitoring register'ları

Offset	Register	Erişim	Sayılan olay
0x000	MON_CTRL	RW	Bit0 ENABLE, bit1 FREEZE, bit2 CLEAR (W1P), bit3 SNAPSHOT (W1P).
0x004	CYCLES	RO	ENABLE=1 ve FREEZE=0 iken geçen core cycle sayısı.
0x008	RETIRED	RO	Mimari olarak retire edilen instruction.
0x00C	STALL	RO	Pipeline stall cycle.
0x010	FLUSH	RO	Redirect/flush olayı (trap/mret, JAL/JALR, mispredict recovery, exception).
0x014	BUBBLE	RO	Pipeline içerisinde geçerli instruction bulunmayan bubble cycle.
0x018	HAZARD	RO	Hazard-unit etkinliği (load-use stall veya forwarding seçimi).
0x01C	LOAD_USE	RO	Load-use interlock oluşumu.
0x020	BRANCH	RO	Koşullu branch instruction sayısı.
0x024	BRANCH_TAKEN	RO	Taken olarak sonuçlanan branch sayısı.
0x028	MISPREDICT	RO	Yanlış branch prediction sayısı, retire-hizalı.
0x02C	IMEM_WAIT	RO	Instruction memory request/response bekleme cycle'ları.
0x030	DMEM_WAIT	RO	Data memory request/response bekleme cycle'ları.
0x034	AXI_WAIT	RO	AXI bridge üzerinde beklenen cycle sayısı.
0x038	BUS_ERROR	RO	Native IMEM/DMEM error response sayısı.
0x03C	IRQ_TAKEN	RO	Core tarafından kabul edilen interrupt sayısı.
0x040	ALU_USE	RO	Retire edilen integer ALU instruction sayısı.
0x044	MUL_USE	RO	Retire edilen multiply instruction sayısı.
0x048	DIV_USE	RO	Retire edilen divide/remainder instruction sayısı.
0x04C	CAPABILITY	RO	Sayaç genişliği, MAP revision 2 ve destek maskesi.
0x050	LOAD_COUNT	RO	Retire edilen load instruction sayısı.
0x054	STORE_COUNT	RO	Retire edilen store instruction sayısı.
0x058	BRANCH_STALL	RO	Mispredict recovery sonrası fetch bekleme cycle'ları.
0x05C	MEM_WAIT_TOTAL	RO	IMEM_WAIT veya DMEM_WAIT aktif olan cycle sayısı.
0x060	PREDICT_CORRECT	RO	Doğru branch prediction sayısı, retire-hizalı.
0x064	SPEC_REDIRECT	RO	Taken prediction sebebiyle yapılan spekülasyon fetch redirect sayısı.
+0x100	Shadow kopya	RO	SNAPSHOT anındaki live sayaç görüntüsü.

8.6 Temel Sayaçların Anlamı

8.6.1 CYCLES

CYCLES, `ENABLE=1` ve `FREEZE=0` iken geçen toplam core cycle sayısını tutar.

Bu sayaç birçok performans metriğinin temelidir.

Örneğin CPI, IPC ve stall oranı hesaplanırken payda veya pay olarak CYCLES kullanılır.

8.6.2 RETIRED

RETIRED, çekirdeğin mimari olarak retire ettiği instruction sayısını gösterir.

Pipeline içerisine alınan her instruction retire edilmiş sayılmaz.

Örneğin:

- branch misprediction sonucu flush edilen instruction,
- exception sebebiyle iptal edilen instruction,
- spekülatif olarak alınmış yanlış-path instruction

retired sayacına eklenmez.

Bu nedenle RETIRED, programın mimari olarak tamamlanmış instruction sayısını ölçmek için kullanılır.

8.6.3 STALL

STALL, pipeline'ın ilerleyemediği cycle'ları sayar.

Stall sebepleri arasında:

- load bekleme,
- load-use hazard,
- MUL/DIV execution beklemesi,
- core dışı store beklemesi,
- memory veya bus response beklemesi

bulunabilir.

Stall sayısının yüksek olması, listedeki bekleme kaynaklarından birinin (veya birkaçının) pipeline'ı sık sık durdurduğunu gösterir; tek başına yalnızca “veri/memory bekliyor” anlamına gelmez.

8.6.4 FLUSH

FLUSH, RTL'de `pipeline_flush_redirect` veya `exception_flush` etkin olduğu cycle/olayları sayar. Bu kümeye trap/mret, JAL/JALR redirect, mispredict recovery ve exception dahildir. Spekülatif prediction redirect'ler ayrı `SPEC_REDIRECT` sayacına gider.

Flush sayısının yüksek olması control-flow yoğun bir uygulamaya, düşük prediction başarısına veya sık trap/exception'a işaret edebilir.

8.6.5 BUBBLE

Bubble, pipeline içerisinde geçerli instruction bulunmayan cycle veya slot durumudur.

Bubble doğrudan stall ile aynı şey değildir.

Bir stall sonucunda bubble oluşabilir ancak pipeline redirect veya hazard kontrolü gibi başka sebeplerle de bubble meydana gelebilir.

Bubble sayısının artması pipeline'ın teorik throughput değerinden uzaklaşmasına sebep olur.

8.6.6 HAZARD ve LOAD_USE

HAZARD, hazard biriminin etkin olduğu cycle'ları sayar: load-use stall oluşması veya operand forwarding seçiminin yapılması. Bu, retired instruction sayısı değil, hazard-unit karar/etkinlik sayacıdır.

LOAD_USE ise bunun daha özel bir alt durumudur (load-use interlock'un pipeline advance ile birlikte görüldüğü durum).

Örneğin:

```
lw  x5, 0(x10)
add x6, x5, x7
```

kodunda ikinci instruction birinci instruction'ın memory'den getireceği x5 değerine ihtiyaç duyar.

Veri zamanında hazır değilse load-use interlock oluşabilir.

Bu durum LOAD_USE sayacından takip edilebilir.

8.7 Branch Sayaçları

Branch predictor performansını incelemek için birden fazla sayaç birlikte kullanılmalıdır.

8.7.1 BRANCH

BRANCH, çalıştırılan koşullu branch instruction sayısını tutar.

Bu değer programın control-flow yoğunluğunu anlamak için kullanılabilir.

8.7.2 BRANCH_TAKEN

BRANCH_TAKEN, gerçek sonucu taken olan branch sayısını tutar.

Taken oranı:

$$\text{Taken Rate} = \frac{\text{BRANCH_TAKEN}}{\text{BRANCH}}$$

şeklinde hesaplanabilir.

8.7.3 MISPREDICT

MISPREDICT, branch predictor'ın yanlış tahmin ettiği branch sayısını tutar.

Misprediction rate:

$$\text{Misprediction Rate} = \frac{\text{MISPREDICT}}{\text{BRANCH}}$$

olarak hesaplanabilir.

8.7.4 PREDICT_CORRECT

PREDICT_CORRECT, doğru tahmin edilmiş branch sayısını tutar.

Prediction accuracy:

$$\text{Prediction Accuracy} = \frac{\text{PREDICT_CORRECT}}{\text{PREDICT_CORRECT} + \text{MISPREDICT}}$$

şeklinde hesaplanır.

BP_ENABLE=1 iken retire-hızlı koşullu branch kümesinde PREDICT_CORRECT + MISPREDICT = BRANCH eşitliği beklenir; bu durumda

$$\text{Prediction Accuracy} = \frac{\text{BRANCH} - \text{MISPREDICT}}{\text{BRANCH}}$$

aynı sonucu verir.

8.7.5 SPEC_REDIRECT

SPEC_REDIRECT, predictor'ın branch'i taken tahmin etmesi sebebiyle Fetch biriminin branch target'a spekülâtif olarak yönlendirildiği olayları sayar.

Bu olay tek başına bir hata değildir.

Prediction doğruysa spekülâtif redirect performans kazandırır.

Prediction yanlışsa daha sonra misprediction recovery gerekir.

Bu nedenle:

SPEC_REDIRECT ve MISPREDICT

sayaçlarının birlikte incelenmesi predictor davranışını anlamak için kullanışlıdır.

8.7.6 BRANCH_STALL

BRANCH_STALL, branch misprediction recovery sonrasında Fetch tarafının tekrar doğru instruction akışını oluşturmasını beklediği cycle'ları tutar.

Bu sayaç yalnızca kaç misprediction olduğunu değil, yanlış tahminlerin pipeline üzerindeki gerçek cycle maliyetini anlamaya yardımcı olur.

8.8 Memory Bekleme Sayaçları

Memory sistemi pipeline performansını doğrudan etkileyebilir.

Bu nedenle instruction ve data memory beklemleri ayrı sayaçlarda tutulur.

8.8.1 IMEM_WAIT

IMEM_WAIT, instruction fetch isteği oluşturulmuş ancak gerekli instruction henüz alınmamış cycle'ları sayar.

Bu değer yüksekse instruction memory, bus veya fetch yolu işlemci performansını sınırlıyor olabilir.

8.8.2 DMEM_WAIT

DMEM_WAIT, data memory request/response bekleme cycle'larını tutar.

Özellikle load işlemlerinde memory cevabı beklenirken pipeline stall oluşabileceği için bu sayaç CPI üzerinde doğrudan etkili olabilir.

8.8.3 MEM_WAIT_TOTAL

MEM_WAIT_TOTAL, IMEM veya DMEM tarafında memory beklemesi olan cycle'ları toplu şekilde ölçer.

Bu sayaç:

IMEM_WAIT + DMEM_WAIT

şeklinde basit toplama değildir.

Aynı cycle içerisinde hem IMEM hem DMEM wait durumu varsa cycle yalnızca bir defa sayılır.

Genel mantık:

MEM_WAIT_TOTAL = count(imem_wait_event | dmem_wait_event)

şeklinde. Buradaki | olay sinyallerinin mantıksal OR'udur; sayaç değerlerinin sayısal toplamı değildir.

8.8.4 AXI_WAIT

AXI_WAIT, core dışındaki AXI erişimlerinin bridge üzerinde beklediği cycle'ları sayar.

Bu değer özellikle external peripheral veya memory erişimlerinin sistem performansına etkisini görmek için kullanılabilir.

8.9 BUS_ERROR ve IRQ_TAKEN

8.9.1 BUS_ERROR

BUS_ERROR, IMEM fetch yanıtındaki error veya DMEM memory response error pulse'larını sayar (RTL: `imem_bus_error_pulse / dmem_bus_error_pulse`).

Bu sayaç normal benchmark çalışmasında genellikle sıfır olması beklenen sayaçlardan biridir.

Beklenmeyen artış:

- geçersiz MMIO adresi (access fault response),
- disabled peripheral erişimi,
- external/native bus error response

gibi durumların araştırılmasını gerektirebilir.

Hızasız load/store erişimleri çekirdekte misaligned exception olarak yakalanabilir ve bus'a çıkmadan trap üretebilir; bu durumda olay BUS_ERROR yerine exception/flush yoluna yansır.

8.9.2 IRQ_TAKEN

IRQ_TAKEN, yalnızca pending olan interrupt'ları değil, core tarafından gerçekten kabul edilmiş interrupt sayısını tutar.

Bu ayırım önemlidir.

Bir interrupt kaynağı uzun süre pending kalabilir ancak yalnızca trap kabul edildiğinde IRQ_TAKEN artırılır.

8.10 Execution Birimi Kullanımı

ALU_USE, MUL_USE ve DIV_USE sayaçları retire edilen instruction sayılarını tutar; cycle kullanımını değil, ilgili birime ait tamamlanan instruction/event sayısını gösterir.

Örneğin:

- yüksek ALU_USE: integer işlem yoğunluğu,
- yüksek MUL_USE: multiplication yoğunluğu,
- yüksek DIV_USE: division/remainder yoğunluğu

olarak yorumlanabilir.

Özellikle DIV_USE sayısı düşük olsa bile DIV instruction'larının latency değeri yüksek olduğu için toplam CPI üzerindeki etkisi büyük olabilir.

Bu nedenle instruction sayısı ile stall cycle sayısı birlikte değerlendirilmelidir.

8.11 Load ve Store Sayaçları

LOAD_COUNT retire edilmiş load instruction sayısını,

STORE_COUNT ise retire edilmiş store instruction sayısını tutar.

Memory instruction oranı:

$$\text{Memory Instruction Rate} = \frac{\text{LOAD_COUNT} + \text{STORE_COUNT}}{\text{RETIRED}}$$

şeklinde hesaplanabilir.

Bu değer yüksek olan uygulamalarda memory sistemi performans üzerinde daha önemli hale gelir.

8.12 CAPABILITY Register'ı

CAPABILITY, monitoring biriminin hangi özellikleri desteklediğini yazılıma bildirir. Register içeriği sayaç genişliğini, monitoring map revision değerini (REF32'de revision 2) ve desteklenen sayaç maskesini içerir.

Bu sayede yazılım belirli bir monitoring register'ının mevcut olup olmadığını sabit varsaymak yerine donanımdan okuyabilir.

8.13 Performans Ölçümleri

Ham sayaç değerleri tek başına her zaman yeterli değildir.

Birçok performans metriği bu sayaçlardan türetilir.

Tablo 8.2: Türetilen performans ölçümleri

Ölçüm	Formül	Raporlama kuralı
IPC	retired / cycles	Cycle başına tamamlanan instruction.
CPI	cycles / retired	Bir instruction için gereken ortalama cycle.
Stall oranı	stall / cycles	Pipeline'in ne kadar süre beklediğini gösterir.
Branch doğruluğu	predict correct / (predict correct + mispredict)	Predictor etkinse anlamlıdır.
Mispredict oranı	mispredict / branch	Branch'lerin ne kadarının yanlış tahmin edildiğini gösterir.
Taken oranı	branch taken / branch	Branch davranışını gösterir.
IMEM wait oranı	imem wait / cycles	Instruction memory etkisini gösterir.
DMEM wait oranı	dmem wait / cycles	Data memory etkisini gösterir.
Memory instruction oranı	(load count + store count) / retired	Programın memory yoğunluğunu gösterir.

8.13.1 IPC

IPC, Instructions Per Cycle anlamına gelir.

Formülü:

$$IPC = \frac{RETIREDD}{CYCLES}$$

şeklindedir.

IPC değeri işlemcinin bir cycle içerisinde ortalama kaç instruction tamamladığını gösterir.

microBee gibi single-issue bir pipeline için uzun workload'larda ideal durumda IPC değeri 1'e yaklaşır. Kısa benchmarklarda başlangıç doldurma ve drain etkileri nedeniyle tam 1 beklenmeyebilir.

Stall, flush, memory wait veya uzun gecikmeli instruction'lar IPC değerini düşürür.

8.13.2 CPI

CPI, Cycles Per Instruction anlamına gelir.

Formülü:

$$CPI = \frac{CYCLES}{RETIREDD}$$

şeklindedir.

IPC ile ters ilişkilidir:

$$CPI = \frac{1}{IPC}$$

Uzun workload'larda ideal single-issue durumda CPI yaklaşık 1'e yaklaşır. CPI'nin yükselmesi işlemcinin instruction başına daha fazla cycle harcadığını gösterir.

8.13.3 Stall Oranı

Pipeline'ın toplam execution süresinin ne kadarını stall durumunda geçirdiği:

$$\text{Stall Rate} = \frac{\text{STALL}}{\text{CYCLES}}$$

ile hesaplanabilir.

Yüzde olarak:

$$\text{Stall Percentage} = \frac{\text{STALL}}{\text{CYCLES}} \times 100$$

kullanılabilir.

8.13.4 Branch Prediction Doğruluğu

Doğrudan sayaçlar kullanılarak:

$$\text{Prediction Accuracy} = \frac{\text{PREDICT_CORRECT}}{\text{PREDICT_CORRECT} + \text{MISPREDICT}}$$

hesaplanabilir.

Bu değer yüksek olsa bile branch predictor'ın toplam performans etkisini anlamak için branch instruction oranına ve misprediction penalty değerine de bakılmalıdır.

8.14 Sayaçlar Birlikte Nasıl Yorumlanır?

Tek bir sayaç üzerinden performans yorumu yapmak çoğu durumda doğru değildir.

Örneğin CPI yüksekse bunun sebebi:

- memory wait,
- branch misprediction,
- DIV kullanımı,
- load-use hazard,
- external bus wait

olabilir.

Bu nedenle sayaçlar birbirleriyle ilişkilendirilerek incelenmelidir.

Örneğin:

$$\text{CPI yüksek} + \text{DMEM_WAIT yüksek}$$

ise data memory latency önemli bir bottleneck olabilir.

$$\text{CPI yüksek} + \text{MISPREDICT yüksek}$$

ise branch predictor veya branch-heavy workload incelenmelidir.

$$\text{CPI yüksek} + \text{DIV_USE yüksek}$$

ise iterative divider toplam execution süresini artırıyor olabilir.

$$\text{STALL yüksek} + \text{LOAD_USE yüksek}$$

ise compiler scheduling veya forwarding yapısı incelenebilir.

Bu şekilde monitoring birimi yalnızca performans ölçmek için değil, mikro-mimarideki darboğazın nereden kaynaklandığını anlamak için de kullanılabilir.

8.15 Ölçüm Yaparken Dikkat Edilmesi Gerekenler

İki farklı benchmark sonucunu karşılaştırabilmek için yalnızca CPI veya IPC değerlerini kaydetmek yeterli değildir.

Aşağıdaki bilgilerin de sonuç ile birlikte tutulması gerekir:

- benchmark binary'si,
- benchmark giriş verisi,
- compiler sürümü,
- compiler optimization flag'leri,
- kullanılan ISA profili,
- core konfigürasyonu,
- clock frekansı,
- IMEM/DMEM wait-state davranışı,
- AXI veya external peripheral konfigürasyonu.

Aynı C kodu farklı compiler flag'leri ile farklı instruction sayısı ve farklı memory erişim davranışı oluşturabilir.

Bu nedenle performans sonucu yalnız işlemciye ait bir değer olarak düşünülmemelidir.

Compiler, memory sistemi ve benchmark birlikte sonucu oluşturur.

8.16 CPI Hedefinin Yorumlanması

Karar matrisinde geçen:

$$\text{CPI} > 0.8$$

koşulu single-issue bir çekirdek için ayırt edici bir hedef değildir. microBee sürekli durumda tipik olarak CPI ≈ 1 veya üzerindedir; CPI'nin 0.8'in altına inmesi beklenen bir durum değildir.

CPI için genel olarak daha düşük değer daha iyi performansı ifade eder; ancak single-issue mimaride anlamlı karşılaştırma için ham sayaçlar ve türetilmiş oranlar birlikte raporlanmalıdır.

microBee single-issue yapıda olduğu için uzun workload'larda ideal IPC yaklaşık 1, ideal CPI ise yaklaşık 1 seviyesindedir. Kısa ölçüm pencerelerinde doldurma/drain etkileri sapma yaratabilir.

Bu nedenle performans sonucu raporlanırken yalnızca hedef koşulu verilmemeli, ham değerler de paylaşılmalıdır.

Önerilen raporlama:

- toplam cycle,
- retired instruction,
- CPI,
- IPC,
- stall cycle,
- flush sayısı,
- branch ve misprediction sayısı,
- IMEM/DMEM wait değerleri

şeklindedir.

Bu yaklaşım tek bir performans metriğine bakmak yerine işlemcinin gerçek davranışının anlaşılmasını sağlar.

8.17 Örnek Ölçüm Akışı

Bir benchmark'ın monitoring birimi ile ölçülmesi genel olarak aşağıdaki şekilde yapılabilir:

1. Monitoring sayaçlarını CLEAR ile sıfırla.
2. Gerekirse FREEZE değerini kaldır.
3. ENABLE bitini aç.
4. Benchmark'ı çalıştır.
5. Benchmark sonunda SNAPSHOT al veya FREEZE uygula.
6. CYCLES ve RETIRED sayaçlarını oku.
7. Stall, branch, memory ve execution sayaçlarını oku.
8. CPI, IPC ve diğer oranları hesapla.

9. Sonuçları benchmark ve compiler bilgileriyle birlikte kaydet.

Bu yapı sayesinde aynı microBee konfigürasyonu üzerinde farklı benchmark'lar veya farklı mikro-mimari değişiklikler karşılaştırılabilir.

8.17.1 Örnek: CYCLES sayacını AXI UART Lite'a yazmak

Aşağıdaki örnek REF32 adreslerini kullanır:

- Monitoring base: 0x40005000, CYCLES offset 0x04
- AXI UART Lite: 0x60000000

Akış: UART TX init, sayaçları CLEAR+ENABLE, ardından yalnız ENABLE, kısa bir iş döngüsü, FREEZE, CYCLES oku, değeri hex olarak UART'a bas.

İki ayrı MON_CTRL yazmasının nedeni: kontrol önceliğinde CLEAR, sayımından yüksektir. 0x5 (ENABLE|CLEAR) yazıldığında sayaçlar temizlenir; ardından 0x1 ile CLEAR pulse'ı bittikten sonra yalnız ENABLE=1 bırakılarak ölçüm aralığı başlatılır.

```
# -----
# microBee: CYCLES sayacını AXI UART Lite @ 0x60000000 üzerinden yaz
# MONITOR @ 0x40005000
# -----

.section .text
.globl _start
_start:
    li    sp, 0x2000FF00

    # ---- AXI UART Lite init (polling TX) ----
    li    s0, 0x60000000
    li    t1, 0x03                # CTRL: TX+RX FIFO reset
    sw    t1, 0x0C(s0)
    sw    x0, 0x0C(s0)            # CTRL: interrupt kapalı

    # ---- Monitoring: CLEAR sonra ENABLE ----
    # 0x5 = ENABLE|CLEAR: CLEAR öncelikli temizler
    # 0x1 = yalnız ENABLE: ölçüm aralığını başlatır
    li    s1, 0x40005000          # MONITOR base
    li    t1, 0x5                 # bit0 ENABLE | bit2 CLEAR (W1P)
    sw    t1, 0(s1)
    li    t1, 0x1                 # ENABLE=1, FREEZE=0
    sw    t1, 0(s1)

    # ---- Ölçülecek iş (örnek döngü) ----
    li    t0, 1000
work_loop:
    addi  t0, t0, -1
    bnez  t0, work_loop

    # ---- FREEZE, CYCLES oku ----
    li    t1, 0x3                 # ENABLE | FREEZE
    sw    t1, 0(s1)
    lw    s2, 0x04(s1)            # s2 = CYCLES

    # ---- "CYCLES=0x" yaz ----
    li    a0, 'C'
    jal   ra, uart_putc
    li    a0, 'Y'
    jal   ra, uart_putc
    li    a0, 'C'
    jal   ra, uart_putc
    li    a0, 'L'
```

```

    jal ra, uart_putc
    li a0, 'E'
    jal ra, uart_putc
    li a0, 'S'
    jal ra, uart_putc
    li a0, '='
    jal ra, uart_putc
    li a0, '0'
    jal ra, uart_putc
    li a0, 'x'
    jal ra, uart_putc

    # 8 hex hane, MSB önce
    mv a0, s2
    jal ra, print_hex32

    li a0, '\r'
    jal ra, uart_putc
    li a0, '\n'
    jal ra, uart_putc

done:
    j done

# a0 = 32-bit değer -> 8 hex karakter
print_hex32:
    addi sp, sp, -8
    sw ra, 4(sp)
    sw s3, 0(sp)
    mv s3, a0
    li t3, 28 # nibble kaydırma
hex_nibble:
    srl t4, s3, t3
    andi t4, t4, 0xF
    li t5, 10
    blt t4, t5, hex_digit
    addi a0, t4, 55 # 10..15 -> 'A'..'F'
    j hex_out
hex_digit:
    addi a0, t4, 48 # 0..9 -> '0'..'9'
hex_out:
    jal ra, uart_putc
    addi t3, t3, -4
    bgez t3, hex_nibble
    lw s3, 0(sp)
    lw ra, 4(sp)
    addi sp, sp, 8
    ret

# a0 = karakter; STAT bit3 (TX full)=0 olana kadar bekle
uart_putc:
    li t0, 0x60000000
1:
    lw t1, 0x08(t0) # STAT
    andi t1, t1, 0x08 # TX full?
    bnez t1, 1b
    sw a0, 0x04(t0) # TX_FIFO
    ret

```

Bölüm 9

Performans Raporunun Hazırlanması

Bölüm 8 monitoring biriminin yapısını, sayaçlarını ve bu sayaçlardan elde edilebilecek metrikleri tanımlar. Bu bölümde ise bu bilgilerin gerçek bir performans ölçümünde nasıl kullanılacağı anlatılır.

Performans ölçümünde amaç yalnızca tek bir CPI veya IPC değeri elde etmek değildir. Asıl amaç işlemcinin neden belirli bir performans gösterdiğini anlayabilmektir.

Örneğin iki farklı microBee konfigürasyonundan biri diğerinden daha yavaşa yalnız:

$$\text{CPI} = 1.8$$

demek yeterli değildir.

Bu farkın:

- bellek beklemesinden (IMEM_WAIT/DMEM_WAIT),
- yanlış dal tahmininden (mispredict),
- pipeline stall'dan,
- DIV gibi uzun gecikmeli (latency) instruction'lardan,
- veya compiler tarafından üretilen farklı instruction akışından

kaynaklanıp kaynaklanmadığının da incelenmesi gerekir.

Bu nedenle performans raporu genel olarak üç parçadan oluşur:

- sistemin nasıl gözlemlendiği,
- ham monitoring sayaçları,
- bu sayaçlardan türetilen performans değerleri ve yorum.

9.1 Sistem Nasıl İzlenir?

microBee performansı üç temel yöntem ile izlenebilir:

- waveform ve debug sinyalleri,
- memory-mapped monitoring sayaçları,
- simülasyon log ve trace çıktıları.

Bu yöntemler birbirinin alternatifi değildir. Her biri farklı bir soruya cevap verir.

Tablo 9.1: microBee performans gözlem yöntemleri.

Yöntem	Ne gösterir?	Ne için uygun değildir?
Waveform / dbg_*	Cycle seviyesinde PC, pipeline aşamalarındaki PC/valid durumu, register yazımı ve CSR değişimleri	Uzun benchmark sonucunu özetlemek
MMIO monitoring	Toplam cycle, retired, stall, branch ve memory bekleme sayıları	Tek bir stall'ın hangi instruction'da oluştuğunu göstermek
Simülasyon log / trace	Commit izi, test sonucu ve ISS karşılaştırması	FPGA üzerindeki fiziksel zamanlama davranışını göstermek

9.1.1 Waveform ve Debug Sinyalleri

Waveform kısa bir instruction akışının neden belirli şekilde davrandığını anlamak için kullanılır.

Örneğin:

“Bu load instruction’ından sonra neden bubble oluştu?”

sorusu waveform üzerinden rahatlıkla cevaplanabilir.

microBee içerisinde gözlem amacıyla farklı `dbg_*` sinyalleri bulunmaktadır.

Başlıca debug sinyalleri:

- `dbg_fetch_pc_o`,
- `dbg_if_pc_o`,
- `dbg_ex_pc_o`,
- `dbg_mem_pc_o`,
- `dbg_wb_pc_o`,
- `dbg_rf_we_o`,
- `dbg_rf_waddr_o`,
- `dbg_rf_wdata_o`.

Bunların yanında `mstatus`, `mcause`, `mip`, `mcycle` ve `minstret` gibi CSR değerleri de izlenebilir.

Waveform kısa süreli davranışı anlamakta oldukça kullanışlıdır. Ancak milyonlarca cycle süren bir benchmark’ı waveform üzerinden analiz etmek pratik değildir.

9.1.2 Monitoring Sayaçları

Uzun performans ölçümlerinde monitoring birimi temel yöntemlerden biridir.

Monitoring birimi:

`0x40005000`

adresinde bulunur. Register tablosundaki değerler offset’tir. Mutlak adres:

`mutlak_adres = 0x40005000 + offset`

Örneğin `CYCLES` için offset `0x04` olduğundan mutlak adres `0x40005004`’tür.

Temel ölçüm sırasında yazılım:

1. sayaçları temizler,
2. monitoring’i enable eder,
3. ölçülecek programı çalıştırır,
4. sayaçları freeze eder veya snapshot alır,
5. sayaç değerlerini okur.

Örneğin:

- `CYCLES`,
- `RETIRED`,
- `STALL`,
- `BRANCH`,
- `MISPREDICT`,
- `IMEM_WAIT`,
- `DMEM_WAIT`

gibi değerler doğrudan yazılım tarafından okunabilir.

Bu yöntemin önemli avantajı özel bir debugger gerektirmemesidir. Yazılım, işlemcinin monitoring sayaçlarını okuyabilir ve sonucu UART üzerinden dışarı aktarabilir.

UART çıktısına dikkat.

Monitoring değerleri UART üzerinden basılacaksa sayaçlar önce freeze edilmeli veya snapshot alınmalıdır.

Aksi durumda UART'a karakter basmak için çalışan instruction'lar ve UART beklemeleri de monitoring sayaçlarına dahil olur. Böylece ölçülen program ile raporlama kodunun maliyeti birbirine karışır.

9.1.3 Simülasyon Log ve Trace

microBee SoC testleri FPGA olmadan da çalıştırılabilir.

Verilator ortamında kullanılan SoC testlerinde yazılım aynı monitoring register'larını kullanabilir ve AXI UART Lite üzerinden üretilen çıktı simülasyon konsolunda görülebilir.

Bu yöntem FPGA kartına geçmeden önce:

- monitoring yazılımını doğrulamak,
- sayaç değerlerini kontrol etmek,
- farklı core konfigürasyonlarını karşılaştırmak

için kullanılabilir.

Ancak simülasyon cycle sayısı doğrudan gerçek zaman değildir.

Örneğin:

CYCLES = 1 000 000

değeri bir milyon mimari clock cycle'ı ifade eder.

Teorik çalışma süresi hesaplanmak istenirse hedeflenen clock frekansı (simülasyonda modellenen frekans veya FPGA clock frekansı) ayrıca bilinmelidir.

9.2 Hangi Yöntem Ne Zaman Kullanılmalı?

Pratikte aşağıdaki sıra kullanılabilir:

1. Önce monitoring sayaçları ile genel performans tablosunu oluşturun.
2. CPI veya IPC beklenenden farklıysa hangi sayaçların yüksek olduğuna bak.
3. Darboğaz hakkında bir fikir oluştuğunda waveform aç.
4. İlgili instruction akışını cycle seviyesinde incele.
5. Son olarak aynı testi FPGA üzerinde tekrar çalıştır.

Örneğin `DMEM_WAIT` çok yüksekse doğrudan bütün benchmark'ın waveform'unu açmak yerine memory erişimlerinin yoğun olduğu küçük bir bölge incelenebilir.

Benzer şekilde `LOAD_USE` yüksekse load instruction'larının hemen ardından gelen bağımlı instruction'lar waveform üzerinde kontrol edilebilir.

Önce sayaç ile problemi bulmak, sonra waveform ile nedenini incelemek daha verimli bir yöntemdir.

9.3 Sistem İzleme Araçları

9.3.1 AXI UART Lite ve Host Terminal

REF32 FPGA konfigürasyonunda AXI UART Lite:

0x60000000

adresinde bulunur.

50 MHz sistem clock'u için tipik baud değeri 115200 olarak kullanılabilir.

Performans sayaçlarını dışarı aktarmak için interrupt kullanılması zorunlu değildir. Basit polling TX yeterlidir.

Temel UART yazma işlemi:

1. `STAT.TX_FULL` kontrol edilir,
2. FIFO doluysa beklenir,

3. FIFO dolu değilse karakter TX_FIFO'ya yazılır.

Örneğin monitoring sonucu aşağıdaki gibi satır satır basılabilir:

```
CYCLES=0x.....
RETIRED=0x.....
STALL=0x.....
```

Tam bir örnek döküm Bölüm 9.6 içindedir. Raporla mümkün olduğunca ham hex değer de saklanmalıdır.

9.3.2 Vivado Waveform

Vivado waveform, monitoring sonuçlarında görülen anomalilerin nedenini cycle seviyesinde araştırmak için kullanılmaktadır.

Örneğin:

LOAD_USE yüksek

sonucu elde edilmişse load instruction ile onu takip eden bağımlı instruction arasındaki pipeline davranışı waveform üzerinden incelenebilir.

Waveform performans tablosunun yerine geçmez. Sayaçların gösterdiği problemin nedenini anlamak için kullanılır.

9.3.3 Hesaplama

Monitoring birimi ham sayaçları üretir.

IPC, CPI ve oranlar bu değerlerden hesaplanır.

$$IPC = \frac{RETIRED}{CYCLES}$$

$$CPI = \frac{CYCLES}{RETIRED}$$

$$Stall Rate = \frac{STALL}{CYCLES}$$

Paydanın sıfır olmadığı kontrol edilmelidir.

Özellikle:

$$RETIRED = 0$$

ise CPI hesaplanamaz.

Bu durum performans sonucu olarak değerlendirilmemeli, programın veya ölçüm kurulumunun neden instruction retire etmediği araştırılmalıdır.

9.4 Performans Raporu Nasıl Hazırlanır?

İki farklı performans sonucunu karşılaştırabilmek için ölçüm şartlarının bilinmesi gerekir.

Yalnızca:

“Konfigürasyon A daha hızlıdır.”

demek yeterli değildir.

CPI ve IPC boyutsuz performans metrikleridir; mikro-mimari karşılaştırma için aynı clock frekansını gerektirmez. Asıl koşul ölçüm penceresinin aynı program, aynı derleme ve aynı bellek/ISA modeli ile alınmasıdır.

Karşılaştırılan CPI/IPC testlerinde mümkün olduğunca:

- aynı binary,
- aynı input,
- aynı compiler,
- aynı optimization flag,
- aynı ISA,
- aynı memory davranışı

kullanılmalıdır.

Gerçek execution time veya throughput (saniye, iş/saniye) karşılaştırılırken frekansların aynı olması şart değildir; farklı frekanslarda süre

$$t = \text{CYCLES} / f_{\text{clk}}$$

ile hesaplanıp karşılaştırılabilir. Ham CPI tek başına süre demek değildir.

Bir mikro-mimari parametre karşılaştırılıyorsa ideal olarak yalnızca o parametre değiştirilmelidir.

9.4.1 Kimlik Bilgileri

Her performans raporunda ölçümün hangi sistem üzerinde yapıldığı belirtilmelidir.

Tablo 9.2: Performans raporunda tutulması gereken kimlik bilgileri.

Alan	Kaydedilecek bilgi
Ortam	Simülasyon veya FPGA
Binary	ELF / HEX adı ve input verisi
Compiler	Compiler sürümü ve optimization flag
ISA	RV32I / RV32IM / RV32IMC vb.
Profil	REF32 veya kullanılan farklı konfigürasyon
Clock	FPGA veya simülasyonda hedeflenen clock frekansı; süre için $t = \text{CYCLES} / f_{\text{clk}}$
Memory	IMEM/DMEM wait-state bilgisi
Core parametreleri	MUL, DIV, predictor, AXI vb.
MONITOR_ENABLE	Monitoring'in açık olup olmadığı

9.4.2 Ham Sayaçlar

Performans yorumundan önce ham değerler rapora yazılmalıdır.

Tablodaki offset'ler 0x40005000 tabanına eklenir. STALL için mutlak adres 0x4000500C, FLUSH için 0x40005010'dur.

Minimum olarak:

Tablo 9.3: Performans raporunda önerilen temel monitoring sayaçları.

Register	Offset	Kullanım
CYCLES	0x04	Toplam ölçüm cycle'ı
RETIRED	0x08	Tamamlanan instruction
STALL	0x0C	Pipeline stall cycle
FLUSH	0x10	Redirect / flush olayı
BUBBLE	0x14	Geçerli instruction olmayan cycle
LOAD_USE	0x1C	Load-use interlock
BRANCH	0x20	Branch sayısı
BRANCH_TAKEN	0x24	Taken branch sayısı
MISPREDICT	0x28	Yanlış prediction
IMEM_WAIT	0x2C	Instruction memory bekleme
DMEM_WAIT	0x30	Data memory bekleme
DIV_USE	0x48	Divide/remainder kullanımı

STALL, FLUSH ve BUBBLE birbirinin yerine geçmez ve toplanarak boş cycle hesabı yapılmaz.

RTL'de her biri ayrı olay bitidir; monitoring birimi bunları aynı cycle içinde birbirinden bağımsız artırabilir. Örneğin `pipeline_advance=1` iken decode/execute stall nedeniyle geçerli instruction ilerlememişse aynı cycle'da hem STALL hem BUBBLE artabilir. BUBBLE üretimi redirect/exception sırasında bastırılır; buna karşılık STALL ile FLUSH olay bitleri bağımsızdır ve aynı cycle'da birlikte 1 olabilir.

CYCLES monitoring açıkken her cycle artar. Bu yüzden:

$$\text{STALL} + \text{FLUSH} + \text{BUBBLE} = \text{CYCLES}$$

eşitliği geçerli değildir. Aynı gerekçe ile IMEM_WAIT ile DMEM_WAIT de toplanmaz; örtüşen cycle'lar için MEM_WAIT_TOTAL vardır.

İhtiyaca göre:

- PREDICT_CORRECT,
- AXI_WAIT,
- BUS_ERROR,
- IRQ_TAKEN,
- LOAD_COUNT,
- STORE_COUNT

gibi sayaçlar da rapora eklenebilir.

9.4.3 Türetilen Metrikler

Ham değerlerden aşağıdaki temel metrikler hesaplanabilir:

Tablo 9.4: Performans raporunda kullanılan temel metrikler.

Metrik	Formül
IPC	retired / cycles
CPI	cycles / retired
Stall oranı	stall / cycles
Mispredict oranı	mispredict / branch
Taken oranı	branch taken / branch
IMEM wait oranı	imem wait / cycles
DMEM wait oranı	dmem wait / cycles

Branch sayısı sıfırsa branch tabanlı oranlar hesaplanmamalıdır.

9.4.4 CPI ve IPC Nasıl Yorumlanır?

microBee single-issue bir core'dur.

İdeal sürekli instruction akışında:

$$\text{IPC} \approx 1$$

$$\text{CPI} \approx 1$$

beklenir.

Pipeline stall, memory wait, branch recovery ve uzun gecikmeli instruction'lar IPC'yi düşürür ve CPI'yi yükseltir.

Bu sebeple CPI için genel olarak:

daha düşük değer daha iyidir

denilebilir.

Ancak CPI tek başına darboğazın nedenini göstermez.

9.5 Sayaçlar Birlikte Nasıl Yorumlanır?

Performans analizi sırasında sayaçlar tek tek değil, birbirleriyle birlikte incelenmelidir.

Örneğin:

CPI yüksek + DMEM_WAIT yüksek

ise data memory erişimleri önemli bir darboğaz olabilir.

CPI yüksek + MISPREDICT yüksek

ise branch predictor veya branch yoğun program akışı incelenmelidir.

CPI yüksek + DIV_USE yüksek

ise divider gecikmesi (latency) toplam çalışma süresini etkiliyor olabilir.

STALL yüksek + LOAD_USE yüksek

ise load-use dependency ve compiler scheduling incelenebilir.

Bu yorumların hiçbiri yalnızca tek bir sayaç değerine bakılarak yapılmamalıdır.

9.6 Örnek Performans Raporu

Aşağıdaki değerler rapor formatını göstermek için verilmiştir. Gerçek bir benchmark sonucu olarak düşünülmemelidir.

Sayaçların program tamamlandıktan sonra freeze edildiği varsayılmıştır.

9.6.1 Host Çıktısı

CYCLES=0x00000B18
 RETIRED=0x000007E4
 STALL=0x00000140
 FLUSH=0x00000008
 BRANCH=0x000003E8
 MISPREDICT=0x00000004
 IMEM_WAIT=0x00000000
 DMEM_WAIT=0x00000000
 LOAD_USE=0x00000000
 DIV_USE=0x00000000

Ondalık değerler:

Tablo 9.5: Örnek monitoring çıktısının ondalık karşılıkları.

Register	Değer
CYCLES	2840
RETIRED	2020
STALL	320
FLUSH	8
BRANCH	1000
MISPREDICT	4
IMEM_WAIT	0
DMEM_WAIT	0

Türetilen değerler:

$$IPC = \frac{2020}{2840} = 0.711$$

$$CPI = \frac{2840}{2020} = 1.406$$

$$\text{Stall Rate} = \frac{320}{2840} = 0.113$$

$$\text{Mispredict Rate} = \frac{4}{1000} = 0.004$$

Bu örnekte IMEM_WAIT, DMEM_WAIT, LOAD_USE ve DIV_USE sıfırdır. Yani bu döküm memory beklemesi, load-use veya bölme gecikmesi iddiasını desteklemez.

CPI'nin 1'in üzerinde olması stall ve flush olaylarının varlığıyla uyumludur; ancak sayaçlar örtüşebildiği için CPI farkının ne kadarının stall veya flush'tan geldiği bu sayılardan tek başına ayrıştırılamaz. BRANCH_STALL bu örnekte yoktur ve 1000'lik döngü gerçek bir benchmark değeridir.

Yorum bu yüzden format örneği olarak kalmalıdır. Asıl darboğaz iddiası Bölüm 9.7 gibi kontrollü bir karşılaştırmaya dayanır.

9.7 Konfigürasyon Karşılaştırması

Karşılaştırma Embench-IoT süiti (19 benchmark) üzerinde yapılmıştır. Ortam: RV32IMC, MONITOR_ENABLE=1, 50 MHz cycle sayımı. Raporlanan CPI ve IPC süit genelinde ağırlıklı değerlerdir.

Kural: bir koşuda yalnız bir mikro-mimari parametre değişir. Binary, derleyici ve bellek modeli aynı kalır. Aksi halde kazancın hangi değişiklikten kaynaklandığı bilinemez.

Yüzde farkı (örneğin CPI için) şu şekilde raporlanır:

$$\frac{CPI_{\text{yeni}} - CPI_{\text{eski}}}{CPI_{\text{eski}}} \times 100$$

9.7.1 Neden Tek Parametre Değiştirilir?

Bir ara koşuda store kaynaklı çekirdek stall'ı kapatıldı. DMEM_WAIT yaklaşık 185 milyon cycle düştü. Aynı koşuda branch predictor fiilen kapalı kaldı (PREDICT_CORRECT=0, MISXPREDICT=0, SPEC_REDIRECT=0). Branch-stall hemen hemen aynı miktarda arttı.

Sonuç: bellek tarafı düzeldiği halde ağırlıklı CPI yaklaşık 2.27'den 2.29'a çıktı. İki değişiklik birbirini gizledi. Predictor yeniden açılınca bellek kazancı CPI üzerinde görünür oldu.

Bu yüzden tablolarda bellek deneyi ile predictor deneyi ayrı satırdır.

9.7.2 Deney 1: predictor kapasitesi (GShare kapalı)

Aynı bellek/LSU konfigürasyonu korunarak yalnız BP_ENTRIES değiştirildi. GShare kapalıdır. Tarama 8 girişten başladı; süit seçimi 128 girişli bimodal oldu. REF32 profili varsayılanı bu seçimdir (BP_ENTRIES=128).

Tablo 9.6: Embench ağırlıklı sonuçlar, GShare kapalı, yalnızca BP_ENTRIES değişir.

BP_ENTRIES	CPI	IPC	Tahmin doğruluğu	Not
8	2.240	0.446	%71.9	indeks düzeltmesi sonrası
16	2.232	0.448	%74.6	mispredict $\approx$ %9.4 azaldı
32	2.218	0.451	%76.7	32 girişli deney baseline'ı
64	2.203	0.454	%81.6	32→64 en büyük sıçrama
128	2.198	0.455	%83.3	taramadaki en düşük süit CPI'si

Yorum:

- Tablo büyüdükçe doğruluk artar, CPI düşer.
- 32→64 kazancı belirgindir (doğruluk +4.96 puan, mispredict $\approx$ %21, cycle $\approx$ %0.70).
- 64→128 kazancı küçüktür (cycle $\approx$ %0.23). Kapasitede azalan verim başlar.
- DMEM_WAIT bu taramada $\approx$ 505 milyon cycle'da sabit kaldı; fark predictor'dandır.
- nsichneu 16-entry ve 128-entry geçişlerinde geriledi. Daha büyük PHT her workload'da otomatik iyileşme demek değildir.

FPGA'da asıl seçim yalnız bu tabloya bakılmaz; LUT, FF ve Fmax ile birlikte 64 ile 128 arasında karar verilir.

9.7.3 Deney 2: GShare, BP_ENTRIES=128

Kapasite 128'de sabitlenip yalnız GShare ve GHR_WIDTH değiştirildi.

Tablo 9.7: Embench ağırlıklı sonuçlar, BP_ENTRIES=128, GShare taraması.

GShare	GHR	CPI	IPC	Doğruluk
kapalı	—	2.198	0.455	%83.3
açık	7	2.212	0.452	%78.8
açık	3	2.204	0.454	%81.0
açık	2	2.202	0.454	%81.7

Yorum:

- Bu süitte GHR kısaldıkça sonuç iyileşir: GHR=7 → CPI 2.212, GHR=3 → 2.204, GHR=2 → 2.202 (her adımda CPI düşer, doğruluk artar).
- GHR=2, denenen saf GShare konfigürasyonlarının en iyisidir.
- Yine de 128-entry bimodal'i geçemez: CPI yaklaşık %0.20 daha yüksek, doğruluk 1.62 puan daha düşük.
- nettle-sha256 ve edn global history'den yararlanır; slre, nbody, minver bimodal'de daha iyidir.

Süit genelinde bu taramanın seçimi **128-entry, GShare kapalıdır**; REF32 profili varsayılanı budur. FPGA LUT/FF/Fmax raporu 64 ile 128 arasında farklı bir ürün kararı verirse o değer ayrıca belgelenir. Tek workload için GShare kazansa bile rapor süit ağırlıklı CPI'ye bakarak yazılır.

Tablo 9.8: Predictor deneylerinin kısa özeti.

Sonuç	Neden?
BP_ENTRIES 8→128 CPI'yi düşürür	Daha büyük PHT aliasing'i azaltır; doğruluk artar.
64→128 kazancı küçüktür	Kapasitede azalan verim; alan/Fmax maliyeti artar.
GShare (bu süitte) bimodal'i geçemez	Kısa GHR bile süit ağırlıklı CPI'yi iyileştirmez.

9.7.4 Deney Sonucunun Özeti

Bu kampanyanın geçerli özeti şudur:

Aynı Embench süiti ve aynı bellek modeli ile BP_ENTRIES 8'den 128'e çıkarıldı; ağırlıklı CPI 2.240'tan 2.198'e indi, predictor doğruluğu %71.9'dan %83.3'e çıktı. 64'ten sonra kazanç azaldı. Aynı 128-entry tabloda GShare (GHR=2, 3, 7) süit CPI'sini iyileştirmede. İki parametrenin aynı anda değiştiği store-stall koşusu ise DMEM kazancını gizlediği için karşılaştırmaya alınmadı.

9.8 Sık Yapılan Hatalar

9.8.1 MONITOR_ENABLE = 0

MONITOR_ENABLE=0 olduğunda monitoring bloğu sentezlenmez. Request router 0x40005000 penceresini geçerli hedef olarak seçmez; erişim error response / access fault ile sonuçlanır.

Bu durumda sayaçlar ne artar ne okunur. Core tarafında access fault görülebilir ama BUS_ERROR monitoring sayacının arttığı söylenemez: o register zaten yoktur.

Geçerli CPI/IPC raporu ancak `MONITOR_ENABLE=1` ve `MON_CTRL` enable iken oluşturulur.

9.8.2 Sayaç Doyumu (Saturation)

REF32 monitoring sayaçları:

`0xFFFF_FFFF`

değerinde doyuma ulaşır (saturate).

Bu değere ulaşan sayaç daha fazla artmaz.

Bu durumda ölçüm penceresi çok uzundur ve gerçek olay sayısı artık bilinmemektedir.

Çözüm olarak:

- ölçüm süresi kısaltılabilir,
- ara snapshot alınabilir,
- benchmark parçalara bölünebilir.

9.8.3 Freeze veya Snapshot Kullanmamak

Live sayaçlar okunurken core çalışmaya devam ederse sayaç değerleri farklı zaman noktalarına ait olabilir.

Örneğin önce `CYCLES`, birkaç instruction sonra `RETIRED` okunursa iki değer aynı anda alınmış değildir.

Bu sebeple ölçüm sonunda:

- `FREEZE` uygulanmalı

veya:

- `SNAPSHOT` alınmalı ve shadow register'lar okunmalıdır.

9.8.4 UART Çıktısını Ölçüme Dahil Etmek

Monitoring sonuçlarını terminale yazdırmak da instruction çalıştırır.

Özellikle UART FIFO dolduğunda polling sebebiyle birçok cycle harcanabilir.

Sayaçlar açık bırakılırsa bu cycle'lar ölçülen benchmark'ın parçasıymış gibi görünür.

Bu nedenle:

`Benchmark -> FREEZE/SNAPSHOT -> UART Print`

sırası kullanılmalıdır.

9.8.5 Farklı Binary'leri Core Karşılaştırması Sanmak

Aynı C kodu:

`-00`

ve:

`-02`

ile farklı instruction akışına dönüşebilir.

Bu durumda `RETIRED`, branch sayısı ve memory erişimleri değişebilir.

Dolayısıyla iki farklı binary kullanılarak yapılan test doğrudan mikro-mimari karşılaştırması değildir.

9.8.6 Simülasyon Cycle'ını Gerçek Zaman Sanmak

Monitoring sayacı cycle sayar.

FPGA'da 50 MHz kullanılıyorsa teorik execution süresi:

$$t = \frac{\text{CYCLES}}{50\,000\,000}$$

şeklinde hesaplanabilir.

Ancak simülasyonun bilgisayar üzerinde kaç saniyede tamamlandığı bundan farklıdır.

Simülasyonun duvar saati süresi ile çekirdeğin simüle edilen çalışma süresi birbirine karıştırılmamalıdır.

9.8.7 Tek Metriğe Bakmak

Yalnızca CPI veya IPC değeri darboğazı göstermez.

Örneğin branch sayısı yüksek olabilir ancak:

$$\text{MISPREDICT} = 0$$

ise predictor performans probleminin temel sebebi değildir.

Aynı şekilde `DIV_USE` düşük olabilir ancak divider gecikmesi (latency) yüksek olduğu için az sayıdaki `DIV` instruction toplam süreyi belirgin şekilde artırabilir.

Bu nedenle ham sayaçlar birlikte yorumlanmalıdır.

9.8.8 Live ve Shadow Sayaçları Karıştırmak

Snapshot sonrasında tutarlı ölçüm shadow register alanındadır.

Örneğin:

$$\text{CYCLES live} = 0x004$$

iken:

$$\text{CYCLES shadow} = 0x104$$

adresinden okunur.

Snapshot alıp daha sonra live register okumak snapshot anındaki değeri vermez.

9.9 Minimum Geçerli Performans Raporu

Bir performans sonucu en az aşağıdaki bilgileri içermelidir:

- test ortamı,
- kullanılan binary,
- compiler ve flag bilgisi,
- core konfigürasyonu,
- ham `CYCLES` ve `RETIRED`,
- gerekli monitoring sayaçları,
- CPI ve IPC,
- sayaçlara dayanan kısa performans yorumu.

Bu bilgiler olmadan yalnızca:

$$\text{“CPI} = 1.4\text{”}$$

şeklinde verilen bir sonuç başka bir testle sağlıklı biçimde karşılaştırılmaz.

Bölüm 10

Debug Portları

Bölüm 8 ve Bölüm 9'da sayaçlar ve performans raporunun hazırlanması anlatıldı. Bu bölüm çekirdeğin dışarı verdiği gözlem portlarını konu alır. Amaç JTAG veya harici bir debug köprüsü tanımlamak değildir. Amaç, simülasyonda ve FPGA waveform'unda boru hattının hangi aşamada hangi PC'yi işlediğini, yazmaç dosyasına ne yazıldığını ve trap CSR'lerinin anlık değerlerini gözlemlemektir.

Bu portlar `tmcu_core` üzerinde tanımlıdır. `tmcu_soc` ve `tmcu_top` portları üst seviyeye çıkarır. `DEBUG_ENABLE=0` iken çıkışlar 0'a çekilir ve Vivado IP'te portlar gizlenebilir. Vivado Block Design simülasyonunda `tmcu_soc_0` örneğinin portları, Verilator `tmcu_top` simülasyonunda ise çekirdek hiyerarşisi üzerinden görülür (`DEBUG_ENABLE` açıkken).

10.1 Debug Portlarının Yapısı

Çekirdekte iki ayrı gözlem ailesi vardır. Bunları karıştırmamak gerekir.

- **dbg_* portları.** `DEBUG_ENABLE` parametresi ile açılır (varsayılan 1). Yalnızca gözlem içindir; çekirdek davranışını değiştirmez. Kapalıyken çıkışlar sabittir (0).
- **trace_* portları.** `tmcu_core` ve `tmcu_top` üzerinde her zaman üretilir. SoC sarmalayıcısında ise `TRACE_ENABLE` parametresi ile dışarı verilir. `tmcu_soc` varsayılanında bu parametre 0'dır. Kapalıyken SoC çıkışları sabittir (`trace_valid_o = 0`).

Bu bölümün asıl konusu `dbg_*` ailesidir. `trace_*` ailesi isteğe bağlı bir commit/trace gözlem arayüzüdür; Bölüm 10.1.3'de kısaca özetlenir.

10.1.1 Boru hattı PC gözlemi

Beş PC çıkışı, o cycle'daki fetch adresini ve dört pipeline register PC'sini yansıtır. `dbg_fetch_pc_o` bir pipeline stage register'ı değildir; IMEM getirme PC'sidir (`fetch_pc_q`). Diğer dördü ilgili pipeline register'larından gelir. Port isimleri aşama adlarının kısaltmasıdır: `if` IF/ID (decode), `ex` ID/EX (execute), `mem` EX/MEM, `wb` MEM/WB.

Tablo 10.1: Debug PC portları ve RTL kaynağı.

Port	Genişlik	Kaynak / aşama
<code>dbg_fetch_pc_o</code>	32	<code>fetch_pc_q</code> (getirme)
<code>dbg_if_pc_o</code>	32	<code>if_id_pc_q</code> (decode)
<code>dbg_ex_pc_o</code>	32	<code>id_ex_pc_q</code> (execute)
<code>dbg_mem_pc_o</code>	32	<code>ex_mem_pc_q</code> (memory)
<code>dbg_wb_pc_o</code>	32	<code>mem_wb_pc_q</code> (writeback)

Geçerlilik bitleri “bu aşamada bir komut var mı?” sorusuna cevap verir. Fetch için ayrı bir valid portu yoktur; `dbg_fetch_pc_o` her cycle yayınlanır.

Tablo 10.2: Debug geçerli bitleri.

Port	Kaynak	Anlam
dbg_if_valid_o	if_id_valid_q	Decode aşamasında komut var.
dbg_ex_valid_o	id_ex_valid_q	Execute aşamasında komut var.
dbg_mem_valid_o	ex_mem_valid_q	Memory aşamasında komut var.
dbg_wb_valid_o	mem_wb_valid_q	Writeback aşamasında komut var.

dbg_fetch_pc_o aynı zamanda IMEM istek adresidir (`imem_req_addr_o = fetch_pc_q`). IMEM kelime (32-bit) getirdiği için istek kabul edilince (`imem_req_valid & imem_req_ready`) `fetch_pc_q` her zaman +4 artar. Bu, mimari instruction PC'sinin her zaman +4 ilerlediği anlamına gelmez; sıkıştırılmış komutlar için mimari sonraki PC (+2/+4) decode/pipeline yolunda ayrı tutulur.

Redirect'te fetch adresi kelime hizalı tabana yazılır (`{target[31:2], 2'b00}`). Hedef 2-byte hizalıysa (`target[1]=1`) ilk IMEM yanıtının alt yarım kelimesi atlanır (`fq_fill_skip_lo`); böylece C extension ile uyumlu 2-byte hedefler desteklenir. İstek yoksa veya bellek hazır değilse `fetch_pc_q` donar. Fetch satırına bakarken "PC değişti mi?" ve sonraki aşamanın `valid` bitine birlikte bakılır. Bir aşamanın `valid` biti 0 ise o satırdaki PC eski bir değer taşıyor olabilir.

10.1.2 Yazmaç dosyası yazma portu

Çekirdek, yazmaç dosyasının *yazma portunu* dışarı verir. Tüm 31 yazmaçın içeriğini okuyan bir tarama yolu yoktur. Tablo 3.2 ile aynı sinyallerdir.

Tablo 10.3: Yazmaç dosyası debug yazma portu.

Port	Kaynak	Anlam
dbg_rf_we_o	rf_write_enable	<code>mem_wb_valid & mem_wb_rd_write & ~mem_wb_exception_valid</code>
dbg_rf_waddr_o	mem_wb_rd_addr_q	Hedef yazmaç
dbg_rf_wdata_o	mem_wb_result_q	Yazılan 32-bit değer

`mem_wb_result_q` load sonucu, ALU sonucu, sonraki PC veya CSR okuma sonucundan gelir. İstisna sırasında yazma kapalıdır.

x0 hedefi yazmaç dosyasında yok sayılır (`rd_write & (rd_addr != 0)`). Debug portu yazma-enable'ı x0 için ayrıca maskeleyemez. Bu nedenle dalga formunda `we=1` ve `waddr=0` görülse bile gerçek RF write olmayabilir; dosyaya yazma olmamıştır. İz portu (`trace_rd_write`) ise x0'ı maskeler.

Gölge bank açıksa (`SHADOW_BANK_ENABLE=1`; REF32 SoC/FPGA profilinde varsayılan 0) donanım trap girişinde shadow bank seçer. Debug portu yine *o cycle*'da *yazılan* adresi ve veriyi gösterir; hangi bankanın seçili olduğu ayrı bir `dbg_` sinyali değildir.

10.1.3 CSR anlık görüntüsü

CSR bloğu (`tmcu_csr_trap`), seçili CSR değerlerini (örneğin `mstatus`, `mepc`, `mcause`) debug portlarına çıkarır. Bu, tüm mimari register dosyasını dışarı vermek değildir; CSR adres uzayına MMIO ile bakmak da değildir. Portlar ayrı shadow register değil, ilgili CSR state'inin anlık gözlem değerleridir.

Tablo 10.4: Debug CSR portları.

Port	Genişlik	Kaynak
dbg_csr_mstatus_o	32	mstatus_q (tutulan alanlar: MIE bit3, MPIE bit7, MPP bit[12:11]=11)
dbg_csr_mie_o	32	mie_q
dbg_csr_mtvec_o	32	mtvec_q
dbg_csr_mscratch_o	32	mscratch_q
dbg_csr_mepc_o	32	mepc_q
dbg_csr_mcause_o	32	mcause_q (bit31 kesme, [30:0] kod)
dbg_csr_mtval_o	32	mtval_q
dbg_csr_mip_o	32	mip_value
dbg_csr_mcycle_o	32	mcycle_q[31:0]
dbg_csr_mcycleh_o	32	mcycle_q[63:32]
dbg_csr_minstret_o	32	minstret_q[31:0]
dbg_csr_minstreth_o	32	minstret_q[63:32]

Reset sonrası `mstatus = 0x00001800` (MPP=M), `mtvec = 0x80000000` (REF32 MTVEC_RESET). `mtvec` yazımında MODE bitleri sıfırlanır; trap vektörü her zaman doğrudan kipte `{mtvec[31:2], 2'b00}` adresidir.

Bu CSR kopyaları, monitoring birimindeki CYCLES ve RETIRED MMIO sayaçları ile aynı register'lar değildir (Ek A).

10.1.4 İsteğe bağlı iz (TRACE_ENABLE)

`tmcu_core` her cycle şu iz kaydını üretir:

```
trace_valid = interrupt_trace_pending_q | mem_wb_valid
```

Normal WB commit izi `mem_wb_valid` ile üretilir. Kesme izi ise bir cycle gecikmeli kaydedilir (`interrupt_trace_pending_q <= interrupt_trap_valid`); böylece interrupt accept ile aynı event akışında, fakat bir cycle kaydırılmış olarak görünür. Kayıt alanları PC, komut sözcüğü, `rd` yazması (`x0` hariç), bellek erişimi ve trap bilgisi içerir.

SoC varsayılanında (`TRACE_ENABLE=0`) bu portlar dışarı bağlanmaz. FPGA'da izi açmak LUT/FF, routing ve istenirse ILA kapasitesi ister; sinyallerin mutlaka dış FPGA pinlerine çıkarılması gerekmez. Ders ölçümlerinde asıl yol monitoring MMIO'sudur; iz, kısa wave-dump veya özel bir iz toplayıcı içindir.

10.2 Debug Portlarının Kullanımı

10.2.1 Ne zaman dbg_*, ne zaman monitoring?

Kısa bir soru, yanlış komut veya beklenmeyen bir dallanma varsa önce `dbg_*` bakılır. Milyon cycle'lık bir benchmark'ın CPI'si aranıyorsa monitoring sayaçları kullanılır. İkisi birbirinin yerine geçmez: debug portu anlık gözlem sağlar; monitoring toplanmış sayaç değerleri üretir.

10.2.2 Vivado Block Design simülasyonu

Arty A7 BD sarmalayıcısında çekirdek `tmcu_soc_0` örneğidir. Hazır dalga betiği:

```
scripts/wave_arty_bd_hello.tcl
```

Üst modül `tb_tmcu_arty_bd_hello` olmalıdır. Betik üç debug grubu ekler:

- 09_Debug_PC — beş PC ve dört valid.
- 10_Debug_RF — `we`, `waddr`, `wdata`.
- 11_Debug_CSR — on iki CSR kopyası (`mstatus ... minstreth`).

Sinyal yolu önce sarmalayıcı portlarından aranır:

```
/tb_tmcu_arty_bd_hello/dut/tmcu_arty_bd_i/tmcu_soc_0/dbg_*
```

Port yoksa hiyerarşide ilgili çekirdek örneğine (`inst`) inilir.

10.2.3 Verilator / `tmcu_top`

SoC sarmalayıcısı olmayan simülasyonda portlar `tmcu_core` üzerindedir ve `tmcu_top` bunları dış arayüze aktarır. Dalga görüntüleyicide (GTKWave, Surfer) aynı isimleri ekleyin. Reset bırakıldıktan sonra (`rst_ni=1`) `dbg_fetch_pc_o` reset vektörünü göstermelidir. REF32'de `RESET_VECTOR = 0x80000000` (IMEM tabanı).

10.2.4 Okuma düzeni

Bir cycle'ı okurken sıra şudur:

1. Reset ve saat: `rst_ni=0` iken PC'ler anlamsızdır; `rst_ni=1` olduğunda reset bırakılmıştır.
2. Fetch PC: çekirdek hangi adresi istiyor?
3. IF valid ve PC: decode'a ne girdi?
4. EX, MEM, WB: komut hangi aşamada?
5. `dbg_rf_we_o`: bu cycle yazmaç güncellendi mi?
6. CSR: trap olduysa `mcause` ve `mepc` değişti mi?

10.3 Debug Portlarının Örnekleri

Aşağıdaki örnekler simülasyon dalga formundan okunacak davranışlardır. Adresler REF32 haritasıdır (Ek A).

10.3.1 Reset sonrası ilk getirme

Reset bırakılınca (`rst_ni=1`) `dbg_fetch_pc_o` `0x80000000` olur. IMEM bu adresten ilk komutu verir. IMEM latency ve fetch kuyruğu doldurma süresine bağlı olarak aynı PC daha sonra `dbg_if_pc_o` üzerinde görünür; `dbg_if_valid_o` 1'e çıkar. Bu, boru hattının dolduğunu gösterir. `fetch_pc_q` IMEM kelime getirdiği için kabul edilen her istekte +4 ilerler. Mimari sıradaki instruction adresi ise sıkıştırılmış komutta +2, 32-bit komutta +4 (veya tahmin edilen dal hedefi) olur; bu değer pipeline içindeki `pc_next` yolunda tutulur.

10.3.2 Bir ALU yazması

`addi x3, x0, 1` commit olduğunda:

- `dbg_wb_valid_o = 1`,
- `dbg_wb_pc_o = 0` komutun PC'si,
- `dbg_rf_we_o = 1`,
- `dbg_rf_waddr_o = 3`,
- `dbg_rf_wdata_o = 1`.

Bu örnekte trap olmadığı için `mcause` değişmez. Yazma yoksa ya komut henüz WB'de değildir ya da istisna yazmayı kesmiştir. `dbg_rf_wdata_o` `we=0` iken de bir değer taşır; yoruma `we` ile birlikte bakılır.

10.3.3 Yükleme

`lw` için WB'deki `wdata`, bellekten (veya MMIO'dan) gelen yüküdür. Adres yolu `dbg_*` portunda yoktur; hangi adresten okunduğu EX/MEM iç sinyallerinde veya iz portunun `trace_mem_*` alanlarındadır. Yükleme `memory_stall` ile uzarsa WB valid'i gecikir; stall devam ettiği cycle'larda komut `dbg_mem_pc_o/dbg_mem_valid_o` üzerinde görünür.

10.3.4 Dallanma

Alınan bir dalda fetch PC, tahmin veya düzeltme ile hedefe atlar. Yanlış tahmin EX'te çözülünce branch'ten daha genç olan IF/ID ve ID/EX geçerli bitleri temizlenir; dal komutunun kendisi MEM/WB yoluna ilerleyebilir. MEM/WB'deki daha eski komutlar bu recovery flush'ından etkilenmez. Flush olay sayımı için monitoring `FLUSH` sayacı vardır; debug portu ise o cycle'da hangi aşama PC'lerinin geçersiz hale getirildiğini gösterir.

10.3.5 Trap

Synchronous exception (örneğin `ecall`) ile interrupt aynı trap CSR portlarını kullanır, ancak `mepc` semantiği farklıdır:

- `dbg_csr_mcause_o` ilgili koda geçer (Ek A, Tablo A.11 ve Tablo A.12),
- Exception'da `dbg_csr_mepc_o` faulting instruction PC'sini; interrupt'ta bir sonraki mimari PC'yi (`fetch_pc_q`) tutar (`mepc[0]=0`),
- `dbg_csr_mstatus_o` içinde MIE kapanır, MPIE önceki MIE'yi saklar, MPP=11 kalır,
- Fetch PC `mtvec` tabanına gider (`{mtvec[31:2], 2'b00}`).

`mret` sonrası `mepc`'ye dönüş, $MIE \leftarrow MPIE$ ve $MPIE=1$ aynı CSR portlarından okunur. Gölge bank açıksa trap'te shadow bank'a, `mret`'te normal bank'a dönülür; bu seçim `dbg_*` üzerinde ayrı bir tel değildir.

10.3.6 UART'a yazılan bir karakter

Yazılım `0x60000004` adresine (AXI UART Lite TX_FIFO) bir `sw` yaptığında store instruction RF write yapmaz; debug RF portu bu komut için yazma göstermez. Çevresel yazma, bellek/MMIO yolundadır. Karakterin git-tiğini doğrulamak için AXI UART Lite STAT bit 3 (TX FIFO full=0) ve/veya simülasyon UART modelinin TX kuyruğuna bakılır. Debug portu “çekirdek hangi komuttaydı?” sorusunu cevaplar; “UART ne gönderdi?” sorusunu cevaplamaz.

10.4 Debug Portlarının Sınırlamaları

1. **RISC-V Debug Specification değildir.** JTAG, halt, abstract command, program buffer veya harici debugger yok. Çekirdek çalışmayı durdurmaz.
2. **Tüm yazmaçlar görünmez.** Yalnızca yazma portu vardır. Debug portları üzerinden `x5`'in mevcut değerini keyfi anda okuyamazsınız; yalnız `x5`'e yapılan sonraki yazmayı gözleyebilir veya yazılım değeri bellek/U-ART'a dökülebilir.
3. **Fetch valid yoktur.** `dbg_fetch_pc_o` her cycle vardır; yeni getirme ile duran fetch PC'si bu porttan ayırt edilmez.
4. **`mcycle` CSR'dir, `CYCLES MMIO` değildir.** Debug portundaki `mcycle/minstret` mimari sayaçlardır. Monitoring `CYCLES/RETIRED` ayrı register'lardır; freeze ve clear yalnızca MMIO tarafını etkiler.
5. **`dbg_csr_mstatus_o` tam `mstatus` değildir.** Port yalnız implemented alanları yansıtır: yazılabilen alanlar MIE ve MPIE'dir; MPP her zaman 11'e zorlanır. Okunan 32 bitin diğer bitleri sıfırdır (reset değeri `0x00001800`).
6. **Trap vektörü yalnızca Direct kip.** `mtvec[1:0]` yazılsa bile 00'a kırılır. `mtvec + 4*cause` davranışı yoktur.
7. **SoC'de `TRACE_ENABLE` varsayılan kapalıdır.** FPGA BD iz portlarına bakmadan önce parametrenin 1 olduğunu doğrulayın. `tmcu_top` simülasyonunda iz her zaman vardır.
8. **`DEBUG_ENABLE` debug pinlerini kapatır.** Varsayılan 1'dir. 0 yapıldığında `dbg_*` çıkışları 0 olur ve Vivado Customize IP'te portlar gizlenir. Waveform gözlemi için açık tutulması önerilir; gözlem portları gerekmiyorsa kapatılabilir.
9. **FPGA'da pin maliyeti.** Tüm `dbg_*` sinyallerini karta çıkarmak pratik değildir. Kullanım yeri simülasyon waveform'u veya ILA'ya seçilmiş birkaç işaretir. Bu depo varsayılan Vivado akışına ILA çekirdeği eklemeyiz.
10. **Zaman çözünürlüğü cycle'dır; toplam değildir.** Uzun bir Embench koşusunu debug portundan CPI'ye çevirmek pratik değildir; bu amaç için monitoring kullanılmalıdır (Bölüm 9).
11. **Gözlem, kontrol değildir.** Bu sinyallere yazmak mümkün değildir. Kesme, stall veya freeze için ilgili MMIO ve pinler kullanılır.

Kısa ayırım:

`dbg_*`: anlık boru hattı / CSR gözlemi; monitoring: toplanmış sayaçlar; `trace_*`: isteğe bağlı commit/trace arayüzü.

Bölüm 11

rv32 Emülatörü ve Assembly Geliştirme Ortamı

Bu bölüm, EHB326 kapsamında assembly programı yazan öğrencinin rv32 aracını kurmasını, programını çalıştırmasını ve elde ettiği sonucu açıklamasını amaçlar. İzlenen sıra; kaynak kodun hazırlanması, assembly işlemi, yazmaç ve bellek gözlemi, hata ayıklama ve bellek görüntüsü üretimidir.

Araç davranışı yerel rv32 1.1.0 uygulaması ve incelenen kaynak kod üzerinden doğrulanmıştır. Ana kitap için kullanılan referans *EHB326_Book_MCU.pdf*, Rev. P1.0'dır. Bu metindeki bölüm numarası birleştirme önerisidir; ana kitabın bölüm sayacıları kullanılarak değiştirilebilir.

Belge Kullanım Kuralları

Kitabın Bölüm 3–10 ve Ek A'sı microBee donanımını tanımlar. Buradaki ekranlar, kısayollar ve örnek adresler ise rv32 yazılım ortamına aittir. Bir ISA işleminin desteklenmesi, çevre birimi register haritasının veya çevrim zamanlamasının da aynı olduğu anlamına gelmez.

Tablo 11.1: Bu bölümde kullanılan bilgi sınıfları

Sınıf	Anlam
Araçta doğrulanan	İncelenen rv32 sürümünde gözlenen veya kaynak kodla doğrulanan davranış.
Yapılandırılabilir	rv32 arayüzü veya desteklenen board.toml anahtarlarıyla seçilen değer.
REF32 bilgisi	Ana kitabın microBee-REF32 profiline ait donanım bilgisi.
Ders kapsamında seçilecek	Teslim dosyaları, kart ve hedef araç akışı gibi öğretim elemanının belirleyeceği koşul.

Öğrenme çıktısı. Öğrenci, bir programın hangi instruction ile hangi yazmacı veya bellek adresini değiştirdiğini gösterebilmeli; rv32'de gözlenen sonuç ile donanımda ölçülen zamanlamayı ayırt edebilmelidir.

Örneklerin kapsamı. Bu paket içindeki programlar rv32 için hazırlanmıştır. REF32'de çalıştırılacak sürümler adres yerleşimi, başlangıç adresi, MMIO erişimi ve durma yöntemi bakımından ayrıca uyarlanmalıdır. Karşılaştırma Bölüm 11.10'da verilmiştir.

11.1 Giriş

11.1.1 Nedir?

rv32; RV32IMC *Zicsr* *Zifencei* komut kümesini destekleyen bir emülatör, assembler ve debugger uygulamasıdır. Kaynak kod aynı pencerede yazılır ve makine koduna dönüştürülür. Yazmaçlar, veri belleği ve komut encoding'i program ilerletilirken incelenebilir. Masaüstü uygulamasının adı **rv32-gui**, terminal uygulamasının adı **rv32**'dir.

11.1.2 Neyi Amaçlar?

Bir assembly satırının etkisini görünür hâle getirir. Öğrenci, aritmetik sonucun yazmaca ne zaman geçtiğini, store işleminin hangi adrese yazdığını ve dallanmanın hangi koşulda alındığını gözler. Ayrı bir RISC-V derleyici zinciri kullanmadan küçük assembly deneyleri yapılabilir.

11.1.3 Nasıl Kullanılır?

1. Çalışılacak makineyi ve örnek kaynak dosyasını seçin.
2. Kaynağı düzenleyip kaydedin; F5 ile assemble edin.
3. F8 ile adım ilerletin veya F9 ile çalıştırın.
4. Beklenen yazmaç, bellek ve Console değerlerini kontrol edin.
5. Gerekliyse kaynağı düzeltin; tekrar assemble edin.
6. Dersin istediği bellek görüntülerini uygun boyutlarla dışa aktarın.

11.1.4 Sınırları Nelerdir?

rv32 bir RTL simülatörü değildir. FPGA pinleri, bus handshake'leri, bellek beklemeleri, timing closure ve fiziksel clock burada doğrulanmaz. Aracın Pipeline görünümü öğretim amaçlıdır; hazard, stall ve forwarding modeli içermez. ELF yükleme, kayan nokta komutları ve user/supervisor mode desteği bulunmaz.

Kitabın Bölüm 2.4'ünde simülasyon ortamları tanıtılırken bu ayrım kullanılabilir: rv32 programın işlevsel etkisini izlemeye; RTL simülasyonu ise microBee'nin donanım sinyallerini ve clock temelli davranışını doğrulamaya hizmet eder. Bölüm 8–9'daki donanım performans ölçümleri için ilgili RTL/FPGA ortamı kullanılmalıdır.

11.2 Kurulum ve İlk Açılış

11.2.1 Windows

1. Ders için sağlanan rv32-windows-x64.zip paketini yazma izniniz olan bir klasöre çıkartın.
2. rv32-gui.exe dosyasını çalıştırın. DLL dosyalarını ve platforms klasörünü uygulamanın yanında tutun.
3. Kaynak dosyanızı Ctrl+O ile açın. Kaydetmek için Ctrl+S kullanın.

11.2.2 Linux

Kurulu uygulama menüden veya **rv32-gui** komutuyla açılır. AppImage ve projeye sağlanan install.sh dosyası kullanılıyorsa aşağıdaki komutta gerçek dosya yollarını yazın:

```
bash /path/to/install.sh ~/Downloads/rv32-x86_64.AppImage
```

Kurulum kullanıcı dizini altında yapılır. Installer AppImage içeriğini açarak FUSE gereksinimini aşar. FUSE desteği bulunan sistemlerde AppImage'ye çalıştırma izni verilerek doğrudan da açılabilir. Kaynaktan oluşturulmuş bir debug build, depo kökünden **./build/debug/rv32-gui** ile çalıştırılır.

11.2.3 Çalışma Dosyalarının Düzeni

```
lab01/
  first_steps.s
  board.toml
  exports/
```

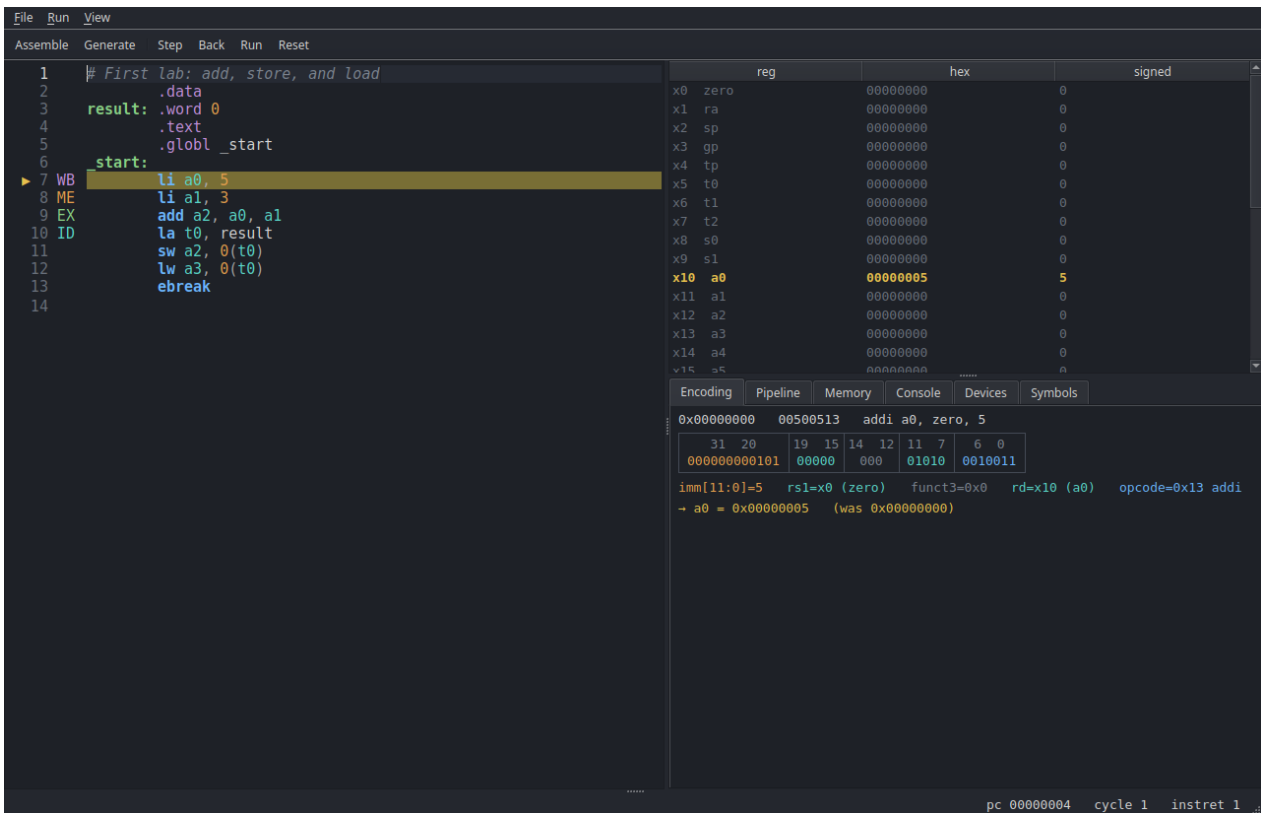
.s düzenlenebilir assembly kaynağıdır. **.toml** makine yapılandırmasıdır. **.mem** bellek görüntüsüdür. Bellek görüntüsü üretildikten sonra kaynak dosya saklanmalıdır. Kitap kökündeki **examples/rv32** dizini dört başlangıç programı ve örnek board dosyası içerir; bunlar lab01 çalışma klasörüne kopyalanarak kullanılabilir.

11.2.4 Açılış Sorunları Nasıl Çözülür?

Tablo 11.2: Kurulum sırasında sık görülen durumlar

Durum	Kontrol
Windows uygulaması açılmıyor	ZIP'in tamamının çıkartıldığını; DLL ve platforms klasörünün taşınmadığını kontrol edin.
Linux komutu bulunamıyor	Uygulamanın gerçek yolunu kullanın veya kullanıcı kurulumunun PATH ayarını kontrol edin.
AppImage açılmıyor	Sağlanan installer ile kurulumu deneyin; doğrudan çalıştırmada izin ve FUSE desteğini kontrol edin.
Dosya bulunamıyor	Ctrl+O ile gerçek .s dosyasını seçin; terminalde çalışma dizinini kontrol edin.

11.3 Arayüz ve Çalıştırma Denetimleri



Şekil 11.1: İlk instruction write-back aşamasına ulaştığında a0 yazmacı 5 olur. Encoding sekmesi gerçekleşen işlemi gösterir.

Tablo 11.3: Temel kısayollar

Kısayol	İşlev
F5 — Assemble	Editördeki kaynağı assemble eder ve yükler.
F8 — Step	Bir pipeline çevrimi ilerler.
F6 — Back	Bir pipeline çevrimini, o çevrimdeki commit etkisiyle birlikte geri alır.
F9 — Run	Çalıştırır; çalışırken tekrar basılması yürütmeyi keser.
F4 — Reset	Yüklenmiş programı yeniden başlatır; breakpoint'leri korur.
Ctrl+E — Generate	En son assemble edilmiş programın bellek görüntülerini üretir.
Ctrl+N / Ctrl+O / Ctrl+S	Yeni dosya / dosya aç / kaydet.

Kaydetme ve assembly farklı işlemlerdir. Değiştirilen kaynağı çalıştırmadan veya görüntü üretmeden önce F5'in başarılı olması gerekir. Reset, düzenlenen metni yeniden assemble etmez. Başarısız assembly, daha önce yüklen-

miş programı korur.

11.4 İlk Program: Aritmetik, Store ve Load

11.4.1 Kullanım Örneği

examples/rv32/first_steps.s dosyasını açın. Program 5 ile 3'ü toplar, sonucu DMEM'e yazar ve tekrar bir yazmaca yükler.

```
# First lab: add, store, and load
.data
result: .word 0
.text
.globl _start
_start:
    li a0, 5
    li a1, 3
    add a2, a0, a1
    la t0, result
    sw a2, 0(t0)
    lw a3, 0(t0)
    ebreak
```

11.4.2 Nasıl Çalıştırılır?

1. F5'e basın. la için iki instruction üretildiğini açıklayan not hata değildir.
2. Yeni yüklenmiş durumdan F8'e beş kez basın. İlk instruction WB'ye ulaştığında a0 = 5 gözlenir.
3. İlerlemeye devam edin. a1 = 3 ve a2 = 8 olur. la satırı iki machine instruction içerir.
4. sw tamamlandığında Memory sekmesinde adres 0'ın +0 hücresi 00000008 olur. lw tamamlandığında a3 = 8 olur.
5. F9 ile programı bitirin. ebreak bu örnekte bilinçli bir durma noktasıdır.

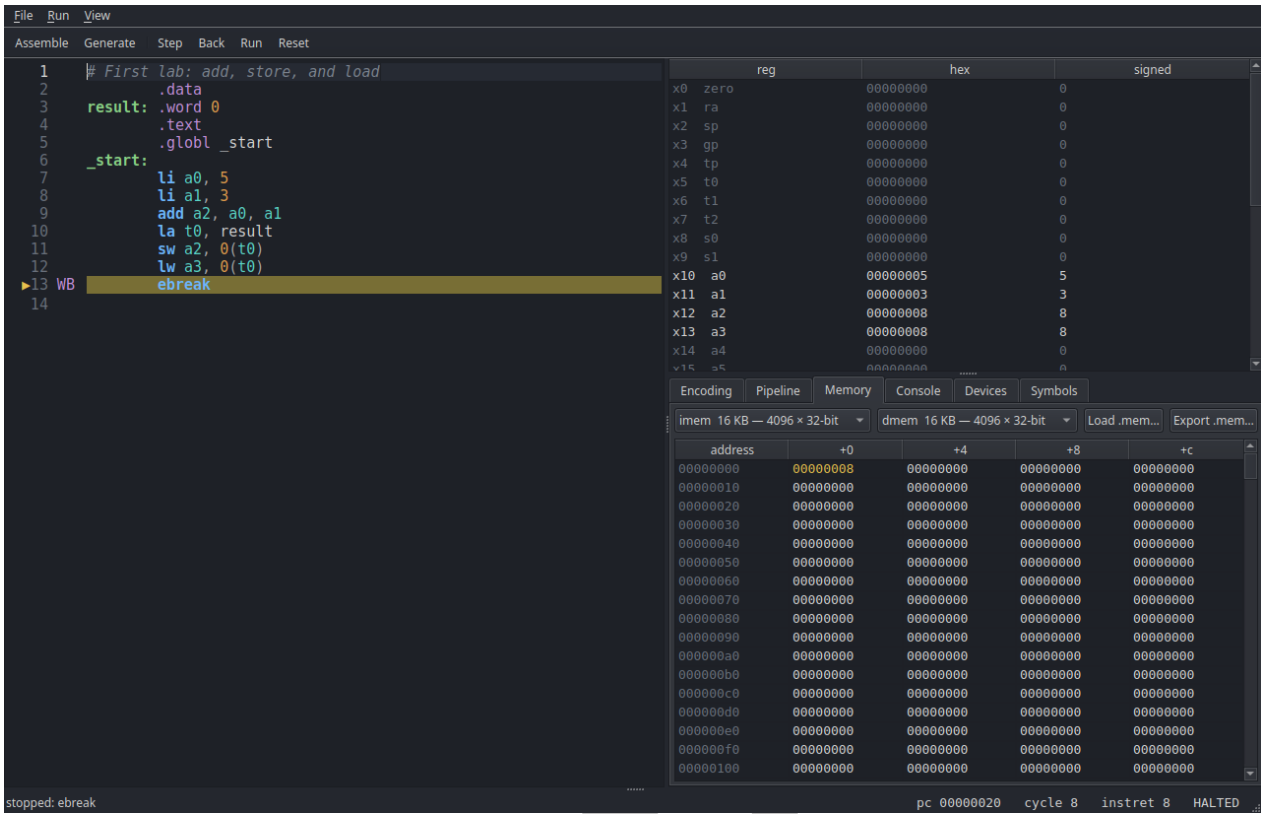
Tablo 11.4: Program sonunda beklenen değerler

Gözlem	Değer
a0 / x10	5
a1 / x11	3
a2 / x12 ve a3 / x13	8 = 0x00000008
t0 / x5	0: result sembolünün DMEM adresi
DMEM[0]	00000008

11.4.3 Öğrenci Uygulaması

İlk sabiti 12 yapın; kaydedin ve tekrar F5, F9 uygulayın. Beklenen sonuç 15'tir. Değişiklikten sonra yalnız Reset kullanmak ile yeniden assemble etmek arasındaki farkı açıklayın.

11.5 Yazmaç, Bellek ve Instruction Gözlemi



Şekil 11.2: İlk program tamamlandıktan sonra veri belleğinde result = 8 gözlenir. Satır adresi ile sütun offset'i birlikte okunur.

11.5.1 Register File Nasıl İzlenir?

Registers görünümü x10 ve a0 gibi iki adlandırmayı birlikte gösterir. Hex ve signed sütunları aynı 32-bit örüntünün farklı yorumlarıdır. Örneğin FFFFFFFF signed gösterimde -1'dir. Son yazılan yazmaç vurgulanır. x0 yazılabilen bir genel amaçlı değer saklamaz; daima sıfır okunur.

11.5.2 DMEM Adresi Nasıl Hesaplanır?

Hücresinin byte adresi, satır adresi ile sütun offset'inin toplamıdır. 00000010 satırının +4 sütunu 0x14 adresidir. Her hücre bir 32-bit word gösterir. lw/sw için adres 4'ün, lh/sh için 2'nin katı olmalıdır. Bellek little-endian'dır: 0x12345678 sözcüğünün artan byte adreslerindeki içeriği 78, 56, 34, 12'dir.

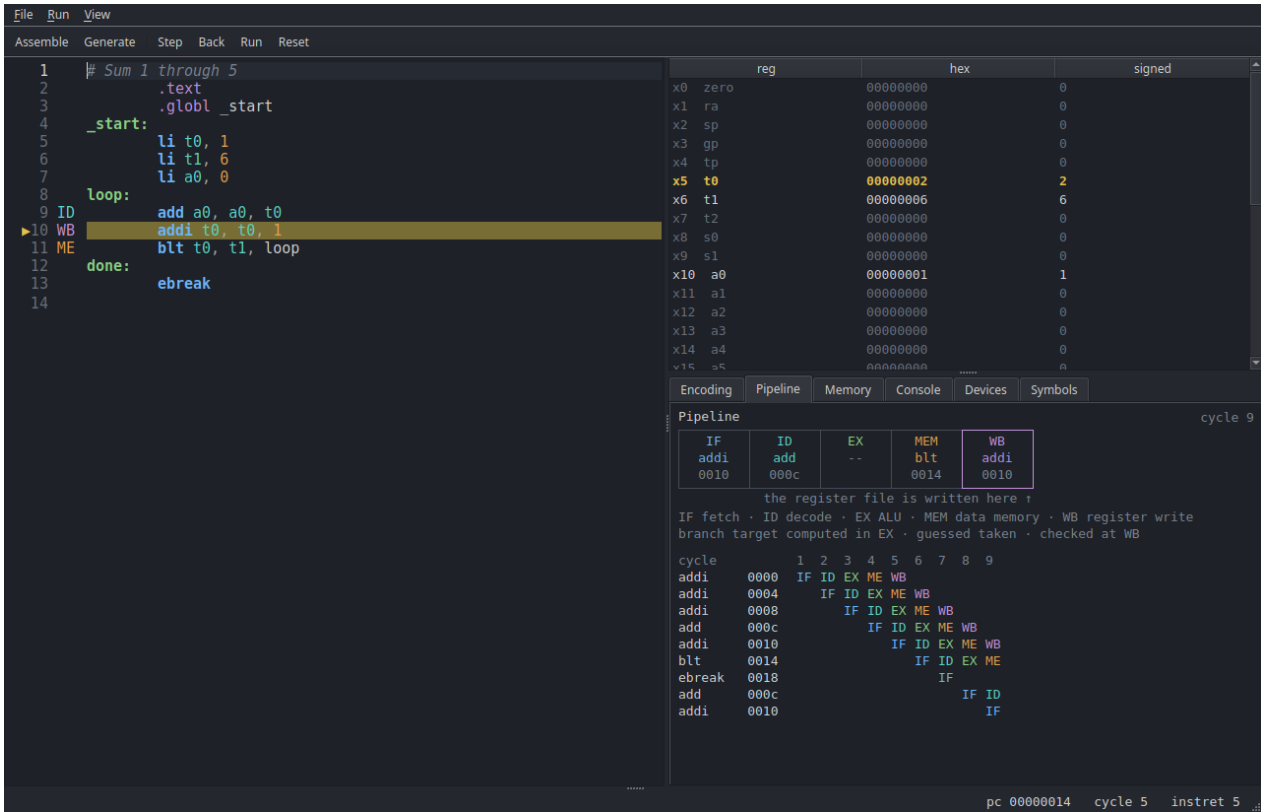
11.5.3 Encoding ve Symbols Ne Gösterir?

Encoding, instruction bit alanlarını ve işlemin etkisini gösterir; stepping sırasında WB'yi veya kodda seçilen satırı izler. Compressed instruction için genişletilmiş temel instruction da görünür. Symbols, etiketin Text veya Data alanında olduğunu gösterir. rv32'de iki bellek de 0'dan başlar; aynı sayısal adres aynı fiziksel yer anlamına gelmez.

11.6 Pipeline ve Hata Ayıklama

11.6.1 Pipeline Nasıl Okunur?

IF, ID, EX, MEM ve WB instruction'ın aşamalarını gösterir. Alt diyagramın satırları instruction'ları, sütunları çevrimleri temsil eder. Sarı kaynak satırı WB'yi gösterir. Çözülmüş branch, alındığında yeşil; alınmadığında kırmızı gösterilir. Ctrl+Shift+P pipeline işaretlerini, Ctrl+Shift+B branch renklerini açıp kapatır.



Şekil 11.3: sum_loop.s programında pipeline görünümü. Son çevrimler instruction bazında izlenebilir.

11.6.2 Bir Adım Ne Anlama Gelir?

F8 tek kaynak satırı yürütmez; bir gösterim çevrimi iletir. Emülatörün mimari durum değişikliği instruction WB'ye geldiğinde uygulanır. Arayüzün çevrim sayısı, yazılımın okuduğu cycle/mcycle/time sayaçlarıyla aynı ölçüm değildir. Kaynak satırı bir pseudo-instruction ise birden fazla instruction'a karşılık gelebilir.

11.6.3 Donanım Pipeline'ından Farkı Nedir?

Aracın öngörüsü always-taken yaklaşımıdır; branch sonucu WB'de kesinleşir ve yanlış öngöründe görünüm temizlenir. Kitabın Bölüm 3 ve 7'sindeki microBee predictor, stall ve hazard özellikleri bu ekranda modellenmez. Bu ekran üzerinden microBee CPI, forwarding veya branch cezası ölçülmez.

11.7 Breakpoint, Geri Adım ve Tanı Mesajları

11.7.1 Döngü Nasıl İncelenir?

sum_loop.s dosyası 1'den 5'e kadar sayıları a0'da toplar. İlk değerler t0 = 1, t1 = 6 ve a0 = 0'dır; gövde add, addi ve blt komutlarından oluşur.

1. F5 ile yükleyin. add a0, a0, t0 satırının sol kenarına tıklayıp breakpoint koyun.
2. F9 ile breakpoint'e gelin. İlerlemeden önce sayaç ve toplamı okuyun.
3. F8 ile add tamamlanana kadar ilerleyin. F6 ile geri gidip yazmacın eski değerine döndüğünü izleyin.
4. Breakpoint'i aynı kenara tıklayarak kaldırın ve F9'a basın. a0 = 15 ve t0 = 6 beklenir.
5. F4 ile yeniden başlatın. Bir fonksiyonu GUI'de step over yapmak için call sonrasındaki çalıştırılabilir satıra breakpoint koyup çalıştırın.

11.7.2 Kod Tamamlama Nasıl Kullanılır?

Mnemonic'in iki harfi yazıldığında öneriler açılır; Ctrl+Space ile açıkça istenebilir. Tab/Shift+Tab seçimler arasında ilerler, Enter seçimi kabul eder, Esc kapatır. Gri operand ipuçları kaynak metnin parçası değildir.

Virgöl, yazmaç ve immediate değerleri kullanıcı tarafından girilir.

Tablo 11.5: Sık yapılan hatalar ve düzeltme yolu

Belirti	İzlenecek yol
Undefined symbol	Etiketın yazımını kontrol edin. Tanı mesajına tıklayarak ilgili satıra gidin.
Immediate sınır dışında	addi aralığı -2048...2047'dir; büyük sabiti li ile yükleyin.
Data etiketine call	Kod hedefi için Text etiketi; veri adresi için la kullanın.
Misaligned load/store	Etkin adresi ve veri yerleşimini kontrol edin.
Kırmızı bellek taşıma işareti	İzin verilen bellek boyutunu büyütün veya programı küçültün; tekrar assemble edin.
Program eski davranışı gösteriyor	F5'in başarıyla tamamlandığını doğrulayın.
Console boş	Programın gerçekten UART'a yazdığını ve cihazın bağlı olduğunu kontrol edin.
Sonsuz döngü	F9 ile kesin; branch operandlarını ve sayaç güncellemesini inceleyin.

Düzeltilme yöntemi: beklenen değeri belirleyin, değişikliği üreten instruction öncesinde durun, ilk farklılığı bulun, kaynağı düzeltin ve aynı başlangıç koşulundan tekrar deneyin.

11.8 Çevre Birimleri ve Interrupt Çalışmaları

11.8.1 UART Kullanım Örneği

uart.s, rv32 varsayılan UART cihazına ASCII 65 ve 10 gönderir. Console'da A ve satır sonu beklenir. Burada ecall tabanlı başka simülatörlerin çıktı servisleri kullanılmaz.

```
li t0, 0xffff0000
li a0, 65
sb a0, 0(t0)
li a0, 10
sb a0, 0(t0)
ebreak
```

Tablo 11.6: rv32 varsayılan çevre birimi adresleri

Adres	Görev
0xFFFF0000	UART; +0 byte gönderimi, +4 status.
0xFFFF0010	LED çıkış register'ı.
0xFFFF0020	Switch giriş register'ı.
0xFFFF0030	mtime düşük/yüksek sözcükleri.
0xFFFF0040	mtimecmp deadline değeri.
0xFFFF0050 / 0xFFFF0054	Software / external interrupt istek register'ları.

11.8.2 Konfigürasyon Nasıl Yüklenir?

File → Load peripherals... ile `examples/rv32/board.toml` yüklenir. Bu işlem cihaz kümesini değiştirir; mevcut cihaza ekleme anlamına gelmez. Dosya bellek boyutlarını da belirleyebilir. `switches_leds.s` bir kez switch okur ve LED'lere yazar. Sağlanan board'un başlangıç girişi 0b1010 olduğundan `a0 = 10` beklenir. Girişi değiştirip Reset ve Run ile yeniden örnekleyin. Switch girişi reset boyunca korunur.

11.8.3 Interrupt Çalışırken Neye Dikkat Edilir?

Önce `mtvec` handler adresi kurulmalı, `mie` içinde ilgili kaynak ve `mstatus.MIE` etkinleştirilmelidir. Handler, kaynağı temizlemeli veya timer deadline'ını iletmelidir. Interrupt dönüşünde `mepc` iletilemez; `ecall exception`'ında ise aynı komutun tekrarını önlemek için dönüş adresi uygun biçimde düzenlenir. Başlangıç için ders deposunda sağlanan timer örneği kullanılabilir; bu kitap paketinde henüz `examples/rv32/` altında `timer.s` bulunmamaktadır.

rv32, kitabın PLIC claim/complete mekanizmasını ve gölge yazmaç bankasını uygulamaz. Kitabın Bölüm 5 örneklerini doğrudan bu varsayılan cihaz haritasıyla çalıştırmayın. Adres ve davranış farkları Bölüm 11.10'da birlikte verilmiştir.

11.9 Bellek Görüntüsü Üretme

11.9.1 Generate Nasıl Kullanılır?

1. İlgili board dosyasını yükleyin ve Memory sekmesinde IMEM/DMEM boyutlarını belirleyin.
2. Kaynağı kaydedin; F5 ile başarılı assembly ve taşıma kontrolü yapın.
3. Generate, Ctrl+E veya File → Generate .mem... seçin.
4. Folder ve Base name alanlarını doldurun. lab01 adı, lab01_imem.mem ve lab01_dmем.mem üretir. Varsayılan ad rv32'dir.
5. İstenen bellekleri işaretleyin. Düz dosya isteniyorsa annotation seçimini kapalı tutun; Export'a basın.

Tablo 11.7: Dışa aktarma işlemlerinin kapsamı

İşlem	Üretilen veri
Generate / Ctrl+E	En son assemble edilen instruction görüntüsü ve başlangıç .data içeriği. Bellek derinliğine tamamlanır.
Memory → Export .mem...	Çalışma ve manuel düzenlemeler sonrasındaki güncel DMEM.
Devices → blok → Export .mem...	Seçili RAM/ROM bloğunun kendi güncel görüntüsü.

İlk program çalıştıktan sonra result = 8 olsa bile Generate başlangıçtaki .word 0 değerini üretir. Güncel değeri saklamak için Memory sekmesinin export işlemi kullanılır. Generate, sonradan yapılan kaynak değişikliğini otomatik assemble etmez.

11.9.2 Dosya Nasıl Okunur?

Her satır 32-bit hexadecimal word'dür. İlk satırın indeksi 0 kabul edilirse n numaralı satırın byte offset'i $4 \times n$ 'dir. 16 KiB bellek 4096×32 -bit derinliğe karşılık gelir. Compressed instruction'lar aynı word akışına paketlenir; bir satır her zaman tek instruction değildir.

```
rv32 asm first_steps.s --pad -o lab01_imem.mem --dmem lab01_dmем.mem
rv32 asm first_steps.s --pad --devices board.toml
```

Terminalde -pad verilmezse yalnız kullanılan kısım dışa aktarılır. Özel bellek boyutları için aynı board dosyasını verin. Donanımın word-adresli ve byte enable içermeyen özel bir belleği kullanılıyorsa -strict-word-mem ile alt-sözcük erişim uyarıları alınabilir.

11.9.3 Vivado'ya Geçiş Koşulu

Bu çıktı \$readmemh biçimindeki word dizisidir. Laboratuvarın simülasyon veya IP initialization akışının istediği biçim ve derinlik doğrulanmalıdır. Dosya adını değiştirmek format dönüşümü değildir. REF32 için yalnız derinliği büyütme yeterli değildir; Bölüm 11.10'daki adres ve cihaz uyarlaması da gerekir.

11.10 microBee-REF32 ile Birlikte Kullanım

Bu karşılaştırma ana kitabın Bölüm 3, 5-9 ve Ek A'sı ile incelenen rv32 kaynaklarından çıkarılmıştır. İşlevsel sonucun karşılaştırılabilmesi için aynı başlangıç verisi, algoritma ve gözlem noktası tanımlanmalıdır.

Tablo 11.8: REF32 ve rv32 arasında taşımayı etkileyen farklar

Konu	Kitaptaki REF32	İncelenen rv32
IMEM	0x80000000; 64 KiB	Ayrı Text alanı, 0'dan başlar; varsayılan 16 KiB.
DMEM	0x20000000; 64 KiB	Ayrı Data alanı, 0'dan başlar; varsayılan 16 KiB.
MMIO	0x40000000 çevresindeki native cihazlar; AXI penceresi 0x60000000	0xFFFF0000–0xFFFF03FF slot penceresi.
GPIO / UART	GPIO register bankası, native UART ve AXI UART Lite	LED/switch ve basitleştirilmiş UART cihazları; register davranışları eşdeğer değildir.
Interrupt	CLINT, PLIC kaynakları; ilgili donanım seçenekleri	Software, timer, external hatları; PLIC claim/complete veya shadow bank yok.
Pipeline	Donanım hazard/stall ve predictor konfigürasyonu	Öğretim görünümü; hazard/stall/forwarding modeli yok.
Zaman / ölçüm	Clock tabanlı mcycle ve monitoring register'ları	Instruction tabanlı sayaçlar ve ayrı GUI çevrim sayımı.
Timer	Donanım timer/CLINT register modeli	Basitleştirilmiş mtime/mtimecmp; mtimecmp 32-bit.
Durma	Hedef debug/trap/testbench akışına bağlı	ebreak varsayılan olarak debugger'a durur.

11.10.1 Bir Board Dosyası REF32 Uyumu Sağlar mı?

Hayır. Desteklenen board anahtarları bellek boyutlarını ve rv32 cihazlarını seçer; REF32'nin bütün bus, GPIO, UART, PLIC veya monitoring modelini oluşturmaz. 64 KiB seçmek IMEM başlangıcını 0x80000000 veya DMEM başlangıcını 0x20000000 yapmaz.

11.10.2 Taşıma Nasıl Hazırlanır?

1. Algoritmayı ve beklenen sonucu rv32'de deneyin; ortamı sonuç kaydına yazın.
2. Hedef assembly/link yerleşiminde Text, Data ve giriş adreslerini REF32'ye göre kurun. la ile oluşan mutlak veri adreslerini ve bütün MMIO erişimlerini yeniden değerlendirin.
3. Cihaz sürücülerini, erişim genişliklerini, interrupt kurulumunu ve program bitiş yöntemini hedefe uyarlayın.
4. Hedef için üretilmiş görüntüyü RTL simülasyonunda doğrulayın. Performans ölçümünü kitabın monitoring/FPGA akışında yapın.

11.11 Komut Satırı ve Tekrarlanabilir Deney

11.11.1 Temel Komutlar

Aşağıdaki komutları örneklerin bulunduğu çalışma klasöründe uygulayın. rv32 PATH üzerinde değilse yürütülebilir dosyanın yolunu yazın. Windows PowerShell'de geçerli klasördeki program `.\rv32.exe` biçiminde çağrılır.

```
rv32 run first_steps.s
rv32 run first_steps.s --trace
rv32 run sum_loop.s --limit 1000
rv32 dis lab01_imem.mem
rv32 run switches_leds.s --devices board.toml
rv32 dbg sum_loop.s
```

Trace, machine instruction'ları ve etkilerini gösterir. Bir pseudo-instruction birden çok satır üretir. Varsayılan çalıştırma sınırı 1.000.000 instruction'dır. Sınıra ulaşmak programın doğru bittiğini kanıtlamaz. Padding içeren IMEM disassembly'si programdan sonraki kullanılmayan alanı da gösterebilir.

11.11.2 Debugger Nasıl Kullanılır?

```
(rv32) break done
(rv32) run
(rv32) info registers
```

```
(rv32) print a0
(rv32) backi
(rv32) si
(rv32) continue
(rv32) quit
```

Tablo 11.9: Terminal debugger komutları

Komut	İşlev
step / si	Bir kaynak satırı / bir machine instruction.
next / finish	Call'ı tamamlayarak kaynak adımı / fonksiyon dönüşüne kadar ilerleme.
back / backi	Bir kaynak satırı / bir instruction geri alma.
run / continue	Baştan reset ederek çalıştırma / mevcut durumdan sürdürme.
watch result	result tanımlı bir programda ilgili veri word'ünün değişimini izleme.
x/4xw 0	0 adresinden dört word'ü hex olarak okuma.
help	Komut listesini ve kullanımını gösterme.

GUI F8/F6 çevrim birimindedir; terminal step/back kaynak satırı, si/backi instruction birimindedir. Terminalde boş satır önceki komutu tekrarlar.

11.12 Öğrenci Uygulamaları ve Sonuç Kaydı

11.12.1 Uygulama Sırası

Tablo 11.10: Başlangıç uygulamaları

Uygulama	Değişiklik / gözlem	Beklenen
Aritmetik	first_steps.s: 5 yerine 12	a2 = a3 = 15; DMEM[0] = 0000000f.
Döngü	sum_loop.s: üst sınır 6 yerine 11	1...10 toplamı; a0 = 55.
UART	uart.s: 65 yerine 66	Console'da B ve satır sonu.
Giriş/çıkış	Switch değerini değiştir, reset et, çalıştır	a0 ve LED'ler örneklenen değeri taşır.
Encoding	addi a0, a0, 1 ile c.addi a0, 1 karşılaştır	Aynı aritmetik etki; 32/16-bit encoding.
Hata bulma	Döngü branch hedefini geçici olarak lop yap	Undefined symbol tanısı; düzeltme sonrası başarılı assembly.

11.12.2 Sonuç Nasıl Raporlanır?

Tablo 11.11: İşlevsel deney kaydı şablonu

Alan	Kaydedilecek bilgi
Kimlik	Ad/numara, tarih, deney adı; dersin istediği biçimde.
Ortam	rv32 sürümü; board dosyası; IMEM/DMEM boyutları.
Program ve giriş	Kaynak dosya adı, değişiklik, giriş sabitleri ve switch değerleri.
Beklenen	Kontrol noktasındaki yazmaç/bellek/Console değeri.
Gözlenen	Aynı kontrol noktasındaki gerçek değer ve gerekiyorsa ekran görüntüsü.
Sonuç	Beklentiyle uyum; ilk fark ve düzeltme açıklaması.
Çıktılar	İstenen .s, board ve .mem dosyalarının adları; initial/current ayrımı.

Bu kayıt işlevsel deney içindir. Ana kitabın Bölüm 9'undaki donanım performans raporu için gerekli clock, monitoring sayaçları ve CPI/IPC ölçümleri rv32 ekranından türetilmez. Teslim kapsamı ve değerlendirme kuralları öğretim elemanı tarafından belirlenir.

11.12.3 Kontrol Soruları

- İlk F8 basışında a0 neden hemen değişmeyebilir?
- la satırı neden birden fazla instruction üretebilir?

- Generate ile Memory export sonucunda result neden farklı olabilir?
- Aynı byte offset'ine sahip iki bellek görüntüsü neden REF32'de doğrudan çalışmayabilir?
- Bir branch sonucunun doğru olması donanımda kaç clock süreceğini gösterir mi?

11.13 Kaynaklar ve Bölümler Arası İlişki

Bu bölümün kaynakları aşağıda verilmiştir. Ana kitap atıflarındaki sayfa numaraları PDF görüntüleyicisinin sıra numarası yerine kitabın basılı sayfa numaralarıdır.

Tablo 11.12: Kullanılan kaynaklar

Kod	Kaynak ve kullanım
K1	EHB326_Book_MCU.pdf, Rev. P1.0: §2.4, s. 2; Bölüm 3, s. 3–17; Bölüm 5, s. 36–49; Bölüm 6–7, s. 51–66; Bölüm 8–10, s. 67–94; Ek A, s. 97–104.
K2	rv32 README.md ve docs/building.md: uygulama açılışı ve paket kullanımı.
K3	docs/keybindings.md; src/gui/main_window.cpp: kısayollar, paneller, export kapsamı ve adımlama.
K4	docs/memory-map.md; src/core/device_config.cpp: emülatör adres alanları ve board kapsamı.
K5	docs/isa-support.md; src/dbg/pipeline.cpp: ISA ve gösterim sınırları.
K6	src/main.cpp ve src/cmd/commands.cpp: CLI seçenekleri ve debugger komutları.
K7	Kitaptaki <code>examples/rv32/</code> ve <code>validation.txt</code> dosyaları: çalıştırılmış başlangıç örnekleri ve sonuçları.

Ana kitapta “Nedir?”, “Ne İşe Yarar?”, “Nasıl Kullanılır?”, kullanım örneği ve debug açıklamalarıyla kurulan anlatım düzeni burada korunmuştur. Yeni içerik, ana kitabın donanım tanımını değiştirmeden araç kullanımını tamamlar.

Birleştirme için önerilen yer, Bölüm 10’dan sonra ve kaynakça/eklerden öncedir. §2.4’e kısa tanıtım eklenerek bu bölüme yönlendirme yapılabilir. Alternatif yerleşimde bölüm numarası ana belgenin sayacıyla güncellenmeli; şekil, tablo ve çapraz atıflar yeniden üretilmelidir.

Kaynakça

- [1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Resmi çevrimiçi belge.
- [2] RISC-V International, *RV32I Base Integer Instruction Set, Version 2.1*. Resmi RV32I referansı.
- [3] RISC-V International, *M Extension for Integer Multiplication and Division, Version 2.0*. Resmi M uzantısı referansı.
- [4] RISC-V International, *C Extension for Compressed Instructions, Version 2.0*. Resmi C uzantısı referansı.
- [5] RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. Resmi privileged ISA belgesi.
- [6] Arm Limited, *Armv8-A Instruction Set Architecture*, Non-Confidential, Issue 1.1, Document ID: ARM062-948681440-3280, 2019–2020.

DRAFT

Ek A

Adres Haritası ve Sistem Kapasitesi

Bu bölümde REF32 profilinde kullanılan bütün adres decode pencereleri ve tanımlı register adresleri artan adres sırasına göre verilmiştir.

Adres decode sırasında seçilen pencere, ilgili çevre biriminin gerçekten kullandığı register alanından daha büyük olabilir. Bu nedenle bir adresin ilgili peripheral penceresi içerisinde bulunması, o adresin mutlaka geçerli bir register'a karşılık geldiği anlamına gelmez.

Tanımlanmamış bir offset'e veya desteklenmeyen hizada bir adrese yapılan erişim error response üretir.

Bir register'ın mutlak adresi:

$$\text{mutlak adres} = \text{taban adresi} + \text{offset}$$

şeklinde hesaplanır.

A.1 REF32 Kapasite Özeti

REF32 profili microBee'nin referans SoC konfigürasyonunu ifade eder. Bu profil içerisinde kullanılan temel memory, peripheral ve mikro-mimari kaynakların kapasiteleri aşağıdaki tabloda verilmiştir.

Tablo A.1: REF32 sistem kapasitesi.

Kaynak	Kapasite	Not
IMEM BRAM	16384 × 32 bit, 64 KiB	Instruction fetch, başlangıç adresi 0x80000000
DMEM BRAM	16384 × 32 bit, 64 KiB	Load/store, başlangıç adresi 0x20000000
Reset / mtvec	0x80000000	RESET_VECTOR
MMIO pencereleri	6 × 4 KiB	CLINT, PLIC, GPIO, TIMER, UART ve MONITOR
GPIO	32 bit, 1 bank	GPIO_WIDTH=32
PLIC	16 interrupt kaynağı, 3-bit priority	ID 1 AXI UART, ID 2 Timer, ID 3 Native UART
Native UART FIFO	16 giriş	8N1 UART
Timer	64-bit sayaç, 2 compare kanalı	PLIC ID 2
Monitoring	32-bit saturating sayaçlar	Shadow register'lar için offset +0x100
Branch predictor	128 giriş, bimodal	BP_ENTRIES=128
AXI penceresi	256 MiB	Başlangıç adresi 0x60000000
FPGA clock / baud	50 MHz / 115200	AXI UART Lite

REF32 profilinde yazılım tarafından kullanılabilen temel memory alanları IMEM, DMEM ve AXI penceresidir. IMEM programın instruction'larını tutar ve Fetch birimi tarafından kullanılır. DMEM ise load/store instruction'larının eriştiği veri belleğidir.

AXI penceresi yalnızca:

$$\text{AXI_ENABLE}=1$$

olduğunda kullanılabilir.

MMIO bölgelerinde ise yalnızca ilgili peripheral için tanımlanmış register adresleri geçerlidir. Decode penceresinin geri kalanına yapılan erişimler error response üretir.

A.2 Global Bellek Haritası

microBee request router içerisinde adres bölgeleri belirli bir öncelik sırasına göre kontrol edilir.

Router önceliği:

CLINT → PLIC → GPIO → TIMER → UART → MONITOR → DMEM → AXI → Error

şeklindedir.

REF32 global memory map aşağıdaki gibidir.

Tablo A.2: REF32 bellek pencereleri, adres sırasıyla.

Başlangıç	Son	Bölge	Erişim
0x00000000	0x1FFFFFFF	Boş	Error
0x20000000	0x2000FFFF	DMEM BRAM	RW, 64 KiB
0x20010000	0x3FFFFFFF	Boş	Error
0x40000000	0x40000FFF	CLINT	Register haritası
0x40001000	0x40001FFF	PLIC	Register haritası
0x40002000	0x40002FFF	GPIO	Register haritası
0x40003000	0x40003FFF	TIMER	Register haritası
0x40004000	0x40004FFF	Native UART	Register haritası
0x40005000	0x40005FFF	MONITOR	Register haritası
0x40006000	0x40006FFF	Ayrılmış mailbox	REF32'de Error
0x40007000	0x5FFFFFFF	Ayrılmış	Error
0x60000000	0x6FFFFFFF	AXI4-Lite	AXI UART Lite @ 0x60000000
0x70000000	0x7FFFFFFF	Boş	Error
0x80000000	0x8000FFFF	IMEM BRAM	Instruction fetch, data alias kapalı
0x80010000	0xFFFFFFFF	Boş	Error

Bir peripheral'in enable parametresi 0 ise ilgili peripheral sistem adres haritasından çıkar.

Örneğin:

MONITOR_ENABLE=0

olduğunda MONITOR penceresine yapılan erişimler geçerli bir peripheral tarafından karşılanmaz ve error response oluşur.

A.3 DMEM — 0x20000000

DMEM, programın load ve store işlemleri için kullandığı 64 KiB büyüklüğündeki RW veri belleğidir.

Başlangıç adresi:

0x20000000

olarak tanımlanmıştır.

Word, half-word ve byte store işlemlerinde hangi byte lane'lerin yazılacağı **wstrb** sinyali ile belirlenir.

Yazılım örneklerinde stack pointer genellikle:

0x2000FF00

civarında başlatılır.

Test firmware’i bazı DMEM adreslerini yazılımsal mailbox olarak kullanabilir. Örneğin:

- 0x20000000: test signature,
- 0x20000100: test phase.

Bu adreslerin mailbox olarak kullanılması ayrı bir donanım decode mekanizması değildir. Adresler normal DMEM alanının bir parçasıdır.

A.4 CLINT — 0x40000000

CLINT, machine software interrupt ve machine timer interrupt üretiminde kullanılan çevre birimidir.

Base adresi:

0x40000000

olarak tanımlanmıştır.

Tablo A.3: CLINT register haritası.

Offset	Mutlak adres	Register	Erişim
0x000	0x40000000	MSIP	RW, bit 0
0x400	0x40000400	MTIMECMP_LO	RW
0x404	0x40000404	MTIMECMP_HI	RW
0xFF8	0x40000FF8	MTIME_LO	RW
0xFFC	0x40000FFC	MTIME_HI	RW

mtime ve mtimecmp 64-bit değerlerdir ancak RV32 sistemde 32-bit register’lar üzerinden iki parça halinde erişilir.

A.5 PLIC — 0x40001000

PLIC, external interrupt kaynaklarının priority, pending, enable ve trigger mode bilgilerinin yönetilmesinde kullanılır.

REF32 profilinde 16 external interrupt kaynağı bulunur.

PLIC base adresi:

0x40001000

olarak tanımlanmıştır.

Tablo A.4: PLIC register haritası.

Offset	Mutlak adres	Register	Erişim
0x000	0x40001000	CAPABILITY	RO
0x004+4(<i>i</i> − 1)	0x40001004	PRIORITY[<i>i</i>]	RW, <i>i</i> = 1..16
	başlangıç		
0x100	0x40001100	PENDING	RO
0x104	0x40001104	SOURCE_MODE	RW
0x108	0x40001108	INPUT_STATE	RO
0x10C	0x4000110C	RESET_LEVEL	RO
0x180	0x40001180	ENABLE	RW
0x300	0x40001300	THRESHOLD	RW
0x304	0x40001304	CLAIM	RO, okuma yan etkili
0x308	0x40001308	COMPLETE	WO

A.6 GPIO — 0x40002000

GPIO çevre birimi genel amaçlı dijital giriş ve çıkış pinlerinin yazılım tarafından kontrol edilmesini sağlar.

Base adresi:

0x40002000

olarak tanımlanmıştır.

Tablo A.5: GPIO register haritası.

Offset	Mutlak adres	Register	Erişim
0x00	0x40002000	IN	RO, yazma no-op
0x04	0x40002004	OUT	RW
0x08	0x40002008	OUT_SET	WO, okuma 0
0x0C	0x4000200C	OUT_CLR	WO, okuma 0
0x10	0x40002010	OUT_TGL	WO, okuma 0
0x14	0x40002014	OE	RW
0x18	0x40002018	VALID	RO
0x1C	0x4000201C	—	Geçersiz, error response

A.7 TIMER — 0x40003000

Native Timer, genel amaçlı zamanlama ve compare işlemleri için kullanılan 64-bit timer çevre birimidir.

Base adresi:

0x40003000

olarak tanımlanmıştır.

Tablo A.6: Native Timer register haritası.

Offset	Mutlak adres	Register	Erişim
0x00	0x40003000	CTRL	RW
0x04	0x40003004	PRESCALE	RW
0x08	0x40003008	COUNT_LO	RW
0x0C	0x4000300C	COUNT_HI	RW
0x10	0x40003010	CMPO_LO	RW
0x14	0x40003014	CMPO_HI	RW
0x18	0x40003018	CMP1_LO	RW
0x1C	0x4000301C	CMP1_HI	RW
0x20	0x40003020	PERIOD_LO	RW
0x24	0x40003024	PERIOD_HI	RW
0x28	0x40003028	STATUS	RW1C
0x2C	0x4000302C	CAPABILITY	RO

Timer interrupt çıkışı REF32 profilinde PLIC ID 2'ye bağlanmıştır.

A.8 Native UART — 0x40004000

Native UART, microBee RTL içerisinde bulunan UART çevre birimidir.

Base adresi:

0x40004000

olarak tanımlanmıştır.

Tablo A.7: Native UART register haritası.

Offset	Mutlak adres	Register	Erişim
0x00	0x40004000	DATA	RW, TX için yazma / RX için okuma
0x04	0x40004004	STATUS	RO
0x08	0x40004008	CTRL	RW
0x0C	0x4000400C	BAUDDIV	RW, minimum 8
0x10	0x40004010	IRQ_STATUS	RW1C
0x14	0x40004014	FIFO_LEVEL	RO
0x18	0x40004018	CAPABILITY	RO

Native UART interrupt çıkışı REF32 profilinde PLIC ID 3'e bağlanmıştır.

A.9 MONITOR — 0x40005000

Monitoring birimi mikro-mimari performans olaylarının donanım sayaçları üzerinden izlenmesini sağlar.

Base adresi:

0x40005000

olarak tanımlanmıştır.

MON_CTRL register'ı RW, performans sayaçları ise RO'dur.

Snapshot alınan shadow register'ların offset'i ilgili live sayaç offset'ine:

+0x100

eklenerek bulunur.

Tablo A.8: Monitoring register haritası.

Offset	Mutlak adres	Register	Erişim
0x000	0x40005000	MON_CTRL	RW
0x004	0x40005004	CYCLES	RO
0x008	0x40005008	RETIRED	RO
0x00C	0x4000500C	STALL	RO
0x010	0x40005010	FLUSH	RO
0x014	0x40005014	BUBBLE	RO
0x018	0x40005018	HAZARD	RO
0x01C	0x4000501C	LOAD_USE	RO
0x020	0x40005020	BRANCH	RO
0x024	0x40005024	BRANCH_TAKEN	RO
0x028	0x40005028	MISPREDICT	RO
0x02C	0x4000502C	IMEM_WAIT	RO
0x030	0x40005030	DMEM_WAIT	RO
0x034	0x40005034	AXI_WAIT	RO
0x038	0x40005038	BUS_ERROR	RO
0x03C	0x4000503C	IRQ_TAKEN	RO
0x040	0x40005040	ALU_USE	RO
0x044	0x40005044	MUL_USE	RO
0x048	0x40005048	DIV_USE	RO
0x04C	0x4000504C	CAPABILITY	RO
0x050	0x40005050	LOAD_COUNT	RO
0x054	0x40005054	STORE_COUNT	RO
0x058	0x40005058	BRANCH_STALL	RO
0x05C	0x4000505C	MEM_WAIT_TOTAL	RO
0x060	0x40005060	PREDICT_CORRECT	RO
0x064	0x40005064	SPEC_REDIRECT	RO
0x104	0x40005104	CYCLES shadow	RO
+0x100	live offset + 0x100	Diğer shadow sayaçlar	RO

A.10 AXI UART Lite — 0x60000000

AXI UART Lite, microBee RTL içerisinde bulunan Native UART'tan farklıdır.

Bu birim FPGA tasarımı Vivado catalog IP olarak sisteme eklenir ve AXI4-Lite üzerinden microBee'ye bağlanır.

REF32 FPGA konfigürasyonunda base adresi:

0x60000000

olarak tanımlanmıştır.

Tablo A.9: AXI UART Lite register haritası.

Offset	Mutlak adres	Register	Erişim
0x00	0x60000000	RX_FIFO	RO
0x04	0x60000004	TX_FIFO	WO
0x08	0x60000008	STAT	RO
0x0C	0x6000000C	CTRL	RW

A.11 IMEM — 0x80000000

IMEM, instruction'ların tutulduğu 64 KiB büyüklüğündeki program belleğidir.

Başlangıç adresi:

0x80000000

olarak tanımlanmıştır.

Reset vector ve varsayılan `mtvec` adresi bu bölgededir.

IMEM yalnız instruction fetch tarafında kullanılır. Data-bus alias kapalı olduğu için normal load/store instruction'ları üzerinden IMEM erişimi yapılmaz.

A.11.1 Hizalama

MMIO erişimleri word aligned olmalıdır.

Geçerli hizalama:

`address[1:0] = 00`

şeklindedir.

Desteklenmeyen hizadaki erişimlerde `rsp_err` üretilir.

A.12 CSR Numaraları

CSR adresleri normal memory adresleri ile karıştırılmamalıdır.

CSR, 12-bit bir register numarasıdır ve load/store adresi değildir.

CSRRW, CSRRS, CSRRC ve bunların immediate biçimlerinde CSR numarası instruction'ın:

`instruction[31:20]`

alanında bulunur.

microBee yalnız machine mode'u uygular.

Desteklenen temel CSR kümesi aşağıdaki tabloda verilmiştir.

Tablo A.10: microBee machine-mode CSR kümesi, numara sırasıyla [5].

Numara	CSR	Erişim	İşlev
0x300	<code>mstatus</code>	MRW/WARL	MIE, MPIE ve MPP
0x304	<code>mie</code>	MRW/WARL	bit3 MSIE, bit7 MTIE, bit11 MEIE
0x305	<code>mtvec</code>	MRW/WARL	Handler base adresi, [1:0]=00, Direct mode
0x340	<code>mscratch</code>	MRW	Trap sırasında yazılım tarafından kullanılabilecek çalışma alanı
0x341	<code>mepc</code>	MRW/WARL	Trap dönüş PC'si, [0]=0
0x342	<code>mcause</code>	MRW	bit31 interrupt bilgisi ve cause kodu
0x343	<code>mtval</code>	MRW	Trap ile ilişkili değer
0x344	<code>mip</code>	MRW/WARL	MSIP, MTIP ve MEIP pending bilgileri
0xB00	<code>mcycle</code>	MRW	Cycle sayacı [31:0]
0xB02	<code>minstret</code>	MRW	Retired instruction sayacı [31:0]
0xB80	<code>mcycleh</code>	MRW	Cycle sayacı [63:32]
0xB82	<code>minstreth</code>	MRW	Retired instruction sayacı [63:32]

Reset sonrasında temel CSR değerleri:

- `mstatus` = 0x00001800,
- `mie` = 0,
- yazılımsal `mip` bitleri = 0,
- `mtvec` = 0x80000000,
- `mcycle` = 0,
- `minstret` = 0.

`mcycle` her clock cycle'da artar.

`minstret` ise mimari olarak tamamlanan instruction'lar ile artar.

Bu CSR sayaçları, monitoring birimindeki memory-mapped `CYCLES` ve `RETIRE` sayaçları ile aynı register'lar değildir.

`CSRRS` veya `CSRRC` instruction'ında kaynak register `x0` ise yalnızca CSR okuma işlemi yapılır.

`CSRWR` ise CSR üzerinde yazma işlemi gerçekleştirir.

A.12.1 mcause Kodları

`mcause`, trap sebebini belirtir.

Bit 31, trap'ın interrupt olup olmadığını belirtir:

- 0 = synchronous exception,
- 1 = interrupt

Tablo A.11: Exception cause kodları, bit31=0.

mcause	Olay
0	Instruction address misaligned
1	Instruction access fault
2	Illegal instruction
3	Breakpoint (<code>ebreak</code>)
4	Load address misaligned
5	Load access fault
6	Store address misaligned
7	Store access fault
11	Environment call, M-mode <code>ecall</code>

Tablo A.12: Interrupt cause kodları, bit31=1.

mcause	Kaynak
0x80000003	Machine Software Interrupt, CLINT <code>MSIP</code>
0x80000007	Machine Timer Interrupt, CLINT <code>mtime/mtimecmp</code>
0x8000000B	Machine External Interrupt, PLIC

microBee içerisinde aynı anda birden fazla interrupt kabul edilebilir durumda olduğunda interrupt önceliği yüksekten düşüğe:

External → Software → Timer

şeklindedir.